

MIXR Technical Report

June 10, 2026

Prepared by:
Framework Team

Prepared for:
ASA Team

Contents

List of Figures		viii
List of Tables		xii
I	Introdução	2
I.1	O que é o MIXR	2
I.2	Objetivo e público-alvo	2
I.3	Versão documentada	2
I.4	Convenções do documento	3
I.5	Organização do manual	4
II	Arquitetura geral	5
II.1	Princípios de projeto	5
II.2	Organização em camadas	5
II.3	Modelo de objetos: os três mecanismos da composição	5
II.4	Ciclo de atualização	8
II.5	Modelo de <i>threading</i>	10
II.6	Um cenário para acompanhar o manual	11
III	A camada base	13
III.1	O modelo de classes e suas macros	13
III.1.1	DECLARE_SUBCLASS e IMPLEMENT_SUBCLASS	13
III.1.2	STANDARD_CONSTRUCTOR e STANDARD_DESTRUCTOR	14
III.1.3	copyData e deleteData	15
III.1.4	MetaObject: o registro estático de uma classe	16
III.1.5	SlotTable: indexação cumulativa de <i>slots</i>	16
III.1.6	Object: despacho de <i>slots</i> e identificação de tipo	18
III.2	Component: a árvore, o tempo e os eventos	18
III.2.1	A árvore de contenção	19
III.2.2	Navegando a árvore: a família <i>find</i>	20
III.2.3	A propagação do tempo, e o que <i>tcFrame()</i> acrescenta	22
III.2.4	Eventos: mensagens nomeadas na árvore	23
III.2.5	Ciclo de vida: <i>reset</i> , <i>freeze</i> e <i>shutdown</i>	25
III.2.6	Seleção, estatísticas e mensagens	26
III.2.7	Os <i>slots</i> de <i>Component</i>	27
III.2.8	Exemplo completo: montar, navegar e instrumentar	27
III.3	O <i>parser</i> EDL: da gramática à árvore	28
III.3.1	Nome de fábrica $\neq$ nome de classe	29
III.3.2	A gramática do EDL	31
III.3.3	<i>Slots</i> posicionais: por que (<i>Feet 10000</i>) funciona	31
III.3.4	Percurso completo de uma linha EDL	33
III.4	Estruturas de contenção	34
III.4.1	List: a lista e sua semântica de posse	34
III.4.2	Pair e PairStream: nomes como cidadãos de primeira classe	36
III.5	Sistema de unidades	37
III.5.1	O padrão de família: classe abstrata + referência + subclasses	37
III.5.2	convertStatic: conversão sem instanciar o tipo de destino	38
III.5.3	Grandezas compostas: LinearVelocity e AngularVelocity	40

III.5.4	Catálogo de famílias	40
III.6	Álgebra: vetores, matrizes e quaternions	41
III.6.1	Vetores	42
III.6.2	Matrizes	42
III.6.3	Quaternions	43
III.6.4	Padrão de uso no código do MIXR	43
III.6.5	Transforms : matrizes declaradas em EDL	44
III.6.6	Matrix e Vectors : a outra álgebra	44
III.7	Geodésia: EarthModel e nav_utils	44
III.7.1	EarthModel : parâmetros do elipsoide de referência	45
III.7.2	Os três referenciais e as conversões entre eles	45
III.7.3	LLA $\leftrightarrow$ ECEF	45
III.7.4	LLA $\leftrightarrow$ NED	46
III.7.5	ECEF $\leftrightarrow$ NED: a matriz mundo	47
III.7.6	Círculo máximo: gll2bd e gbd211	47
III.7.7	Matrizes de orientação e utilitários angulares	48
III.7.8	Fluxo completo: posição de um <i>player</i> em todos os referenciais	49
III.8	<i>Threading</i>	49
III.8.1	AbstractThread : a base comum	50
III.8.2	PeriodicThread : agendamento por tempo absoluto	51
III.8.3	SyncThread : sinalização por <i>mutex</i> pré-bloqueado	53
III.9	Máquinas de estado	55
III.9.1	O ciclo de execução por <i>frame</i>	55
III.9.2	Definindo estados: STATE_FUNC e STATE_MACH	57
III.9.3	Funções de transição: além do switch	58
III.9.4	Composição via EDL: comportamento principal e submáquinas	59
III.9.5	Eventos: ON_ENTRY , ON_EXIT , ON_RETURN	61
III.10	<i>Framework</i> UBF	61
III.10.1	As três abstrações: State , Behavior e Action	61
III.10.2	O Agent : ciclo de controle	63
III.10.3	O Arbiter : composição por votação	63
III.10.4	Configuração em EDL	65
III.11	Interpoladores: tabelas e LFI	66
III.11.1	A fórmula fundamental: interpolação linear 1D	66
III.11.2	Extensão a múltiplas dimensões: redução dimensional	67
III.11.3	Ordem dos <i>breakpoints</i> , delta e cache	68
III.11.4	Nível alto: família Table1–Table5	70
III.11.5	Caso completo: $C_L(\alpha, M)$ em EDL e uso em C++	70
III.11.6	Func1–Func5 e Polynomial : a função como <i>slot</i>	71
III.12	Cores e espaços de cor	72
III.12.1	A hierarquia: Color , Hsv e Hsva	72
III.12.2	Configuração em EDL	74
III.12.3	Conversão HSV $\leftrightarrow$ RGB	75
III.12.4	Hsva::colorInterpolate : mapas de cor	75
III.13	Utilitários e arquivos de plataforma	76
III.13.1	safe_ptr : referências duráveis sem vazamento	77
III.13.2	Timer : cronômetros registrados globalmente	78
III.13.3	Arquivos pareados por plataforma	80
III.13.4	Onde ficam as constantes, e o que não foi coberto	81

III.14	Tipos de valor: <i>strings</i> , números e operadores	82
III.14.1	String e Identifier	83
III.14.2	Number e as especializações do <i>parser</i>	84
III.14.3	Complex	84
III.14.4	LatLon : coordenadas como se escrevem numa carta	85
III.14.5	Operators : aritmética declarativa em EDL	85
III.15	Comunicação em rede	86
III.15.1	NetHandler : o contrato	87
III.15.2	PosixHandler : implementação e fluxo de inicialização	88
III.15.3	Os tipos de <i>socket</i> concretos	89
III.16	Interfaces abstratas de E/S (<i>linkage</i>)	91
III.16.1	AbstractIoDevice : interface com o hardware	92
III.16.2	AbstractIoData : o <i>buffer</i> de valores	92
III.16.3	AbstractIoHandler : o orquestrador	93
IV	A camada simulation	97
IV.1	Visão geral: a espinha dorsal de execução	97
IV.2	Interfaces abstratas de infraestrutura	98
IV.2.1	AbstractRecorderComponent e AbstractDataRecorder	98
IV.2.2	AbstractIgHost	100
IV.2.3	AbstractNetIO	101
IV.2.4	AbstractNib : o bloco de interface de rede	101
IV.3	AbstractPlayer : a interface de toda entidade simulada	102
IV.4	Simulation : o <i>executive</i>	103
IV.4.1	Dois relógios: computador e simulado	104
IV.4.2	O ciclo de quatro fases	105
IV.4.3	Dois níveis de lista de <i>players</i>	107
IV.4.4	Distribuição <i>round-robin</i> entre <i>threads</i>	109
IV.5	Station : a raiz da simulação	110
IV.5.1	Configuração via EDL	110
IV.5.2	O quadro updateTC(dt) : sete passos em ordem fixa	111
IV.5.3	O quadro updateData(dt) : ramos condicionais e gravador	112
IV.5.4	<i>Fast-forward</i> e <i>ownership</i>	114
IV.6	As <i>threads</i> do simulation	115
IV.6.1	As três PeriodicThread de Station	115
IV.6.2	O <i>pool</i> SyncThread de Simulation	116
IV.6.3	O protocolo de despacho por fase	116
IV.6.4	A distribuição <i>round-robin</i>	117
IV.7	Fábrica do módulo	118
V	A camada terrain	120
V.1	Visão geral e hierarquia	120
V.2	Terrain : a interface	120
V.3	DataFile : um arquivo de elevação em grade regular	122
V.4	Formatos de arquivo: DTED, SRTM e DED	125
V.5	QuadMap : agregação em grade 2×2	127
VI	A camada models	131
VI.1	Visão geral: o que está sendo simulado	131
VI.2	WorldModel : a simulação situada no mundo	131
VI.2.1	O ponto de referência e a <i>world matrix</i>	131

VI.2.2	Configuração via EDL	133
VI.2.3	O <i>reset</i> como ponto de inicialização	134
VI.3	Player : a entidade simulada	135
VI.3.1	Estado cinemático completo: três representações simultâneas	135
VI.3.2	Descoberta automática de sistemas	137
VI.3.3	O ciclo de quatro fases	138
VI.3.4	SynchronizedState : o <i>snapshot</i> seguro entre <i>threads</i>	139
VI.4	Dynamics : modelos de física de voo	140
VI.4.1	DynamicsModel : o contrato	140
VI.4.2	RacModel : comportamento sem aerodinâmica	142
VI.4.3	LaeroModel : quatro graus de liberdade	143
VI.4.4	JSBSimModel : fidelidade completa (6DOF)	143
VI.5	Sistemas embarcados: base e cadeia de RF	144
VI.5.1	O percurso completo de uma detecção	145
VI.5.2	Gimbal e Tdb : apontamento e preenchimento	146
VI.5.3	Antenna : transmissão e cálculo de ERP	148
VI.5.4	RfSystem , RfSensor e Radar	149
VI.6	<i>Track managers</i>	151
VI.7	Sensores especializados: IR, SAR e variantes de radar	152
VI.7.1	Cadeia IR: IrSeeker e IrQueryMsg	152
VI.7.2	Variantes de radar: Tws , Stt e Gmti	153
VI.7.3	Sar : imagem sintética de abertura	153
VI.8	Comunicações e gerenciamento de armazenamento	154
VI.8.1	Cadeia de comunicações: Radio , CommRadio e Datalink	154
VI.8.2	Iff : identificação amigo ou inimigo	155
VI.8.3	StoresMgr : gerenciamento de armas	155
VI.9	Navegação: rotas, <i>steerpoints</i> e sistemas inerciais/GPS	157
VI.9.1	Navigation : o sistema base	157
VI.9.2	Steerpoint e Route	158
VI.10	Piloto, autopiloto e computador de bordo	159
VI.10.1	Pilot e Autopilot	159
VI.10.2	OnboardComputer : <i>shoot list</i> e ações de missão	161
VI.10.3	CollisionDetect	162
VI.11	Assinaturas RF e IR	163
VI.11.1	Assinaturas RF: RfSignature e subclasses	163
VI.11.2	Assinaturas IR: IrSignature e AircraftIrSignature	164
VI.12	Ambiente: atmosfera e propagação IR	166
VI.12.1	IrAtmosphere : modelo simples	166
VI.12.2	IrAtmosphere1 : modelo de alta fidelidade	167
VI.13	Armas e efeitos	169
VI.13.1	AbstractWeapon : o ciclo de vida de uma arma	169
VI.13.2	Subtipos de arma	172
VI.13.3	Contramedidas: Chaff , Flare e Decoy	172
VI.14	Tipos de <i>player</i> : ar, solo, mar e espaço	173
VI.14.1	Veículos aéreos	174
VI.14.2	Veículos terrestres e o lançador dinâmico	174
VI.14.3	LifeForm : o infante simulado	175
VI.14.4	SpaceVehicle e BoosterSpaceVehicle	177
VI.15	Dados e mensagens entre sistemas	177
VI.15.1	Emission : pacote de energia RF em trânsito	177

VI.15.2	SensorMsg e IrQueryMsg	178
VI.15.3	Tdb: o buffer de geometria por quadro	179
VI.15.4	Track: o estado de um alvo rastreado	179
VI.15.5	Message, TargetData, Image e Designator	180
VI.16	Agentes (UBF) e fábrica do módulo	181
VI.16.1	SimAgent e MultiActorAgent	181
VI.16.2	Actions: integração UBF com lógica de missão	183
VI.16.3	Fábrica do módulo	185
VII	A camada linearsystem	195
VII.1	Visão geral: discretização de Tustin	195
VII.2	ScalerFunc e DiffEquation	195
VII.3	Funções de transferência de primeira ordem	197
VII.4	Atrasos, <i>sample-and-hold</i> e limitadores	199
VIII	A camada linkage	202
VIII.1	Visão geral: as quatro camadas de E/S	202
VIII.2	IoData: o <i>buffer</i> central	203
VIII.3	IoDevice e dispositivos concretos	204
VIII.4	Adaptadores: mapeamento canal a canal	205
VIII.5	Geradores de sinal sintético	206
VIII.6	IoHandler: o orquestrador	206
IX	A camada recorder	210
IX.1	Visão geral e Protocol Buffers	210
IX.2	DataRecord.proto e DataRecordHandle	210
IX.3	DataRecorder	211
IX.4	<i>Handlers</i> de saída	214
IX.5	<i>Handlers</i> de entrada: leitura e análise	216
X	A camada interop	218
X.1	Visão geral: DIS e HLA	218
X.2	interop/common: Nib e <i>dead reckoning</i>	218
X.3	interop/common: NetIO	221
X.4	interop/dis: NetIO e tipos de PDU	222
X.5	interop/dis: EntityType e o mapeamento por Ntm	225
X.5.1	Ntm: a correspondência entre tipos de rede e <i>players</i>	226
XI	Sistema de compilação: Meson e Conan	230
XI.1	Compilando o próprio MIXR	230
XI.2	Estrutura do conanfile.py	231
XI.3	Descrição do <i>build</i> com Meson	232
XI.4	Fluxo de configuração e compilação	232
XI.5	Dependências com código C++ interdependente: o caso <i>models/</i>	233
XII	Apêndice A: uma aplicação completa	235
XII.1	A árvore do cenário	235
XII.2	O arquivo de configuração	235
XII.3	A aplicação em C++	239
XII.4	Compilando e executando	240
XII.5	O que observar rodando	241

XII.6 Como este manual é verificado	242
XIII Apêndice B: nomes de fábrica que divergem do nome da classe	243
Referências	245
Glossário	246
Índice remissivo	248

List of Figures

1	Organização em camadas e sentido das dependências.	6
2	A cadeia de herança fundamental e o que cada nível agrega.	7
3	Isomorfismo entre o arquivo EDL e a árvore de componentes.	8
4	Hierarquia temporal: ciclo, <i>frame</i> e fase.	10
5	Distribuição de <i>players</i> em <i>pool</i> de <i>threads</i> com barreira de sincronização por fase.	11
6	Do texto EDL à árvore de objetos: as três etapas de <code>parse()</code> .	33
7	As três estruturas de contenção do MIXR: <code>List</code> (lista encadeada de <code>Object*</code> com semântica de posse via <code>ref/unref</code>), <code>Pair</code> (par nome/objeto que corresponde a uma entrada <code>nome: valor</code> no EDL) e <code>PairStream</code> (lista de <code>Pairs</code> navegável por nome e por tipo).	35
8	Família <code>Distance</code> : a referência <code>Meters</code> é o ponto de passagem obrigatório de toda conversão entre unidades.	39
9	Os dois mundos de tipos do MIXR e o critério de escolha entre eles.	41
10	Os três referenciais geoespaciais do MIXR. As conversões entre eles são providas por <code>nav_utils</code> e descritas nas subseções seguintes.	45
11	<code>PeriodicThread</code> (autônoma, à esquerda) e <code>SyncThread</code> (sincronizada, à direita). Os mutexes de <code>SyncThread</code> são criados já bloqueados; <code>signal*</code> é <code>unlock</code> e <code>wait*</code> é <code>lock</code> .	50
12	<code>StateMachine</code> : ciclo de execução por <i>frame</i> (topo), grafo de estados com transições e sub-máquinas (centro) e funções de transição disponíveis (baixo).	56
13	Ciclo percepção–decisão–atuação do <code>Agent</code> (topo) e composição por arbitragem com <code>Arbiter</code> (baixo). O <code>Arbiter</code> é ele próprio um <code>AbstractBehavior</code> , o que permite aninhamento ilimitado.	62
14	Geometria da interpolação linear 1D. A fração m mede o quanto x avançou dentro do intervalo $[x_1, x_2]$; o mesmo m é então aplicado ao salto de valor $a_2 - a_1$. É essa razão entre segmentos — e não a inclinação da reta — que o código de <code>lfi_1D</code> calcula.	66
15	Redução dimensional em <code>lfi_2D</code> : ① <code>lfi_1D</code> na linha y_1 (azul) produz $a_1 = 0.444$; ② <code>lfi_1D</code> na linha y_2 (verde) produz $a_2 = 0.490$; ③ interpolação linear em y combina os dois em $f(x, y) = 0.462$. O padrão repete-se recursivamente: <code>lfi_3D</code> chama <code>lfi_2D</code> duas vezes e interpola em z ; e assim até <code>lfi_5D</code> .	67
16	Topo: <i>breakpoints</i> crescentes (<code>delta = +1</code> , busca avança <code>x2++</code>) e decrescentes (<code>delta = -1</code> , busca recua <code>x2--</code>) — o código detecta a ordem automaticamente. Baixo: sem cache, a busca percorre todos os índices desde o início ($O(n)$); com cache (<code>xbp</code> âmbar aponta para o índice do <i>frame</i> anterior), uma única comparação basta — $O(1)$.	69
17	Hierarquia <code>Color</code> $\rightarrow$ <code>Hsv</code> $\rightarrow$ <code>Hsva</code> com os dois vetores internos (RGBA e HSVA) e a conversão bidirecional automática. Os operadores de conversão herdados por <code>Hsv</code> e <code>Hsva</code> sempre retornam o vetor RGBA.	73

18	<code>safe_ptr</code> automatiza <code>ref/unref</code> (topo); hierarquia <code>Timer/UpTimer/DownTimer</code> com <code>updateTimers()</code> global (centro); arquivos genérico + plataforma que devem ser compilados juntos (baixo).	77
19	Hierarquia de tipos de valor: <code>String/Identifier</code> (texto), família <code>Number</code> com as três especializações que o <i>parser</i> instancia automaticamente, <code>Complex</code> e os <code>Operators</code> aritméticos declarativos.	82
20	Hierarquia de rede: <code>NetHandler</code> define o contrato; <code>PosixHandler</code> implementa sobre POSIX; subclasses UDP e TCP especializam o tipo de <i>socket</i> . O fluxo de <code>initNetwork()</code> é compartilhado, com <code>init()</code> e <code>bindSocket()</code> como pontos de extensão.	87
21	As três interfaces abstratas de E/S (topo), o fluxo de dados no sentido entrada e saída (centro) e os dois modos de operação do <code>IoHandler</code> — síncrono e assíncrono (baixo). A nota explica por que as interfaces ficam em <code>base/</code> e não em <code>linkage/</code> .	91
22	Arquitetura do módulo <code>simulation</code> : <code>Station</code> no topo com seus cinco <i>slots</i> principais (esquerda); as <i>threads</i> periódicas criadas por <code>Station</code> e as <i>threads</i> síncronas criadas por <code>Simulation</code> (direita). Setas tracejadas indicam qual componente cria cada <i>thread</i> . Os cinco grupos do módulo aparecem na faixa inferior.	98
23	As quatro interfaces abstratas de infraestrutura. A seta de <code>AbstractNetIO</code> para <code>AbstractNib</code> indica que o <code>NetIO</code> é o criador e dono dos NIBs. A caixa inferior mostra como <code>AbstractPlayer</code> carrega até <code>MAX_NETWORD_ID</code> NIBs de saída e no máximo um NIB de entrada.	99
24	<code>Simulation</code> : os dois relógios independentes (topo), o ciclo de quatro fases por <i>frame</i> com a progressão de fase (centro), os dois níveis de lista de <i>players</i> e a relação entre <code>origPlayers</code> e <code>players</code> (baixo-esq), e a distribuição <i>round-robin</i> entre <i>threads</i> (baixo-dir).	104
25	Os dois quadros de <code>Station</code> : <code>updateTC()</code> com a ordem fixa de sete passos (esquerda) e <code>updateData()</code> com os três ramos condicionais e a chamada incondicional ao gravador (direita). A faixa inferior mostra <code>fastForwardRate</code> e <code>ownship</code> .	111
26	Uma fase do quadro TC com $N = 4$ <i>threads</i> e 12 <i>players</i> . A <i>thread</i> principal chama <code>start0()</code> nos três <i>workers</i> (que dormiam em <code>waitForStart()</code>) e imediatamente processa sua própria fatia em paralelo. A barreira <code>waitForAllCompleted()</code> bloqueia até todos concluírem, antes da fase seguinte. A lista de <i>players</i> (baixo) mostra a distribuição <i>round-robin</i> : Worker 1 processa P1, P5, P9; Worker 2 processa P2, P6, P10; Worker 3 processa P3, P7, P11; e a <i>thread</i> principal (<code>idx = 4</code>) processa P4, P8, P12.	117
27	Hierarquia do módulo <code>terrain</code> : <code>Terrain</code> define o contrato com dois métodos; <code>DataFile</code> implementa a leitura de arquivo único em grade regular com três subclasses de formato; <code>QuadMap</code> agrega até quatro arquivos em grade 2×2 . Os dois métodos de consulta e seus consumidores são resumidos nas faixas inferiores.	120
28	Estrutura interna de <code>DataFile</code> : matriz <code>columns[lon][lat]</code> de <code>short</code> (topo-esq); os três passos de <code>getElevation()</code> (topo-dir); comparação entre <code>interp=false</code> (post mais próximo) e <code>interp=true</code> (bilinear de quatro vizinhos) (centro); utilitários estáticos <code>vbwShadowChecker()</code> e <code>aac()</code> (baixo).	123

- 29 **WorldModel** estende **Simulation** com o ponto de referência geodésico (topo-esq) e a *world matrix* (topo-dir), mais os *slots* de configuração (centro) e a sequência obrigatória do **reset()** (rodapé). 132
- 30 **Player**: hierarquia de subtipos (topo), três representações de posição atualizadas a cada fase 0 (centro-esq), sistemas descobertos por tipo no **reset()** (baixo-esq) e o ciclo de quatro fases por *frame* (baixo-dir). 136
- 31 Hierarquia de **DynamicsModel** (topo): **AerodynamicsModel** especializa com métricas aerodinâmicas e ramifica em **RacModel**, **LaeroModel** e **JSBSimModel** — os três derivam dela. Os três grupos de métodos (centro) definem o contrato: integração, comandos do autopilot e entradas manuais. A faixa inferior compara os modelos concretos. 141
- 32 Hierarquia de sistemas embarcados (topo), fluxo da cadeia RF por fase — Fase 1: **Gimbal** preenche o **Tdb** e **Antenna** transmite **Emissions**; Fase 2: **RfSensor** aplica a equação do radar e alimenta o **TrackManager** (centro). Estado de um **Track** e filtro $\alpha\text{-}\beta\text{-}\gamma$ (baixo). Resumo da cadeia IR equivalente (rodapé). 187
- 33 Sistemas de comunicação e **StoresMgr** (topo); hierarquia de navegação e fluxo por *frame* (centro); exemplo EDL de **Steerpoint** com encadeamento via **next:** e nota sobre **directTo()** (baixo). 188
- 34 Os seis modos independentes do Autopilot com o algoritmo **flyCRS** (topo); **OnboardComputer** com *shoot list* e **actionManager** ao lado de **CollisionDetect** (centro); *slots* EDL do Autopilot (baixo). 189
- 35 Quatro subclasses de **RfSignature** (topo-esq) e **IrSignature** com **IrShape** (topo-dir); os três componentes físicos independentes de **AircraftIrSignature** com suas tabelas de *lookup* (centro) e o fluxo de saída por banda espectral até o sensor (rodapé). 190
- 36 **IrAtmosphere** (simples, topo-esq) com **transmissivityTable1** e fundo constante; **IrAtmosphere1** (alta fidelidade, topo-dir) com três tabelas multidimensionais. O fluxo completo da cadeia IR desde o **IrSeeker** até o cálculo de SNR (baixo). 190
- 37 Ciclo de vida de **AbstractWeapon** com as transições reais (topo); hierarquia de subtipos e *slots* com valores padrão (centro); comparação entre as três contramedidas (baixo). 191
- 38 Os quatro domínios de *player* concreto (topo); o lançador dinâmico de **GroundVehicle** com campos reais e EDL de **SamVehicle** (centro); **LifeForm** com a atualização automática de postura e o mapeamento para o campo *appearance* do PDU DIS (baixo). 192
- 39 Os seis tipos de objeto de dados: **Emission** e **SensorMsg/IrQueryMsg** (topo, cadeia de sensores); **Tdb** e **Track** (centro, geometria e rastreamento); **Message/TargetData** e **Image/Designator** (baixo, comunicação e missão). O rodapé sintetiza a invariante comum e **SynchronizedState**. 193
- 40 **SimAgent** e **MultiActorAgent** (topo): integração do UBF com o contexto da simulação. Ciclo de vida de **Action** com as quatro subclasses concretas (centro). Fábrica do módulo (baixo). 194
- 41 Hierarquia do módulo **linearsystem**: **ScalerFunc** como raiz (herda de **Object**, não de **Component**); **DiffEquation** como motor numérico com suas subclasses; **LimitFunc** e **SaH** como blocos auxiliares. As faixas inferiores detalham os três passos de **DiffEquation::g()** e os coeficientes Tustin calculados por **FirstOrderTf::initialize()**. 196

- 42 As quatro camadas do módulo `linkage`: fluxo estrutural de `Station` até `Adapter` (topo); tabelas internas de `IoData` e pipeline de `AnalogInput` (centro-esq e centro-dir); `UsbJoystick` e catálogo de adaptadores (baixo); EDL mínimo compondo as quatro camadas. 202
- 43 Fluxo do módulo `recorder`: pipeline de gravação em tempo real (topo); esquema `DataRecord.proto` e `DataRecordHandle` como wrapper de ciclo de vida entre threads (centro); cadeia de `OutputHandlers` com os três destinos concretos (baixo-esq); leitura e análise pós-missão (baixo-dir). 210
- 44 Arquitetura do módulo `interop`: `Nib` com o estado de *dead reckoning* e `NetIO` como repositório de NIBs (`interop_common`, topo); `dis::NetIO` e o *Entity State PDU* (`interop_dis`, centro); o tipo `EntityType` e o `Ntm` como mapeamento MIXR↔DIS (baixo). Tipos de PDU implementados no rodapé. 229

List of Tables

1	As quatro fases de um <i>frame</i> e a razão de cada uma.	9
2	Os grupos de <i>tokens</i> de <code>eventTokens.hpp</code> .	23
3	A tabela de <i>slots</i> de <code>Component</code> . Por serem os primeiros da cadeia, ocupam os índices globais 1 a 7 de qualquer classe <i>derivada de Component</i> .	27
4	Os cinco padrões de divergência entre nome de classe e nome de fábrica. O catálogo completo está no apêndice.	30
5	Famílias de unidades, referência interna e domínio de uso típico.	95
6	Mapeamento da prioridade portátil [0,1] para os agendadores de Linux e Windows.	95
7	Resumo dos dois níveis de LFI: interface, caso de uso e custo.	95
8	As subclasses de <code>Color</code> em <code>base/colors</code> .	96
9	Os parâmetros globais de <code>include/mixr/config.hpp</code> .	96
10	Arquivos pareados: o genérico declara a interface; o de plataforma implementa. Neste <i>fork</i> , apenas a coluna Linux entra no <code>meson.build</code> .	96
11	As cinco <i>threads</i> do módulo <code>simulation</code> e suas responsabilidades.	97
12	As quatro fases de <code>Simulation::updateTC()</code> e seus papéis.	106
13	Níveis de resolução do formato DTED.	125
14	Os oito domínios funcionais de <code>libmixr_models.so</code> .	131
15	Comparação dos modelos de dinâmica de voo do MIXR.	144
16	Modos de varredura de <code>ScanGimbal</code> . Não existem <code>BAR_SCAN</code> nem <code>SECTOR_SCAN</code> .	148
17	Slots de <code>AbstractWeapon</code> com os valores padrão efetivamente usados pelo código (membros de <code>AbstractWeapon.hpp</code>).	171
18	Os valores de <code>msgHdrOptn</code> . <code>ON_CHANGE</code> é o que reage à troca de tipo; <code>NEW_MSG</code> usa um <i>flag</i> por tipo de mensagem, reiniciado a cada arquivo.	215
19	O <code>enum DeadReckoning</code> de <code>include/mixr/interop/common/Nib.hpp</code> . As letras da sigla são, em ordem: <i>Fixed/Rate</i> de rotação, <i>Position/Velocity</i> de translação, e <i>World/Body</i> de referencial.	220
20	Os limiares de <i>dead reckoning</i> e seus padrões (<code>src/interop/common/NetIO.cpp</code>).	221
21	Tipos de PDU efetivamente despachados por <code>dis::NetIO::netInputHandler()</code> . Há ainda o tratamento de IFF/ATC-NAVAIDS, e qualquer outro tipo cai em <code>processUserPDU()</code> , o ponto de extensão da aplicação.	223

22	A regra de melhor casamento na saída, com o exemplo do próprio <code>include/mixr/interop/common/Ntm.hpp</code> .	227
23	Dependências externas do MIXR e quem as consome.	231

Aviso

Este relatório foi construído com o auxílio de inteligência artificial generativa e revisado manualmente por um grupo de especialistas humanos.

Todo o conteúdo técnico foi conferido contra o código-fonte da versão documentada (Seção I.3); os blocos de configuração EDL são verificados automaticamente contra as fábricas e *slots* do *framework* (Seção XII.6). A responsabilidade pelo texto final é da equipe revisora.

I Introdução

I.1 O que é o MIXR

O **MIXR** (*Mixed Reality Simulation*) [1] é um *framework* em C++ para a construção de aplicações de modelagem e simulação (M&S) de sistemas — plataformas, sensores, armamento e redes de interoperabilidade. Ele não é um simulador pronto, mas um conjunto de bibliotecas em camadas a partir das quais o desenvolvedor monta a simulação que precisa.

A decisão de projeto que distingue o MIXR de uma biblioteca convencional, e que governa praticamente todo o restante deste manual, é a *separação entre estrutura e comportamento*: o *comportamento* de cada peça é implementado em C++, mas a *estrutura* de uma simulação — quais peças existem, como se aninham e com que parâmetros — é descrita em arquivos de configuração na linguagem **EDL** (*English Description Language*). Em tempo de carga, um *parser* lê esse arquivo e constrói a árvore de objetos correspondente. A consequência prática é direta e poderosa: **reconfigurar uma simulação não exige recompilar a aplicação** — basta editar o arquivo de configuração.

Essa escolha só funciona porque três mecanismos do núcleo a sustentam, e os três reaparecerão em todas as seções deste manual:

- **Fábrica por nome** (*factory*) — permite criar um objeto a partir de uma *string* de identificação lida do arquivo.
- **Sistema de *slots*** — conecta os atributos nomeados do arquivo às funções C++ que os configuram.
- **Contagem de referências** — gerencia automaticamente o ciclo de vida de uma árvore de objetos montada dinamicamente, sem **delete** explícito.

Na prática, toda simulação MIXR é uma árvore de objetos cuja raiz é um **Station** — o componente que organiza o *executive*, as redes de interoperabilidade, o *hardware* de entrada/saída e o gravador de dados. A topologia completa dessa árvore é declarada em um arquivo EDL; o *parser* a constrói em tempo de carga e a mantém viva pela contagem de referências.

I.2 Objetivo e público-alvo

Este documento é um **manual técnico de referência** voltado a desenvolvedores que pretendem *usar* o MIXR para construir simulações ou *estender* o *framework* com novos modelos. Pressupõe familiaridade com C++ moderno e com conceitos gerais de M&S; não é um tutorial de usuário final.

O manual privilegia as *decisões arquiteturais* — o porquê de cada mecanismo — em vez da exposição exaustiva de código. O objetivo é que o leitor compreenda *como pensar* dentro do *framework*, de modo que o código-fonte se torne legível por consequência.

I.3 Versão documentada

Este manual descreve o *fork* do MIXR mantido em github.com/ASA-Simulation/mixr, versão **1.0.5**, cuja diferença principal em relação ao MIXR original ([mixr.dev](https://github.com/mixr-dev/mixr) [1]) é a adoção de **Meson** e **Conan 2** no lugar

do sistema de compilação anterior (Seção XI). Todos os trechos de código, nomes de *slot*, constantes e valores padrão citados ao longo do texto foram conferidos contra essa árvore de fontes.

Três características desse *fork* são consequência do estado atual do *build* e merecem ser conhecidas desde já, porque afetam o planejamento de uma integração:

- **A compilação é apenas para Linux.** As fontes específicas de Windows estão presentes na árvore, mas não são selecionadas pelo `meson.build` (Seção III.13.3).
- **HLA e RPR-FOM não são compilados.** Dos protocolos de interoperabilidade, apenas o DIS entra no *build* (Seção X.1).
- **O padrão de linguagem é C++11**, com `-fpermissive` (Seção XI.1).

Onde o texto descreve algo que existe no código mas não está operacional — como as classes HLA ou o `SecondOrderTf` da Seção VII.3 — isso é dito explicitamente no ponto correspondente.

I.4 Convenções do documento

Nomes de classes, funções e identificadores aparecem em fonte monoespaçada (`Object`, `updateTC()`). O código do MIXR organiza-se em *namespaces* aninhados sob `mixr`; o caminho completo (`mixr::base::Component`) é indicado quando relevante. Termos técnicos consagrados em inglês (*slot*, *factory*, *frame*) são mantidos no original, e os principais estão definidos no glossário, ao final.

Blocos de código. Uma convenção importa mais que as outras: **quando a legenda de um bloco traz um caminho de arquivo**, como `(src/base/Component.cpp)`, **o conteúdo é transcrição do código-fonte** — eventualmente condensado, com as omissões marcadas por `// ...`, mas nunca reescrito. Blocos sem caminho são ilustrativos: mostram como usar um mecanismo, não como ele está escrito.

Caixas de destaque. Além da caixa cinza de aviso que abre o manual — preliminar e de ocorrência única —, três tipos de quadro interrompem o texto, sempre com o mesmo papel:

Regra — o que é uma Regra

Uma norma do *framework* — algo que o desenvolvedor precisa fazer, ou não fazer, para que o mecanismo funcione.

Armadilha — o que é uma Armadilha

Um comportamento real do código que contraria a expectativa razoável: um valor padrão surpreendente, um nome que não é o que parece, um comentário de cabeçalho desatualizado, uma funcionalidade que existe pela metade. São o resultado da conferência do texto contra o fonte, e vale lê-las mesmo em diagonal.

Por que — o que é um Por quê

A motivação de uma decisão de projeto — a pergunta que o mecanismo responde. São opcionais para usar o *framework*, e essenciais para entendê-lo.

I.5 Organização do manual

A Seção II estabelece a visão arquitetural: os princípios de projeto, a organização em camadas, o modelo de objetos, o ciclo de execução e o paralelismo. As seções seguintes descem ao detalhe de cada módulo, na ordem de dependência — de **base** para as camadas superiores: **base** (Seção III), **simulation** (Seção IV), **terrain** (Seção V), **models** (Seção VI), **linearsystem** (Seção VII), **linkage** (Seção VIII), **recorder** (Seção IX) e **interop** (Seção X). A Seção XI aborda o sistema de compilação baseado em Meson e Conan.

O **Seção XII** fecha o manual com uma aplicação completa — arquivo de configuração, **main()** e comandos de *build* — que exercita quase todos os mecanismos discutidos. Quem prefere partir do concreto pode lê-lo primeiro e usar as remissões para voltar aos capítulos.

Ao final estão o **glossário** dos termos do *framework* e um **índice remissivo** de classes e *slots*.

Sobre a exatidão deste texto. Todo bloco de configuração EDL do manual é verificado automaticamente contra o código-fonte: um script confere se cada nome de fábrica existe e está registrado, e se cada *slot* pertence de fato à classe em que aparece (Seção XII.6). Isso elimina a classe de erro mais frustrante num manual — o exemplo que não carrega — mas não substitui a leitura do fonte quando a dúvida for de comportamento.

II Arquitetura geral

Esta seção apresenta o MIXR de cima: primeiro as decisões de projeto que moldam tudo, depois a estrutura estática (camadas e objetos) e, por fim, o comportamento dinâmico (ciclo de execução e paralelismo). O detalhamento de cada peça fica para as seções de módulo; aqui interessa o vocabulário e a lógica do conjunto.

II.1 Princípios de projeto

Quatro decisões arquiteturais permeiam o *framework*. Mantê-las em mente torna o restante do manual previsível.

1. **Composição dirigida por dados.** Estrutura em EDL, comportamento em C++ (Seção I.1). Favorece experimentação rápida e reaproveitamento, ao custo de um núcleo de *runtime* mais elaborado — o preço que o *framework* paga uma vez para que toda aplicação se beneficie.
2. **Tudo é um Component.** Da estação no topo ao menor subsistema, todos compartilham a mesma maquinaria de atualização, contenção e eventos. Essa uniformidade significa que um mecanismo aprendido uma vez se aplica em toda a árvore.
3. **Dependências unidirecionais entre camadas.** Cada camada conhece apenas as inferiores. Isso fixa uma ordem de construção clara, isola mudanças e mantém as camadas baixas testáveis de forma independente.
4. **Atualização determinística e faseada.** O avanço do tempo é dividido em fases com papéis fixos (Seção II.4), de modo que o resultado de um *frame* independa da ordem em que os participantes são processados — condição necessária para reprodutibilidade e para paralelizar com segurança.

II.2 Organização em camadas

O MIXR empilha-se em camadas com dependência fluindo de cima para baixo. Na fundação, **base** reúne a infraestrutura comum (modelo de objetos, *parser*, matemática, geodésia, *threading*). Sobre ela, **simulation** fornece a *engine* de execução. As camadas de domínio (**models**, **terrain**, **linearsystem**) implementam o conteúdo simulado, e as camadas de topo (**interop**, **recorder**, **linkage**) tratam de interoperabilidade em rede, registro de dados e ligação com dispositivos.

A direção única das dependências é o que permite, por exemplo, compilar e validar **base** sem nenhuma das camadas acima, e o que explica por que a integração e o estudo do *framework* seguem naturalmente essa mesma ordem ascendente.

II.3 Modelo de objetos: os três mecanismos da composição

A composição dirigida por dados exige um modelo de objetos que saiba ser criado, configurado e gerenciado a partir de texto. No MIXR isso se concretiza em uma cadeia de herança curta — **Referenced** → **Object** → **Component** — em que cada nível acrescenta exatamente uma capacidade.

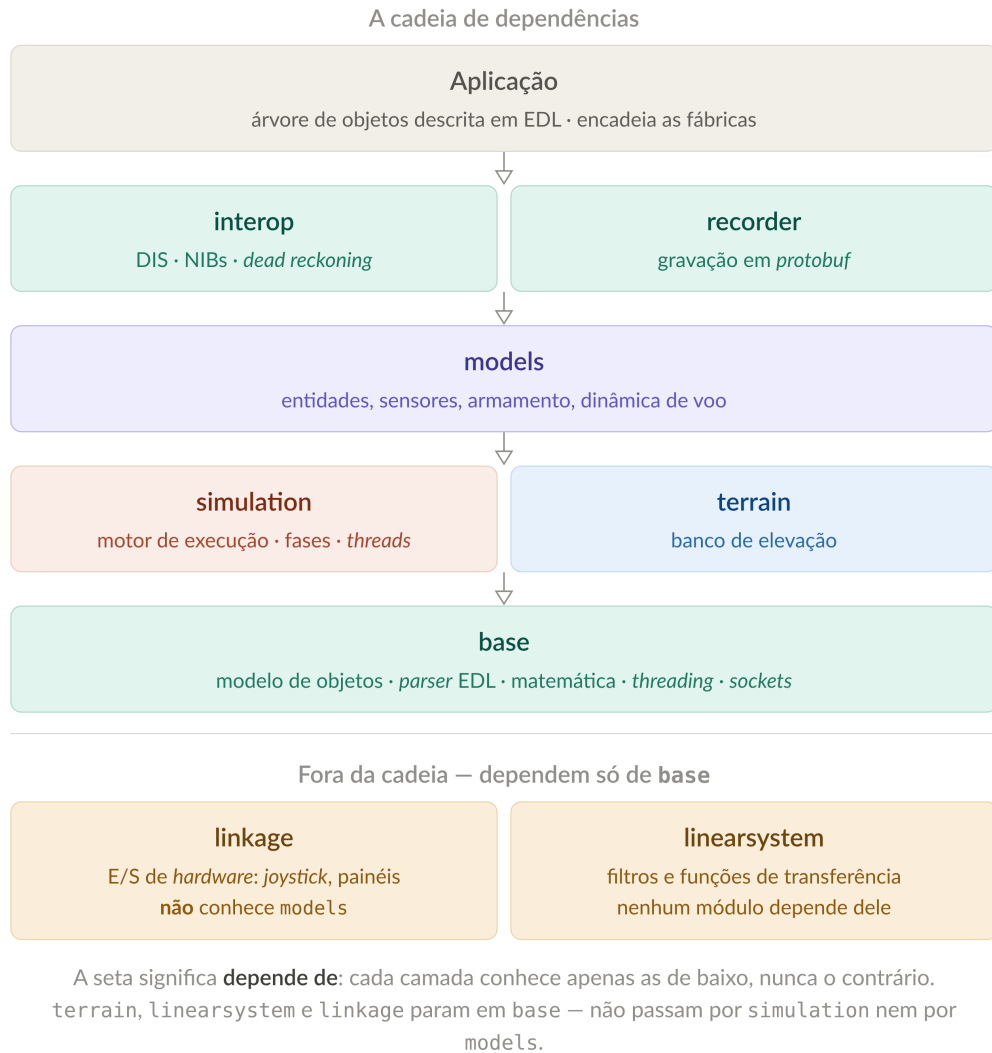


Figure 1: Organização em camadas e sentido das dependências.

Por que contagem de referências. Uma árvore montada dinamicamente pelo *parser*, com objetos compartilhados entre vários donos (um mesmo modelo de assinatura referido por diversos *players*, por exemplo), torna o gerenciamento manual de memória frágil. A decisão do MIXR é que **Referenced** controle o ciclo de vida por contagem: cada objeto nasce com contador 1, cada novo dono chama `ref()`, cada dono que se desliga chama `unref()`, e o objeto se destrói sozinho ao chegar a zero. A regra que daí decorre — **nunca usar delete, apenas unref()** — é a primeira que todo desenvolvedor do *framework* internaliza.

Referenced é o topo abstrato dessa cadeia, e deliberadamente magro: não conhece nada do *framework* além do próprio ciclo de vida. Internamente guarda dois campos privados — um contador `refCount` (que nasce em 1) e um `semaphore` usado apenas para travar o acesso a esse contador — e expõe três operações: `ref()` incrementa, `unref()` decrementa e, ao chegar a zero, executa `delete this`, e `getRefCount()` expõe o valor para diagnóstico. O travamento em si (funções `lock()/unlock()` de `atomics.hpp`) não usa *mutex* do sistema operacional: o `semaphore` é manipulado por instruções atômicas do processador, sem chamada de sistema. A razão é de frequência — `ref()` e `unref()` disparam a cada atribuição de ponteiro em toda a árvore, potencialmente a partir de *threads* diferentes (Seção II.5), e um *mutex* de sistema seria custoso demais para uma operação tão comum. É essa primitiva atômica, e não um mecanismo de mais alto nível,

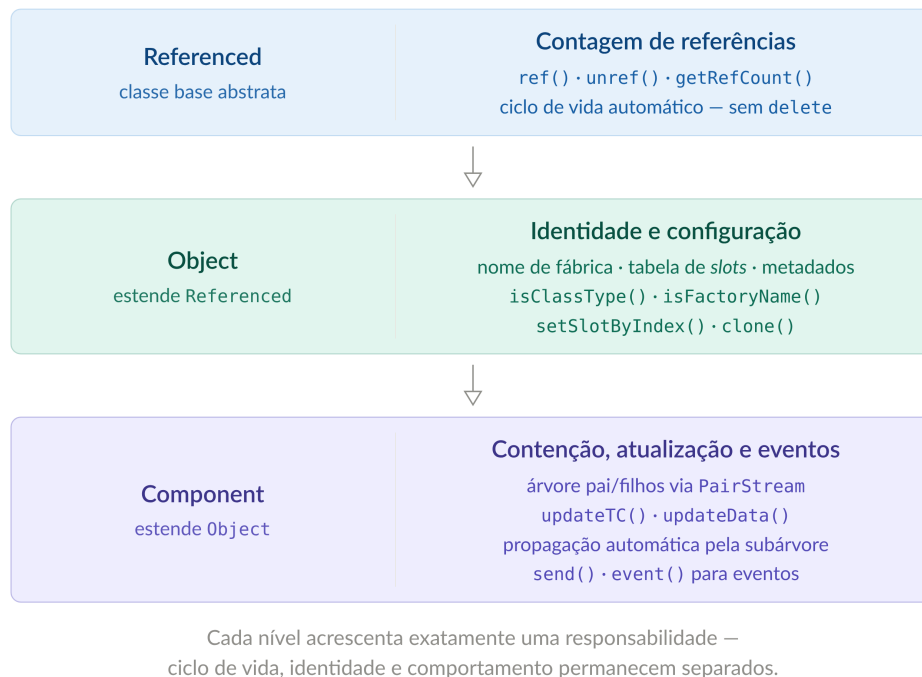


Figure 2: A cadeia de herança fundamental e o que cada nível agrega.

que torna a contagem de referências segura entre *threads*.

Dois detalhes de C++ reforçam a disciplina de uso. O construtor de cópia e o operador de atribuição de `Referenced` são explicitamente removidos (`= delete`): copiar um `Referenced` duplicaria um contador que descreve donos compartilhados, o que não tem sentido semântico. Cada subclasse define sua própria noção de cópia por meio de `copyData()` (Seção III.1), que clona dados, não identidade referenciada. Adicionalmente, `Referenced` não é instanciável diretamente — toda instância real pertence a uma subclasse com identidade própria obtida pelo par de macros `DECLARE_SUBCLASS/IMPLEMENT_SUBCLASS` (Seção III.1).

Por que fábrica por nome. Para o *parser* traduzir o texto “(*Aircraft* ...)” em um objeto, é preciso criar uma instância da classe certa a partir apenas do nome. `Object` atribui a cada classe um *nome de fábrica*, e a aplicação fornece uma função que mapeia nome em objeto. Como a aplicação encadeia as fábricas das bibliotecas que usa, ela controla a precedência: um modelo específico pode sobrepor-se a um genérico de mesmo nome.

Por que slots. Configurar um objeto a partir de texto requer mapear atributos nomeados (“`type: “F-16”`”) às funções C++ que os recebem. O sistema de *slots* é esse mapeamento; ele é herdado pela cadeia de classes, de modo que atributos comuns são definidos uma vez na base. Ao terminar de preencher os *slots*, o *parser* pergunta ao objeto se ele está íntegro (`isValid()`), dando-lhe a chance de recusar configurações incompletas.

A árvore de componentes. `Component` fecha o modelo ao permitir que objetos contenham e sejam contidos, formando a árvore que estrutura toda simulação. A correspondência entre o texto EDL e a árvore resultante é direta — cada parêntese é um objeto; cada nome seguido de dois-pontos, um *slot*:


```

1 ( Station
2   simulation:
3   ( Simulation
4     players: {
5       ownship:
6       ( Aircraft
7         type: "F-16" )
8       bandit:
9       ( Aircraft
10        type: "Su-27" )
11     }
12   )
13   ioHandler:
14   ( MeuIoHandler ... ) -- handler concreto da aplicacao
15 )

```

Listing 1: A estrutura de uma simulação é a estrutura do arquivo EDL.

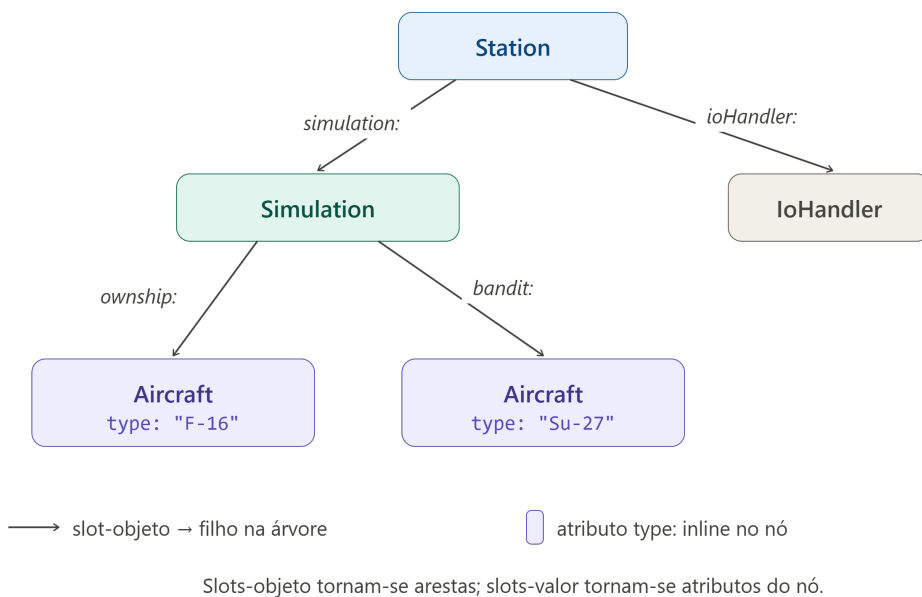


Figure 3: Isomorfismo entre o arquivo EDL e a árvore de componentes.

II.4 Ciclo de atualização

Definida a estrutura, resta o comportamento no tempo. O MIXR conduz toda a dinâmica por dois fluxos que percorrem a árvore: `updateTC()`, a atualização *crítica em tempo* (taxa fixa e alta, para dinâmica de voo e sensores), e `updateData()`, a atualização de *fundo* (para tarefas sem cadência rígida, como entrada/saída). A propagação é automática: cada componente atualiza-se e repassa a chamada aos filhos.

A decisão arquitetural central aqui é *subdividir o tempo em fases com papéis fixos*. Cada chamada de `updateTC()` é um *frame*; 16 *frames* formam um *ciclo*; e cada *frame* divide-se em 4 *fases*:

A documentação interna de `Simulation.hpp` registra esse contrato com precisão:

```

1 // Cycles, frames and phases:
2 //
3 // Each call to updateTC() by its manager (e.g., a Station class) is one
4 // frame, and there are 16 frames, which are numbered 0 to 15, to one cycle.

```

Fase	Papel	Motivação
0	Dinâmica	move os <i>players</i> antes de qualquer interação
1	Sensores transmitem	todas as emissões do <i>frame</i> são geradas
2	Sensores recebem	só então cada <i>player</i> processa o que recebeu
3	Lógica e controle	decisões sobre um estado já estabilizado

Table 1: As quatro fases de um *frame* e a razão de cada uma.

```

5 //
6 // Each frame is broken into 4 phases, which are numbered 0 to 3, and are
7 // used to process dynamics, transmit sensor queries (e.g., emissions),
8 // receive sensor queries, and information or control processing,
9 // respectively. These phases allow for the synchronized flow of data
10 // between players within a given frame. The player list is traversed for
11 // each phase, therefore the player list is traversed 4 times the core
12 // frame rate (e.g., 50 Hz frame rate is 200 Hz phase rate).
13 //
14 // Use cycle(), frame() and phase() to get the current values, and use
15 // getExecCounter() to get the total number of phases since the start
16 // of the exec.

```

Listing 2: Documentação do modelo ciclo/*frame*/fase em `include/mixr/simulation/Simulation.hpp`.

O ganho dessa separação é o **determinismo**: ao garantir que *toda* emissão (fase 1) ocorra antes de *qualquer* recepção (fase 2), o resultado de um *frame* deixa de depender da ordem em que os *players* aparecem na lista. Sem isso, um *player* processado antes de outro “veria” um mundo diferente, e a simulação não seria reproduzível nem segura para paralelizar. Pela mesma lógica de consistência, valores de tempo e data são fixados apenas no início de cada *frame*.

A estrutura em ciclo/*frame*/fase também serve de relógio para agendamento em sub-taxas: lógica pode rodar a cada *frame*, a cada *N frames* do ciclo, ou até a cada ciclo inteiro.

O *dt* das fases: dividido na descida, recomposto na chegada. Este é o ponto do modelo mais frequentemente mal interpretado, e vale enunciá-lo com precisão porque metade dele costuma ser omitida. O *dt* que `Simulation` passa à lista de *players* em cada fase é de fato $\Delta t_{\text{frame}}/4$ — a chamada é `updateTcPlayerList(lista, dt0/4.0, ...)`, repetida quatro vezes por *frame*. A leitura ingênua a partir daí seria concluir que a dinâmica de voo integra quatro vezes por *frame*, com um quarto do passo. **Não é o que acontece.**

A razão é que cada método de fase roda em *uma única* das quatro fases, não em todas. Um subsistema que só age na fase 0 é chamado uma vez por *frame*; se recebesse $\Delta t/4$, integraria com um quarto do tempo que de fato passou. Por isso `Player::updateTC()` e `System::updateTC()` — os dois pontos onde o despacho por fase realmente ocorre — começam recompondo o intervalo do *frame*:

```

1 // Delta time for methods that are running every fourth phase
2 double dt4{dt * 4.0};
3
4 switch (sim->phase()) {
5     case 0 : dynamics(dt4); break; // Dinamica
6     case 1 : transmit(dt4); break; // Sensores transmitem
7     case 2 : receive(dt4); break; // Sensores recebem
8     case 3 : process(dt4); break; // Logica e controle
9 }
10
11 // A propagacao aos subcomponentes usa 'dt' (o dt/4 original), porque
12 // cada subcomponente refaz este mesmo raciocinio para a sua propria fase.
13 BaseClass::updateTC(dt);

```

Listing 3: A recomposição do Δt do *frame* em `src/models/system/System.cpp` (o mesmo padrão aparece em `Player.cpp`).

O enunciado correto, portanto, tem duas metades: **a travessia da lista de *players* ocorre a quatro vezes a taxa do *frame*, mas cada método de fase recebe o Δt *integral* do *frame* e é executado uma única vez por *frame***. A divisão por quatro na descida e a multiplicação por quatro na chegada se cancelam; o que a subdivisão compra não é um passo de integração menor, e sim a *ordenação* — a garantia de que toda transmissão (fase 1) precede qualquer recepção (fase 2) dentro do mesmo *frame*.

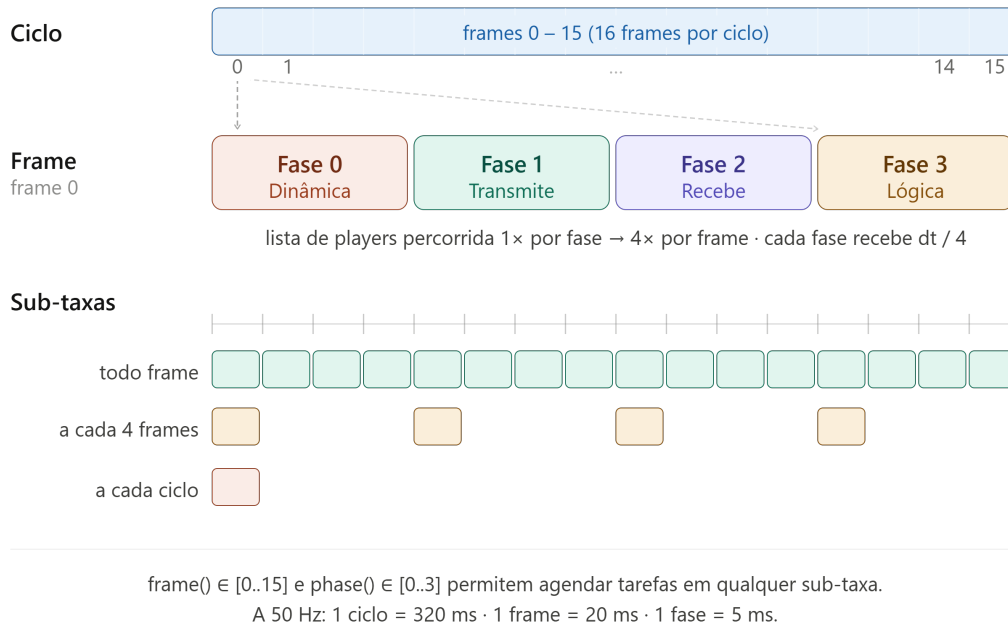


Figure 4: Hierarquia temporal: ciclo, *frame* e fase.

II.5 Modelo de *threading*

O par `updateTC()`/`updateData()` é também a fronteira do paralelismo. A *engine* pode distribuir a travessia da lista de *players* por várias *threads* (configuráveis pelos slots `numTcThreads` e `numBgThreads`), cada uma cuidando de um subconjunto dos participantes.

A decisão que mantém o paralelismo correto é o *reagrupamento por fase*: as *threads* críticas em tempo se sincronizam ao final de cada fase antes de seguir para a próxima. Assim, o determinismo estabelecido na Seção II.4 é preservado — nenhuma *thread* entra na fase de recepção enquanto outra ainda transmite. O paralelismo, portanto, não relaxa as garantias do modelo de fases; apenas as executa em paralelo dentro de cada fase.

Como há custo em gerenciar *threads*, o ganho só aparece com muitos *players*, e o *framework* adota uma regra de bom-senso: não solicitar mais *threads* do que o número de CPUs menos um, preservando um núcleo para o sistema operacional e demais tarefas.



Figure 5: Distribuição de *players* em *pool* de *threads* com barreira de sincronização por fase.

A infraestrutura de baixo nível que sustenta esse modelo (classes de *thread* periódica e sincronizada sobre primitivas POSIX) reside em **base** e é detalhada na Seção III.

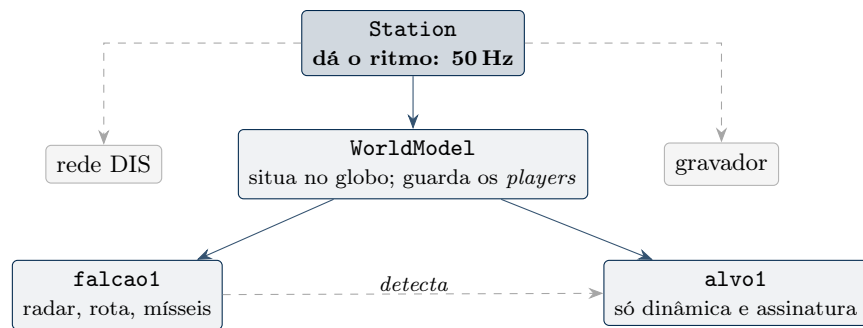
II.6 Um cenário para acompanhar o manual

As seções seguintes descem, uma a uma, às camadas do *framework*. Para que elas não fiquem sendo um catálogo de mecanismos isolados, vale fixar desde já um **cenário concreto**, ao qual cada capítulo voltará para mostrar a fatia que lhe cabe.

O cenário é uma **interceptação**, e é pequeno de propósito:

Um caça sobre o Vale do Paraíba segue uma rota de dois *waypoints* sob autopiloto. O seu radar varre à frente. Um segundo avião, em rota de colisão, entra no alcance. A simulação grava tudo em disco e publica o estado do caça numa rede DIS.

Dois *players*, um radar, um míssil, terreno, gravador e rede — o suficiente para exercitar quase todo o *framework*, e pequeno o bastante para caber em duas páginas de configuração.



Cada capítulo de módulo termina com um parágrafo curto — “*no cenário da interceptação...*” — ligando o que acabou de ser explicado a esse desenho. O arquivo de configuração inteiro, o `main()` que o carrega e os comandos de *build* estão no Seção XII. Quem preferir partir do concreto pode lê-lo agora e voltar.

III A camada base

A Seção II apresentou os três mecanismos da composição dirigida por dados — fábrica por nome, *slots* e contagem de referências — como conceitos. Esta seção desce ao código: como cada mecanismo é implementado, quais classes o compõem e quais regras o desenvolvedor deve seguir ao estender o *framework*.

III.1 O modelo de classes e suas macros

Para que cada objeto participe da composição dirigida por dados ele precisa satisfazer um contrato com o *framework*: ser contado por referência, clonar-se corretamente, identificar seu tipo em tempo de execução e expor sua tabela de *slots*. Implementar esse contrato à mão em cada classe seria repetitivo e frágil — um `clone()` mal escrito corrompe a contagem de referências de uma subárvore inteira; um `isFactoryName()` que esquece de chamar a base quebra silenciosamente a resolução de tipos.

A decisão do MIXR é **gerar essa maquinaria por macros**, tornando a adesão correta ao contrato automática em vez de opcional.

III.1.1 DECLARE_SUBCLASS e IMPLEMENT_SUBCLASS

Cada classe participante usa um par complementar de macros: a primeira no cabeçalho, a segunda na implementação:

```
1 // -- Foo.hpp --
2 class Foo : public Component {
3     DECLARE_SUBCLASS(Foo, Component)
4     // DECLARE_SUBCLASS gera as declaracoes de:
5     // clone(), copyData(), deleteData(),
6     // isClassType(), isFactoryName(),
7     // setSlotByIndex(), SlotTable, MetaObject
8     // e define o alias: typedef Component BaseClass;
9 public:
10     Foo();
11     // ... resto da interface propria de Foo
12 };
13
14 // -- Foo.cpp --
15 IMPLEMENT_SUBCLASS(Foo, "Foo")
16 // IMPLEMENT_SUBCLASS implementa os metodos declarados acima
17 // e fixa o nome de fabrica como a string "Foo"
```

Listing 4: O par de macros: tudo o que liga uma nova classe ao *framework*.

O alias `BaseClass` merece atenção: todo código interno de `Foo` deve usar `BaseClass::método()` em vez de `Component::método()`. Se a hierarquia mudar — `Foo` passar a herdar de outra classe — somente a linha da macro muda, não o corpo dos métodos.

`DECLARE_SUBCLASS` expande para um conjunto fixo de declarações. Conhecer a expansão ajuda a depurar e a entender o que o *framework* exige:

```
1 typedef Component BaseClass;
2
3 public:
4     Foo(const Foo& org);           // construtor de copia
5     virtual ~Foo();               // destrutor virtual
6     Foo* clone() const override;  // clonagem profunda
```

```

7   Foo& operator=(const Foo& org);    // atribuicao
8   protected:
9       void copyData(const Foo& org, const bool cc = false);
10      void deleteData();
11   public:
12      bool isClassType(const std::type_info&) const override;
13      bool isFactoryName(const char[]) const override;
14      static const MetaObject* getMetaObject();
15      static const char* getFactoryName();
16      static const SlotTable& getSlotTable(); // usado pelo BEGIN_SLOT_MAP da subclasse
17   protected:
18      bool setSlotByIndex(const int, Object* const) override;
19      static const SlotTable slottable;
20   private:
21      static MetaObject metaObject; // UM por classe, nao por instancia
22      static const char* slotnames[];
23      static const int    nslots;
24   private: // <-- a macro TERMINA em 'private:'

```

Listing 5: Expansão simplificada de DECLARE_SUBCLASS: o que a macro gera.

Vale reparar na última linha: DECLARE_SUBCLASS encerra com `private:`, de modo que tudo escrito logo abaixo da macro no corpo da classe é privado por padrão. É por isso que o exemplo anterior reabre explicitamente com `public:` antes de declarar o construtor de Foo — omitir esse `public:` torna a classe não instanciável de fora, com um erro de acesso que costuma desconcertar quem vê a macro pela primeira vez.

Além do par principal, existem variantes para casos especiais. IMPLEMENT_ABSTRACT_SUBCLASS atende classes com métodos virtuais puros: gera tudo o que IMPLEMENT_SUBCLASS gera, exceto que `clone()` retorna `nullptr`. IMPLEMENT_PARTIAL_SUBCLASS gera apenas a identidade de tipo (`metaObject`, `isFactoryName()`, `isClassType()`, `getSlotTable()`) e deixa a cargo do desenvolvedor o construtor de cópia, o destrutor, o `clone()` e o `operator=` — quatro membros, não três.

Há ainda três macros de *boilerplate* que é útil não confundir: EMPTY_COPYDATA e EMPTY_DELETEDATA servem a classes sem dados próprios a copiar ou liberar, enquanto EMPTY_SLOTTABLE é a que atende classes sem *slots* próprios — ela declara uma `SlotTable` vazia encadeada à da base e um `setSlotByIndex()` que apenas repassa. A combinação frequente das duas primeiras com IMPLEMENT_SUBCLASS tem inclusive um atalho dedicado, IMPLEMENT_EMPTY_SLOTTABLE_SUBCLASS.

III.1.2 STANDARD_CONSTRUCTOR e STANDARD_DESTRUCTOR

Duas macros adicionais devem ser chamadas *dentro de cada construtor e destrutor* de uma classe MIXR. Elas realizam as tarefas administrativas que conectam a instância ao sistema de metadados:

```

1   Foo::Foo() : Component()
2   {
3       STANDARD_CONSTRUCTOR()
4       // O que a macro faz:
5       //   slotTable = &Foo::slottable; // aponta para a SlotTable estatica da classe
6       //   metaObject.count++;           // registra mais uma instancia viva
7       //   if (count > mc) mc = count;   // atualiza pico historico
8       //   metaObject.tc++;               // incrementa total historico
9
10      // ... inicializacao propria de Foo
11  }
12
13  Foo::~~Foo()
14  {
15      STANDARD_DESTRUCTOR()
16      // O que a macro faz:

```

```

17 | // deleteData();           // libera recursos proprios
18 | // metaObject.count--;     // desregistra a instancia
19 | }

```

Listing 6: STANDARD_CONSTRUCTOR e STANDARD_DESTRUCTOR: tarefas automáticas em cada construção e destruição.

III.1.3 copyData e deleteData

A motivação dessas duas funções é o `clone()`: o *parser* e vários modelos clonam protótipos para gerar instâncias novas, e o *framework* exige que cada clone seja **completamente independente** do original — sem ponteiros compartilhados disfarçados de cópia.

O parâmetro `cc` (*copy constructor*) distingue dois contextos de chamada: construção por cópia, em que o objeto acabou de nascer e ainda não tem estado próprio a preservar, e atribuição, em que os membros já valem alguma coisa. O idioma do MIXR é usar `cc` para *reinicializar* nesse primeiro caso — `if (cc) initData();` aparece em 36 arquivos do fonte, e a forma negada `if (!cc)` não aparece em nenhum:

```

1 | void Foo::copyData(const Foo& org, const bool cc)
2 | {
3 |     // REGRA: sempre chama a base ANTES de copiar os proprios dados
4 |     BaseClass::copyData(org);
5 |
6 |     // construação por copia: poe os membros num estado conhecido
7 |     if (cc) initData();
8 |
9 |     // copia valores escalares normalmente
10 |    speed  = org.speed;
11 |    altitude = org.altitude;
12 |
13 |    // subobjetos: libera o antigo e CLONA o novo
14 |    if (mySubObj != nullptr) { mySubObj->unref(); mySubObj = nullptr; }
15 |    if (org.mySubObj != nullptr) {
16 |        // clone() ja devolve o objeto com contador 1 -- NAO chame ref() aqui
17 |        mySubObj = org.mySubObj->clone();
18 |    }
19 | }

```

Listing 7: `copyData()` no idioma do *framework*: `cc` marca a construção por cópia.

Armadilha — `clone()` já vem referenciado

O erro mais fácil de cometer nesta função é acrescentar um `ref()` depois do `clone()`. `IMPLEMENT_SUBCLASS` define `clone()` como `new ThisType(*this)`, e todo `Referenced` nasce com `refCount` igual a 1 (Seção II.3) — o objeto já chega *pré-referenciado*. Um `ref()` extra o deixa em 2, o `unref()` do dono o devolve a 1, e ele nunca é destruído.

O fonte é explícito a respeito: `Pair::copyData()` comenta a linha com “*already ref() by constructor in clone()*”. O mesmo vale para `PairStream::findName()` e para `Component::copyData()`.

`deleteData()` deve ser **idempotente** (pode ser chamada mais de uma vez sem efeito colateral) e libera recursos na ordem *inversa* à construção — próprios primeiro, base depois:

```

1 | void Foo::deleteData()
2 | {
3 |     // proprios primeiro
4 |     if (mySubObj != nullptr) { mySubObj->unref(); mySubObj = nullptr; }
5 |
6 |     // base por ultimo
7 |     BaseClass::deleteData();
8 | }

```

Listing 8: `deleteData()` idempotente: anulação de ponteiro após `unref()`.

III.1.4 MetaObject: o registro estático de uma classe

Cada classe participante possui, como membro estático, um único `MetaObject` que existe **uma vez por tipo** — independentemente de quantas instâncias dessa classe existam em memória. Ele não herda de `Object`, não participa da contagem de referências e vive em memória estática durante toda a execução.

```
1 // include/miixr/base/MetaObject.hpp
2 class MetaObject {
3 public:
4     const char* getClassName() const; // nome C++ (de typeid), util para diagnostico
5     const char* getFactoryName() const; // nome EDL, ex.: "Aircraft"
6
7     const SlotTable* slottable {}; // tabela de slots desta classe
8     const MetaObject* baseMetaObject {}; // MetaObject da classe base (forma a cadeia)
9
10    int count {}; // instancias vivas agora
11    int mc {}; // pico historico de instancias simultaneas
12    int tc {}; // total de instancias criadas desde o inicio
13 };
```

Listing 9: Campos do `MetaObject`: identidade, estrutura e contadores de instância.

O encadeamento via `baseMetaObject` espelha em dados a mesma hierarquia que `isFactoryName()` percorre recursivamente em código. Essa redundância é proposital: a cadeia de `MetaObjects` pode ser percorrida em tempo de depuração para inspecionar a hierarquia sem executar nenhuma chamada virtual.

Os três contadores são uma ferramenta de diagnóstico integrada e sem custo adicional, pois são atualizados automaticamente pelas macros `STANDARD_CONSTRUCTOR`/`STANDARD_DESTRUCTOR`:

- `count` nunca retornando a zero ao final de uma simulação indica instâncias vazando.
- `mc` ajuda a dimensionar pré-alocação de memória.
- `tc` elevado em uma classe simples pode indicar criação e destruição excessivas — candidata a *object pooling*.

`Object::metaObject` é o único cujo `baseMetaObject` é `nullptr` — é a raiz da cadeia, fechando a recursão de `isClassType()/isFactoryName()`.

III.1.5 SlotTable: indexação cumulativa de *slots*

Cada classe que aceita *slots* declara, via `BEGIN_SLOTTABLE/END_SLOTTABLE`, um array local de nomes. Em tempo de inicialização estática, a macro instancia uma `SlotTable` que guarda esse array e um ponteiro para a `SlotTable` da classe base — formando uma cadeia que espelha a hierarquia.

A decisão de projeto crítica é que a indexação seja **global e cumulativa**: se a classe base já define N *slots*, os *slots* declarados pela subclasse começam no índice $N + 1$. Isso garante que um índice passado de uma subclasse para a base signifique o mesmo *slot* em qualquer nível da cadeia — condição necessária para o repasse funcionar:

```

1 // -- Foo.cpp --
2
3 // declara os slots proprios de Foo (indices locais 1, 2, ...)
4 BEGIN_SLOTTABLE(Foo)
5     "speed",      // indice local 1 -> indice global N+1
6     "altitude",   // indice local 2 -> indice global N+2
7 END_SLOTTABLE(Foo)
8
9 // implementa setSlotByIndex(): cast, teste e repasse
10 BEGIN_SLOT_MAP(Foo)
11     ON_SLOT(1, setSlotSpeed, Number) // indice local 1 -> funcao setSlotSpeed()
12     ON_SLOT(2, setSlotAltitude, Number) // indice local 2 -> funcao setSlotAltitude()
13 END_SLOT_MAP()
14
15 // a funcao de configuracao verifica o tipo e armazena o valor
16 bool Foo::setSlotSpeed(const Number* const n)
17 {
18     speed = n->getReal();
19     return true;
20 }

```

Listing 10: Declaração de *slots* e despacho em `setSlotByIndex()`.

`BEGIN_SLOT_MAP/ON_SLOT/END_SLOT_MAP` geram o corpo de `setSlotByIndex()`. O detalhe que vale conhecer é que o repasse à classe base acontece **antes** da cadeia de testes, não depois — e é decidido por comparação de índice, não por falha de despacho:

```

1 bool Foo::setSlotByIndex(const int slotindex, Object* const obj)
2 {
3     // 1. Quantos slots existem ATE a classe base?
4     const int _n {BaseClass::getSlotTable().n()};
5
6     // 2. Indice dentro da faixa da base -> repassa IMEDIATAMENTE
7     if (slotindex <= _n) {
8         return BaseClass::setSlotByIndex(slotindex, obj);
9     }
10
11     // 3. Caso contrario, o slot e nosso: converte global -> local
12     bool _ok {};
13     int _n1 {slotindex - _n};
14
15     // -- um bloco ON_SLOT por slot proprio --
16     if ( !_ok ) { // ON_SLOT(1, setSlotSpeed, Number)
17         const auto _msg = dynamic_cast<Number*>(obj);
18         if (1 == _n1 && _msg != nullptr) {
19             _ok = setSlotSpeed(_msg);
20         }
21     }
22     // ...
23
24     return _ok; // 4. Nada casou -> false. NAO ha novo repasse a base aqui.
25 }

```

Listing 11: Expansão real das macros de *slot map* (resumo de `include/mixr/base/macros.hpp`). O repasse à base é a *primeira* decisão, e a conversão global→local vem dela.

Duas consequências práticas decorrem dessa forma. Primeira: a subtração `_n1 = slotindex - _n` é exatamente a contrapartida da indexação cumulativa — é ela que traduz o índice global de volta ao índice local que o `ON_SLOT` conhece, e é por isso que a numeração precisa ser cumulativa. Segunda: um índice que pertence a esta classe mas cujo `dynamic_cast` falha — *slot* correto, tipo errado — retorna `false` sem tentar a base. É esse `false` que sobe até `setSlotByName()` e vira a mensagem "error while setting slot name: ..." do *parser*. A recursão que fecha a cadeia é a do passo 2, e sua base é `Object::setSlotByIndex()`, que retorna `false`.

III.1.6 Object: despacho de *slots* e identificação de tipo

Object é o nível onde os mecanismos acima ganham uma interface operável. Dois métodos são a ponte entre o *parser* e o sistema de *slots*:

```
1 // chamado pelo parser: recebe o nome textual do slot e o objeto de valor
2 bool setSlotByName(const char* slotname, Object* obj);
3
4 // virtual, sobrescrito por cada subclasse via BEGIN_SLOT_MAP
5 bool setSlotByIndex(int slotindex, Object* obj);
```

Listing 12: Os dois métodos de configuração por *slots* em **Object**.

A separação é deliberada: `setSlotByName()` resolve o nome para um índice global (via `SlotTable::index()`), e `setSlotByIndex()` despacha para a função de configuração correta. O *parser* conhece apenas `setSlotByName()`; as subclasses conhecem apenas `setSlotByIndex()`.

A identificação de tipo sem `dynamic_cast` segue o mesmo padrão de recursão que os *slots*. Cada subclasse sobrescreve `isClassType()` e `isFactoryName()`: testa o próprio tipo/nome e, se não corresponder, repassa para `BaseClass`. A cadeia sobe até **Object**, que é a raiz. O resultado prático é que `obj->isFactoryName("Aircraft")` retorna `true` tanto para **Aircraft** quanto para qualquer classe que herde de **Aircraft** — o que o *framework* precisa para verificar compatibilidade de *slots*.

III.2 Component: a árvore, o tempo e os eventos

A Seção II.1 elegeu “tudo é um **Component**” a princípio de projeto, e a Seção II.3 apresentou a cadeia **Referenced** → **Object** → **Component** como a espinha do modelo de objetos. Vimos os dois primeiros elos: **Referenced** resolve o ciclo de vida, **Object** resolve a configuração por slot e a identificação de tipo. Falta o terceiro, e ele é o mais consequente — é o que transforma uma coleção de objetos configuráveis em uma *simulação*.

Component acrescenta quatro capacidades, e vale enunciá-las de saída porque o resto desta seção é o detalhamento de cada uma:

1. **Contenção:** um componente contém outros e conhece o seu continente, formando a árvore que o arquivo EDL descreve.
2. **Tempo:** `updateTC()` e `updateData()` descem automaticamente por essa árvore, sem que nenhum componente precise saber quem são os seus filhos.
3. **Eventos:** mensagens nomeadas trafegam pela árvore — para baixo por chamada direta, para cima por um mecanismo de propagação restrito a teclas.
4. **Ciclo de vida:** `reset()`, `freeze` e `shutdown` são coordenados ao longo de toda a árvore.

Armadilha — **Component** não é instanciável em EDL

Apesar de ser a base de tudo, **Component** **não** está registrada em `base/factory.cpp`. Escrever `(Component ...)` num arquivo EDL produz `undefined factory name: Component`. A classe existe para ser herdada, não instanciada — o que é coerente: um componente sem comportamento próprio não teria o que fazer numa simulação.

III.2.1 A árvore de contenção

Todo o estado da contenção cabe em dois campos:

```
1 private:
2     safe_ptr<PairStream> components; // filhos -- CONTADO por referencia
3     Component* containerPtr; // pai -- ponteiro CRU, sem ref()
```

Listing 13: Os dois únicos campos que sustentam a árvore (resumo de `include/mixr/base/Component.hpp`).

A assimetria entre os dois é deliberada e é o detalhe mais importante do mecanismo. O ponteiro para o pai é **cru, não contado**. Se um filho mantivesse uma referência contada para o seu continente, toda árvore do MIXR seria um ciclo de referências — pai referencia filho, filho referencia pai — e nenhuma delas jamais seria destruída. Contando apenas para baixo, a posse flui numa direção só: destruir a raiz destrói a árvore inteira.

Por que — o pai é um ponteiro cru

Contagem de referências resolve posse, não navegação. O pai *possui* o filho; o filho apenas *sabe* quem é o seu pai. Marcar essa distinção no tipo do ponteiro é o que impede o ciclo. A contrapartida é que `container()` pode devolver um ponteiro para um objeto em vias de ser destruído — e é por isso que existe o `shutdownNotification()` da Seção III.2.5, que dá a cada componente a chance de largar ponteiros crus antes que a destruição comece.

Coerentemente, `copyData()` **anula** o ponteiro de continente ao copiar — o comentário no fonte é direto ao ponto: “*Copied doesn’t mean contained in the same container!*”. Um clone nasce órfão, e será o código que o clonou a decidir onde inseri-lo.

Como a árvore é montada. O *slot* que constrói a árvore é `components`, e ele aceita duas formas: uma `PairStream` com vários filhos nomeados, ou um único `Component` solto. As duas convergem para o mesmo lugar:

```
1 // Forma 1 -- varios filhos nomeados
2 components: {
3     radar1: ( Radar )
4     nav1:   ( Gps   )
5     piloto: ( Autopilot )
6 }
7
8 // Forma 2 -- um filho unico, sem nome explicito
9 components: ( Radar )
10 // o framework o embrulha num Pair de nome "1", como se fosse posicional
```

Listing 14: As duas formas do *slot* `components`.

Toda mutação da lista de filhos — pelo *slot*, por `addComponent()` em tempo de execução, ou por remoção — passa por um único método, `processComponents()`. Conhecer o seu algoritmo explica várias regras que de outra forma pareceriam arbitrárias:

```
1 void Component::processComponents(
2     PairStream* const list, // lista nova
3     const std::type_info& filter, // tipo exigido dos filhos
4     Pair* const add, // um filho a acrescentar (opcional)
5     Component* const remove) // um filho a remover (opcional)
6 {
7     PairStream* oldList {components.getRefPtr()};
8
9     // 1. Uma lista NOVA e construida do zero -- a antiga nunca e editada
```

```

10  const auto newList = new PairStream();
11
12  for (/* cada par da lista recebida */) {
13      const auto cp = dynamic_cast<Component*>(pair->object());
14
15      // 2. Nao-Components sao descartados SILENCIOSAMENTE (porta de tipo)
16      if (cp != nullptr && cp != remove && (skipFilter || cp->isClassType(filter))) {
17          newList->put(pair);
18          cp->container(this);      // 3. adotado: passa a apontar para nos
19      } else if (cp == remove) {
20          cp->container(nullptr);    // removido: fica orfao
21      }
22  }
23
24  if (add != nullptr) { /* ... mesmo filtro ... */ } // 4. acrescentado no FIM
25
26  components = newList;          // 5. troca ATOMICA via safe_ptr
27  newList->unref();
28
29  // 6. a selecao e reaplicada sobre a lista nova (ver sec:base-component-select)
30  oldList->unref();
31  }

```

Listing 15: `Component::processComponents()`: o funil por onde passa toda alteração da árvore (`src/base/Component.cpp`, essência).

Seis consequências, na ordem dos passos:

1. A lista antiga nunca é modificada no lugar. Quem estiver percorrendo-a — a *thread* de tempo crítico, por exemplo — continua vendo uma lista íntegra até o fim da travessia.
2. O `dynamic_cast` é a porta de tipo do *framework*: um objeto que não seja `Component` é descartado *sem erro*. Esse silêncio é uma armadilha real (veja o quadro adiante).
3. Adotar um filho é o que estabelece `container()`. Antes de entrar numa lista de componentes, um objeto não tem pai.
4. O parâmetro `filter` é o ponto de extensão: uma classe que só aceite certos filhos sobrescreve `processComponents()` e passa um `typeid` mais estreito. É assim que o `OutputHandler` do capítulo do gravador garante que todo filho seja outro `OutputHandler`.
5. A troca é uma única atribuição a um `safe_ptr`, e é isso que torna a mutação segura enquanto outra *thread* lê.
6. A seleção sobrevive à mutação, porque é reaplicada por nome ou índice sobre a lista nova.

Armadilha — filho de tipo errado desaparece sem aviso

Se um *slot* `components:` receber um objeto que não deriva de `Component` — um `Table1`, digamos — o `dynamic_cast` falha e o objeto é simplesmente **omitido da árvore**. Não há mensagem de erro, o *parser* não acusa nada, e `isValid()` continua verdadeiro. O sintoma aparece muito depois, como um subsistema que “não faz nada”. O mesmo vale para o filtro de tipo das classes que o estreitam.

III.2.2 Navegando a árvore: a família `find`

Quatro métodos localizam objetos na árvore. Todos devolvem ponteiros **emprestados** — não contados, não precisam de `unref()` — o que os torna baratos e é a razão de aparecerem dentro de laços de tempo real:

```

1 // DESCER na arvore:
2 Pair* findByName (const char* nome); // por nome, com caminho pontilhado
3 Pair* findByIndex(int indice); // por posicao, 1-based, so filhos diretos
4 Pair* findByType (const std::type_info&); // por tipo, recursivo, primeiro que casar
5
6 // SOBE na arvore:
7 Component* findContainerByType(const std::type_info&); // primeiro ANCESTRAL do tipo

```

Listing 16: A família `find`: três descem, uma sobe.

`findContainerByType()` é o que sustenta todo o resto do *framework*: é assim que um radar aninhado a três níveis de profundidade encontra o *player* que o carrega, e como esse *player* encontra a simulação em que vive. Note que ele **não** considera o próprio objeto — começa pelo pai.

O caminho pontilhado de `findByName()`. Esta é a única das quatro com sintaxe própria, e a semântica surpreende:

```

1 // (a) nome simples -> busca em PROFUNDIDADE na subarvore inteira
2 findByName("radar1"); // acha mesmo que esteja varios niveis abaixo
3
4 // (b) nome com ponto inicial -> APENAS filhos diretos ("nome duro")
5 findByName(".radar1"); // nao desce
6
7 // (c) caminho pontilhado -> o PRIMEIRO segmento so busca filhos diretos,
8 // e todos os seguintes tambem
9 findByName("sms.estacao1.missil");

```

Listing 17: As três formas de `findByName()` e o que cada uma percorre.

A assimetria entre (a) e (c) é a parte não óbvia: um nome simples faz busca profunda, mas assim que existe um ponto, *todos* os segmentos passam a ser buscas rasas. Internamente, a recursão repassa a cauda do caminho já começando com o ponto, o que reativa a regra do “nome duro” em cada nível.

Armadilha — só o “nome duro” não tem retrocesso

Se o primeiro segmento de “a.b.c” não estiver entre os filhos diretos, a busca **não** termina ali: o mesmo *fallback* do caso (a) entra em ação e o caminho *inteiro* é procurado em profundidade, filho a filho. A guarda no fonte é `if (q == nullptr && slotname[0] != '.')` — e o primeiro caractere de “a.b.c” não é um ponto.

Quem realmente não retrocede é a forma (b): num nome iniciado por ponto a guarda falha, e um “.radar1” ausente entre os filhos diretos devolve `nullptr` sem mais tentativas. É também o que acontece nos segmentos seguintes de um caminho pontilhado, porque a recursão os repassa já com o ponto.

Um detalhe à parte, e esse sim sem rede: o *buffer* interno que copia o primeiro segmento tem 128 bytes, sem verificação de limite.

Há ainda a operação inversa, `findNameOfComponent()`, que devolve o caminho completo de um componente. Ela é a única da família que devolve um objeto **contado** — o chamador precisa chamar `unref()`, exatamente como em `PairStream::findName()` (Seção III.4.2).

Regra — quem devolve emprestado e quem devolve contado

Na árvore de componentes: `findByName()`, `findByIndex()`, `findByType()` e `findContainerByType()` devolvem ponteiros **emprestados** — não chame `unref()`. Já `getComponents()` e `findNameOfComponent()` devolvem objetos **contados** — `unref()` é obrigatório.

III.2.3 A propagação do tempo, e o que `tcFrame()` acrescenta

A Seção II.4 descreveu dois fluxos de atualização que “percorrem a árvore automaticamente”. O mecanismo é este:

```
1 void Component::updateTC(const double dt)
2 {
3     PairStream* subcomponents {getComponents()}; // contado: precisa unref()
4     if (subcomponents != nullptr) {
5         if (selection != nullptr) {
6             // Ha uma selecao: SO o filho selecionado e atualizado
7             if (selected != nullptr) selected->tcFrame(dt);
8         } else {
9             // Sem selecao: todos os filhos, na ordem da lista
10            List::Item* item{subcomponents->getFirstItem()};
11            while (item != nullptr) {
12                const auto pair = static_cast<Pair*>(item->getValue());
13                const auto obj = static_cast<Component*>(pair->object());
14                obj->tcFrame(dt); // <-- tcFrame(), NAO updateTC()
15                item = item->getNext();
16            }
17        }
18        subcomponents->unref();
19    }
20 }
```

Listing 18: `Component::updateTC()` — a propagação, transcrita (`src/base/Component.cpp`).

Três coisas merecem atenção. A primeira é que a lista é obtida com `getComponents()`, que a devolve *contada* — a lista fica viva durante toda a travessia mesmo que outra *thread* a substitua no meio dela. É a contrapartida direta da troca atômica do `processComponents()`.

A segunda é que o `dt` atravessa **sem modificação**. `Component` não zera o `dt` quando congelado; quem faz isso é cada subclasse, e por isso o congelamento não é automático (veja Seção III.2.5).

A terceira é a mais consequente: a chamada aos filhos é `tcFrame(dt)`, e não `updateTC(dt)`. Essa distinção é a resposta à pergunta “o que `tcFrame()` acrescenta?”:

```
1 void Component::tcFrame(const double dt) // NAO e virtual
2 {
3     double tcStartTime {};
4     if (isTimingStatsEnabled()) tcStartTime = getComputerTime();
5
6     this->updateTC(dt); // o trabalho de fato
7
8     if (isTimingStatsEnabled()) {
9         double dtime {(getComputerTime() - tcStartTime) * 1000.0}; // em MS
10        timingStats->sigma(dtime);
11        if (isTimingStatsPrintEnabled()) printTimingStats();
12    }
13 }
```

Listing 19: `Component::tcFrame()` — instrumentação opcional em torno de `updateTC()` (`src/base/Component.cpp`, condensado).

`tcFrame()` é um invólucro **não virtual** de medição em torno de `updateTC()`. Se as estatísticas estiverem desligadas — o padrão — ele é um repasse puro, e o custo é zero. Se estiverem ligadas, ele cronometra a chamada e alimenta um `Statistic`. Como a propagação passa por `tcFrame()` em todos os níveis, **qualquer nó da árvore pode ser instrumentado isoladamente** ligando um único *slot* nele — e o número que sai já inclui toda a sua subárvore.

Armadilha — só o caminho de tempo crítico é instrumentado

Não existe um `dataFrame()`. `updateData()` chama `updateData()` nos filhos diretamente, sem invólucro e sem medição. As estatísticas de tempo cobrem exclusivamente o laço de tempo crítico.

III.2.4 Eventos: mensagens nomeadas na árvore

Além do tempo, componentes trocam *eventos*: mensagens identificadas por um inteiro simbólico, opcionalmente carregando um objeto. O despacho se escreve com um trio de macros que espelha o de *slots*:

```
1 BEGIN_EVENT_HANDLER(MeuSistema)
2   ON_EVENT_OBJ( RESET_EVENT, aoResetarCom, Number ) // com objeto tipado
3   ON_EVENT_ ( RESET_EVENT, aoResetar ) // sem objeto
4   ON_EVENT_OBJ( DATALINK_MESSAGE, aoReceber, Message )
5 END_EVENT_HANDLER()
```

Listing 20: O tratador de eventos de uma classe qualquer.

A expansão segue exatamente o padrão do `BEGIN_SLOT_MAP` da Seção III.1.5: uma cadeia de testes sobre uma variável `_used`, em que **o primeiro que casar vence**, e `END_EVENT_HANDLER()` repassa a `BaseClass::event()` o que ninguém tratou.

Regra — a forma com objeto vem antes da forma sem

Para um mesmo *token*, `ON_EVENT_OBJ` deve ser escrito **antes** de `ON_EVENT`. Como o primeiro teste que casar encerra a cadeia, a ordem inversa faria a versão sem objeto capturar também os eventos que trazem um — e o objeto seria descartado silenciosamente.

Os *tokens* vêm de `eventTokens.hpp`, que é incluído *dentro* do corpo da classe `Component`. A consequência é agradável: todo componente herda os nomes e pode escrevê-los sem qualificação. Os grupos são:

Faixa	Grupo	Exemplos
1–127	teclas ASCII	ENTER_KEY (13), ESC_KEY (27), DELETE_KEY (127)
128–152	mouse e edição	ON_SINGLE_CLICK, ON_DOUBLE_CLICK, F1_KEY... F12_KEY
200–239	teclas de MFD	OSB_T1... OSB_L10 (botões da moldura)
999	— MAX_KEY_EVENT: a fronteira que decide a propagação	
1001	sistema	SHUTDOWN_EVENT
1201–1235	gráficos	ON_ENTRY, ON_EXIT, ON_RETURN, SELECT
1301–1323	simulação	RESET_EVENT, FREEZE_EVENT, KILL_EVENT, CRASH_EVENT, RF_EMISSION, IR_QUERY, DATALINK_MESSAGE
1400–1423	HOTAS	TRIGGER_SW_EVENT, WPN_REL_EVENT, TMS_*, DMS_*
≥2000	da aplicação	USER_EVENTS

Table 2: Os grupos de *tokens* de `eventTokens.hpp`.

A direção da propagação. O `event()` do próprio `Component` é escrito à mão, e não com as macros — justamente porque precisa de um final não padrão:


```

1  bool Component::event(const int _event, Object* const _obj)
2  {
3      bool _used {};
4
5      ON_EVENT_OBJ(SELECT,      select, Number)
6      ON_EVENT_OBJ(SELECT,      select, String)
7      ON_EVENT(RESET_EVENT,     onEventReset)
8      ON_EVENT(SHUTDOWN_EVENT,  shutdownNotification)
9      ON_EVENT_OBJ(FREEZE_EVENT, setSlotFreeze, Number)
10
11     // *** Special handling of the end of the EVENT table ***
12     // Pass only key events up to our container
13     if (_event <= MAX_KEY_EVENT && container() != nullptr) {
14         _used = container()->event(_event, _obj);
15     }
16     return _used;
17 }

```

Listing 21: `Component::event()` transcrito: repare no final (`src/base/Component.cpp`).

Regra — eventos de tecla sobem; os demais morrem

Um evento não tratado só é repassado ao continente se o seu *token* for $\leq \text{MAX_KEY_EVENT}$ (999) — ou seja, se for uma tecla. Eventos de simulação como `RESET_EVENT` ou `RF_EMISSION` **não sobem**: se ninguém os tratou naquele nível, eles se perdem ali.

A razão é de interface: uma tecla pressionada tem que encontrar quem a queira, e quem a queira pode estar acima na hierarquia de telas. Já um evento de simulação é endereçado — quem o envia sabe a quem.

send(): eventos com valor, e por que existe um *cache*. Enviar um evento a um componente nomeado é frequente o bastante para ter atalho:

```

1  // Forma simples: localiza por nome e dispara
2  send("radar1", RESET_EVENT);
3
4  // Forma com valor: exige um SendData, que deve ser MEMBRO da classe
5  class Paine1 : public Component {
6      SendData sdAltitude;    // um por destino/valor -- persiste entre frames
7  };
8
9  void Paine1::updateData(const double dt)
10 {
11     send("mostradorAlt", UPDATE_VALUE, getAltitudeFt(), sdAltitude);
12 }

```

Listing 22: A família `send()` e o objeto `SendData` que a acompanha.

O `SendData` guarda duas coisas, cada uma resolvendo um problema diferente:

1. **O destinatário já resolvido.** `findByName()` é uma busca recursiva com comparação de *strings*; refazê-la a cada quadro, para cada mostrador de um painel, seria proibitivo. O `SendData` guarda o ponteiro e só busca na primeira vez.
2. **O último valor enviado.** O evento só é disparado se o valor *mudou*. Um mostrador acionado a 50 Hz com um número parado não gera evento nenhum.

Por que — `send()` é um filtro de mudança

A segunda função é a que surpreende, e é a mais valiosa: a família `send()` não é apenas um atalho para `findByName() + event()`, é um **detector de mudança**. É o que permite escrever a atualização de um painel inteiro como uma lista de `send()` incondicionais, chamada a cada quadro, sem inundar a árvore de eventos redundantes.

Armadilha — `SendData` guarda um ponteiro cru

O destinatário em *cache* não é contado por referência. Se a árvore mudar — um componente removido, uma lista reconstruída — o ponteiro fica pendurado. Quem altera a árvore em tempo de execução precisa chamar `empty()` nos `SendData` afetados. Por isso eles devem ser membros da classe, e não variáveis locais: uma variável local perderia o *cache* a cada quadro, anulando as duas vantagens.

III.2.5 Ciclo de vida: *reset*, *freeze* e *shutdown*

Três operações coordenam o estado da árvore inteira, e as três têm comportamentos de propagação diferentes — o que é fonte recorrente de engano.

`reset()` **propaga, mas não reinicia nada por conta própria**. `Component::reset()` percorre os filhos e chama `reset()` em cada um (respeitando a seleção, se houver). Ele **não toca em nenhum campo do próprio Component** — nem *freeze*, nem estatísticas, nem *shutdown*. Restaurar condições iniciais é responsabilidade de cada subclasse, que faz o seu trabalho e chama `BaseClass::reset()` para que a propagação continue. O evento `RESET_EVENT` chega aqui pela ponte `onEventReset()`, que apenas chama `reset()`.

`freeze()` **não propaga**. Esta é a assimetria que mais engana:

```
1 void Component::freeze(const bool f) { frz = f; }
2 bool Component::isFrozen() const { return frz; }
```

Listing 23: `freeze()` é apenas um par de acessores.

Armadilha — congelar um componente não congela os filhos

`freeze()` escreve um `bool` e nada mais. Ele não desce pela árvore, e `updateTC()` não zera o `dt` dos filhos por causa dele. Cada subclasse é que precisa consultar `isFrozen()` e agir — o padrão do *framework* é a linha `double dt{dt0}; if (isFrozen()) dt = 0.0;` que aparece em `Simulation`, `Player` e `System`. Um componente próprio que se esqueça dessa linha continuará rodando com a simulação congelada.

`shutdownNotification()` **propaga e deve ser idempotente**. O encerramento é o oposto do *reset*: desce pela árvore por meio de *eventos*, não por chamada direta.

```
1 bool Component::shutdownNotification()
2 {
3     PairStream* subcomponents {getComponents()};
4     if (subcomponents != nullptr) {
5         List::Item* item {subcomponents->getFirstItem()};
6         while (item != nullptr) {
7             const auto pair = static_cast<Pair*>(item->getValue());
8             const auto p = static_cast<Component*>(pair->object());
```

```

9      p->event(SHUTDOWN_EVENT);    // vira shutdownNotification() no filho
10     item = item->getNext();
11   }
12   subcomponents->unref();
13 }
14 shutdown = true;
15 return shutdown;
16 }

```

Listing 24: shutdownNotification() transcrito (src/base/Component.cpp).

Regra — shutdownNotification() tem duas obrigações

Toda sobrescrita precisa (a) chamar `BaseClass::shutdownNotification()`, sob pena de a subárvore inteira nunca ser notificada, e (b) ser **idempotente**: `deleteData()` a chama de novo “por via das dúvidas”, de modo que ela roda ao menos duas vezes em toda destruição normal.

É aqui que um componente larga os ponteiros crus que mantém para os seus pares — justamente aqueles que a contagem de referências não protege, como o `container()` da Seção III.2.1.

III.2.6 Seleção, estatísticas e mensagens

Seleção: um filho de cada vez. Os *slots* `select` e o par `selection/selected` permitem eleger **um único** filho para receber as atualizações. Todos os demais deixam de custar qualquer coisa. É o mecanismo por trás da troca de páginas de um MFD: as dez formatações existem na árvore, uma só é atualizada.

Armadilha — seleção que não resolve silêcia a árvore

Se o nome ou índice selecionado não for encontrado, o *framework* imprime “`Component::select: name not found!`” **mas mantém a seleção ativa**. Como `updateTC()` pergunta “há seleção?” antes de perguntar “ela resolveu?”, o resultado é que **nenhum** filho passa a ser atualizado — a subárvore inteira congela em silêncio. `isComponentSelected()` continua devolvendo `true`.

Estatísticas de tempo. Ligadas pelos *slots* `enableTimingStats` e `printTimingStats`, alimentam o `Statistic` descrito na Seção III.2.3. “Habilitado” é literalmente “o ponteiro para o `Statistic` não é nulo” — não há um *flag* separado. As amostras são em milissegundos.

Tipos de mensagem, e a herança pela árvore. As constantes vivem em `Object`: `MSG_ERROR`, `MSG_WARNING`, `MSG_INFO`, `MSG_DEBUG`, `MSG_DATA`, `MSG_USER`. O padrão habilita apenas as duas primeiras. O que `Component` acrescenta é uma única regra, e ela é elegante:

```

1  bool Component::isMessageEnabled(const unsigned short msgType) const
2  {
3      bool enabled {BaseClass::isMessageEnabled(msgType)};
4
5      // Se nao estava habilitada aqui, e nao foi explicitamente DESABILITADA aqui,
6      // e temos um continente: pergunte a ele.
7      if ( !enabled && !isMessageDisabled(msgType) && containerPtr != nullptr) {
8          enabled = containerPtr->isMessageEnabled(msgType);
9      }
10     return enabled;
11 }

```

Listing 25: Component::isMessageEnabled() transcrito (src/base/Component.cpp).

Ou seja: a política de mensagens é **herdada para baixo pela árvore de contenção**, e qualquer componente pode vetá-la localmente. Ligar MSG_DEBUG na Station torna a simulação inteira verbosa; ligar num único radar, apenas aquele ramo. Note que MSG_ERROR não pode ser desabilitada — o *slot* `disableMessageType` aceita apenas WARNING, INFO, DEBUG, USER e DATA.

III.2.7 Os slots de Component

Sete *slots*, herdados por absolutamente toda classe do *framework*:

#	Slot	Tipo	Efeito
1	<code>components</code>	PairStream Component	monta a árvore de filhos
2	<code>select</code>	String Number	elege um único filho a atualizar
3	<code>enableTimingStats</code>	Number	liga a cronometragem de <code>tcFrame()</code>
4	<code>printTimingStats</code>	Number	imprime as estatísticas a cada quadro
5	<code>freeze</code>	Number	estado inicial do <i>flag</i> de congelamento
6	<code>enableMessageType</code>	Identifier Number	habilita categorias de mensagem
7	<code>disableMessageType</code>	Identifier Number	desabilita (exceto ERROR)

Table 3: A tabela de *slots* de *Component*. Por serem os primeiros da cadeia, ocupam os índices globais 1 a 7 de qualquer classe *derivada de Component*.

Como *Component* está na base de toda a árvore de simulação, esses sete são os primeiros índices globais da *SlotTable* de qualquer classe que dele derive (Seção III.1.5) — o que explica por que os *slots* próprios de uma classe como *Radar* começam num índice bem acima de 1.

A ressalva importa porque nem tudo no MIXR é um *Component*. *Object* não declara *slot* nenhum — a sua *SlotTable* é construída com tamanho zero — e classes que herdam diretamente dele (*Number*, *String*, *Color*, *Table1*, *Timer*) começam a numeração no 1 por conta própria. É justamente isso que faz (*Feet* 10000) funcionar: naquela cadeia o *slot* global 1 é o *value* de *Number*, e não *components* (Seção III.3.3).

III.2.8 Exemplo completo: montar, navegar e instrumentar

Reunindo tudo, eis uma árvore declarada em EDL e o código que a percorre:

```

1  ( Aircraft
2    type: "F-16C"
3
4    // Liga a cronometragem DESTA no e de toda a sua subarvore
5    enableTimingStats: 1
6    printTimingStats: 1
7
8    // Deixa a subarvore inteira verbosa
9    enableMessageType: DEBUG
10
11    components: {
12      sensores: ( Rwr
13        components: {
14          antena1: ( Antenna )
15        }
16      )
17      piloto: ( Autopilot )
18    }

```

19 |)

Listing 26: Uma árvore de componentes com nomes, seleção e instrumentação.

```
1 // Busca PROFUNDA: acha "antena1" mesmo dois níveis abaixo
2 base::Pair* p { aircraft ->findByName("antena1") };
3
4 // Caminho pontilhado: so filhos diretos em cada segmento
5 base::Pair* q { aircraft ->findByName("sensores.antena1") }; // mesma antena
6
7 // Busca RASA: nao desce, entao NAO acha
8 base::Pair* r { aircraft ->findByName(".antena1") }; // nullptr
9
10 // Por tipo, recursivo -- o idioma usado por Player::updateSystemPointers()
11 base::Pair* s { aircraft ->findByType(typeid(Antenna)) };
12
13 // Subindo: de dentro da antena, achar o Player que a carrega
14 Player* dono { static_cast<Player*>(
15     antena->findContainerByType(typeid(Player))) };
16
17 // Nenhum dos ponteiros acima precisa de unref() -- todos sao emprestados.
```

Listing 27: Navegando a mesma árvore a partir do C++.

Esse último idioma — subir com `findContainerByType()` até encontrar o *player* dono — reaparecerá dezenas de vezes na Seção VI. É o que permite que um subsistema seja escrito sem nunca receber um ponteiro para o seu contexto: ele o encontra.

III.3 O parser EDL: da gramática à árvore

O *parser* EDL é o ponto onde todos os mecanismos anteriores convergem: ele lê o texto, chama a fábrica, configura os *slots* e organiza os objetos em estruturas de contenção. Como foi introduzido na Seção II.3, seu acoplamento com o domínio é zero — toda ligação com classes concretas chega por injeção de uma única função:

```
1 typedef Object* (*factory_func)(const std::string& name);
2
3 Object* edl_parser(const std::string& filename,
4     factory_func f,
5     int* num_errors = nullptr);
```

Listing 28: Interface do *parser* EDL: gramática desacoplada do domínio.

A função interna `parse()` — chamada a cada forma (Nome ...) reconhecida — executa a tríade fábrica / *slots* / validação que a Seção II.3 descreveu conceitualmente. Aqui está sua implementação real:

```
1 // src/base/edl_parser/edl_parser.y
2 static Object* parse(const std::string& name, PairStream* arg_list)
3 {
4     Object* obj {};
5
6     if (factory != nullptr) { // 'factory' e o ponteiro injetado
7
8         // 1. FABRICA: instancia o objeto pelo nome
9         obj = factory(name);
10
11         // 2. SLOTS: configura cada atributo pelo nome
12         if (obj != nullptr && arg_list != nullptr) {
13             List::Item* item {arg_list->getFirstItem()};
14             while (item != nullptr) {
15                 Pair* p {static_cast<Pair*>(item->getValue())};
16                 bool ok {obj->setSlotByName(*p->slot(), p->object())};
```

```

17         if (!ok) {
18             yyerror(("error while setting slot name: "
19                     + std::string(*p->slot()).c_str());
20         }
21         item = item->getNext();
22     }
23
24     // 3. VALIDACAO: o objeto tem a palavra final sobre sua integridade
25     if (!obj->isValid()) {
26         yyerror(("error: invalid object: " + name).c_str());
27     }
28 }
29 else if (obj == nullptr) {
30     yyerror(("undefined factory name: " + name).c_str());
31 }
32 }
33 return obj;
34 }

```

Listing 29: A função `parse()` real de `edl_parser.y`: triade em código.

Falhas em qualquer etapa são reportadas com arquivo e número de linha — o *scanner* mantém essas informações durante toda a análise — mas **nenhuma delas interrompe a análise**. `yyerror()` apenas imprime a mensagem e incrementa um contador; `parse()` segue configurando os *slots* restantes e devolve o objeto de qualquer forma. Quem decide o que fazer é o chamador, através do parâmetro de saída `num_errors`:

```

1  int nErrs {};
2  Object* raiz { base::edl_parser("cenario.edl", minhaFactory, &nErrs) };
3
4  // Um ponteiro nao-nulo NAO significa que o arquivo estava correto:
5  // pode haver dezenas de slots mal configurados. Verifique sempre:
6  if (raiz == nullptr || nErrs > 0) {
7      std::cerr << "configuracao invalida (" << nErrs << " erros)\n";
8      return EXIT_FAILURE;
9  }

```

Listing 30: O contrato de erro de `edl_parser()`: um ponteiro devolvido e uma contagem de erros a verificar.

Ignorar `num_errors` é o engano mais comum ao integrar o *parser*: a simulação sobe, a árvore existe, e apenas os parâmetros mal escritos ficaram silenciosamente com seus valores padrão.

A aplicação encadeia as fábricas das bibliotecas que utiliza. Um nome ambíguo é resolvido pela primeira fábrica da cadeia que o reconhece, o que permite que um modelo específico se sobreponha a um genérico homônimo sem tocar no *parser*.

III.3.1 Nome de fábrica ≠ nome de classe

Há uma suposição que quase todo leitor faz ao ver (`Aircraft ...`) resolver para a classe `Aircraft`, e que é a origem da maior parte dos arquivos EDL que não carregam:

Regra — o nome de fábrica é uma *string* independente

O nome de fábrica é o **segundo argumento** de `IMPLEMENT_SUBCLASS` — uma *string* escolhida livremente. Ela coincide com o nome da classe C++ na maioria dos casos, mas isso é uma *convenção*, não uma regra do *framework*. Em **51 classes** do MIXR os dois divergem.

E o comentário `// Factory name: ...` no topo dos cabeçalhos **não é confiável**: vários estão desatualizados em relação à macro logo abaixo. A fonte de verdade é sempre o `IMPLEMENT_*SUBCLASS` no

.cpp.

As divergências não são aleatórias — caem em cinco padrões:

Padrão	Exemplos	Razão
Símbolo	Add→"+", Divide→"/"	operadores ficam legíveis em notação prefixada (Seção III.14.5)
Minúscula	Rgb→"rgb", Hsva→"hsva", Integer→"int"	tipos “de valor” parecem palavras-chave, não classes
Prefixo de módulo	FileWriter→ "RecorderFileWriter", NetIO→"DisNetIO", Ntm→"DisNtm"	desambiguar nomes que se repetem entre módulos
Prefixo Abstract/Base	Distance→ "AbstractDistance", IoHandler→"BaseIoHandler", StoresMgr→"BaseStoresMgr"	liberar o nome “bonito” para a classe concreta de uso comum
Renomeação	Aam→"AamMissile", Image→"SarImage", Tdb→"Gimbal_Tdb"	o nome EDL é mais descritivo que o nome C++

Table 4: Os cinco padrões de divergência entre nome de classe e nome de fábrica. O catálogo completo está no apêndice.

Armadilha — o quarto padrão é o perigoso

Os quatro primeiros padrões produzem erro imediato e claro: (Decibel7.0) falha com `undefined factory name: Decibel`, e o autor corrige para (dB7.0). O quinto caso do quarto padrão não falha — e é por isso que é o pior. StoresMgr→"BaseStoresMgr" liberou o nome "StoresMgr", que foi então tomado por SimpleStoresMgr. Escrever (StoresMgr ...) num arquivo EDL **funciona** — e constrói silenciosamente um objeto de outra classe, com outro comportamento. Um erro assim não aparece no carregamento; aparece como um comportamento inesperado horas depois.

Registrado é diferente de existir. Uma segunda condição, independente da primeira: mesmo com o nome certo, a classe só é construtível se alguma `factory.cpp` a registrar. Dos 334 nomes de fábrica declarados no fonte, **224 estão registrados**. Os demais são classes abstratas, classes internas — ou omissões: AgentTC, IoHandler e *todo* o módulo linearsystem, que sequer tem uma `factory.cpp` (Seção VII).

Regra — como descobrir o nome de fábrica correto

Diante de qualquer classe, o procedimento é sempre o mesmo, e leva dez segundos:

1. Abra o .cpp da classe e leia a *string* do IMPLEMENT_*SUBCLASS — esse é o nome.
2. Confirme que ele aparece na `factory.cpp` do módulo.

Não confie no nome da classe, nem no nome do arquivo, nem no comentário do cabeçalho.

III.3.2 A gramática do EDL

O *scanner* (gerado por *flex*) categoriza o texto em identificadores (**IDENT**), *slot-ids* (identificador seguido de **:**, **SLOT_ID**), *strings* entre aspas ou chevrons (**STRING_LITERAL**), constantes inteiras, de ponto flutuante e booleanas (**INTEGERconstant**, **FLOATINGconstant**, **BOOLconstant**) e os delimitadores literais (**(**, **)**, **{**, **}**, **[**, **]**). A gramática que o *parser* (gerado por *bison*) reconhece sobre essa sequência é, transcrita de `edl_parser.y` e levemente simplificada:

```
1 // check-fontes: esquematico
2 file      ::= form
3           | SLOT_ID form
4
5 arglist    ::= /* vazio */           // -> PairStream() vazia
6           | arglist form           // item POSICIONAL (objeto sem nome)
7           | arglist prim           // item POSICIONAL (literal sem nome)
8           | arglist slot_value     // item NOMEADO
9
10 form      ::= '(' IDENT arglist ')' // objeto: ( Tipo ... )
11           | '{' arglist '}'        // lista de itens (vira uma PairStream)
12
13 slot_value ::= SLOT_ID prim         // nome: literal
14           | SLOT_ID form           // nome: ( Tipo ... )
15
16 prim       ::= STRING_LITERAL      // "texto" ou <texto> -> String
17           | IDENT                  // palavra nua -> Identifier
18           | BOOLconstant           // true / false -> Boolean
19           | '[' numlist ']'        // lista de numeros -> List
20           | number                 // Integer ou Float
21
22 numlist    ::= number | numlist number
23 number     ::= INTEGERconstant | FLOATINGconstant
```

Listing 31: Gramática do EDL: as produções de `src/base/edl_parser/edl_parser.y`, reescritas em notação BNF. Note que **arglist** aceita itens *sem* nome.

Três pontos dessa gramática merecem comentário.

O primeiro é que um bloco entre chaves — a produção `'{' arglist '}'` — é construído como uma *PairStream* na ordem em que os itens aparecem. É exatamente esse mecanismo que torna possível a sintaxe do *slot players*: na *Simulation*: um único *slot* recebe uma lista completa de *players* nomeados.

O segundo é que um identificador nu, sem aspas, é um *valor* válido — vira um *Identifier*, não um erro de sintaxe. É assim que `earthModel: wgs84` funciona sem aspas.

O terceiro é o mais importante, e é o assunto da subseção seguinte: **arglist** aceita **form** e **prim** **sem** um **SLOT_ID** à frente.

III.3.3 Slots posicionais: por que `(Feet 10000)` funciona

Praticamente todo exemplo de EDL neste manual usa, em algum ponto, a forma `(Feet 10000)`, `(Meters 1.0)` ou `(Float 600.0)`. Nenhuma delas nomeia um *slot* — e, no entanto, o valor chega ao objeto. O mecanismo que faz isso funcionar é o de **slots posicionais**, e ele opera em dois tempos.

Primeiro, o *parser* dá um nome a cada item sem nome: a sua posição ordinal, como texto. A ação semântica da produção **arglist prim** é literalmente esta:

```
1 | arglist prim {
2     int i = $1->entries(); // quantos itens ja ha
```



```

3      char cbuf[20] {};
4      std::sprintf(cbuf, "%i", i+1); // "1", "2", "3", ...
5      Pair* p {new Pair(cbuf, $2)}; // nome = a posicao
6      $2->unref();
7      $1->put(p);
8      p->unref();
9      $$ = $1;
10     }

```

Listing 32: A regra que nomeia itens posicionais em `edl_parser.y`: o i -ésimo item anônimo vira o `Pair` de nome `"i"`.

Segundo, `Object::slotName2Index()` reconhece nomes puramente numéricos e os interpreta como **índices globais diretos**, saltando a busca por nome na `SlotTable`:

```

1  int Object::slotName2Index(const char* const slotname) const
2  {
3      int slotindex {};
4      if (slotname == nullptr) return slotindex;
5
6      const int n {slotTable->n()};
7
8      // a) o nome e composto so de digitos? (ex.: "1", "12")
9      bool isNum {true};
10     for (int i = 0; isNum && slotname[i] != '\0'; i++) {
11         if ( !std::isdigit (slotname[i]) ) isNum = false;
12     }
13
14     if (isNum) {
15         // b) SIM: o proprio numero E o indice global do slot
16         int j {std::atoi(slotname)};
17         if (j > 0 && j <= n) slotindex = j;
18     } else {
19         // c) NAO: resolucao normal por nome
20         slotindex = slotTable->index(slotname);
21         if (slotindex <= 0)
22             std::cerr << "slot not found: " << slotname << std::endl;
23     }
24     return slotindex;
25 }

```

Listing 33: `Object::slotName2Index()`: o ramo que fecha o mecanismo (`src/base/Object.cpp`).

Juntando as duas pontas: em `(Feet 10000)`, o literal `10000` é o primeiro item anônimo, recebe o nome `"1"`, e `"1"` resolve para o `slot` global de índice 1. Na cadeia `Object → Number → Distance → Feet`, o `slot` de índice 1 é `"value"`, declarado por `Number` — que é precisamente onde o valor numérico deve ir. As duas formas abaixo são, portanto, rigorosamente equivalentes:

```

1  altitude: ( Feet 10000 )      -- item anonimo -> Pair("1", Integer)
2  altitude: ( Feet value: 10000 ) -- identico, com o slot nomeado

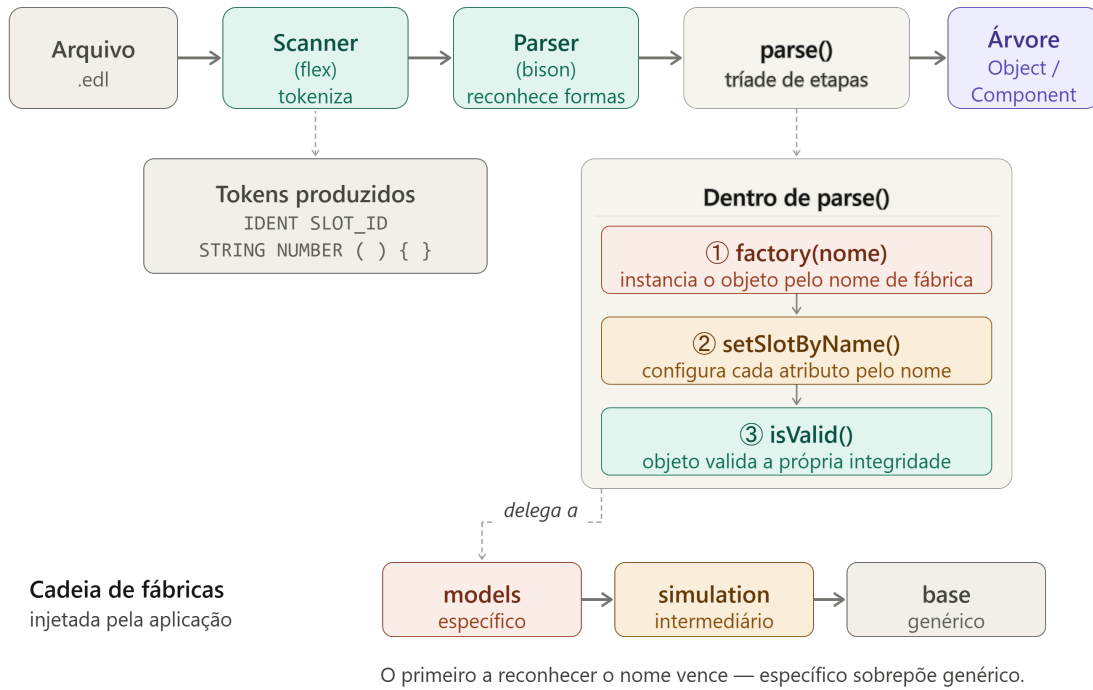
```

Listing 34: Forma posicional e forma nomeada: o mesmo objeto, o mesmo `slot`.

A economia de notação que isso compra é grande — é o que permite escrever `(Meters 3048)` em vez de `(Meters value: 3048)` em cada grandeza física de um arquivo de configuração. O preço é uma dependência silenciosa da *ordem de declaração* dos `slots` na hierarquia: uma classe que insira um `slot` novo antes dos existentes muda o significado de todo EDL posicional que a referencie. Na prática, a forma posicional é usada quase exclusivamente para o `slot` 1 de `Number` e suas famílias de unidades, onde essa ordem é estável por construção.

Arquivos gerados. Os arquivos `EdlScanner.cpp` e `EdlParser.cpp` são distribuídos **já gerados** no repositório, ao lado das fontes `edl_scanner.l` e `edl_parser.y` que os originaram — *flex* e *bison* não são

necessários para compilar o MIXR, mas as fontes estão presentes para quem precisar alterar a gramática. Os cabeçalhos `EdlParser.hpp` e `EdlScanner.hpp` completam o conjunto; sem eles o *scanner* não compila.



O parser conhece apenas a gramática — as classes concretas chegam exclusivamente pela fábrica injetada. Cada forma (Nome ...) reconhecida aciona a tríade uma vez, produzindo um nó na árvore.

Figure 6: Do texto EDL à árvore de objetos: as três etapas de `parse()`.

III.3.4 Percurso completo de uma linha EDL

Para solidificar a compreensão, vale acompanhar o percurso completo de uma linha concreta através de todos os mecanismos. Considere:

```
1 ownship: ( Aircraft type: "F-16" )
```

Listing 35: Uma linha EDL e os mecanismos que ela aciona em sequência.

1. **Scanner** produz a sequência de *tokens*: `SLOT_ID("ownship"), '(', IDENT("Aircraft"), SLOT_ID("type"), STRING("F-16"), ')'`.
2. **Parser** reconhece a forma e chama `parse("Aircraft", {type: String("F-16")})`.
3. `parse()` executa:
 - (a) `factory("Aircraft")` retorna `new Aircraft()`, cujo `STANDARD_CONSTRUCTOR()` registra a instância no `MetaObject` e aponta `slotTable` para a tabela de *slots* de `Aircraft`.

- (b) `aircraft->setSlotByName("type", String("F-16"))`: `Object` consulta a `SlotTable` de `Aircraft` via `index("type")` → obtém o índice global → chama `Aircraft::setSlotByIndex()` → `ON_SLOT` despacha para `setSlotType()` → armazena "F-16".
 - (c) `aircraft->isValid()` retorna `true`.
4. O objeto retornado é embrulhado em `Pair("ownership", aircraft)` e inserido na `PairStream` de filhos da `Simulation`.

Todo esse percurso ocorre sem que o *parser* conheça a classe `Aircraft`. Ele conhece apenas `Object`; as classes concretas chegam exclusivamente pela fábrica injetada.

III.4 Estruturas de contenção

O *parser* produz, a partir do texto EDL, uma árvore de objetos de tipos heterogêneos — `Aircraft`, `Sensor`, `Simulation` e dezenas de outros. O *framework* precisa de um mecanismo capaz de carregar qualquer combinação desses tipos sem conhecê-los de antemão. A solução é deliberadamente simples: como todo objeto deriva de `Object`, basta uma **lista homogênea de `Object*`**. Um único mecanismo de contenção serve a toda a árvore.

Sobre essa lista constrói-se uma segunda decisão: **nomes como cidadãos de primeira classe**. Cada entrada da lista pode ser um par (**nome**, **objeto**) — exatamente a estrutura que o EDL expressa com **nome: (Tipo ...)**. Essa correspondência direta entre arquivo e estrutura em memória é o que torna a árvore navegável pelos mesmos identificadores que o desenvolvedor escreveu no arquivo de configuração.

A Figura 7 mostra a relação entre as três classes que implementam esse modelo.

III.4.1 List: a lista e sua semântica de posse

`List` é uma lista duplamente encadeada de `Object*`. Cada nó é um `List::Item` que guarda o ponteiro para o objeto e os links `next/previous`. O que distingue `List` de um contêiner genérico é a **semântica de posse** que ela impõe sobre os objetos que guarda:

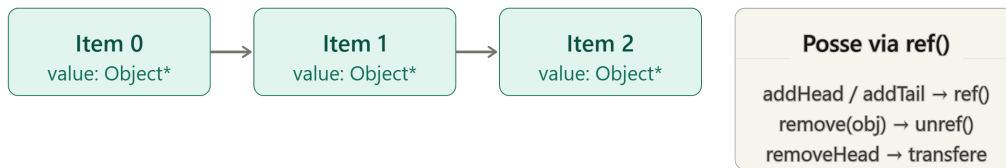
```

1  // Inserir: a lista torna-se coproprietaria -- chama ref()
2  void List::addHead(Object* const obj)
3  {
4      auto* d = new Item;
5      d->value = obj;
6      obj->ref();    // lista assume co-posse
7      addHead(d);
8  }
9
10 // Remover por valor: a lista libera a posse -- chama unref()
11 bool List::remove(const Object* const obj)
12 {
13     // ... localiza o Item ...
14     remove(d);
15     obj->unref(); // lista libera sua co-posse
16     return true;
17 }
18
19 // Remover da cabeça: transfere a posse para o chamador -- NAO chama unref()
20 Object* List::removeHead()
21 {
22     // ... remove o Item ...
23     return obj;    // chamador recebe o objeto JA possuido; nao deve ref()

```

List

lista encadeada de qualquer Object* — semântica de posse via ref/unref



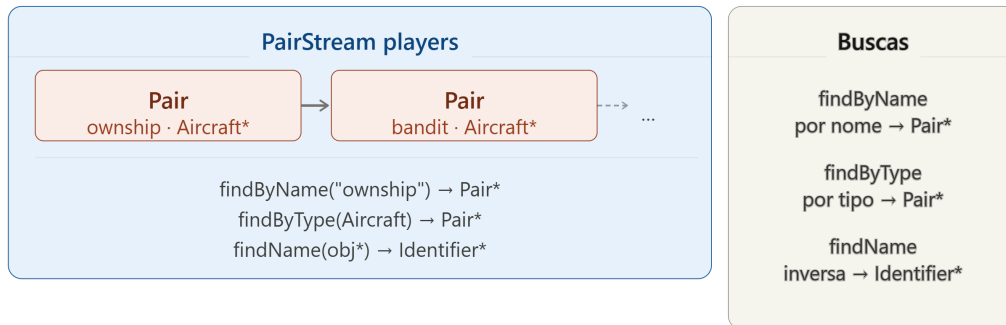
Pair

associa um nome a um objeto — cada entrada nome: valor no EDL vira um Pair



PairStream

List especializada em Pair* — representa um bloco { ... } do EDL em memória



PairStream representa players, subcomponentes e qualquer bloco { ... } do EDL.

É navegável pelos mesmos identificadores escritos no arquivo de configuração.

Figure 7: As três estruturas de contenção do MIXR: **List** (lista encadeada de **Object*** com semântica de posse via **ref/unref**), **Pair** (par nome/objeto que corresponde a uma entrada **nome: valor** no EDL) e **PairStream** (lista de **Pairs** navegável por nome e por tipo).

24 | }

Listing 36: Posse explícita em **List**: inserção adquire, remoção libera.

A distinção entre **remove(obj)** e **removeHead()** é fundamental:

- **addHead()/addTail()** chamam **ref()** — a lista torna-se co-proprietária do objeto.
- **remove(obj)** chama **unref()** — a lista abre mão da co-posse; se ninguém mais tiver referência, o objeto é destruído.
- **removeHead()/removeTail()/get()** *não* chamam **unref()** — **transferem** a posse ao chamador. O chamador *não deve* chamar **ref()** adicionalmente; o objeto já lhe pertence.

Respeitar essa assimetria evita tanto vazamentos (objeto nunca destruído) quanto **unref()** excessivo (objeto

destruído enquanto ainda em uso). O padrão correto de travessia é:

```
1 // Percorrer sem remover -- getItem() nao transfere posse
2 List::Item* item = minhaLista->getItem();
3 while (item != nullptr) {
4     Object* obj = item->getValue();
5     // ... processa obj ...
6     item = item->getNext();
7 }
```

Listing 37: Padrão canônico de travessia de `List`: obtém item, lê valor, avança.

Além da travessia sequencial, `List` oferece: `getPosition(n)` para acesso por índice (custo $O(n)$), `getIndex(obj)` para localizar um objeto por ponteiro, e `insert(newItem, refItem)` para inserção antes de um item de referência. O método `getNumberList(values[], max)` extrai os valores numéricos de uma lista de `Number` para um array C — a ponte entre a lista heterogênea do *framework* e o código numérico que espera dados contíguos:

```
1 // EDL: gains: [ 1.0  0.5  0.25 ]
2 // O parser produz uma List de Float; recuperamos assim:
3 double gains[8]{};
4 unsigned int n = slotList->getNumberList(gains, 8);
5 // gains[0]=1.0, gains[1]=0.5, gains[2]=0.25, n=3
```

Listing 38: `getNumberList()`: extraindo valores numéricos de uma lista EDL para array C.

III.4.2 `Pair` e `PairStream`: nomes como cidadãos de primeira classe

`Pair` é a unidade básica de configuração nomeada do *framework*. Ele associa um `Identifier*` (o nome do *slot*) a um `Object*` (o valor), e corresponde diretamente a uma entrada `nome: valor` no arquivo EDL:

```
1 ( Simulation
2   players: {
3     ownship: ( Aircraft  type: "F-16" ) // -> Pair{ "ownship", Aircraft* }
4     bandit:  ( Aircraft  type: "Su-27" ) // -> Pair{ "bandit",  Aircraft* }
5   }
6 )
```

Listing 39: Cada entrada nomeada no EDL vira exatamente um `Pair` na árvore.

O construtor de `Pair` exige ambos os campos e chama `ref()` no objeto recebido. `deleteData()` chama `unref()` em ambos os campos e os anula — idempotente, como exigido de toda `deleteData()` (Seção III.1.3). Em `copyData()`, `Pair` não compartilha ponteiros: tanto o `Identifier` quanto o `Object` são clonados via `clone()`, tornando cada `Pair` proprietário exclusivo de cópias próprias dos seus dois campos.

`PairStream` herda de `List` e especializa-a para `Pair*`. É a estrutura que representa, na árvore de objetos, qualquer bloco delimitado por chaves no EDL — os *players* de uma *Simulation*, os subcomponentes de um *Component*, a lista de comportamentos de um *arbiter*. A especialização acrescenta três buscas que tornam a árvore navegável pelos nomes do arquivo-fonte:

```
1 // Localiza o primeiro Pair cujo slot() == "ownship"
2 Pair* p = players->findByName("ownship");
3 if (p != nullptr) {
4     auto* ac = dynamic_cast<Aircraft*>(p->object());
5 }
6
7 // Localiza o primeiro Pair cujo object() e do tipo Aircraft
8 Pair* p2 = players->findByType(typeid(Aircraft));
```

```

9
10 // Dado um ponteiro de objeto ja na lista, devolve o nome associado
11 // ATENCAO: retorna clone -- o chamador deve chamar unref()
12 const Identifier* nome = players->findName(ac);
13 if (nome != nullptr) {
14     std::cout << *nome << std::endl; // imprime "ownship"
15     nome->unref();                    // OBRIGATORIO
16 }

```

Listing 40: As três buscas de *PairStream*: por nome, por tipo e inversa.

O detalhe de `findName()` merece atenção: ele devolve um *clone* do *Identifier* — não o ponteiro original guardado no *Pair*. O chamador é, portanto, responsável por chamar `unref()` ao terminar. Esse padrão de transferência de posse é o mesmo de `List::removeHead()`: o comentário no cabeçalho o documenta explicitamente.

O idioma de travessia de uma *PairStream* percorrendo todos os *players* aparece repetidamente no código do MIXR:

```

1 // Verificar que todos os elementos sao do tipo esperado
2 List::Item* item = players->getFirstItem();
3 while (item != nullptr) {
4     const auto* pair = static_cast<const Pair*>(item->getValue());
5     const auto* ac = dynamic_cast<const Aircraft*>(pair->object());
6     if (ac == nullptr) {
7         std::cerr << "slot " << *pair->slot() << " nao e Aircraft\n";
8     }
9     item = item->getNext();
10 }

```

Listing 41: Idioma canônico de travessia de *PairStream*: presente em todo o *framework*.

Reconhecer esse idioma — `getFirstItem()`, `getValue()`, `static_cast<Pair*>`, `pair->object()`, `getNext()` — é essencial para ler qualquer trecho de código do *framework* que processe configuração ou listas de *players*.

III.5 Sistema de unidades

A primeira decisão da camada matemática é tratar grandezas físicas como **objetos tipados**, e não como *doubles* nus. A motivação é dupla: elimina uma classe inteira de erros (não se passa um valor em pés onde se esperam metros, porque o tipo carrega a unidade) e serve à composição dirigida por dados, já que cada grandeza é um *Object* instanciável pelo *parser*. Um valor pode ser escrito em EDL na unidade mais conveniente, ficando a conversão a cargo do *framework*.

III.5.1 O padrão de família: classe abstrata + referência + subclasses

O desenho que viabiliza a conversão automática é uma *família* por grandeza: uma classe base abstrata que fixa uma **unidade de referência**, e várias classes concretas — uma por unidade — que implementam dois métodos virtuais puros:

```

1 // include/mixr/base/units/Distances.hpp
2 class Distance : public Number {
3     DECLARE_SUBCLASS(Distance, Number)
4 public:
5     // converte o valor armazenado PARA metros (unidade de referencia)
6     virtual double toDistance() const = 0;

```

```

7
8 // converte um valor EM metros DE VOLTA para esta unidade
9 virtual double fromDistance(const double) const = 0;
10
11 // conveniencia: converte qualquer Distance para esta unidade
12 double convert(const Distance& n) { return fromDistance(n.toDistance()); }
13 };

```

Listing 42: Interface da classe abstrata `Distance`: dois métodos puros definem toda a família.

Cada subclasse concreta implementa o par com suas constantes de conversão próprias. A seguir, três subclasses de `Distance` do repositório:

```

1 class Meters : public Distance {
2 public:
3 // Meters e a referencia: nao ha conversao
4 double toDistance() const override { return val; }
5 double fromDistance(const double a) const override { return a; }
6 static double convertStatic(const Distance& n) { return n.toDistance(); }
7 };
8
9 class Feet : public Distance {
10 public:
11 // 1 pe = 0.3048 metros
12 double toDistance() const override { return val * distance::FT2M; } // 0.3048
13 double fromDistance(const double a) const override { return a * distance::M2FT; }
14 static double convertStatic(const Distance& n) {
15     return n.toDistance() * distance::M2FT;
16 }
17 };
18
19 class NauticalMiles : public Distance {
20 public:
21 // 1 milha nautica = 1852 metros
22 double toDistance() const override { return val * distance::NM2M; } // 1852.0
23 double fromDistance(const double a) const override { return a * distance::M2NM; }
24 static double convertStatic(const Distance& n) {
25     return n.toDistance() * distance::M2NM;
26 }
27 };

```

Listing 43: Três subclasses de `Distance`: a referência e duas concretas.

Toda conversão se reduz a **ir à referência e voltar**: `Feet` chama `toDistance()` para obter metros, e `NauticalMiles` aplica `fromDistance()` sobre esse metro. Não há conversão direta Pés→NM — o caminho sempre passa por metros, de forma transparente. Acrescentar uma unidade nova exige apenas implementar esse par de funções; a conversão para todas as demais vem de graça.

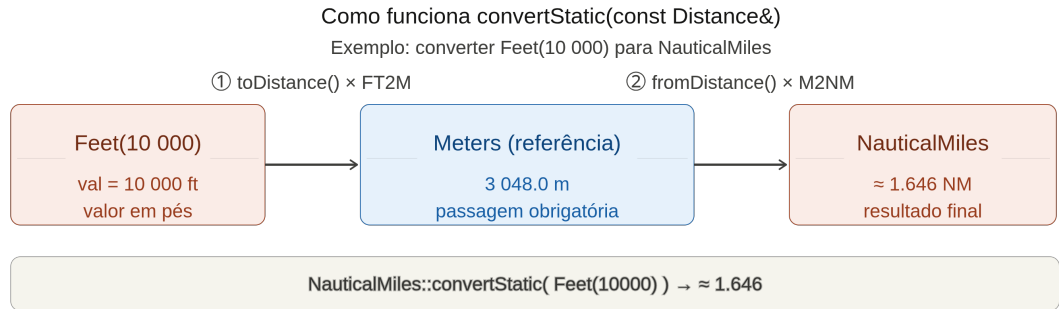
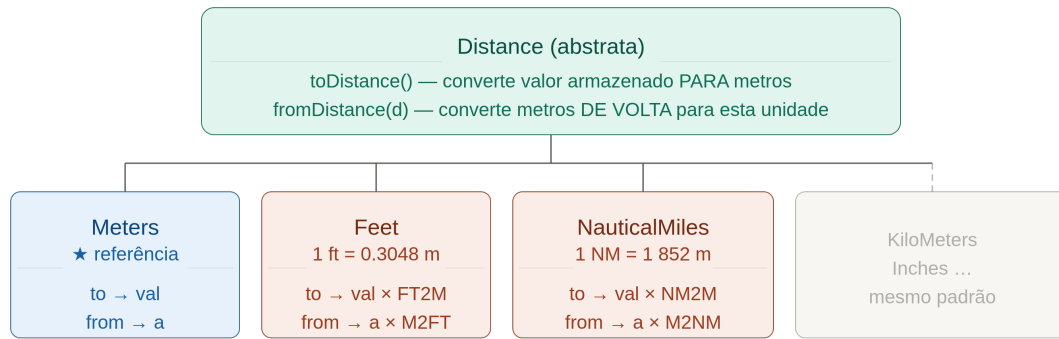
III.5.2 `convertStatic`: conversão sem instanciar o tipo de destino

Cada subclasse concreta oferece um método estático `convertStatic(const Grandeza&)` que recebe *qualquer* instância da grandeza e devolve o valor na unidade da classe chamada, sem que o chamador precise instanciar o tipo de destino. Esse é o padrão mais frequente no código do MIXR. O exemplo abaixo é retirado diretamente de `RacModel.cpp`:

```

1 bool RacModel::setSlotCmdAltitude(const base::Distance* const msg)
2 {
3     if (msg != nullptr) {
4         // aceita Feet, Meters, NauticalMiles, etc. -- converte para metros
5         double value = base::Meters::convertStatic(*msg);
6         cmdAltitude = value; // armazenado internamente em metros
7         return true;
8     }

```



Toda conversão passa pela referência — sem conversão direta entre subclasses.
 Nova unidade: implementar apenas `toDistance()` e `fromDistance()`.

Figure 8: Família `Distance`: a referência `Meters` é o ponto de passagem obrigatório de toda conversão entre unidades.

```

9 |     return false;
10 | }
11 |
12 | bool RacModel::setSlotCmdHeading(const base::Angle* const msg)
13 | {
14 |     if (msg != nullptr) {
15 |         // aceita Degrees, Radians, Semicircles -- converte para graus
16 |         double value = base::Degrees::convertStatic(*msg);
17 |         cmdHeading = value; // armazenado internamente em graus
18 |         return true;
19 |     }
20 |     return false;
21 | }

```

Listing 44: `convertStatic` em uso real: um `slot` aceita qualquer unidade de distância e normaliza para metros internamente (`src/models/dynamics/RacModel.cpp`).

O `slot cmdAltitude`: declara apenas que espera um `base::Distance*` — qualquer subclasse serve. O EDL pode escrever:

```

1 | ( RacModel
2 |   cmdAltitude: ( Feet 10000 ) // convertStatic -> 3048.0 metros
3 |   cmdHeading: ( Degrees 270 ) // convertStatic -> 270.0 graus
4 | )

```

Listing 45: O autor escolhe a unidade mais conveniente; o *framework* normaliza.

O mesmo *slot* aceitaria (`Meters 3048`) ou (`NauticalMiles 1.646`) sem qualquer alteração no código C++.

III.5.3 Grandezas compostas: `LinearVelocity` e `AngularVelocity`

Algumas grandezas são razões entre duas unidades — velocidade é distância por tempo, velocidade angular é ângulo por tempo. O MIXR representa essas grandezas com classes dedicadas que guardam internamente dois valores normalizados (metros e segundos, ou radianos e segundos) e aceitam *slots* separados para cada componente:

```
1 velocidade:
2   ( LinearVelocity
3     distance: ( Feet 90 )      -- setSlotDistance converte para metros
4     time:     ( MilliSeconds 1 ) -- setSlotTime converte para segundos
5   )
6   -- armazenado internamente como 90*0.3048 / 0.001 = 27432 m/s
```

Listing 46: Velocidade linear configurada com unidades distintas para distância e tempo.

Internamente, `setSlotDistance()` e `setSlotTime()` aplicam `convertStatic` de `Meters` e `Seconds`, respectivamente, e armazenam em m/s. O método `getMetersPerSecond()` devolve esse valor normalizado. A conversão para outras unidades de velocidade usa `convert(Distance*, Time*)`, que encapsula a composição:

```
1 // Consulta em m/s (unidade interna)
2 double mps = vel->getMetersPerSecond();
3
4 // Converte para pés por segundo
5 Feet d; Seconds t;
6 double fps = vel->convert(&d, &t);
7
8 // Converte para nós (milhas nauticas por hora)
9 NauticalMiles d2; Hours h;
10 double kts = vel->convert(&d2, &h);
```

Listing 47: `LinearVelocity`: armazena em m/s, converte para qualquer par de unidades.

O mesmo padrão se aplica a `AngularVelocity`: armazena em rad/s e converte para qualquer par `Angle/Time`.

III.5.4 Catálogo de famílias

A Tabela 5 resume as famílias disponíveis. A unidade de referência é sempre o SI; as demais subclasses acomodam as convenções históricas de cada subsistema — aviónica, radar e dinâmica de voo. **Angle** é um caso digno de nota: sua referência interna são **Semicircles** (não radianos), que é o formato nativo de hardware GPS e INS; **Radians** e **Degrees** convertem para e de semicírculos. Isso é invisível ao código que consome a família — basta chamar `Degrees::convertStatic(angulo)` para sempre obter graus, independentemente da unidade original.

Duas ausências merecem nota, porque contrariam a expectativa de quem vem do domínio aeronáutico. **Não existem classes Knots, Mach ou RPM**: velocidade não é uma família de unidades no MIXR, e sim

a grandeza composta `LinearVelocity` da Seção III.5.3. Nós e Mach aparecem no *framework* como *métodos* sobre valores em m/s — `Player::getTotalVelocityKts()`, `Player::getMach()` — e não como tipos instanciáveis em EDL. O mesmo vale para `AngularVelocity` e RPM.

Decibel também é um caso à parte: deriva de `Number` sem pertencer a nenhuma família, e representa uma *escala* logarítmica em vez de uma grandeza. É usada onde um valor em dB precisa ser aceito diretamente — por exemplo, o *slot* `rsc` de `SigConstant` (Seção VI.11.1), que aceita indiferentemente um `Number` em m², uma `Area` em qualquer unidade, ou um `Decibel` em dBsm. As classes `DecibelWatts`, `DecibelMilliWatts` e `DecibelSquareMeters` são independentes dela: pertencem, respectivamente, às famílias `Power` e `Area`, e apenas aplicam a conversão logarítmica nos seus `toPower()`/`toArea()`.

A profusão de famílias reflete a natureza do MIXR como *framework* aeroespacial/militar que precisa interoperar com hardware e modelos legados: radar em dBm e dBsm, motores em lbf e lbs/hora, navegação em pés e nós, dinâmica inteiramente em *slugs* e pés. O sistema de unidades garante que cada `Component` receba, via EDL, qualquer unidade da família e a normalize — via `convertStatic()` — para a unidade interna que de fato utiliza.

III.6 Álgebra: vetores, matrizes e quaternions

Para geometria e cinemática, o MIXR incorpora os tipos algébricos do *OpenSceneGraph* (OSG, licença OSGPL) em vez de reimplementá-los. Mais importante que a origem é a decisão de projeto que governa o uso desses tipos:

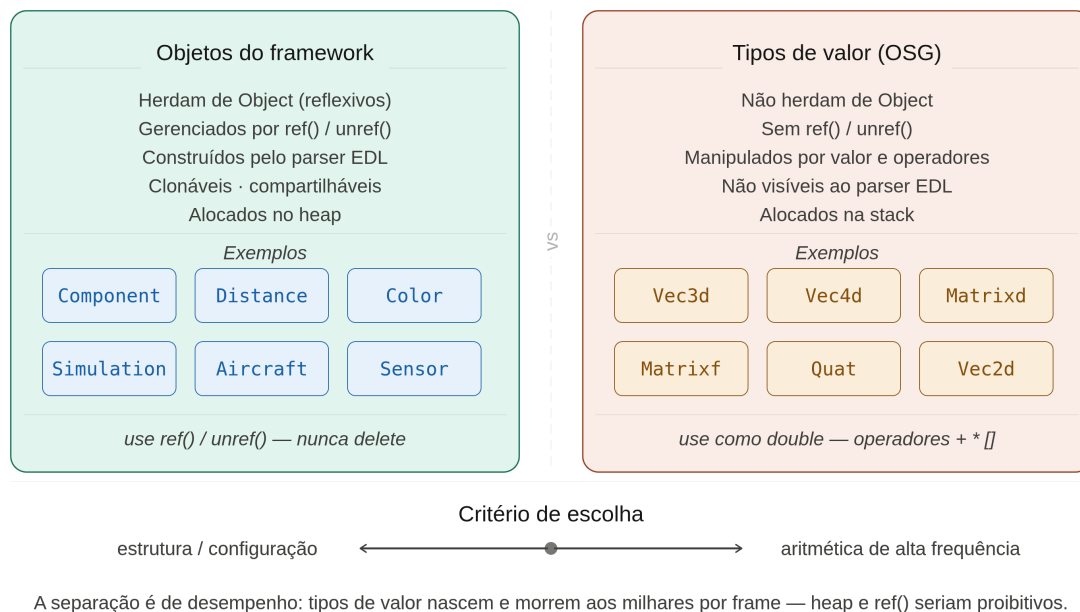


Figure 9: Os dois mundos de tipos do MIXR e o critério de escolha entre eles.

O MIXR convive com dois mundos de tipos completamente distintos. De um lado, os `Objects` reflexivos — contados por referência, clonáveis, visíveis ao *parser*, compartilháveis. De outro, os tipos algébricos — `Vec3d`, `Matrixd`, `Quat` — que são **valores leves**: sem herança de `Object`, sem `ref()`/`unref()`, manipulados por valor e operadores como qualquer `double`. A razão é de desempenho: vetores e matrizes nascem e morrem aos milhares dentro de laços numéricos a cada *frame* e por *player*; impor alocação em *heap* e contagem de referências seria proibitivo.

III.6.1 Vetores

Os vetores existem em dimensões 2, 3 e 4, em precisão simples (f) e dupla (d). Para dinâmica de voo e simulação, **Vec3d** é o tipo mais usado: representa posições e velocidades em referenciais locais (NED) e globais (ECEF).

```
1  #include "mixr/base/osg/Vec3d"
2
3  // construçao por valor -- nada de new/ref
4  base::Vec3d pos(1000.0, 2000.0, -500.0); // x, y, z em metros
5  base::Vec3d vel(100.0, 0.0, 0.0);      // m/s
6
7  // aritmetica por operadores
8  base::Vec3d pos2 = pos + vel * dt;    // integracao Euler simples
9
10 // produto escalar e produto vetorial
11 double dp = pos * vel;                // dot product
12 base::Vec3d cross = pos ^ vel;        // cross product
13
14 // comprimento e normalizacao
15 double len = pos.length();
16 base::Vec3d unit = pos;
17 unit.normalize();                     // modifica in-place
18
19 // acesso por indice ou por nome
20 double x = pos[0]; // equivalente a pos.x()
21 double y = pos.y();
22 double z = pos.z();
```

Listing 48: Operações típicas com **Vec3d**: construção, aritmética e produto vetorial.

III.6.2 Matrices

As matrizes 4×4 (**Matrixf**/**Matrixd**) cuidam de transformações de coordenadas e rotações. A convenção é **vetor-linha à esquerda** — $V_{\text{novo}} = V_{\text{antigo}} \cdot M$ — herdada do OSG. Os métodos **makeRotate()**, **makeTranslate()** e **makeScale()** constroem matrizes compostas, e o operador ***** encadeia transformações:

```
1  #include "mixr/base/osg/Matrixd"
2
3  // rotacao de 45 graus em torno do eixo Z
4  base::Matrixd rot;
5  rot.makeRotate(base::PI/4.0, 0.0, 0.0, 1.0); // angulo, x, y, z
6
7  // aplicar a rotacao a um vetor (vetor-linha)
8  base::Vec3d v(1.0, 0.0, 0.0);
9  base::Vec3d vRot = v * rot; // v transformado
10
11 // encadear duas transformacoes: primeiro rotacao, depois translacao
12 base::Matrixd trans;
13 trans.makeTranslate(100.0, 200.0, 0.0);
14 base::Matrixd combined = rot * trans; // ordem: rot primeiro, trans depois
15
16 // extrair translacao e rotacao de uma matriz composta
17 base::Vec3d t;
18 base::Quat q;
19 base::Vec3d scale;
20 base::Quat so;
21 combined.decompose(t, q, scale, so); // decomposicao TRS
```

Listing 49: Rotação de um vetor e encadeamento de transformações com **Matrixd**.

III.6.3 Quaternions

Os quaternions (Quat) representam orientações e são preferidos às matrizes de rotação por duas razões práticas:

- **Sem gimbal lock** — os ângulos de Euler perdem um grau de liberdade quando dois eixos se alinham; quaternions não têm esse problema.
- **Interpolação suave** — `slerp()` interpola orientações ao longo do caminho geodésico, essencial para suavizar trajetórias e animações.

Um quaternion de identidade tem $w = 1$, $x = y = z = 0$. A conversão de e para matrizes é bidirecional:

```
1  #include "mixr/base/osg/Quat"
2
3  // rotacao de 30 graus em torno do eixo Y
4  base::Quat q;
5  q.makeRotate(base::D2RCC * 30.0, 0.0, 1.0, 0.0); // angulo (rad), eixo
6
7  // compor duas rotacoes (multiplicacao de quaternions)
8  base::Quat q2;
9  q2.makeRotate(base::D2RCC * 45.0, 0.0, 0.0, 1.0);
10 base::Quat qTotal = q * q2; // primeiro q, depois q2
11
12 // converter para matriz (para transformar vetores)
13 base::Matrixd m;
14 qTotal.get(m);           // Quat -> Matrixd
15 base::Quat q3(m);        // Matrixd -> Quat
16
17 // interpolacao esferic (slerp): t=0 retorna qa, t=1 retorna qb
18 base::Quat qa, qb, qInterp;
19 qInterp.slerp(0.5, qa, qb); // orientacao a meio caminho entre qa e qb
20
21 // aplicar quaternion a um vetor
22 base::Vec3d v(1.0, 0.0, 0.0);
23 base::Vec3d vRot = qTotal * v;
```

Listing 50: Quaternion: rotação, conversão com matriz e interpolação `slerp()`.

III.6.4 Padrão de uso no código do MIXR

No código do framework, os tipos algébricos aparecem tipicamente como variáveis locais em métodos de dinâmica, nunca como membros gerenciados por `ref()/unref()`. O fragmento abaixo, extraído de `navDR_utils.hpp`, ilustra o padrão canônico: `Vec3d` e `Matrixd` passados por referência constante — sem heap, sem contagem:

```
1  // include/mixr/base/util/navDR_utils.hpp -- dead reckoning
2  bool deadReckoning(
3      const double dT, // incremento de tempo (s)
4      const unsigned int drNum, // codigo de modelo DR [0..9]
5      const Vec3d& p0, // posicao inicial (metros, ECEF)
6      const Vec3d& v0, // velocidade inicial (m/s)
7      const Vec3d& a0, // aceleracao inicial (m/s^2)
8      const Vec3d& rpy0, // angulos de Euler iniciais (rad)
9      const Vec3d& av0, // velocidade angular inicial (rad/s)
10     Vec3d* const pNewPw, // OUT: nova posicao no mundo (metros)
11     Vec3d* const pNewRPY // OUT: novo roll/pitch/yaw de corpo (rad)
12 );
```

Listing 51: Padrão de uso real em `navDR_utils.hpp`: tipos algébricos por valor/referência.

Nenhum `new`, nenhum `ref()`, nenhuma herança de `Object` — apenas aritmética de ponto flutuante sobre valores na *stack*. Esse é o contrato que separa os dois mundos de tipos do MIXR.

III.6.5 Transforms: matrizes declaradas em EDL

Os tipos algébricos acima vivem no lado C++ da fronteira e, por construção, não são visíveis ao *parser*. Isso deixaria uma lacuna: como declarar uma rotação num arquivo de configuração? A resposta é a família `Transform`, que é a ponte entre os dois mundos.

`Translation`, `Rotation` e `Scale` são `Objects` com *slots* — e portanto declaráveis em EDL — cujo produto é uma `Matrixd`:

```
1 // Rotacao: os slots x/y/z aceitam Number (radianos) ou Angle (qualquer unidade)
2 ( Rotation z: ( Degrees 45 ) )
3
4 // Translacao em metros
5 ( Translation x: 100.0 y: 200.0 )
6
7 // Escala uniforme ou por eixo
8 ( Scale x: 2.0 y: 2.0 z: 1.0 )
```

Listing 52: Transformações declaradas em EDL. Cada uma vira uma `osg::Matrixd`, e a composição é a ordem da lista.

O detalhe que torna a família agradável de usar é que os *slots* aceitam tanto um `Number` cru quanto uma `Angle` (Seção III.5): `z: 0.785` e `z: (Degrees 45)` produzem a mesma matriz.

III.6.6 Matrix e Vectors: a outra álgebra

Armadilha — existem duas classes chamadas “matriz” no MIXR

Tudo o que esta seção descreveu até aqui refere-se aos tipos do OSG — `Matrixd`, `Matrixf` — que são **fixos** em 4×4 e existem para transformações geométricas.

Há uma segunda, sem relação alguma com a primeira: `base::Matrix` (`include/mixr/base/Matrix.hpp`), uma matriz numérica **geral** $N \times M$, com decomposição LU, inversão e autovalores. É um `Object`, tem nome de fábrica “`Matrix`”, e acompanha-se de `RVector` e `CVector` (vetores linha e coluna).

As duas são facilmente confundidas ao ler o código, porque a palavra *matrix* aparece nas duas. O critério é simples: se é geometria, é OSG e tem tamanho fixo; se é álgebra linear numérica, é `base::Matrix`.

III.7 Geodésia: `EarthModel` e `nav_utils`

A geodésia ilustra uma decisão de estilo recorrente no MIXR: como as conversões entre referenciais são *transformações matemáticas sem estado*, elas são oferecidas como **funções livres** no namespace `base::nav`, e não como uma hierarquia de classes. O único estado relevante — a forma da Terra — é isolado em `EarthModel` e injetado como parâmetro opcional, com o WGS-84 como padrão. Quem não precisa de outro modelo simplesmente omite o argumento.

III.7.1 EarthModel: parâmetros do elipsoide de referência

EarthModel encapsula os quatro parâmetros que definem um elipsoide de referência:

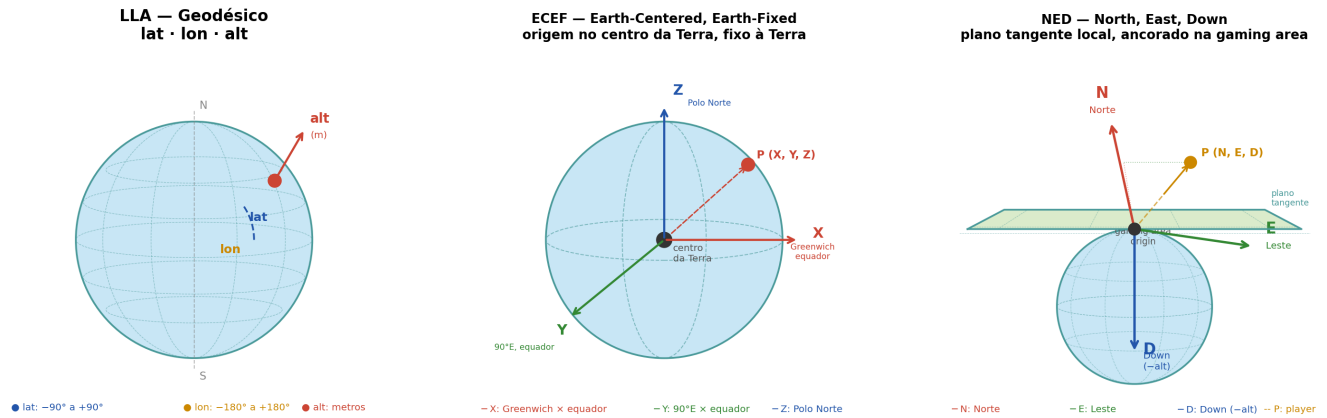
```
1 // include/mixr/base/EarthModel.hpp
2 class EarthModel : public Object {
3     DECLARE_SUBCLASS(EarthModel, Object)
4 public:
5     double getA() const; // semieixo maior a (metros) -- WGS84: 6 378 137.0
6     double getB() const; // semieixo menor b (metros) -- WGS84: 6 356 752.3142
7     double getF() const; // achatamento f = (a-b)/a -- WGS84: 1/298.257
8     double getE2() const; // excentricidade^2 e^2 = 1-b^2/a^2
9
10    static const EarthModel wgs84; // instancia estatica -- passada como padrao
11 };
```

Listing 53: EarthModel: parâmetros e instância estática WGS-84.

O WGS-84 é o modelo de referência usado pelo GPS e é o padrão implícito em todas as funções de `nav_utils`. Passar `nullptr` equivale a usar `EarthModel::wgs84`. Para simulações que exigem outro datum (Clarke 1866, GRS 80, etc.), basta passar a instância correspondente.

III.7.2 Os três referenciais e as conversões entre eles

Toda simulação geoespacial trabalha com três referenciais fundamentais, cada um com seu papel bem definido.



(a) **LLA** — geodésico. Latitude e longitude em graus, altitude em metros acima do elipsoide. Representação natural de posições geográficas; usada em arquivos EDL e interfaces de usuário.

(b) **ECEF** — cartesiano, centrado e fixo à Terra. Eixo *X* aponta para Greenwich no equador, *Z* para o polo norte. Referencial universal: independe de ponto local e é o que o GPS produz internamente.

(c) **NED** — plano tangente local. Norte, Leste e Down (positivo para baixo), ancorado na *gaming area origin*. Referencial de trabalho de sensores, radar e dinâmica de voo.

Figure 10: Os três referenciais geoespaciais do MIXR. As conversões entre eles são providas por `nav_utils` e descritas nas subseções seguintes.

III.7.3 LLA ↔ ECEF

A conversão entre coordenadas geodésicas e ECEF usa as fórmulas clássicas do elipsoide WGS-84. Cada função tem três sobrecargas: escalares, `Vec3d` e `array C`:

```

1  #include "mixr/base/util/nav_utils.hpp"
2
3  // LLA -> ECEF (Vec3d)
4  base::Vec3d lla{36.12, -86.67, 3000.0}; // lat(deg), lon(deg), alt(m)
5  base::Vec3d ecef;
6  base::nav::convertGeod2Ecef(lla, &ecef); // WGS-84 implicit
7
8  // resultado: ecef ~ (299762, -5151871, 3740724) metros
9
10 // ECEF -> LLA (escalares)
11 double lat{}, lon{}, alt{};
12 base::nav::convertEcef2Geod(
13     ecef[0], ecef[1], ecef[2], // X, Y, Z em metros
14     &lat, &lon, &alt // lat/lon em graus, alt em metros
15 );
16 // lat ~ 36.12 lon ~ -86.67 alt ~ 3000.0
17
18 // Com modelo de Terra alternativo
19 base::nav::convertGeod2Ecef(lla, &ecef, &myEarthModel);

```

Listing 54: Conversão LLA ↔ ECEF: três sobrecargas, mesmo `EarthModel` opcional.

O algoritmo iterativo de `convertEcef2Geod()` é o de Bowring, com tratamento especial para pontos polares. `convertGeod2Ecef()` é não-iterativo, usando diretamente o raio de curvatura da normal $N(\phi)$: $N = a/\sqrt{1 - e^2 \sin^2 \phi}$.

III.7.4 LLA ↔ NED

A projeção de LLA para NED (e o inverso) ancora-se em um ponto de referência (*gaming area origin*) e tem duas variantes que expõem o compromisso entre precisão e custo:

```

1  // Ponto de referencia da gaming area
2  const double refLat{36.12};
3  const double refLon{-86.67};
4  const double cosRlat{std::cos(base::angle::D2RCC * refLat)};
5  const double sinRlat{std::sin(base::angle::D2RCC * refLat)};
6
7  // Posicao do alvo em LLA
8  const double tgtLat{36.15};
9  const double tgtLon{-86.60};
10 const double tgtAlt{1000.0}; // metros
11
12 base::Vec3d posNED;
13
14 // VARIANTE E -- elipsoide WGS-84: mais precisa, recomendada para areas grandes
15 base::nav::convertLL2PosVecE(
16     refLat, refLon, sinRlat, cosRlat, // referencia (com sin/cos pre-calculados)
17     tgtLat, tgtLon, tgtAlt, // destino
18     &posNED // OUT: NED em metros (N, E, D=-alt)
19 );
20 // posNED[0] = Norte (m), posNED[1] = Leste (m), posNED[2] = -alt (Down)
21
22 // VARIANTE S -- Terra esferica: mais barata, suficiente para areas ate ~100 km
23 base::nav::convertLL2PosVecS(
24     refLat, refLon, cosRlat,
25     tgtLat, tgtLon, tgtAlt,
26     &posNED
27 );

```

Listing 55: NED ↔ LLA: variante elíptica (E) versus esférica (S).

A variante esférica (S) usa a aproximação de plano tangente puro ($\Delta N \approx \Delta \phi \times 60 \times \text{NM2M}$, $\Delta E \approx \Delta \lambda \times 60 \times \text{NM2M} \times \cos \phi$), que é linear e muito barata. A variante elíptica (E) corrige para a curvatura do

elipsoide e é necessária para áreas de jogo grandes (centenas de quilômetros).

O código de `Player.cpp` usa exatamente esse par de variantes, escolhendo com base num flag da `Simulation`:

```

1 // src/models/player/Player.cpp
2 if (s->isGamingAreaUsingEarthModel()) {
3     base::nav::convertLL2PosVecE(refLat, refLon, sinRlat, cosRlat,
4                                 latitude, longitude, altitude,
5                                 &posVecNED, em);
6 } else {
7     base::nav::convertLL2PosVecS(refLat, refLon, cosRlat,
8                                 latitude, longitude, altitude,
9                                 &posVecNED);
10 }

```

Listing 56: Escolha entre variante elíptica e esférica em `Player.cpp` (código real).

III.7.5 ECEF ↔ NED: a matriz mundo

A conversão direta entre ECEF e NED passa pela **matriz mundo** (*world matrix*), que expressa a orientação do plano NED local em relação ao referencial ECEF. Ela é computada uma vez por *player* e reutilizada:

```

1 // Computa M = Ry[-(90+lat)] * Rz[lon]
2 // tal que: V_NED = M * V_ECEF e V_ECEF = V_NED * M
3 base::Matrixd wm;
4 base::nav::computeWorldMatrix(latitude, longitude, &wm);
5
6 // converter vetor NED para ECEF -- vetor-linha a esquerda
7 base::Vec3d velNED{100.0, 0.0, 0.0}; // 100 m/s para o Norte
8 base::Vec3d velECEF = velNED * wm; // V_ECEF = V_NED * M
9
10 // converter vetor ECEF para NED -- a forma equivalente com a transposta
11 base::Matrixd wmT;
12 wmT.transpose(wm);
13 base::Vec3d velNED2 = velECEF * wmT; // = M * V_ECEF

```

Listing 57: `computeWorldMatrix`: da latitude/longitude à matriz de rotação ECEF ↔ NED.

Regra — a *world matrix* leva de ECEF para NED, não o contrário

O cabeçalho de `nav_utils.hpp` enuncia as duas formas de uso lado a lado, e vale copiá-las literalmente porque a troca é fácil de fazer: $\mathbf{V}_i = \mathbf{M} * \mathbf{V}_w$ e $\mathbf{V}_w = \mathbf{V}_i * \mathbf{M}$, onde $\mathbf{V}_i$ é o vetor *inertial* (NED) e $\mathbf{V}_w$ o vetor *world* (ECEF). Ou seja, `wm` mapeia **ECEF** → **NED**; é a multiplicação por vetor-linha ($\mathbf{V} * \mathbf{M}$) que faz o caminho de volta.

Na prática, `Player` mantém `wm` e a matriz combinada `rmW2B = rm * wm` — *world to body*, isto é, ECEF→corpo — que é recomputada sempre que a posição ou a atitude mudam.

III.7.6 Círculo máximo: `gll2bd` e `gbd2ll`

As funções de círculo máximo calculam distâncias e azimutes sobre a superfície da Terra entre dois pontos dados em LLA. Também existem em duas variantes:

```

1 // De ponto A para ponto B: qual o azimuth e a distancia?
2 double brg{}; // azimuth verdadeiro (graus)
3 double dist{}; // distancia de superficie (milhas nauticas)
4
5 // Variante eliptica (precisa)
6 base::nav::gll2bd(

```



```

7      36.12, -86.67,    // ponto A: lat, lon (graus)
8      48.85,  2.35,    // ponto B: lat, lon (graus)
9      &brg, &dist      // OUT: azimuth (deg), distancia (NM)
10 );
11 // brg ~ 47.6 deg  dist ~ 3779 NM (Nashville -> Paris)
12
13 // Variante esferica (mais rapida, sufixo S)
14 base::nav::gll2bdS(36.12, -86.67, 48.85, 2.35, &brg, &dist);
15
16 // Dado ponto de origem, azimuth e distancia: qual o ponto de destino?
17 double dlat{}, dlon{};
18 base::nav::gbd2ll(
19     36.12, -86.67,    // origem
20     47.2,            // azimuth (graus)
21     500.0,           // distancia (NM)
22     &dlat, &dlon
23 );
24
25 // Com altitude: inclui alcance inclinado e angulo de elevacao
26 double slantRng{}, elev{};
27 base::nav::glla2bd(
28     36.12, -86.67, 0.0,    // origem: lat, lon, alt (m)
29     36.50, -86.00, 5000.0, // destino: lat, lon, alt (m)
30     &brg, &slantRng, &dist, &elev
31 );

```

Listing 58: Círculo máximo: ponto-a-ponto e ponto-dado-rumo-distância.

III.7.7 Matrizes de orientação e utilitários angulares

`computeRotationalMatrix()` constrói a matriz de cossenos diretores a partir dos ângulos de Euler (rolagem ϕ , arfagem θ , guinada ψ), que é a representação de atitude usada internamente por cada *player*:

```

1 // angulos em radianos
2 const double phi {base::angle::D2RCC * 0.0}; // roll (graus -> rad)
3 const double theta{base::angle::D2RCC * 10.0}; // pitch
4 const double psi {base::angle::D2RCC * 270.0}; // yaw
5
6 base::Matrixd rm;
7 base::nav::computeRotationalMatrix(phi, theta, psi, &rm);
8 // rm converte vetores NED para o referencial de corpo (body)
9
10 // versao em graus diretamente
11 base::Vec2d scPhi, scTht, scPsi; // sin/cos opcionais (cache para uso posterior)
12 base::nav::computeRotationalMatrixDeg(0.0, 10.0, 270.0, &rm,
13                                     &scPhi, &scTht, &scPsi);

```

Listing 59: `computeRotationalMatrix`: ângulos de Euler $\rightarrow$ matriz NED $\rightarrow$ corpo.

Os utilitários angulares completam o conjunto:

```

1 // conversao graus <-> radianos
2 const double D2RCC{base::angle::D2RCC}; // pi/180
3 const double R2DCC{base::angle::R2DCC}; // 180/pi
4
5 double rad = 45.0 * D2RCC; // 45 graus em radianos
6 double deg = rad * R2DCC; // de volta para graus
7
8 // normalizacao de angulo para [-180, +180] graus
9 double hdg = base::angle::aepcdDeg(370.0); // -> 10.0 graus
10 double hdg2 = base::angle::aepcdDeg(-200.0); // -> 160.0 graus
11
12 // em radianos: [-pi, +pi]
13 double ang = base::angle::aepcdRad(4.0); // -> 4.0 - 2*pi ~ -2.28 rad

```

Listing 60: Utilitários angulares de `nav_utils`: conversão e normalização.

`aepcdDeg()` (e a variante em radianos) é usada em toda operação que manipula diferenças de longitude ou azimutes, evitando descontinuidades na transição $\pm 180^\circ$.

III.7.8 Fluxo completo: posição de um *player* em todos os referenciais

Para consolidar, o trecho abaixo reproduz (simplificado) o que `Player` faz toda vez que sua posição geodésica muda — o caminho completo `LLA` $\rightarrow$ `ECEF` $\rightarrow$ `NED` $\rightarrow$ corpo:

```
1 // 1. Armazena a posicao geodesica
2 latitude = newLat;
3 longitude = newLon;
4 altitude = newAlt;
5
6 // 2. LLA -> ECEF (para interoperabilidade e dead reckoning)
7 double lla[3]{ latitude, longitude, altitude };
8 double ecef[3]{};
9 base::nav::convertGeod2Ecef(lla, ecef, em);
10 posVecECEF.set(ecef[0], ecef[1], ecef[2]);
11
12 // 3. Computa a matriz mundo (orientacao do plano NED no ECEF)
13 base::nav::computeWorldMatrix(latitude, longitude, &wm);
14
15 // 4. LLA -> NED (relativo ao ponto de referencia da gaming area)
16 if (s->isGamingAreaUsingEarthModel())
17     base::nav::convertLL2PosVecE(refLat, refLon, sinRlat, cosRlat,
18                                 latitude, longitude, altitude, &posVecNED, em);
19 else
20     base::nav::convertLL2PosVecS(refLat, refLon, cosRlat,
21                                 latitude, longitude, altitude, &posVecNED);
22
23 // 5. Computa a matriz combinada NED->corpo (para transformar vetores de sensores)
24 rmW2B = rm * wm; // rm: NED->corpo (de Euler) wm: NED->ECEF
```

Listing 61: Fluxo completo de atualização de posição em `Player.cpp` (simplificado).

Esse fluxo executa a cada chamada de `setGeodPosition()` — que por sua vez é acionada pelo modelo de dinâmica a cada *frame*. A separação em funções livres torna cada etapa testável e substituível de forma independente.

III.8 Threading

A Seção II.5 estabeleceu *por que* o paralelismo do MIXR preserva o determinismo. Esta seção desce ao código: como `base` implementa as duas abstrações de *thread* que o materializam, quais decisões de projeto estão por trás de cada escolha e como o desenvolvedor as usa.

O ponto de partida é reconhecer que há **dois regimes de trabalho** fundamentalmente diferentes em uma simulação em tempo real:

- **Cadência própria** — tarefas que precisam executar a uma taxa fixa e autônoma, independentemente do ciclo da simulação. O tratador de entrada/saída e o processamento de rede são exemplos típicos: eles têm seu próprio compasso e não deveriam esperar nem ser esperados pelas fases de simulação.
- **Sincronização com fases** — tarefas que precisam marchar em sincronia com o ciclo *ciclo/frame/fase* descrito na Seção II.4. O processamento paralelo de *players* é o exemplo canônico: cada *thread* do *pool* deve processar sua fatia *exatamente dentro de uma fase*, nem antes nem depois.

Para cada regime, **base** oferece uma abstração: **PeriodicThread** para o primeiro e **SyncThread** para o segundo. A Figura 11 mostra os dois laços lado a lado.

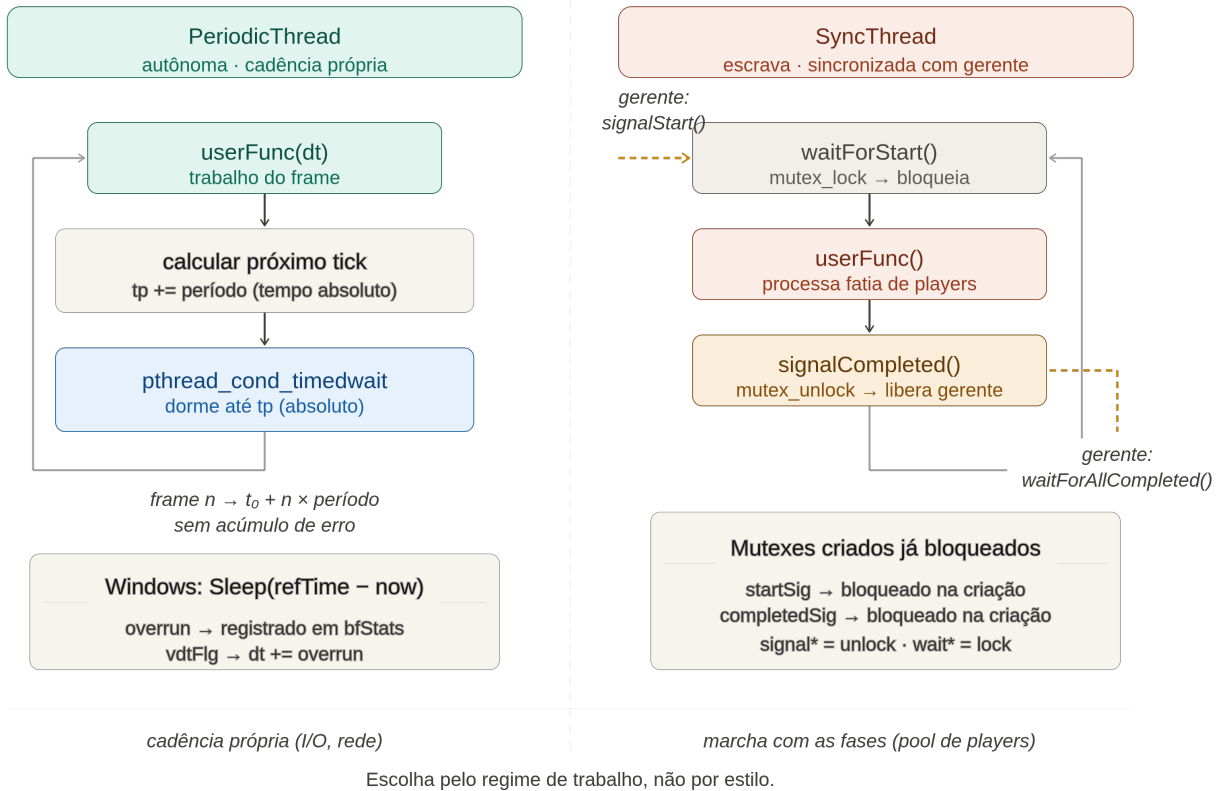


Figure 11: **PeriodicThread** (autônoma, à esquerda) e **SyncThread** (sincronizada, à direita). Os mutexes de **SyncThread** são criados já bloqueados; **signal*** é **unlock** e **wait*** é **lock**.

III.8.1 AbstractThread: a base comum

Ambas as especializações herdam de **AbstractThread**, que não herda de **Object** e sim diretamente de **Referenced** (Seção II.3): uma *thread* é infraestrutura, não um componente configurável via EDL.

```

1 // include/mirr/base/threads/AbstractThread.hpp
2 class AbstractThread : public Referenced // nao herda de Object!
3 {
4 public:
5     AbstractThread(Component* const parent); // parent: ponteiro FRACO (nao ref())
6
7     bool start(const double priority); // [0.0, 1.0] -> agendador do SO
8     virtual bool terminate();
9     bool isTerminated() const;
10    bool setStackSize(const std::size_t); // deve ser chamado ANTES de start()
11    static int getNumProcessors();
12
13 private:
14     virtual unsigned long mainThreadFunc() = 0; // laço real (subclasse)
15     static void* staticThreadFunc(void* lpParam); // ponto de entrada do SO
16
17     Component* parent {}; // ponteiro FRACO -- nao ref()'ado no construtor
18     double priority {}; // prioridade portavel [0..1]
19     bool killed {}; // flag de encerramento
20     void* theThread {}; // pthread_t* (Linux) ou HANDLE (Windows)
    
```

```
21 |};
```

Listing 62: Interface de `AbstractThread`: o essencial que toda *thread* do MIXR expõe.

O ponteiro fraco e o truque do `ref()` estático. A decisão mais sutil de `AbstractThread` está no `parent`: ele é guardado como um ponteiro **fraco** — não `ref()`-eado no construtor. Por quê? Criar uma *thread* não deveria, por si só, manter o componente dono vivo artificialmente. A *thread* é um servo do componente; se o componente decidir morrer, a *thread* deve poder morrer com ele.

O `ref()` acontece apenas no instante em que a *thread* efetivamente começa a executar, dentro da função estática de entrada:

```
1  // src/base/threads/platform/AbstractThread_linux.cpp
2  void* AbstractThread::staticThreadFunc(void* lpParam)
3  {
4      const auto thread = static_cast<AbstractThread*>(lpParam);
5      Component* parent { thread->getParent() };
6
7      // Garante que nenhum dos dois seja destruído enquanto a thread roda
8      thread->ref();
9      parent->ref();
10
11     // Executa o laço real (PeriodicThread ou SyncThread)
12     unsigned long rtn { thread->mainThreadFunc() };
13
14     thread->setTerminated();
15
16     // Libera a posse -- se o contador chegar a zero, o objeto se destroi aqui
17     parent->unref();
18     thread->unref();
19
20     return reinterpret_cast<void*>(rtn);
21 }
```

Listing 63: `staticThreadFunc` real de `AbstractThread_linux.cpp`: o padrão `ref`/`trabalho`/`unref` que protege ambos os objetos durante a execução.

Por que uma função *estática*? Toda API de *threading* de sistema operacional (`pthread_create`, `CreateThread`) exige um ponto de entrada estático — não pode ser um método virtual, pois a *vtable* do objeto pode não estar estabelecida no instante da criação em todas as plataformas. O MIXR contorna isso passando `this` como `void*` e recuperando dentro da função estática. O método virtual `mainThreadFunc()` — onde vive o laço real — só é chamado *depois* dos dois `ref()`, quando ambos os objetos já estão protegidos.

Prioridade portátil `[0, 1]`. `start(priority)` traduz o valor para o agendador do sistema operacional:

No Linux, qualquer prioridade > 0 usa `SCHED_FIFO` (tempo real). Isso significa que a *thread* não será preemptada por *threads* de prioridade menor — essencial para simulações com restrições de tempo real.

III.8.2 PeriodicThread: agendamento por tempo absoluto

`PeriodicThread` implementa o laço autônomo: chama `userFunc(dt)` a uma taxa fixa em Hz indefinidamente até que o componente pai seja encerrado.

```
1  // include/mixr/base/threads/PeriodicThread.hpp
2  class PeriodicThread : public AbstractThread
3  {
```

```

4 public:
5     PeriodicThread(Component* const parent, const double rate);
6
7     double getRate() const;           // taxa em Hz
8     int    getTotalFrameCount() const; // total de frames executados
9
10    // Estatísticas de overrun (apenas Windows)
11    const Statistic& getBustedFrameStats() const;
12
13    // Se true, dt é ajustado para compensar overruns (apenas Windows)
14    bool setVariableDeltaTimeFlag(const bool enable);
15
16 private:
17     unsigned long mainThreadFunc() final; // implementado pela plataforma
18
19    // O DESENVOLVEDOR IMPLEMENTA APENAS ISSO:
20    virtual unsigned long userFunc(const double dt) = 0;
21
22    double rate {0}; // taxa em Hz
23    int    tcnt {0}; // contador de frames
24    bool   vdtFlg {0}; // variable delta time flag
25    Statistic bfStats{}; // estatísticas de overrun
26 };

```

Listing 64: Interface de `PeriodicThread`: o que o desenvolvedor implementa e o que o *framework* cuida.

O desenvolvedor cria uma subclasse e implementa apenas `userFunc(dt)`. Dentro dela vivem tipicamente `Timer::updateTimers(dt)` e `root->updateTC(dt)` — este é, na prática, o motor de tempo real da Seção II.4. Um exemplo real é `StationBgPeriodicThread`:

```

1 unsigned long StationBgPeriodicThread::userFunc(const double dt)
2 {
3     Station* station { static_cast<Station*>(getParent()) };
4     station->processBackgroundTasks(dt);
5     return 0;
6 }

```

Listing 65: `StationBgPeriodicThread`: a subclasse mais simples possível — delegar e sair (`src/simulation/StationBgPeriodicThread.cpp`).

Como evitar *drift*: tempo absoluto vs. relativo. A diferença mais importante entre as implementações Linux e Windows de `PeriodicThread` está em *como* o laço dorme entre *frames*. O *drift* acontece quando cada *sleep* relativo acumula o erro do *frame* anterior:

```

1 // src/base/threads/platform/PeriodicThread_linux.cpp
2 unsigned long PeriodicThread::mainThreadFunc()
3 {
4     const double dt { 1.0 / getRate() };
5     int sec0 { static_cast<int>(dt) };
6     int nsec0 { static_cast<int>((dt - sec0) * 1e9) };
7
8     // tp: instante absoluto de referencia (CLOCK_REALTIME)
9     struct timespec tp;
10    clock_gettime(CLOCK_REALTIME, &tp);
11
12    pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
13    pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
14    pthread_mutex_lock(&mutex);
15
16    while (!getParent()->isShutdown()) {
17
18        this->userFunc(dt); // trabalho do frame
19        tcnt++;
20
21        // Avança tp por EXATAMENTE um periodo (absoluto)

```

```

22     // Frame n sempre agendado para t0 + n*periodo
23     tp.tv_nsec += nsec0;
24     if (tp.tv_nsec >= 1000000000) {
25         tp.tv_nsec++;
26         tp.tv_nsec -= 1000000000;
27     }
28
29     // Dorme ate o instante absoluto -- nao ate "daqui a dt segundos"
30     pthread_cond_timedwait(&cond, &mutex, &tp);
31 }
32
33 pthread_mutex_unlock(&mutex);
34 return 0;
35 }

```

Listing 66: Laço real de `PeriodicThread_linux.cpp`: `tp` é um instante *absoluto* que avança por períodos fixos — sem acumulação de erro.

A consequência é que o *frame* n está sempre agendado para $t_0 + n \times \text{período}$, independentemente de quanto o *frame* anterior demorou. No Windows, a estratégia é diferente: calcula-se o tempo de espera como `refTime - now` e chama-se `Sleep()`. Se o resultado for negativo, o *frame* atrasou (*overrun*), o excesso é registrado em `bfStats` e, se `vdTFlg` estiver ativo, o próximo `dt` é aumentado para compensar.

III.8.3 SyncThread: sinalização por *mutex* pré-bloqueado

`SyncThread` implementa o laço escravo: dorme até receber um sinal de início, executa `userFunc()` uma vez e sinaliza conclusão.

```

1  class SyncThread : public AbstractThread
2  {
3  public:
4      void signalStart(); // gerente -> thread
5      void waitForCompleted(); // gerente espera
6      static void waitForAllCompleted(SyncThread**, int num); // espera N threads
7      static int waitForAnyCompleted(SyncThread**, int num); // espera qualquer
8
9      bool terminate() override; // chama signalCompleted() ANTES de terminar!
10
11 protected:
12     void waitForStart(); // thread espera o gerente
13     void signalCompleted(); // thread avisa que terminou
14
15 private:
16     virtual unsigned long userFunc() = 0; // SEM dt -- sincrono por natureza
17
18     void* startSig {}; // mutex/semaforo de inicio (criado JA BLOQUEADO)
19     void* completedSig {}; // mutex/semaforo de conclusao (criado JA BLOQUEADO)
20 };

```

Listing 67: Interface de `SyncThread`: os sinais e o ponto de extensão (resumo de `include/mixr/base/threads/SyncThread.hpp`).

O laço que anima essa interface cabe em uma tela:

```

1  unsigned long SyncThread::mainThreadFunc()
2  {
3      unsigned long rtn{};
4
5      // Configure this thread
6      bool ok{configThread()};
7
8      // Main start-complete loop ...
9      while ( ok && getParent()->isNotShutdown() ) {

```

```

10
11     // Wait for the start signal
12     waitForStart();
13
14     // Just in case we've been shutdown while we were waiting
15     if (getParent()->isShutdown()) {
16         signalCompleted();
17         break;
18     }
19
20     // User defined tasks
21     this->userFunc();
22
23     // Signal that we've completed
24     signalCompleted();
25 }
26
27 return rtn;
28 }

```

Listing 68: `SyncThread::mainThreadFunc()` (`src/base/threads/SyncThread.cpp`).

O `signalCompleted()` dentro do `if` de encerramento não é redundância: sem ele, um gerente parado em `waitForCompleted()` no instante do desligamento esperaria para sempre por uma *thread* que já decidiu sair. É a mesma preocupação que faz `terminate()` sinalizar antes de encerrar, discutida adiante.

O mecanismo de sinalização no Linux: *mutex* pré-bloqueado. A implementação Linux é elegante e incomum: usa `pthread_mutex_t` não como exclusão mútua, mas como mecanismo de sinalização. Os dois *mutexes* são criados *já bloqueados*:

```

1 // src/base/threads/platform/SyncThread_linux.cpp
2
3 bool SyncThread::createSignals()
4 {
5     // startSig: criado JA BLOQUEADO -- waitForStart() vai bloquear ate
6     // signalStart() chamar unlock
7     pthread_mutex_t* m1 { new pthread_mutex_t };
8     pthread_mutex_init(m1, nullptr);
9     pthread_mutex_lock(m1);    // <-- bloqueia imediatamente
10    startSig = m1;
11
12    // completedSig: idem para o sinal de conclusao
13    pthread_mutex_t* m2 { new pthread_mutex_t };
14    pthread_mutex_init(m2, nullptr);
15    pthread_mutex_lock(m2);    // <-- bloqueia imediatamente
16    completedSig = m2;
17    return true;
18 }
19
20 // signalStart(): mutex_unlock --> libera a thread que dorme em lock
21 void SyncThread::signalStart() { pthread_mutex_unlock(static_cast<pthread_mutex_t*>(startSig)); }
22 void SyncThread::waitForStart() { pthread_mutex_lock (static_cast<pthread_mutex_t*>(startSig)); }
23
24 void SyncThread::signalCompleted(){ pthread_mutex_unlock(static_cast<pthread_mutex_t*>(completedSig)); }
25 void SyncThread::waitForCompleted(){ pthread_mutex_lock (static_cast<pthread_mutex_t*>(completedSig)); }

```

Listing 69: Sinalização por *mutex* pré-bloqueado em `SyncThread_linux.cpp`: `lock` bloqueia, `unlock` libera.

O raciocínio é: `lock` em um *mutex* já bloqueado bloqueia o chamador até outro chamar `unlock`. Esse uso inverte a semântica habitual de exclusão mútua — o *mutex* vira um canal de sinalização de custo mínimo. No Windows, o mesmo protocolo usa semáforos (`CreateSemaphore/ReleaseSemaphore/WaitForSingleObject`).

Padrão de uso: *pool* com barreira por fase. O código abaixo mostra como um gerente dispara N `SyncThreads` e espera todas concluírem — o padrão exato usado por `Simulation` a cada fase:

```
1 // Criar e iniciar os workers (tipicamente em reset())
2 SyncThread* workers[3];
3 for (int i = 0; i < 3; i++) {
4     workers[i] = new MyWorker(parent, i); // MyWorker subclasse de SyncThread
5     workers[i]—>ref();
6     workers[i]—>start(0.9); // prioridade alta -- SCHED_FIFO no Linux
7     // o worker entra imediatamente em waitForStart() e dorme
8 }
9
10 // A cada fase (tipicamente em updateTC()):
11 for (int i = 0; i < 3; i++) {
12     workers[i]—>signalStart(); // acorda o worker i
13 }
14
15 // Thread principal faz sua propria fatia de trabalho enquanto workers rodam
16 doMyShare();
17
18 // Espera todos os workers concluirem -- barreira da fase
19 SyncThread::waitForAllCompleted(workers, 3);
20 // so aqui a proxima fase comeca
```

Listing 70: Padrão completo de uso de `SyncThread`: disparar, trabalhar e esperar.

Detalhe de encerramento. `SyncThread::terminate()` chama `signalCompleted()` antes de delegar a `AbstractThread::terminate()`. Sem isso, qualquer chamador bloqueado em `waitForCompleted()` esperando por uma `SyncThread` que está sendo encerrada ficaria bloqueado para sempre — um *deadlock* silencioso.

Arquivos pareados por plataforma. O código sensível ao sistema operacional é isolado em arquivos nomeados consistentemente: `AbstractThread_linux.cpp` / `AbstractThread_msvc.cpp`, `PeriodicThread_linux.cpp` / `PeriodicThread_msvc.cpp`, `SyncThread_linux.cpp` / `SyncThread_msvc.cpp`. A classe genérica e o arquivo de plataforma *ambos* devem ser compilados — omitir o arquivo de plataforma causa erros de ligação. A regra vale para todo componente do MIXR com código específico de plataforma.

III.9 Máquinas de estado

`StateMachine` oferece máquinas de estado finitas como `Components` do *framework*. A decisão de projeto tem três pilares: estados são **funções membro** mapeadas por número, transições usam um conjunto de **funções com pilha de retorno** (não um `switch` simples) e a máquina admite **hierarquia** — estados podem ser implementados por outras `StateMachines` filhas, definidas em EDL e ativadas pelo nome.

Como `StateMachine` herda de `Component`, ela recebe `updateTC()` e `updateData()` de graça, participa da árvore de contenção e responde a eventos (`ON_ENTRY`, `ON_EXIT`, `ON_RETURN`). Um comportamento sequencial complexo torna-se, assim, um componente reutilizável como qualquer outro.

III.9.1 O ciclo de execução por *frame*

A cada chamada de `updateTC(dt)`, a máquina executa `step(dt)`, que percorre **cinco** etapas fixas. A primeira é a mais importante, e é a que não se adivinha lendo apenas a interface:

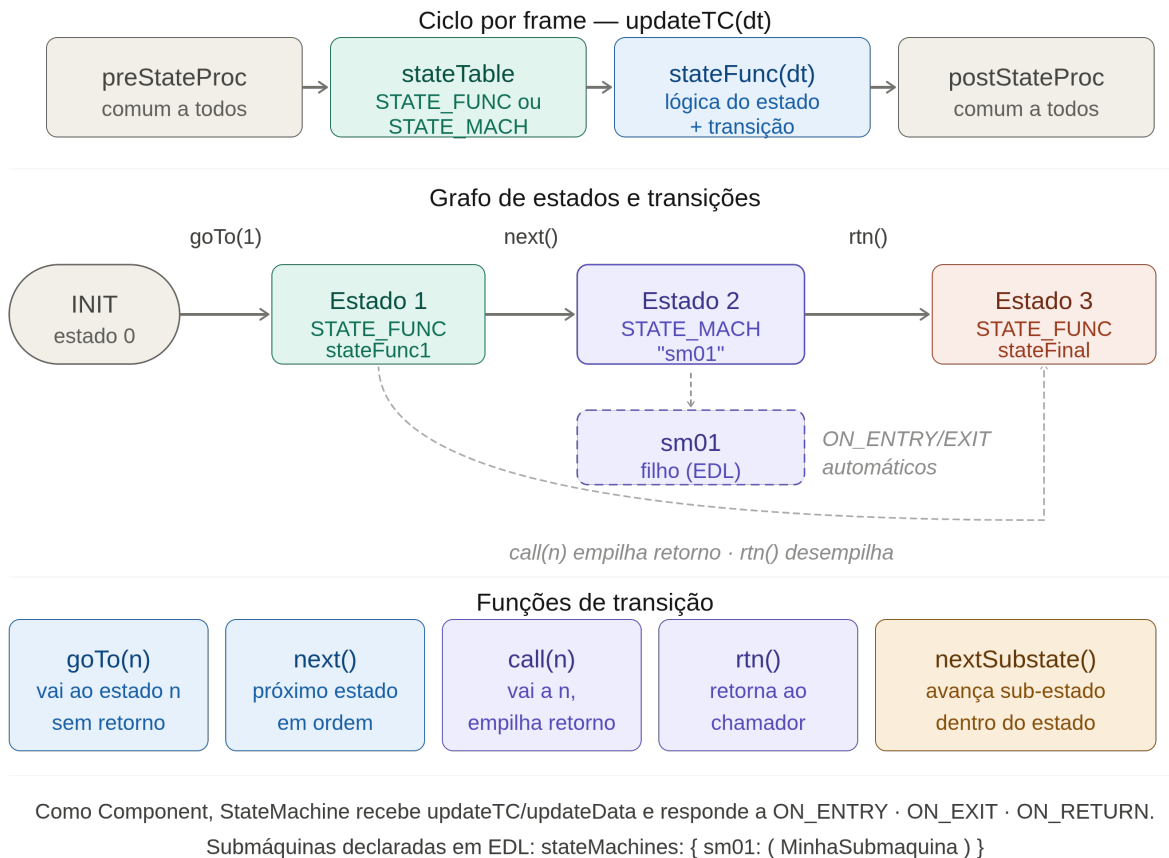


Figure 12: StateMachine: ciclo de execução por *frame* (topo), grafo de estados com transições e submáquinas (centro) e funções de transição disponíveis (baixo).

1. **Efetivação da transição pedida no quadro anterior.** `goTo()`, `next()`, `call()` e `rtn()` *não mudam o estado*. Elas escrevem num conjunto de campos sombra — `nState`, `nSubstate`, `nArg`, `nMode` — que só é consumido aqui, no topo do quadro **seguinte**.
2. `preStateProc(dt)` — função comum a todos os estados, independente do estado atual. Serve para coletar entradas, atualizar sensores, ou qualquer preparação que toda iteração exige.
3. `stateTable(state, CURR_STATE, dt)` — despacha para a função ou submáquina do estado atual. É implementada via macros (`BEGIN_STATE_TABLE/STATE_FUNC/STATE_MACH/ END_STATE_TABLE`).
4. **função de estado ou submáquina** — executa a lógica do estado, verifica condições e dispara transições.
5. `postStateProc(dt)` — função comum pós-estado, para envio de saídas e contabilidade.

Regra — transições são diferidas em um quadro

Uma função de estado que chame `goTo(3)` **continua sendo a função do estado atual até o fim daquele quadro** — e continua executando as linhas que vierem depois da chamada. A troca só

acontece na próxima passagem por `step()`.
Três consequências que decorrem disso:

- `getMode()` devolve `NEW_STATE` no quadro *posterior* à transição, não naquele em que ela foi pedida. É por isso que o idioma de inicialização de estado se escreve `if (getMode() == Mode::NEW_STATE) { ... }` dentro da função do estado *novo*.
- Duas transições no mesmo quadro não se acumulam: a última sobrescreve a anterior, e nenhuma das duas gera erro.
- `getArgument()` só tem valor útil nos modos `NEW_STATE` e `RTN_STATE`; num quadro de `HOLD_STATE` o argumento é zerado por `step()`.

Por que — por que diferir a transição

Executar a troca no meio de uma função de estado significaria que o resto daquela função rodaria num estado que já não é o dela — com `getMode()`, `getArgument()` e a submáquina ativa já trocados sob os seus pés. Diferir garante que cada quadro execute inteiramente num único estado, o que é a mesma disciplina de “estado estável durante o quadro” que a Seção II.4 aplica às fases da simulação.

III.9.2 Definindo estados: `STATE_FUNC` e `STATE_MACH`

O mapeamento de número de estado para comportamento é feito em dois passos: a **tabela de estados** (no `.cpp`) e as **funções de estado** (métodos membros). A tabela aceita dois tipos de entrada:

```

1 BEGIN_STATE_TABLE(MeuComportamento)
2   STATE_FUNC( INIT_STATE, stateInicio ) // estado inicial (obrigatorio)
3   STATE_FUNC( 1,      stateIdle ) // estado 1 -> chama stateIdle(dt)
4   STATE_MACH( 2,      "sm01" ) // estado 2 -> delega a "sm01"
5   STATE_FUNC( 3,      stateAtaque ) // estado 3 -> chama stateAtaque(dt)
6   STATE_FUNC( 9,      stateFinal ) // estado 9 -> estado terminal
7 END_STATE_TABLE()
```

Listing 71: Tabela de estados: `STATE_FUNC` para funções membro e `STATE_MACH` para submáquinas filhas.

`STATE_FUNC` mapeia um número para um método membro cuja assinatura é sempre `void f(const double dt)`. `STATE_MACH` mapeia um número para o nome de uma `StateMachine` filha declarada no *slot* `stateMachines`: em EDL. Há uma terceira entrada, `ANY_STATE_FUNC`, que não recebe número: ela casa com *qualquer* estado que nenhuma linha anterior tenha reconhecido — o *default* do `switch`.

Dois números são reservados, e ambos são constantes estáticas de `StateMachine`:

Constante	Valor	Significado
<code>INIT_STATE</code>	0	estado para o qual <code>reset()</code> leva a máquina
<code>INVALID_STATE</code>	0xFFFF	“nenhum estado” — valor de partida e de falha

Uma máquina recém-construída está em `INVALID_STATE`: ela não executa nada até o primeiro `reset()`, que a leva a `INIT_STATE` deixando `INVALID_STATE` como estado anterior. É por isso que `STATE_FUNC(INIT_STATE, ...)` é obrigatório — sem ele a máquina sai do *reset* para um estado que a tabela não reconhece.

Por que — a “tabela de estados” não é uma tabela: é uma função chamada com três perguntas

O nome engana. `BEGIN_STATE_TABLE` não constrói estrutura de dados nenhuma — ele abre a definição do método virtual `stateTable(cstate, code, dt)`, e cada `STATE_FUNC/STATE_MACH` expande para um `if/else if` dentro dele. O que a torna versátil é o segundo parâmetro, um `StateTableCode` com três valores, porque a mesma cadeia de `if` responde a três perguntas diferentes:

Código	Pergunta que a chamada faz
<code>CURR_STATE</code>	“execute o estado <code>cstate</code> ” — chama a função membro
<code>TEST_STATE</code>	“o estado <code>cstate</code> existe?” — devolve-o, ou <code>INVALID_STATE</code>
<code>FIND_NEXT_STATE</code>	“qual o estado seguinte a <code>cstate</code> na tabela?”

Isso explica cada comportamento da armadilha adiante. `goTo(n)` chama a tabela com `TEST_STATE` antes de mudar — daí falhar silenciosamente para um estado inexistente. `next()` chama com `FIND_NEXT_STATE`, que devolve a *próxima linha escrita na tabela*, não `state + 1`: numa tabela que salta de 3 para 9, `next()` a partir do estado 3 vai para 9.

E é também o que distingue `ANY_STATE_FUNC`: sob `FIND_NEXT_STATE` ele devolve literalmente `cstate + 1`, porque não há “próxima linha” — ele é a última.

O *framework* cuida de enviar `ON_ENTRY` quando a submáquina é ativada, de chamar `tcFrame(dt)` nela a cada *frame* e de enviar `ON_EXIT` quando ela é desativada.

```
1 void MeuComportamento::stateIdle(const double dt)
2 {
3     if (alvoDetectado()) {
4         goTo(3);           // vai direto ao estado 3
5     }
6     if (missaoConcluida()) {
7         goTo(9);           // vai ao estado terminal
8     }
9 }
10
11 void MeuComportamento::stateAtaque(const double dt)
12 {
13     if (precisaDeRotina()) {
14         call(2);           // vai ao estado 2 (STATE_MACH "sm01"),
15                             // empilha 3 como retorno
16     }
17 }
```

Listing 72: Funções de estado com transições: lógica e modos.

III.9.3 Funções de transição: além do `switch`

O que distingue `StateMachine` de um `switch` em `updateTC()` é o conjunto de funções de transição com pilha de retorno:

```
1 // -- transicoes de estado --
2 goTo(n);           // vai direto ao estado n (sem retorno)
3 next();            // vai para o proximo estado em ordem na tabela
4 call(n);           // vai ao estado n e EMPILHA o estado atual para retorno
5 rtn();             // retorna ao estado que chamou call() -- desempilha
6
7 // -- transicoes de sub-estado (granularidade dentro de um estado) --
8 nextSubstate();    // avanca substate++
9 goToSubstate(n);   // vai ao sub-estado n
10
11 // -- transicoes para o estado pai (quando rodando como submaquina) --
```

```

12 nextState();    // chama next() na máquina PAI
13 goToState(n);   // chama goTo(n) na máquina PAI
14 callState(n);   // chama call(n) na máquina PAI
15 rtnState();     // chama rtn() na máquina PAI

```

Listing 73: Funções de transição: estado, sub-estado e pai.

O par `call()/rtn()` é particularmente poderoso: permite que um estado “chame” uma sub-rotina de comportamento e retorne ao ponto exato de onde saiu — mas ao longo de múltiplos *frames*. Esse “ponto exato” é uma **pilha de dez posições**, com entradas paralelas para estado e sub-estado; `callState()` empilha tanto na pilha da máquina pai quanto na da filha.

As funções `*State()` fazem o mesmo, porém escalando a transição para a máquina pai — é a forma pela qual uma submáquina filha sinaliza conclusão ao seu controlador.

Armadilha — todas as transições devolvem `bool`, e podem falhar

As dez funções de transição devolvem `bool`, e os exemplos deste manual — como quase todo código real — ignoram esse retorno. Vale saber quando ele é `false`: `goTo(n)` falha se o estado `n` não existir na tabela; `next()` falha se não houver estado seguinte; `call()` falha se a pilha de dez já estiver cheia; `rtn()` falha se a pilha estiver vazia.

Em todos os casos o pedido é **silenciosamente descartado** e a máquina permanece no estado atual. Um `goTo()` para um número de estado inexistente não gera erro nem trava: apenas não acontece.

III.9.4 Composição via EDL: comportamento principal e submáquinas

O que torna `StateMachine` mais do que um `switch` glorificado é a capacidade de compor comportamentos hierárquicos inteiramente em EDL, sem recompilar. O exemplo a seguir percorre um caso completo: uma máquina principal com três estados — dois implementados por funções e um delegado a uma submáquina filha — e duas submáquinas distintas intercambiáveis via EDL.

O cenário. Um *player* autônomo precisa de três fases de comportamento: aguardar ordem (`stateAguarda`), executar uma rotina de busca (delegada a uma submáquina) e, ao concluir, reportar resultado (`stateReporta`). A rotina de busca varia por missão e deve ser configurável em EDL.

Tabela de estados e funções em C++. A tabela fixa a estrutura. Os estados 1 e 3 são funções membro; o estado 2 é delegado a qualquer objeto chamado “busca” que aparecer no EDL:

```

1 BEGIN_STATE_TABLE(PlayerAutonomo)
2   STATE_FUNC( INIT_STATE, stateInit )
3   STATE_FUNC( 1,      stateAguarda ) // aguarda ordem de inicio
4   STATE_MACH( 2,      "busca" ) // delega a SM nomeada "busca"
5   STATE_FUNC( 3,      stateReporta ) // reporta resultado e encerra
6 END_STATE_TABLE()
7
8 void PlayerAutonomo::stateInit(const double dt)
9 {
10   goTo(1);
11 }
12
13 void PlayerAutonomo::stateAguarda(const double dt)
14 {
15   if (getMode() == Mode::NEW_STATE) timer = 0.0;
16   timer += dt;

```

```

17     if (ordemRecebida()) next(); // avanca para o estado 2 ("busca")
18 }
19
20 void PlayerAutonomo::stateReporta(const double dt)
21 {
22     if (getMode() == Mode::NEW_STATE) {
23         // getArgument() recebe o objeto passado por rtnState() da submaquina
24         const auto* r = dynamic_cast<const Identifier*>(getArgument());
25         if (r != nullptr) enviarRelatorio(*r);
26     }
27     // estado terminal: nao transiciona
28 }

```

Listing 74: Tabela de PlayerAutonomo: estrutura fixa em C++, conteúdo configurável em EDL.

O estado 2 não tem função membro em PlayerAutonomo. A cada *frame* em que está ativo, o *framework* chama `tcFrame(dt)` na submáquina. Quando ela chama `rtnState()`, o controle retorna ao estado 3.

Submáquinas em C++. Cada submáquina é uma *StateMachine* independente com sua própria tabela. Duas implementações da rotina de busca com a mesma interface:

```

1 // BuscaLinear.cpp
2 BEGIN_STATE_TABLE(BuscaLinear)
3     STATE_FUNC( INIT_STATE, stateInit )
4     STATE_FUNC( 1,      stateVarrer ) // varre em linha reta
5     STATE_FUNC( 2,      stateConcluir )
6 END_STATE_TABLE()
7
8 void BuscaLinear::stateConcluir(const double dt)
9 {
10     rtnState(); // sinaliza conclusao ao pai; pai avanca para stateReporta
11 }
12
13 // BuscaEspiral.cpp
14 BEGIN_STATE_TABLE(BuscaEspiral)
15     STATE_FUNC( INIT_STATE, stateInit )
16     STATE_FUNC( 1,      stateEspiral ) // varre em espiral
17     STATE_FUNC( 2,      stateConcluir )
18 END_STATE_TABLE()

```

Listing 75: Duas submáquinas intercambiáveis: mesma interface, comportamentos distintos.

Composição em EDL. A escolha de qual submáquina usar é feita inteiramente em EDL. Ambas usam o mesmo nome de *slot* ("busca"), mas apontam para classes C++ diferentes:

```

1 ( PlayerAutonomo
2     stateMachines: {
3         busca: ( BuscaLinear ) -- estado 2 usa BuscaLinear
4     }
5 )

```

Listing 76: PlayerAutonomo na missão de reconhecimento: usa BuscaLinear.

```

1 ( PlayerAutonomo
2     stateMachines: {
3         busca: ( BuscaEspiral ) -- estado 2 agora usa BuscaEspiral
4     }
5 )

```

Listing 77: A mesma classe C++ com comportamento diferente: apenas o EDL muda.

O C++ de `PlayerAutonomo` não muda entre as duas missões. O *framework* localiza a submáquina pelo nome "busca" na `PairStream` de `stateMachines` e a ativa quando o estado 2 é alcançado. Uma nova variante de busca exige apenas uma nova subclasse de `StateMachine` e uma linha de EDL — sem recompilação.

III.9.5 Eventos: `ON_ENTRY`, `ON_EXIT`, `ON_RETURN`

`StateMachine` envia automaticamente três eventos à submáquina ao longo das transições. Tratá-los garante inicialização limpa a cada ativação:

```
1 BEGIN_EVENT_HANDLER(BuscaLinear)
2   ON_EVENT(ON_ENTRY, onEntry) // estado 2 foi ativado
3   ON_EVENT(ON_EXIT, onExit)  // estado 2 foi desativado
4   ON_EVENT(ON_RETURN, onReturn) // retorno de um callState() interno
5 END_EVENT_HANDLER()
6
7 bool BuscaLinear::onEntry(Object* const)
8 {
9     // chamado TODA VEZ que o estado 2 é ativado -- nao apenas na primeira
10    goTo(INIT_STATE);
11    distanciaPercorrida = 0.0;
12    return true;
13 }
14
15 bool BuscaLinear::onExit()
16 {
17     // libera recursos ao sair
18    return true;
19 }
```

Listing 78: Ciclo de vida de uma submáquina via eventos: inicialização a cada ativação.

O evento `ON_ENTRY` é enviado *toda vez* que o estado é ativado — não apenas na primeira. Isso garante que a submáquina reinicie do zero mesmo que seja reutilizada em múltiplas passagens, sem que o chamador precise saber disso. A submáquina **não é destruída** ao ser desativada — é apenas pausada, pronta para ser reusada quando o estado for ativado novamente. É essa semântica que torna a composição de `StateMachines` segura, previsível e reutilizável.

III.10 Framework UBF

O UBF (*Unified Behavior Framework*, em `base::ubf`) é uma arquitetura de controle baseada em comportamentos cuja decisão central é **separar percepção, decisão e atuação** em três abstrações desacopladas. Um comportamento não sabe *como* uma ação se executa; uma ação não sabe *quem* a decidiu. Esse desacoplamento é o que permite trocar comportamentos e ações de forma independente, sem alterar o controlador.

III.10.1 As três abstrações: `State`, `Behavior` e `Action`

Cada abstração tem uma responsabilidade única e uma interface mínima:

```
1 // AbstractState: le o ator e constroi a visao do mundo.
2 // NAO e virtual pura -- tem corpo, e o corpo e a parte interessante.
3 class AbstractState : public Component {
4 public:
5     // Chamado pelo Agent antes de genAction. A implementacao base percorre
6     // os componentes filhos, faz dynamic_cast para AbstractState e recorre --
7     // honrando isComponentSelected(). E isso que permite COMPOR estados:
```

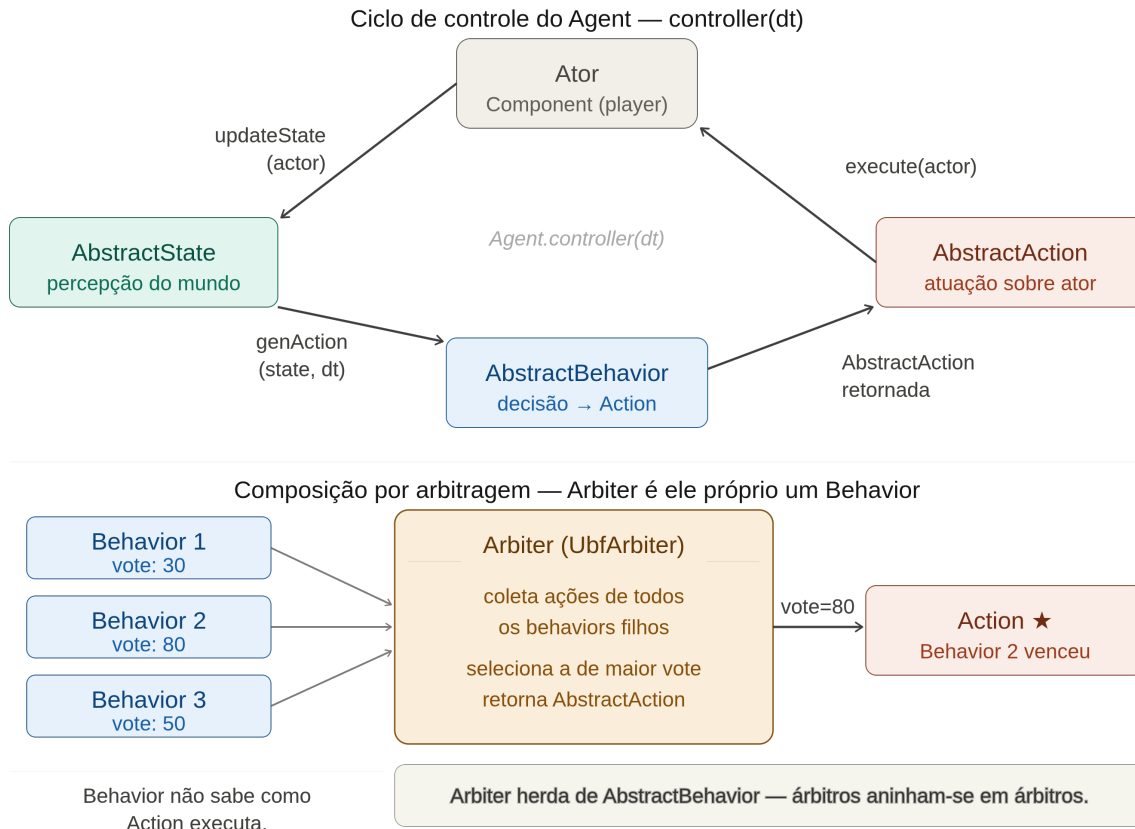


Figure 13: Ciclo percepção–decisão–atuação do Agent (topo) e composição por arbitragem com Arbiter (baixo). O Arbiter é ele próprio um AbstractBehavior, o que permite aninhamento ilimitado.

```

8  // um AbstractState pode ser a raiz de uma arvore de sub-estados.
9  virtual void updateState(const base::Component* const actor);
10
11 // Segunda travessia, independente de ator (contexto global da simulacao)
12 virtual void updateGlobalState();
13
14 // acesso tipado a sub-estados compostos
15 const AbstractState* getUbfStateByType(const std::type_info&) const;
16 };
17
18 // AbstractBehavior: decide, a partir do estado, qual acao tomar
19 class AbstractBehavior : public Component {
20 public:
21     // retorna uma Action pre-ref'd, ou nullptr se nao ha acao
22     virtual AbstractAction* genAction(const AbstractState* const state,
23                                     const double dt) = 0;
24 protected:
25     int getVote() const;           // PROTEGIDO em AbstractBehavior
26     virtual void setVote(const int); // (em AbstractAction, os dois sao publicos)
27 };
28
29 // AbstractAction: sabe executar-se sobre o ator
30 class AbstractAction : public Object {
31 public:
32     virtual bool execute(base::Component* actor) = 0; // sem o 'const' do ponteiro
33     int getVote() const; // voto herdado do behavior que a gerou
34     void setVote(const int);
35 };

```

Listing 79: As três interfaces abstratas do UBF: uma responsabilidade cada.

O ponto crítico é que `AbstractAction::execute()` recebe o ator como parâmetro — a ação não guarda uma referência ao ator, não o conhece em tempo de construção e não depende de nenhum estado global. Ela é um objeto de valor transitório: nasce em `genAction()`, executa em `execute()` e é destruída (via `unref()`) em seguida.

III.10.2 O Agent: ciclo de controle

`Agent` (`UbfAgent`) costura as três abstrações sobre um *ator* (um `Component` — tipicamente um *player* ou subsistema). O ciclo completo, extraído diretamente de `Agent.cpp`:

```

1 void Agent::controller(const double dt)
2 {
3     base::Component* actor { getActor() };
4
5     if (actor != nullptr && getState() != nullptr && getBehavior() != nullptr) {
6
7         // 1. PERCEPCAO: atualiza o estado a partir do ator
8         getState()->updateState(actor);
9
10        // 2. DECISAO: pede ao behavior uma acao
11        AbstractAction* action { getBehavior()->genAction(state, dt) };
12
13        // 3. ATUACAO: executa a acao sobre o ator (se houver)
14        if (action != nullptr) {
15            action->execute(actor);
16            action->unref(); // Action e transitoria -- destruida aqui
17        }
18    }
19 }
```

Listing 80: `Agent::controller(dt)`: o ciclo percepção–decisão–atuação real (`src/base/ubf/Agent.cpp`).

Dois pontos merecem atenção. Primeiro, `genAction()` pode retornar `nullptr` — o comportamento tem o direito de não recomendar ação alguma num dado instante, sem que isso seja um erro. Segundo, `AgentTC` é uma subclasse trivial que apenas sobrescreve `updateTC()` em vez de `updateData()`, movendo o ciclo de controle para a *thread* crítica em tempo:

```

1 class AgentTC : public Agent {
2     DECLARE_SUBCLASS(AgentTC, Agent)
3 public:
4     AgentTC();
5     void updateTC(const double dt = 0.0) override; // chama controller(dt)
6     // updateData() NAO e sobrescrito -- nao roda na thread de fundo
7 };
```

Listing 81: `AgentTC`: o mesmo ciclo na *thread* crítica em tempo.

III.10.3 O Arbiter: composição por votação

A composição de comportamentos é a segunda decisão elegante do UBF. O `Arbiter` (`UbfArbiter`) é, **ele próprio, um `AbstractBehavior`** — e contém uma lista de comportamentos filhos. Quando `genAction()` é chamado no árbitro, ele:

1. Chama `genAction()` em cada comportamento filho, coletando as ações recomendadas em uma lista (*action set*).
2. Seleciona a ação de maior `vote` como a ação resultante (`genComplexAction()`).

3. Retorna essa ação ao chamador — que pode ser o **Agent** ou outro **Arbiter**.

O código real de `Arbiter::genAction()` ilustra o mecanismo:

```
1 AbstractAction* Arbiter::genAction(const AbstractState* const state, const double dt)
2 {
3     // 1. coleta acoes de todos os behaviors filhos
4     const auto actionSet = new base::List();
5     base::List::Item* item { behaviors->getFirstItem() };
6     while (item != nullptr) {
7         const auto behavior = static_cast<AbstractBehavior*>(item->getValue());
8         AbstractAction* action { behavior->genAction(state, dt) };
9         if (action != nullptr) {
10             actionSet->addTail(action);
11             action->unref();
12         }
13         item = item->getNext();
14     }
15
16     // 2. arbitragem: seleciona a acao de maior voto
17     AbstractAction* complexAction { genComplexAction(actionSet) };
18     actionSet->unref();
19
20     return complexAction; // nullptr se nenhum behavior gerou acao
21 }
22
23 // O criterio de selecao padrao: maior vote vence
24 AbstractAction* Arbiter::genComplexAction(base::List* const actionSet)
25 {
26     AbstractAction* complexAction {};
27     int maxVote {};
28     base::List::Item* item { actionSet->getFirstItem() };
29     while (item != nullptr) {
30         const auto action = static_cast<AbstractAction*>(item->getValue());
31         if (maxVote == 0 || action->getVote() > maxVote) {
32             if (complexAction != nullptr) complexAction->unref();
33             complexAction = action;
34             complexAction->ref();
35             maxVote = action->getVote();
36         }
37         item = item->getNext();
38     }
39
40     // *** A LINHA QUE FAZ O ANINHAMENTO FUNCIONAR ***
41     // Se ESTE arbitro tem um voto proprio, ele o CARIMBA na acao vencedora.
42     if (getVote() > 0 && complexAction != nullptr)
43         complexAction->setVote(getVote());
44
45     return complexAction; // pre-ref'd para o chamador
46 }
```

Listing 82: `Arbiter::genAction()` real: coleta, arbitragem e retorno (`src/base/ubf/Arbiter.cpp`).

A última linha é o que transforma o aninhamento numa hierarquia de prioridade de verdade, e sem ela a composição não funcionaria:

Por que — o árbitro carimba o próprio voto na ação vencedora

Sem esse carimbo, um `Arbiter` aninhado devolveria ao árbitro pai uma ação com o voto do *comportamento neto* que a produziu — e o pai compararia maçãs com laranjas, já que os votos dos netos foram calibrados numa escala interna àquele subgrupo. Com o carimbo, cada árbitro apresenta-se ao nível de cima com *o seu próprio peso*, e a escala de cada nível é independente da dos demais.

É o mesmo princípio de encapsulamento dos *slots*: um nível não precisa conhecer a numeração interna do outro.

Armadilha — o primeiro comportamento vence o desempate com voto zero

O teste é `maxVote == 0 || action->getVote() > maxVote`. Na primeira iteração `maxVote` vale zero, de modo que a **primeira** ação da lista é sempre adotada como incumbente — mesmo que o seu voto seja zero. Se depois nenhum outro comportamento superar zero, ela vence. Um comportamento sem `vote`: declarado, colocado no topo da lista, torna-se assim o padrão efetivo do árbitro.

III.10.4 Configuração em EDL

Como todas as classes do UBF são **Components** com *slots*, a composição completa — agente, estado, comportamento e submáquina de árbitros — é declarada em EDL:

```
1 agente: ( UbfAgent
2   state:  ( MeuEstado )    -- AbstractState concreto
3   behavior:
4     ( UbfArbiter           -- Arbiter e um Behavior
5       behaviors: {
6         b1: ( EvitarColisao  vote: 90 ) -- alta prioridade
7         b2: ( BuscarAlvo     vote: 40 ) -- prioridade baixa
8         b3: ( PadraoDeVoo    vote: 10 ) -- comportamento de fundo
9       }
10    )
11 )
```

Listing 83: Configuração completa de um **Agent** em EDL: estado, árbitro com dois comportamentos votantes.

A cada chamada de `updateData(dt)`, o **Agent** atualiza `MeuEstado` com o ator, passa o estado ao `UbfArbiter`, que consulta os três comportamentos. Se `EvitarColisao` gera uma ação (vote 90) e `BuscarAlvo` também (vote 40), o árbitro retorna a ação de `EvitarColisao` — colisão tem prioridade. Se `EvitarColisao` não gera ação (`nullptr`), `BuscarAlvo` vence automaticamente.

Para mover o ciclo à *thread* crítica em tempo existe `AgentTC`, cujo nome de fábrica é "`UbfAgentTC`".

Armadilha — `UbfAgentTC` não é registrado pelo *framework*

`base/factory.cpp` registra "`UbfAgent`" e "`UbfArbiter`" — e mais nada do UBF. `AgentTC` existe como classe, tem nome de fábrica declarado, mas **nenhuma fábrica do MIXR o constrói**. Escrever (`UbfAgentTC...`) em EDL falha, a menos que a aplicação o registre na sua própria fábrica (Seção III.3.1).

Armadilha — um **Agent** não propaga atualizações aos filhos

Há uma segunda surpresa, e ela vale para os dois agentes. Nem `Agent::updateData()` nem `AgentTC::updateTC()` chamam `BaseClass::update*()`: os dois chamam *apenas* `controller(dt)`. Como `setState()` insere o estado na lista de componentes do agente, o efeito é que **o state e qualquer outro filho do agente nunca recebem `updateTC()` nem `updateData()`**. O próprio cabeçalho de `Agent.hpp` registra a intenção: as chamadas “não são repassadas ao restante do *framework* de comportamento”.

Quem escreve um `AbstractState` que dependa de ser atualizado pelo ciclo normal de componentes não será servido — o estado só é atualizado por `updateState(actor)`, dentro de `controller()`.

A camada `models` estende o UBF com agentes cientes do contexto da simulação (`SimAgent`, `MultiActorAgent`), que herdam o mesmo ciclo e o mesmo mecanismo de arbitragem, adicionando acesso à `Simulation` e à lista de *players* — tratados na Seção VI.

III.11 Interpoladores: tabelas e LFI

Modelos de engenharia dependem de funções tabeladas — coeficientes aerodinâmicos (C_L , C_D), curvas de motor, perfis de atmosfera. O MIXR oferece interpolação linear multidimensional (LFI, *Linear Function Interpolation*) em dois níveis, por uma razão que ecoa a dos “dois mundos” da Seção III.6: velocidade e configurabilidade exigem estratégias diferentes.

III.11.1 A fórmula fundamental: interpolação linear 1D

Toda a maquinaria de LFI do MIXR reduz-se, em última instância, a uma única fórmula aplicada repetidamente. Dados dois *breakpoints* consecutivos $x_1 < x_2$ com valores tabelados a_1 e a_2 , e um ponto de consulta $x \in [x_1, x_2]$:

$$m = \frac{x - x_1}{x_2 - x_1}, \quad f(x) = m \cdot (a_2 - a_1) + a_1 \quad (1)$$

$m \in [0, 1]$ é a fração de interpolação: quanto x avançou dentro do intervalo $[x_1, x_2]$. Quando $m = 0$, $f(x) = a_1$; quando $m = 1$, $f(x) = a_2$; no meio do intervalo, $f(x)$ está a meio caminho entre a_1 e a_2 . É o triângulo de semelhança clássico.

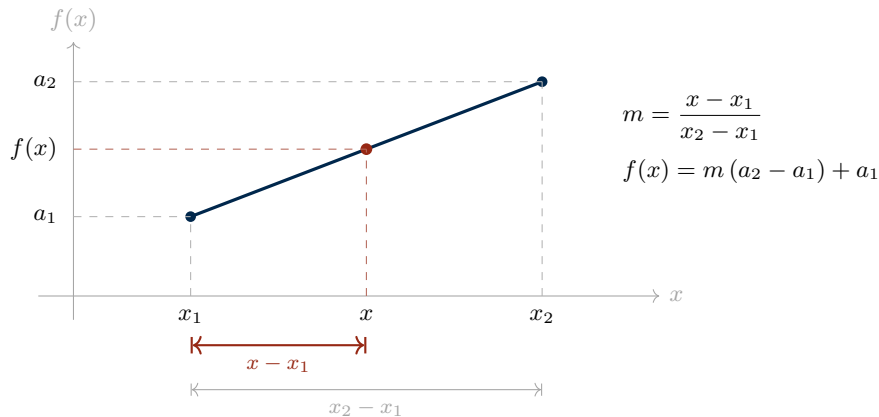


Figure 14: Geometria da interpolação linear 1D. A fração m mede o quanto x avançou dentro do intervalo $[x_1, x_2]$; o mesmo m é então aplicado ao salto de valor $a_2 - a_1$. É essa razão entre segmentos — e não a inclinação da reta — que o código de `lfi_1D` calcula.

O código real de `lfi_1D` implementa exatamente a Equação 1, com dois detalhes adicionais: localizar o intervalo $[x_1, x_2]$ que contém x (busca em `x_data`) e tratar os casos de borda (extrapolação ou *clamping*):

```
1 // Localiza x2: primeiro breakpoint ACIMA de x
2 // (versao sem cache -- busca linear)
3 unsigned int x2 { low + delta };
4 while (x > x_data[x2]) { x2 += delta; }
5
6 // Aplica a formula de interpolacao linear
7 const unsigned int x1 { x2 - delta };
8 const double m { (x - x_data[x1]) / (x_data[x2] - x_data[x1]) };
9 return m * (a_data[x2] - a_data[x1]) + a_data[x1];
```

Listing 84: Núcleo de `lfi_1D`: localização do intervalo e fórmula 1 (`src/base/util/lfi.cpp`).

Quando x está abaixo de `x_data[0]` ou acima de `x_data[nx-1]`, o comportamento depende da flag `eFlg`:

- **eFlg = false** (padrão): retorna o valor do extremo mais próximo (*clamping*). $f(x < x_0) = a_0$ e $f(x > x_{n-1}) = a_{n-1}$.
- **eFlg = true**: extrapola linearmente além dos *breakpoints*, usando a reta definida pelo primeiro ou último par de pontos.

III.11.2 Extensão a múltiplas dimensões: redução dimensional

O salto de 1D para ND é feito por **redução dimensional**. Para calcular $f(x, y)$ em 2D, o algoritmo:

1. Encontra os dois *breakpoints* consecutivos de y que encaixotam o ponto: $y_1 \leq y < y_2$.
2. Calcula $a_1 = \text{lfi_1D}(x, \text{linha } y_1)$ — a linha inteira de valores para y_1 é uma subtabela 1D em x .
3. Calcula $a_2 = \text{lfi_1D}(x, \text{linha } y_2)$ — idem.
4. Interpola linearmente entre a_1 e a_2 :

$$m_y = \frac{y - y_1}{y_2 - y_1}, \quad f(x, y) = m_y \cdot (a_2 - a_1) + a_1 \quad (2)$$

A Figura 15 ilustra os três passos com valores concretos: a coluna vermelha (x atual) cruza a linha azul (y_1) produzindo a_1 , cruza a linha verde (y_2) produzindo a_2 , e a interpolação final em y combina os dois.

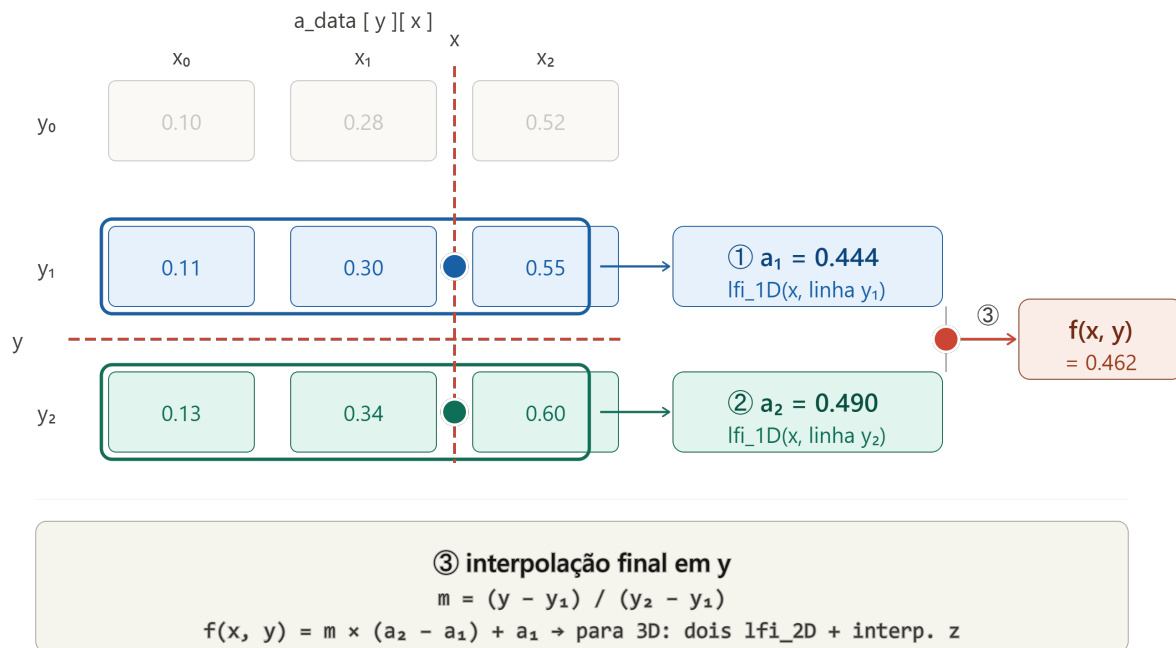


Figure 15: Redução dimensional em `lfi_2D`: ① `lfi_1D` na linha y_1 (azul) produz $a_1 = 0.444$; ② `lfi_1D` na linha y_2 (verde) produz $a_2 = 0.490$; ③ interpolação linear em y combina os dois em $f(x, y) = 0.462$. O padrão repete-se recursivamente: `lfi_3D` chama `lfi_2D` duas vezes e interpola em z ; e assim até `lfi_5D`.

O código real de `lfi_2D` espelha diretamente a Equação 2:

```

1 // Passo 1: interpola na dimensao x para os dois breakpoints de y
2 const unsigned int y1 { y2 - delta };
3 const unsigned int ax1 { nx * y1 };
4 const double a1 { lfi_1D(x, x_data, nx, &a_data[ax1], eFlg, xbp) };
5
6 const unsigned int ax2 { nx * y2 };
7 const double a2 { lfi_1D(x, x_data, nx, &a_data[ax2], eFlg, xbp) };
8
9 // Passo 2: interpolacao linear final em y (Eq. lfi2d)
10 const double m { (y - y_data[y1]) / (y_data[y2] - y_data[y1]) };
11 return m * (a2 - a1) + a1;

```

Listing 85: `lfi_2D` real: dois `lfi_1D` + interpolação final em y (`src/base/util/lfi.cpp`).

O padrão é estritamente recursivo. `lfi_3D` chama `lfi_2D` duas vezes (uma por *breakpoint* de z) e interpola em z . `lfi_4D` chama `lfi_3D` duas vezes e interpola em w . E assim até `lfi_5D`. Em geral:

$$f(x_1, \dots, x_N) = m_N \cdot [\text{lfi}_{N-1}(x_1, \dots, x_{N-1}, \text{slice}_{i_N+1}) - \text{lfi}_{N-1}(x_1, \dots, x_{N-1}, \text{slice}_{i_N})] + \text{lfi}_{N-1}(\dots, \text{slice}_{i_N}) \quad (3)$$

onde i_N é o índice do *breakpoint* inferior na dimensão N e m_N é a fração de interpolação nessa dimensão.

III.11.3 Ordem dos *breakpoints*, `delta` e `cache`

Os *breakpoints* precisam ser ordenados? Sim — mas em **qualquer** das duas ordens. Os arrays `x_data`, `y_data` etc. devem ser estritamente monótonos (crescentes *ou* decrescentes), e o código detecta a ordem automaticamente em tempo de execução comparando os dois primeiros elementos:

```

1 unsigned int low{};
2 unsigned int high{ nx - 1 };
3 int delta{ 1 }; // crescente por padrao
4
5 if (x_data[1] < x_data[0]) { // decrescente?
6     low = nx - 1; // inverte: busca comeca no fim
7     high = 0;
8     delta = -1; // passo da busca: x2--
9 }

```

Listing 86: Detecção automática de ordem e adaptação de `delta` (`src/base/util/lfi.cpp`).

Com `delta = +1`, a busca avança (`x2++`) até encontrar o *breakpoint* acima de x ; com `delta = -1`, recua (`x2--`). A fórmula de interpolação fica invariante porque $y1 = y2 - \text{delta}$ produz sempre o índice *inferior* na ordem lógica, independentemente da direção física do array.

O cache `xbp`: de $O(n)$ para $O(1)$. Localizar o intervalo correto é uma busca no array — $O(n)$ sem informação prévia. O parâmetro opcional `xbp` armazena o índice da última consulta e a busca parte dele, subindo ou descendo conforme necessário:

```

1 x2 = *xbp;
2 if (x2 >= nx) x2 = 0; // seguranca
3
4 while (x > x_data[x2]) { x2 += delta; } // sobe se passou
5 while (x < x_data[x2 - delta]) { x2 -= delta; } // desce se voltou
6
7 *xbp = x2; // persiste para a proxima chamada

```

Listing 87: Busca com cache: parte do índice anterior e ajusta bidireccionalmente (`src/base/util/lfi.cpp`).

A Figura 16 mostra os dois casos lado a lado. Em uma simulação em tempo real, altitude, Mach e ângulo de ataque evoluem continuamente — o *breakpoint* correto raramente muda entre *frames*. A busca termina em uma ou duas comparações, tornando-se efetivamente $O(1)$.

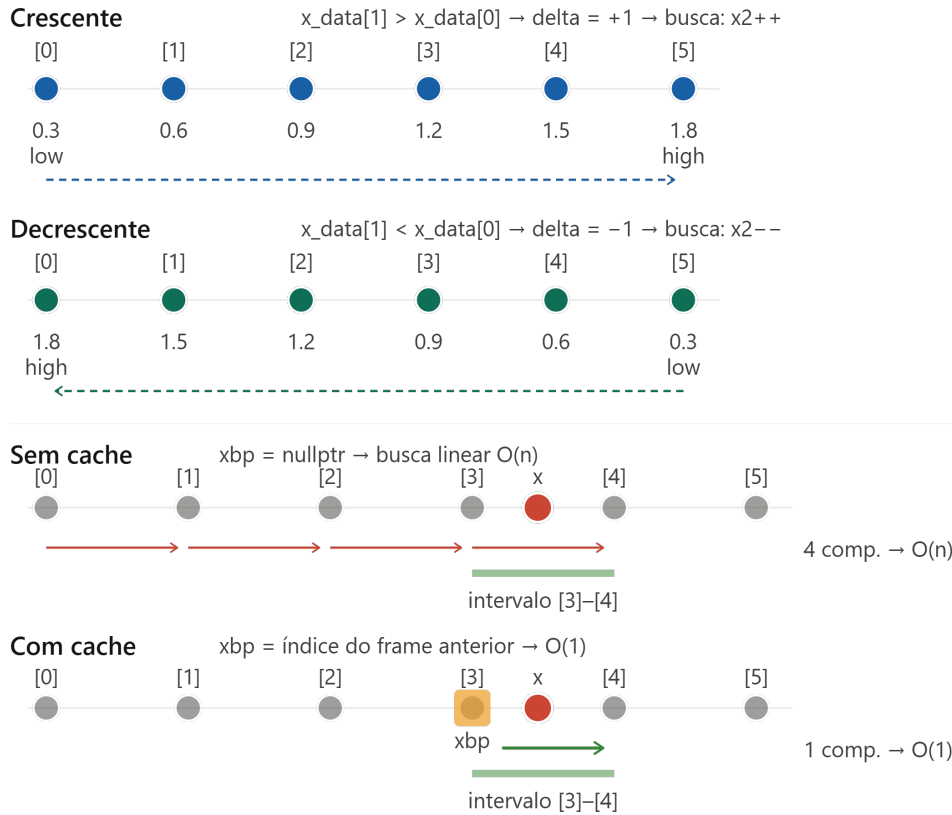


Figure 16: Topo: *breakpoints* crescentes ($\delta = +1$, busca avança $x2++$) e decrescentes ($\delta = -1$, busca recua $x2--$) — o código detecta a ordem automaticamente. Baixo: sem cache, a busca percorre todos os índices desde o início ($O(n)$); com cache (xbp ambar aponta para o índice do *frame* anterior), uma única comparação basta — $O(1)$.

FStorage encapsula os índices de cache para todas as dimensões, para uso seguro com a família Table:

```

1 // include/mixr/base/functors/TableStorage.hpp
2 class TableStorage : public FStorage {
3     DECLARE_SUBCLASS(TableStorage, FStorage)
4 public:
5     TableStorage();
6     unsigned int xbp {}; // cache x (dimensao 1)
7     unsigned int ybp {}; // cache y (dimensao 2)
8     unsigned int zbp {}; // cache z (dimensao 3)
9     unsigned int wbp {}; // cache w (dimensao 4)
10    unsigned int vbp {}; // cache v (dimensao 5)
11 };

```

Listing 88: TableStorage: os cinco índices de cache, um por dimensão.

III.11.4 Nível alto: família Table1–Table5

A família `Table` encapsula dados e *breakpoints* como `Objects` com *slots* configuráveis em EDL, e delega o cálculo às funções livres. A hierarquia é estritamente aditiva: cada classe acrescenta um eixo à anterior (`Table2` herda `Table1`, `Table3` herda `Table2`, etc.).

O método `Table1::lfi()` real mostra exatamente como o `FStorage` é desempacotado e passado para `lfi_1D`:

```

1  double Table1::lfi(const double iv1, FStorage* const f) const
2  {
3      if (!valid) throw new ExpInvalidTable(); // tabela invalida
4
5      if (f != nullptr) {
6          const auto s = dynamic_cast<TableStorage*>(f);
7          if (s == nullptr) throw new ExpInvalidFStorage();
8
9          // COM cache: passa &s->xbp para lfi_1D
10         return lfi_1D(iv1, getXData(), getNumXPoints(),
11                       getDataTable(), isExtrapolationEnabled(),
12                       &s->xbp);
13     } else {
14         // SEM cache: busca linear simples
15         return lfi_1D(iv1, getXData(), getNumXPoints(),
16                       getDataTable(), isExtrapolationEnabled());
17     }
18 }

```

Listing 89: `Table1::lfi()`: delegação para `lfi_1D` com ou sem cache (`src/base/functors/Table1.cpp`).

III.11.5 Caso completo: $C_L(\alpha, M)$ em EDL e uso em C++

O exemplo a seguir mostra o fluxo completo de uma tabela 2D de coeficiente de sustentação C_L em função do ângulo de ataque α e do número de Mach M , do EDL até a consulta eficiente a cada *frame*.

Declaração em EDL. Os dados são armazenados em *row-major*: cada linha de `data`: corresponde a um valor de y (Mach), e as colunas a valores de x (α):

```

1  clTable: ( Table2
2      data: [
3          -- alpha:  -2      4      10      16  (graus)
4              0.10  0.28  0.52  0.80  -- Mach 0.3
5              0.11  0.30  0.55  0.85  -- Mach 0.6
6              0.13  0.34  0.60  0.90  -- Mach 0.9
7          ]
8      x: [ -2.0  4.0  10.0  16.0 ]  -- breakpoints alpha (graus)
9      y: [  0.3  0.6  0.9          ]  -- breakpoints Mach
10 )

```

Listing 90: Tabela 2D $C_L(\alpha, M)$ em EDL: `data` em linha-por-linha, x para α (graus), y para Mach.

Uso em C++. O `TableStorage` é criado **uma vez** por instância de ator e reutilizado a cada *frame*. Desta forma, o cache `xbp/ybp` é mantido entre consultas e a busca converge em $O(1)$:

```

1  // Em reset(): cria o storage UMA VEZ por instancia de ator
2  base::TableStorage* fs { new base::TableStorage() };
3  fs->ref();
4

```

```

5 // Em updateTC() ou userFunc() -- a cada frame:
6 const double alpha_deg { getAlphaDegs() }; // angulo de ataque em graus
7 const double mach      { getMach() };      // numero de Mach
8
9 // Consulta eficiente: xbp e ybp quase nao mudam entre frames
10 double cl { clTable->lfi(alpha_deg, mach, fs) };
11
12 // Ao destruir o ator (em deleteData()):
13 fs->unref();
14 fs = nullptr;

```

Listing 91: Uso de Table2 com TableStorage: criado uma vez por ator, reutilizado a cada *frame*.

Para laços internos onde **Object** é peso excessivo — como integração numérica de trajetórias — as funções livres são a escolha correta, com cache gerenciado manualmente:

```

1 static const double alpha_bp[] { -2.0, 4.0, 10.0, 16.0 };
2 static const double mach_bp[] { 0.3, 0.6, 0.9 };
3 static const double cl_data[] {
4     0.10, 0.28, 0.52, 0.80, // Mach 0.3
5     0.11, 0.30, 0.55, 0.85, // Mach 0.6
6     0.13, 0.34, 0.60, 0.90, // Mach 0.9
7 };
8
9 unsigned int xbp {}; // cache de alpha -- persiste entre iteracoes
10 unsigned int ybp {}; // cache de Mach -- persiste entre iteracoes
11
12 // Laço de integracao: alpha e mach variam suavemente
13 for (int i = 0; i < N; i++) {
14     cl[i] = base::lfi_2D(
15         alpha[i], mach[i],
16         alpha_bp, 4,
17         mach_bp, 3,
18         cl_data,
19         false, // sem extrapolacao: clamp nas extremidades
20         &xbp, &ybp // cache compartilhado entre iteracoes
21     );
22 }
23 // cl[i] = C_L(alpha[i], mach[i]) com custo ~O(1) por iteracao

```

Listing 92: Uso direto de lfi_2D em laço numérico: zero alocação, cache manual, custo mínimo por chamada.

III.11.6 Func1–Func5 e Polynomial: a função como *slot*

Há um terceiro nível que os dois anteriores deixam de fora, e ele resolve um problema diferente: **como declarar que um *slot* aceita “uma função de duas variáveis”**, sem fixar se ela será uma tabela ou uma fórmula.

Func1 a Func5 são exatamente isso — objetos chamáveis de N variáveis:

```

1 class Func2 : public Function {
2 public:
3     virtual double f(const double iv1, const double iv2,
4                     FStorage* const = nullptr) const;
5 };

```

Listing 93: A interface de Func2: uma chamada, e o *storage* opcional de cache (resumo de include/mixr/base/functors/Func2.hpp).

A implementação da base é um **adaptador de tabela**: se nenhuma subclasse tratar a chamada, ela delega à Table anexada pelo *slot table*, verificando a aridade (um Func2 rejeita qualquer tabela que não seja Table2). Isso é o que dá a propriedade útil:

Por que — um *slot* Func2 aceita tabela ou fórmula

Um sistema que precise de um padrão de ganho de antena pode declarar o seu *slot* como **Func2** — e o autor do cenário decide se fornece uma **Table2** medida em câmara anecoica ou uma classe C++ com a fórmula analítica. **Antenna** faz exatamente isso (Seção VI.5.3). Nenhum dos dois lados precisa negociar: o *framework* entrega uma interface chamável.

Polynomial é a alternativa fechada mais simples — deriva de **Func1** e avalia $f(x) = a_0 + a_1x + \dots + a_Nx^N$, com o grau inferido do tamanho da lista (máximo 32):

```
1 // f(x) = 0.02 - 0.15x + 1.1x^2
2 ( Polynomial coefficients : [ 0.02 -0.15 1.1 ] )
```

Listing 94: Polynomial: uma curva suave sem tabela.

Onde a curva é suave e conhecida, um polinômio evita dezenas de *breakpoints* — e, ao contrário da tabela, não usa **FStorage** nenhum, porque não há intervalo a localizar.

Regra — obtenha o *storage* pela função, não à mão

A Seção III.11.5 constrói um **TableStorage** diretamente. Funciona, mas o caminho previsto é **Function::storageFactory()**, que devolve o *storage* apropriado ao objeto concreto — **TableStorage** para tabelas, **FStorage** simples para o resto, nada para **Polynomial**. Usá-lo mantém o código correto se a fórmula for depois trocada por uma tabela, ou vice-versa.

Armadilha — as exceções de tabela são lançadas por ponteiro

Table1::lfi() e companhia lançam **throw new ExpInvalidTable()** — um **ponteiro**, não um objeto. A cláusula de captura tem de ser **catch (ExpInvalidTable* e)**, e quem captura fica dono da exceção. São três, não duas: **ExpInvalidTable**, **ExpInvalidFStorage** e **ExpInvalidVector** — esta última lançada ao carregar uma lista de *breakpoints* malformada.

III.12 Cores e espaços de cor

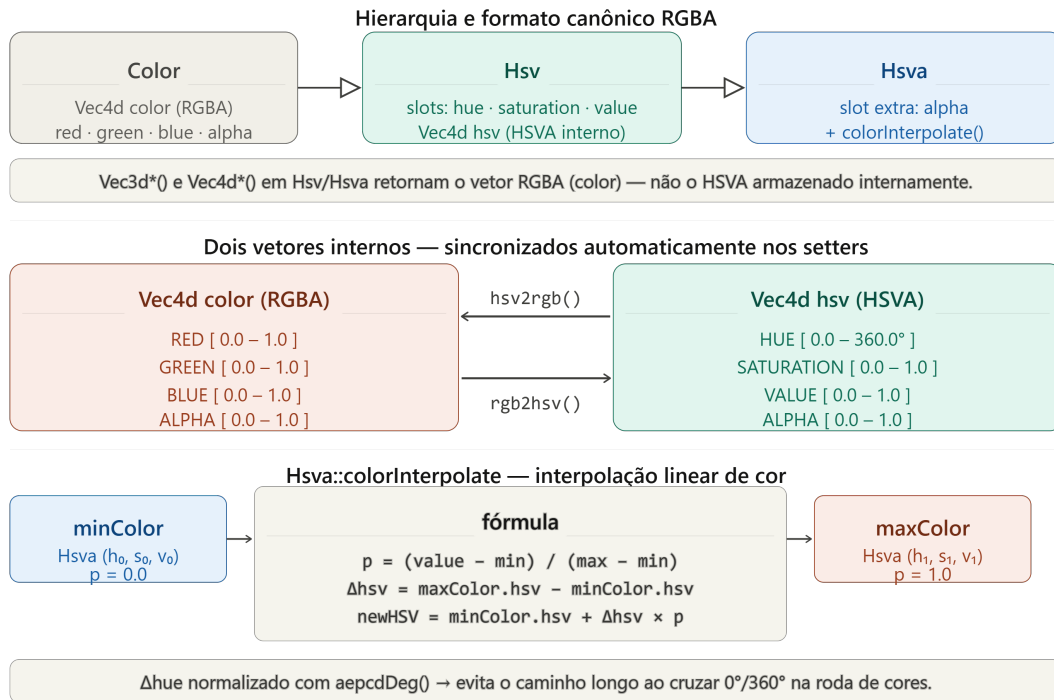
O sistema de cores do MIXR é, também ele, baseado em **Objects** configuráveis via EDL. **Color** armazena a cor internamente como um vetor **RGBA** (**Vec4d color**); **Hsv** e **Hsva** adicionam um segundo vetor **HSVA** (**Vec4d hsv**) e mantêm os dois sincronizados via conversão automática nos *setters*.

O módulo **base/colors** traz oito subclasses de **Color**, todas seguindo o mesmo princípio — cada uma aceita, via *slots*, as coordenadas do seu espaço de cor e converte para o vetor **RGBA** canônico de **Color**:

As subseções seguintes detalham a linha **Color** → **Hsv** → **Hsva**, que é a mais usada e a que melhor ilustra o padrão de sincronização entre os dois vetores internos; as demais o repetem com outras fórmulas de conversão.

III.12.1 A hierarquia: **Color**, **Hsv** e **Hsva**

Color é a classe base: armazena um vetor **Vec4d color[RED, GREEN, BLUE, ALPHA]** com componentes em $[0.0, 1.0]$ e expõe os accessors **red()**, **green()**, **blue()**, **alpha()** e os operadores de conversão **const Vec3d*()** (**RGB**) e **const Vec4d*()** (**RGBA**):



ter (setSlotHue, setSlotSaturation, setSlotValue, setSlotAlpha) chama hsv2rgb() automaticamente, mantendo color (RGBA) sempre sincronizado.

Figure 17: Hierarquia Color → Hsv → Hsva com os dois vetores internos (RGBA e HSVA) e a conversão bidirecional automática. Os operadores de conversão herdados por Hsv e Hsva sempre retornam o vetor RGBA.

```

1 // include/mirr/base/colors/Color.hpp
2 class Color : public Object {
3 public:
4     enum { RED, GREEN, BLUE, ALPHA }; // indices do vetor
5
6     double red() const; // [0.0, 1.0]
7     double green() const;
8     double blue() const;
9     double alpha() const;
10
11     operator const Vec3d*() const; // retorna RGB (sem alpha)
12     operator const Vec4d*() const; // retorna RGBA
13
14     const Vec3d* getRGB() const;
15     const Vec4d* getRGBA() const;
16
17 protected:
18     Vec4d color {}; // [RED, GREEN, BLUE, ALPHA] -- formato canonico
19 };

```

Listing 95: Interface de Color: o vetor RGBA e os operadores de conversão.

Hsv herda Color e acrescenta um segundo vetor interno Vec4d hsv[HUE, SATURATION, VALUE, ALPHA]. A decisão arquitetural central: **os operadores Vec3d*() e Vec4d*() herdados continuam retornando o vetor RGBA**, não o HSVA. Quem recebe um Hsv* e o converte com static_cast<const Vec4d*>(…) obtém RGBA — o mesmo vetor de qualquer outro Color:

```

1 // include/mirr/base/colors/Hsv.hpp
2 class Hsv : public Color {
3 public:
4     enum { HUE, SATURATION, VALUE }; // indices do vetor hsv
5

```

```

6   double hue()      const;  // [0.0, 360.0] graus na roda de cores
7   double saturation() const; // [0.0, 1.0] 0 = branco, 1 = cor pura
8   double value()    const;  // [0.0, 1.0] 0 = preto, 1 = maximo
9
10  // conversoes estaticas
11  static void hsv2rgb(Vec4d& rgb, const Vec4d& hsv);
12  static void rgb2hsv(Vec4d& hsv, const Vec4d& rgb);
13
14  // os operadores Vec3d*() e Vec4d*() HERDAM Color
15  // e retornam 'color' (RGBA), NAO 'hsv'
16
17  protected:
18      Vec4d hsv {}; // [HUE, SATURATION, VALUE, ALPHA]
19  };

```

Listing 96: Hsv: dois vetores internos; operadores retornam RGBA.

Hsva estende Hsv com um *slot* alpha: adicional e o método `colorInterpolate()`.

III.12.2 Configuração em EDL

Aqui a divergência entre nome de classe e nome de fábrica (Seção III.3.1) é sistemática: **todas** as cores concretas registram-se em minúsculas — "rgb", "rgba", "hsv", "hsva", "hls", "cie", "cmy", "yiq". A única em maiúscula é "Color", a classe base — que tem `EMPTY_SLOTTABLE` e por isso não aceita nenhum componente de cor. As cores são declaradas nos *slots* dos componentes que as consomem:

```

1  -- cor RGB: componentes diretos em [0.0, 1.0].
2  -- E ( rgb ), minusculo: 'Color' existe como nome de fabrica, mas e a classe
3  -- base, que tem EMPTY_SLOTTABLE e portanto nao aceita red/green/blue.
4  corFundo: ( rgb red: 0.2 green: 0.4 blue: 0.8 )
5
6  -- cor HSV: hue em graus, saturation e value em [0.0, 1.0]
7  corAlerta: ( hsv hue: 0.0 saturation: 1.0 value: 1.0 ) -- vermelho puro
8
9  -- cor HSVA: como HSV com canal alpha
10 corOverlay: ( hsva hue: 120.0 saturation: 0.8 value: 0.9 alpha: 0.6 )

```

Listing 97: Cores em EDL: RGB direto, HSV e HSVA com alpha.

Cada *setter* de Hsv (por exemplo, `setSlotHue`) chama `hsv2rgb(color, hsv)` automaticamente, mantendo o vetor `color` (RGBA) sempre sincronizado com o vetor `hsv`:

```

1  bool Hsv::setSlotHue(const Number* const msg)
2  {
3      if (msg == nullptr) return false;
4      double value{msg->getReal()};
5      bool ok{value >= 0 && value <= 360}; // a faixa e VERIFICADA
6      if (ok) {
7          hsv[HUE] = value;
8          hsv2rgb(color, hsv); // atualiza o vetor RGBA canonico
9      } else {
10         std::cerr << "Hsv::setHue: invalid entry(" << value
11             << " ), valid range: 0 to 360" << std::endl;
12     }
13     return ok; // false => o parser conta um erro
14 }

```

Listing 98: `setSlotHue()`: setter real que dispara a conversão automática (`src/base/colors/Hsv.cpp`).

O mesmo padrão aplica-se a `setSlotSaturation()`, `setSlotValue()` e `setSlotAlpha()`: toda modificação em qualquer componente HSV dispara `hsv2rgb()` e mantém `color` sempre consistente.

Armadilha — um *hue* fora da faixa é recusado, não normalizado

É tentador supor que o *framework* normalize o matiz, já que `hsv2rgb()` chama `aepcdDeg()` logo adiante. Não é o que acontece no *setter*: fora da faixa ele imprime a mensagem acima, devolve **false** e **não altera nada**. Esse **false** sobe até o *parser*, que incrementa `num_errors` (Seção III.3) — e a cor fica com o matiz anterior. Escrever `hue: -30` ou `hue: 400` não dá o mesmo resultado que `hue: 330` ou `hue: 40`.

III.12.3 Conversão HSV ↔ RGB

A conversão `hsv2rgb()` divide o círculo de matiz em seis setores de 60° cada, determinando quais componentes RGB atingem o máximo e calculando a interpolação linear dentro do setor:

```
1 void Hsv::hsv2rgb(Vec3d& rgb, const Vec3d& hsv)
2 {
3     double h { angle::aepcdDeg(hsv[HUE]) }; // normaliza para [0, 360)
4     if (h < 0.0) h += 360.0;
5     const double s { hsv[SATURATION] };
6     const double v { hsv[VALUE] };
7
8     if (s == 0.0) {
9         // saturacao zero: cinza de intensidade v
10        rgb[RED] = rgb[GREEN] = rgb[BLUE] = v;
11        return;
12    }
13
14    h /= 60.0; // setor [0..6)
15    const auto i { static_cast<int>(h) };
16    const double f { h - i }; // fracao dentro do setor
17    const double p { v * (1.0 - s) }; // projecao minima
18    const double q { v * (1.0 - s * f) }; // descida suave
19    const double t { v * (1.0 - s * (1.0 - f)) }; // subida suave
20
21    // i=0: vermelho->amarelo i=1: amarelo->verde i=2: verde->ciano
22    // i=3: ciano->azul i=4: azul->magenta i=5: magenta->vermelho
23    switch (i) {
24        case 0: rgb[RED]=v; rgb[GREEN]=t; rgb[BLUE]=p; break;
25        case 1: rgb[RED]=q; rgb[GREEN]=v; rgb[BLUE]=p; break;
26        case 2: rgb[RED]=p; rgb[GREEN]=v; rgb[BLUE]=t; break;
27        case 3: rgb[RED]=p; rgb[GREEN]=q; rgb[BLUE]=v; break;
28        case 4: rgb[RED]=t; rgb[GREEN]=p; rgb[BLUE]=v; break;
29        case 5: rgb[RED]=v; rgb[GREEN]=p; rgb[BLUE]=q; break;
30    }
31 }
```

Listing 99: `hsv2rgb()` real: seis setores de 60° na roda de cores (`src/base/colors/Hsv.cpp`).

O uso de `aepcdDeg()` no início normaliza qualquer ângulo de matiz para $[-180^\circ, +180^\circ]$ antes de somar 360° se negativo, garantindo que hues fora do intervalo $[0^\circ, 360^\circ]$ sejam tratados corretamente sem erros de índice no `switch`.

III.12.4 `Hsva::colorInterpolate`: mapas de cor

`Hsva::colorInterpolate()` interpola linearmente entre duas cores HSV, dado um valor escalar e seus limites. É o bloco de construção de mapas de cor — por exemplo, mapear altitude para uma graduação de azul (baixo) a vermelho (alto):

```
1 bool Hsva::colorInterpolate(
2     const double value,
```

```

3   const double minValue,
4   const double maxValue,
5   const Hsva& minColor, // cor no limite inferior
6   const Hsva& maxColor // cor no limite superior
7   )
8   {
9       // fracao de interpolacao: 0.0 = minColor, 1.0 = maxColor
10      const double p { (value - minValue) / (maxValue - minValue) };
11
12      Vec4d deltaColor { maxColor.hsv - minColor.hsv };
13
14      // normaliza a diferenca de hue para [-180, +180]
15      // evita o caminho longo ao cruzar 0/360 graus
16      deltaColor[Hsv::HUE] = angle::aepcdDeg(deltaColor[Hsv::HUE]);
17
18      Vec4d newColor { minColor.hsv + deltaColor * p };
19      newColor[Hsv::HUE] = angle::aepcdDeg(newColor[Hsv::HUE]);
20      if (newColor[Hsv::HUE] < 0.0) newColor[Hsv::HUE] += 360.0;
21
22      setHsva(newColor); // atualiza hsv e dispara hsv2rgb()
23      return true;
24  }

```

Listing 100: `colorInterpolate()` real: interpolação linear em espaço HSV com tratamento de descontinuidade do hue (`src/base/colors/Hsva.cpp`).

O detalhe crítico é a normalização do *delta* de hue com `aepcdDeg()`: sem ela, uma interpolação de 350° (vermelho escuro) para 10° (vermelho claro) percorreria 340° pelo caminho longo (passando por azul, verde e amarelo) em vez dos 20° pelo caminho curto. Com a normalização, o delta é $-340^\circ \rightarrow 20^\circ$, e a interpolação é correta.

Exemplo de uso.

```

1  // cores nos extremos
2  const Hsva corMinima { 240.0, 1.0, 1.0, 1.0 }; // hue=240 (azul)
3  const Hsva corMaxima { 0.0, 1.0, 1.0, 1.0 }; // hue=0 (vermelho)
4
5  // altitude atual e limites
6  const double alt { getAltitude() };
7  const double altMin { 0.0 };
8  const double altMax { 10000.0 };
9
10 // cor interpolada para a altitude atual
11 Hsva corAtual;
12 corAtual.colorInterpolate(alt, altMin, altMax, corMinima, corMaxima);
13
14 // obter o RGBA resultante para passar ao renderizador
15 const Vec4d* rgba { static_cast<const Vec4d*>(&corAtual) };
16 // rgba[0]=R, rgba[1]=G, rgba[2]=B, rgba[3]=A

```

Listing 101: Mapa de cor altitude → HSV: azul (baixo) a vermelho (alto).

III.13 Utilitários e arquivos de plataforma

`base/util` reúne o apoio transversal da camada: `safe_ptr` (ponteiro inteligente thread-safe), `Timer/UpTimer/DownTimer` (cronômetros globais), primitivas atômicas, funções matemáticas e de *string*, e as funções de navegação da Seção III.7. Fecha-se também aqui a questão dos arquivos pareados por plataforma.

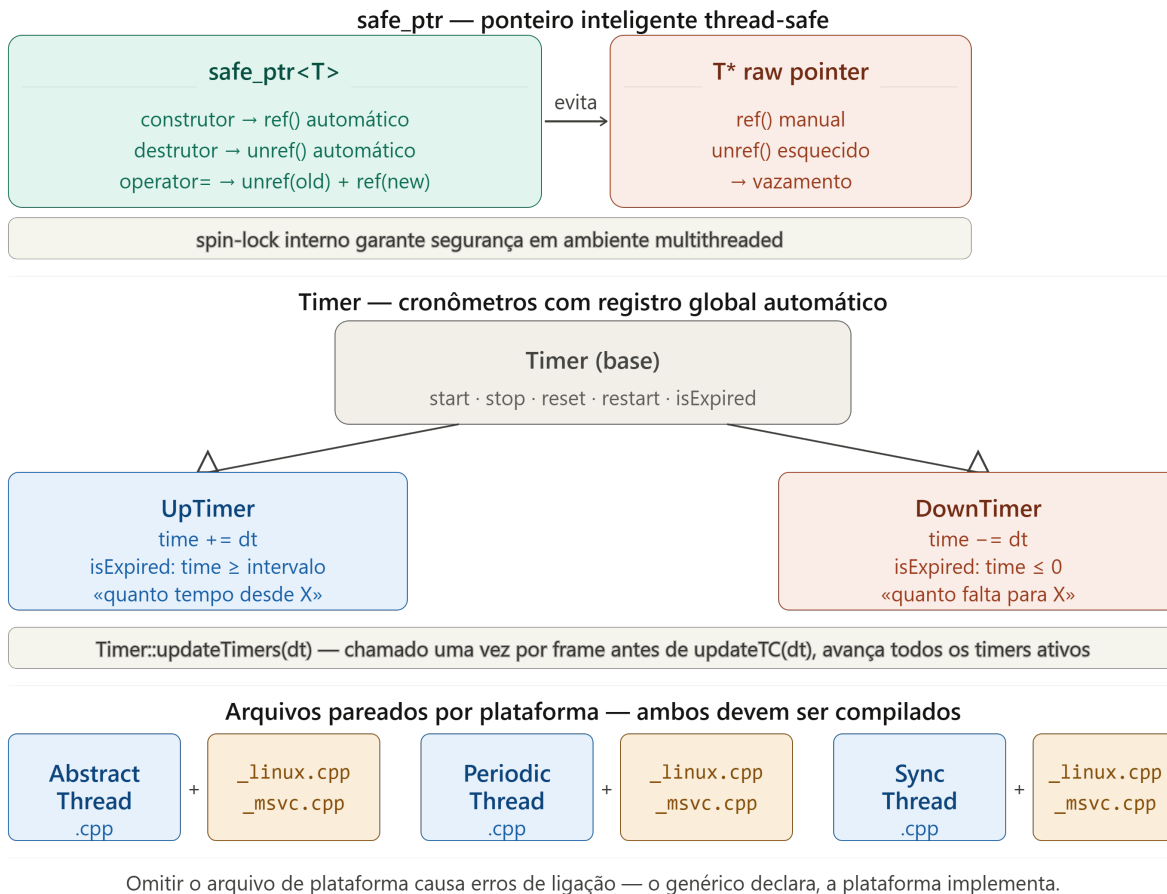


Figure 18: `safe_ptr` automatiza `ref/unref` (topo); hierarquia `Timer/UpTimer/DownTimer` com `updateTimers()` global (centro); arquivos genérico + plataforma que devem ser compilados juntos (baixo).

III.13.1 `safe_ptr`: referências duráveis sem vazamento

O idioma de referência manual (`ref()`/`unref()`) funciona bem em *slots* e passagens de parâmetro, mas é frágil para referências duráveis — membros de classe que apontam para outros objetos. `safe_ptr<T>` é a solução: um ponteiro inteligente que automatiza o ciclo de vida e usa um *spin-lock* para segurança em ambiente *multithreaded*:

```

1  template <class T>
2  class safe_ptr {
3  public:
4      // construtor: chama ref() por padrao (refThis=true)
5      safe_ptr(T* x, const bool refThis = true) : ptr(x) {
6          if (ptr != nullptr && refThis) ptr->ref();
7      }
8
9
10     // destrutor: chama unref() automaticamente
11     ~safe_ptr() {
12         if (ptr != nullptr) ptr->unref();
13     }
14
15     // operador=: unref() no antigo, ref() no novo, com spin-lock
16     safe_ptr<T>& operator=(T* x) {
17         if (ptr != x) {
18             lock();
19             if (ptr != nullptr) ptr->unref();
20             ptr = x;

```

```

21         if (ptr != nullptr) ptr->ref();
22         unlock();
23     }
24     return *this;
25 }
26
27 T* operator->()          { return ptr; }
28 operator T*()           { return ptr; }
29 bool operator==(const T* x) { return (ptr == x); }
30
31 private:
32     T* ptr          {}; // ponteiro gerenciado
33     mutable long semaphore {}; // spin-lock para thread-safety
34 };

```

Listing 102: `safe_ptr`: construção, atribuição e destruição automáticas (`include/mixr/base/safe_ptr.hpp`).

O método `getRefPtr()` retorna uma cópia já `ref()`-cada do ponteiro, para passagens entre *threads*: quem recebe torna-se temporariamente co-proprietário, evitando que o objeto seja destruído enquanto a outra *thread* ainda o usa.

Usos práticos.

```

1 // Padrao 1: membro de classe -- substitui T* + ref()/unref() manual
2 class MeuComponente : public base::Component {
3     base::safe_ptr<base::Table2> tabela; // gerenciado automaticamente
4 };
5
6 // Padrao 2: new sem ref() extra (refThis=false -- ref cnt permanece 1)
7 base::safe_ptr<base::Object> sp;
8 sp.set(new base::Object(), false); // nao chama ref(); sp assume a posse
9
10 // Padrao 3: passagem thread-safe
11 base::safe_ptr<Sensor> sensor { getSensor() }; // ref() no construtor
12 // ... usa sensor em outra thread ...
13 // destrutor chama unref() automaticamente ao sair do escopo

```

Listing 103: Três padrões de uso de `safe_ptr` no MIXR.

III.13.2 Timer: cronômetros registrados globalmente

`Timer` é um cronômetro que se auto-registra em um array estático global ao ser construído e se remove ao ser destruído. Uma única chamada estática `Timer::updateTimers(dt)` percorre o array e avança todos os cronômetros ativos — nenhum componente precisa chamar `update()` no seu próprio cronômetro.

```

1 class Timer : public Object {
2 public:
3     // criacao: registra automaticamente no array estatico
4     // destruicao: remove automaticamente
5
6     void start(); // inicia a contagem
7     void stop(); // pausa sem resetar
8     void reset(); // volta ao valor inicial
9     void restart(); // reset() + start()
10
11     double getCurrentTime() const; // tempo corrente
12     double getTimerValue() const; // intervalo configurado
13     bool isRunning() const; // esta contando?
14     bool isExpired() const; // atingiu o limite?
15
16     bool alarm(const double atime); // dispara em atime
17     bool alarm() const; // verifica alarme
18
19     // CHAMADA GLOBAL: avanca todos os timers de uma vez
20     static void updateTimers(const double dt);

```

```

21
22 private:
23     double ctime {};           // tempo corrente
24     double timerValue {};      // intervalo (reset value)
25     bool active {};           // esta rodando?
26     Type dir { Type::DOWN };  // UP ou DOWN
27
28     static Timer* timers[];    // array estatico (max 500)
29     static unsigned int nTimers;
30     static long semaphore;     // protege o array
31 };

```

Listing 104: Timer: interface real com auto-registro global (resumo de `include/mixr/base/Timers.hpp`).

UpTimer e DownTimer. Duas especializações cobrem os dois padrões de uso:

```

1 // UpTimer: sobe de 0 ate o intervalo
2 // isExpired() verdadeiro quando time >= timerValue
3 // uso: "quanto tempo desde X?"
4 class UpTimer : public Timer {
5 public:
6     UpTimer(const double rtime = 0.0); // rtime = intervalo
7     // update(): time += dt (implementado pela plataforma)
8 };
9
10 // DownTimer: desce do intervalo ate 0
11 // isExpired() verdadeiro quando time <= 0
12 // uso: "quanto falta para X?"
13 class DownTimer : public Timer {
14 public:
15     DownTimer(const double rtime = 0.0); // rtime = contagem inicial
16     // update(): time -= dt
17 };

```

Listing 105: UpTimer e DownTimer: as duas especializações.

Integração com o laço de simulação. A chamada a `updateTimers(dt)` ocorre *antes* de `root->updateTC(dt)`, que por sua vez antecede `root->updateData(dt)`. Quando o `updateTC()` de qualquer componente executa, todos os Timers já refletem o estado do *frame* atual — o componente apenas consulta `isExpired()`:

```

1 // Em initData() ou reset():
2 reloadTimer = new base::DownTimer(2.5); // 2.5 segundos
3 reloadTimer->ref();
4 reloadTimer->start();
5
6 // Em updateTC(): -- updateTimers(dt) ja foi chamado antes
7 if (reloadTimer->isExpired()) {
8     executarRecarga();
9     reloadTimer->restart(); // reset + start
10 }

```

Listing 106: Padrão de uso de DownTimer em `updateTC()`.

A escolha de um array estático (em vez de uma lista dinâmica) elimina alocação durante o laço de tempo real. O limite padrão de `MIXR_CONFIG_MAX_INTERVAL_TIMERS` (500) é generoso e redefinível em tempo de compilação.

O arquivo `config.hpp`. Esse limite é um de oito parâmetros globais reunidos em `include/mixr/config.hpp`. Todos seguem o mesmo padrão — um `#ifndef` seguido de `#define`

— de modo que qualquer um pode ser redefinido pela linha de comando do compilador (`-DMIXR_CONFIG_MAX_TRACKS=400`) sem editar o arquivo. Como todos dimensionam arrays estáticos, aumentá-los é trocar memória por capacidade; a alocação continua fora do laço de tempo real.

Vale não confundir os dois maiores: `MAX_PLAYERS_OF_INTEREST` (4000) é o teto de alvos que um *gimbal* pode considerar num quadro, enquanto `RF_MAX_EMISSIONS` (800) limita as emissões RF em trânsito — números próximos em espírito, mas de ordens e finalidades diferentes.

III.13.3 Arquivos pareados por plataforma

O código dependente de sistema operacional é isolado em arquivos nomeados com sufixo de plataforma, que *devem ser compilados junto* com o arquivo genérico correspondente:

`OneShotThread` completa o trio de especializações de `AbstractThread`: executa `userFunc()` exatamente uma vez e encerra. É a abstração adequada para inicializações caras que não devem bloquear o laço de tempo real — carregar um banco de terreno, abrir uma conexão de rede — e, ao contrário das duas anteriores, não tem arquivo de plataforma próprio: todo o seu corpo cabe na classe genérica.

O arquivo genérico declara a classe e a interface; o arquivo de plataforma implementa os métodos que dependem do SO (`pthread_create` no Linux, `CreateThread` no Windows). Omitir o arquivo de plataforma causa erros de ligação — funções declaradas mas não definidas. A regra vale para qualquer componente do MIXR com código específico de plataforma.

A seleção de qual arquivo compilar, porém, merece uma ressalva importante sobre *este* repositório. Seria natural esperar que o `meson.build` escolhesse as fontes por `host_machine.system()`. Não é o que acontece: `src/base/meson.build` lista as fontes Linux **incondicionalmente**, em uma única lista plana ao lado de todas as demais:

```
1 source_files = [  
2     # ...  
3     './threads/AbstractThread.cpp',  
4     './threads/PeriodicThread.cpp',  
5     './threads/OneShotThread.cpp',  
6     './threads/SyncThread.cpp',  
7     './threads/platform/AbstractThread_linux.cpp', # <-- sempre Linux  
8     './threads/platform/PeriodicThread_linux.cpp',  
9     './threads/platform/SyncThread_linux.cpp',  
10    # ...  
11    './util/platform/system_linux.cpp',             # <-- idem  
12 ]
```

Listing 107: `src/base/meson.build` (trecho real): as fontes de plataforma aparecem sem qualquer condicional.

As contrapartidas Windows (`AbstractThread_msvc.cpp`, `PeriodicThread_msvc.cpp`, `SyncThread_msvc.cpp`, `system_msvc.cpp`, `system_mingw.cpp`) existem na árvore de fontes e os cabeçalhos correspondentes continuam sendo selecionados por pré-processador — `atomics.hpp` escolhe entre `atomics_linux.hpp`, `atomics_mingw.hpp` e `atomics_msvc.hpp` conforme a plataforma — mas **nenhuma delas está ligada ao *build***. Na prática, **este fork do MIXR compila apenas em Linux**; portar para Windows exigiria introduzir a seleção condicional no `meson.build`, não apenas escrever código. É um ponto a ter em conta ao planejar a integração.

Feita a ressalva, é esse isolamento que mantém todo o restante do MIXR independente de plataforma no nível do código-fonte: a simulação nunca vê `pthread_t` nem `HANDLE` — apenas a interface genérica de `AbstractThread`.

III.13.4 Onde ficam as constantes, e o que não foi coberto

Este manual já usou dezenas de nomes como `base::angle::D2RCC`, `base::PI` ou `base::nav::fll2bd()` sem dizer de onde vêm. O mapa é curto e vale memorizá-lo, porque é a diferença entre achar uma constante em segundos e reimplementá-la. Todos os cabeçalhos abaixo estão em `include/mixr/base/util/`:

Cabeçalho	O que traz
<code>constants.hpp</code>	PI, LIGHTSPEED, BOLTZMANN, ETHG, UNDEFINED_VALUE
<code>math_utils.hpp</code>	<code>alim()</code> , <code>sign()</code> , <code>nint()</code> , funções trigonométricas auxiliares
<code>nav_utils.hpp</code>	geodésia: <code>fll2bd()</code> , <code>convertLL2PosVec()</code> , <code>gll2bd()</code>
<code>navDR_utils.hpp</code>	<i>dead reckoning</i> : <code>deadReckoning()</code> e os dez códigos de modelo DR
<code>str_utils.hpp</code>	<code>utStrcpy()</code> , <code>utStrcat()</code> — envoltórios com limite de tamanho
<code>lfi.hpp</code>	interpolação linear de 1 a 5 dimensões (Seção III.11)
<code>atomics.hpp</code>	incremento/decremento atômicos — a base do <code>refCount</code>
<code>system_utils.hpp</code>	<code>getComputerTime()</code> , <code>msleep()</code> , prioridades de <i>thread</i>

Note que estes são **cabeçalhos de funções livres em *namespaces***, não classes: nada aqui herda de `Object`, nada tem *slots*, nada aparece em EDL. As unidades de medida, ao contrário, são classes (Seção III.5) — e é por isso que `base::angle::D2RCC` (uma constante) e `Degrees` (uma classe) coexistem sem conflito.

Fica, por fim, o registro honesto do que *não* está neste manual. A camada `base` tem seis classes que nenhuma outra parte do MIXR usa, e que portanto só interessam a quem for escrever código novo:

- **Statistic** — calculadora estatística incremental: some pontos com `sigma()` e leia média, variância, desvio padrão, RMS, máximo e mínimo. Útil para instrumentar um modelo sem guardar histórico.
- **Locus** — uma linha de pontos igualmente espaçados a partir de uma referência, numa dada direção e alcance. É a forma que um *radial* de terreno teria se alguém a usasse; nenhum consumidor existe no fonte.
- **Stack** e **safe_stack** — pilhas, a segunda com trava para uso entre *threads*. Esta última *tem* consumidores: **Antenna** e **IrSeeker** a usam como reserva de objetos de emissão livres, evitando alocação no laço de tempo crítico.
- **MonitorMetrics** (nome de fábrica `monitorMetrics`) — caracterização colorimétrica de um monitor, com curvas de luminância por canal e coordenadas CIE dos fósforos, para correção de cor fisicamente correta em cabines simuladas.
- **FileReader** — leitor de arquivos de **registro de comprimento fixo**, com *slots* `pathname`, `filename` e `recordLength`, e acesso posicional por número de registro.

Armadilha — existem dois `FileReader`, e é o nome de fábrica que os separa

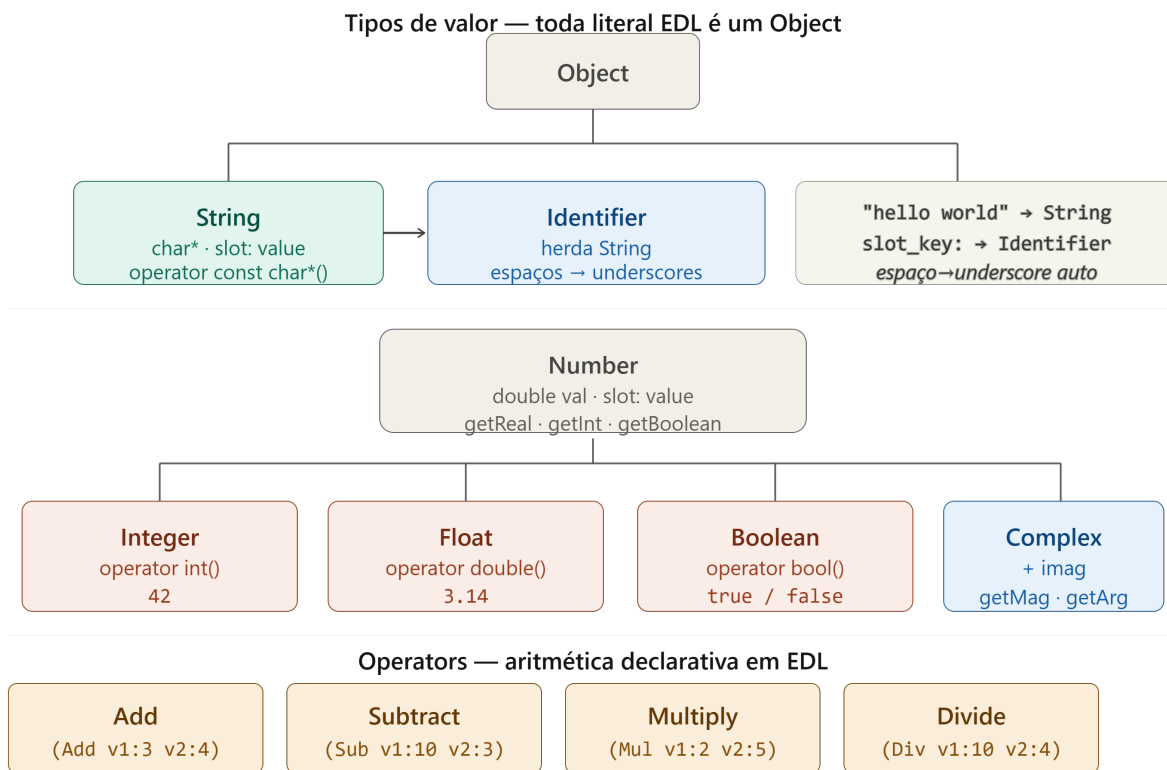
`base::FileReader` (acima) e `recorder::FileReader` (Seção IX) são classes **sem parentesco**: uma lê registros de tamanho fixo de um arquivo qualquer; a outra lê mensagens *protobuf* de um arquivo de gravação.

Em C++ os *namespaces* bastam para distingui-las. Em EDL não há *namespaces* — e a colisão foi resolvida exatamente pelo mecanismo da Seção III.3.1: a de `base` registra-se como "FileReader", e a de `recorder` como "RecorderFileReader". Escrever (`FileReader ...`) num arquivo de configuração cria a de `base`, mesmo dentro de um bloco de gravação.

Com isso, a camada `base` está coberta em sua totalidade. As seções seguintes sobem na pilha de dependências, a começar pelo motor de execução em `simulation`.

III.14 Tipos de valor: *strings*, números e operadores

Os literais que aparecem em um arquivo EDL — uma *string* entre aspas, um número inteiro, um booleano — não são tipos primitivos de C++: cada um se torna uma instância de uma subclasse de `Object`, com a mesma cidadania das demais (contada por referência, clonável, configurável por *slot*). Isso é necessário para que o despacho de `setSlotByIndex()` receba literais e objetos de forma uniforme.



Operators herdam Number — o resultado pode alimentar qualquer slot que aceita Number, sem código C++ adicional.

Figure 19: Hierarquia de tipos de valor: `String/Identifier` (texto), família `Number` com as três especializações que o *parser* instancia automaticamente, `Complex` e os `Operators` aritméticos declarativos.

III.14.1 String e Identifier

String herda de Object e gerencia um `char*` alocado no *heap*, com tamanho em uso (`n`) e tamanho alocado (`nn`) separados. A interface combina operações de manipulação pura com integração ao *framework*:

```
1  class String : public Object {
2  public:
3      // Manipulacao
4      virtual void setStr(const char* s);      // copia s para o buffer interno
5      virtual void catStr(const char* s);      // concatena s ao final
6      std::size_t len() const;                // comprimento atual
7      bool isEmpty() const;                  // len() == 0?
8
9      // Integracao com C++
10     operator char*();                       // uso onde C++ espera char*
11     operator const char*() const;           // idem, const
12
13     // Integracao numerica
14     bool isInteger() const;                  // "42" -> true
15     int getInteger() const;                  // retorna o valor inteiro
16     bool isNumber() const;                  // "3.14" -> true
17     double getNumber() const;
18
19     // Copia com largura e justificacao
20     void setString(const String& str,
21                   const std::size_t width,
22                   const Justify j = Justify::NONE); // LEFT RIGHT CENTER NONE
23
24 private:
25     char* str {}; // buffer
26     std::size_t n {}; // tamanho em uso
27     std::size_t nn {}; // tamanho alocado
28 };
```

Listing 108: String: interface completa com operadores de integração (resumo de `include/mixr/base/String.hpp`).

Identifier herda de String e acrescenta uma única regra: toda atribuição substitui espaços por *underscores*, tornando a *string* um identificador válido sem validação externa. É a implementação real de `setStr()` em `Identifier.cpp` que faz essa normalização:

```
1  void Identifier::setStr(const char* string)
2  {
3      if (string == nullptr || std::strlen(string) == 0) {
4          BaseClass::setStr(string);
5          return;
6      }
7
8      // copia substituindo espacos por underscores
9      const std::size_t len { std::strlen(string) };
10     const auto newStr { new char[len + 1] };
11     for (unsigned int i = 0; i < len; i++) {
12         newStr[i] = (string[i] == ' ') ? '_' : string[i];
13     }
14     newStr[len] = '\0';
15
16     BaseClass::setStr(newStr); // delega ao buffer de String
17     delete[] newStr;
18 }
```

Listing 109: `Identifier::setStr()`: normalização automática de espaços (`src/base/Identifier.cpp`).

A mesma normalização é aplicada em `catStr()`, garantindo que nunca haja espaço em um `Identifier`, independentemente de como foi construído. É por isso que `Identifier` é o tipo das chaves de `Pair` e dos nomes de *slot*.

III.14.2 Number e as especializações do parser

Number armazena um único `double` `val` e expõe uma interface de leitura uniforme: qualquer subclasse pode ser lida como `double`, `float`, `int`, `int64_t` ou `bool`, sem que o consumidor precise saber qual tipo foi declarado em EDL:

```
1 class Number : public Object {
2 public:
3     // Constructores: aceitam qualquer tipo numerico, convertem para double
4     Number(const double value) { val = value; }
5     Number(const float value) { val = static_cast<double>(value); }
6     Number(const int value) { val = static_cast<double>(value); }
7     Number(const int64_t value) { val = static_cast<double>(value); }
8     Number(const bool value) { val = (value ? 1.0 : 0.0); }
9
10    // Leitores uniformes -- funcionam em QUALQUER subclasse
11    double getReal() const { return val; }
12    float getFloat() const { return static_cast<float>(val); }
13    int getInt() const { return static_cast<int>(val); }
14    int64_t getInt64() const { return static_cast<int64_t>(val); }
15    bool getBoolean() const { return (val != 0.0); }
16
17    virtual void setValue(const double nv) { val = nv; }
18
19 protected:
20     double val {}; // valor canonico
21 };
```

Listing 110: Number: constructores e leitores — o `double` é o tipo canônico interno (resumo de `include/mixr/base/numeric/Number.hpp`).

O *parser* EDL instancia automaticamente a subclasse correta conforme o literal encontrado:

```
1 -- inteiro    -> Integer (operator int())
2 altitude: 10000
3
4 -- ponto flutuante -> Float (operator double())
5 ganho: 3.14
6
7 -- booleano    -> Boolean (operator bool())
8 ativo: true
9
10 -- qualquer slot "aceita Number" recebe qualquer dos tres acima
```

Listing 111: Literais EDL e as classes que o *parser* instancia.

Cada especialização acrescenta operadores para o tipo que representa. `Integer` e `Float` oferecem `+=`, `-=`, `*=` e `/=` (com proteção contra divisão por zero). `Boolean` oferece `!`, `&&` e `||`. O efeito é que, na maior parte do código consumidor, eles se comportam como tipos nativos ao mesmo tempo em que herdam toda a maquinaria de `Object`.

III.14.3 Complex

`Complex` estende `Number` com uma parte imaginária (`double imag`), exposta pelo *slot* `"imag"` em EDL:

```
1 class Complex : public Number {
2     // slot: "imag"
3 public:
4     Complex(const double r = 0.0, const double i = 0.0);
5
6     double getImag() const;           // parte imaginaria
7     double getMag() const;           // modulo: sqrt(r^2 + i^2)
```

```

8   double getArg() const;           // argumento: atan2(imag, real)
9
10  // aritmetica unaria
11  const Complex& operator+=(const Complex& z);
12  const Complex& operator-=(const Complex& z);
13  const Complex& operator*=(const Complex& z);
14  const Complex& operator/=(const Complex& z);
15
16  // aritmetica binaria (friend, inline em Complex.inl)
17  friend Complex operator+(const Complex& z1, const Complex& z2);
18  friend Complex operator*(const Complex& z1, const Complex& z2);
19  // ...
20 };

```

Listing 112: Complex: interface com módulo, argumento e aritmética (include/mixr/base/numeric/Complex.hpp).

```

1  impedancia: ( Complex value: 50.0 imag: 25.0 ) -- 50 + j25

```

Listing 113: Complex em EDL: parte real via "value", imaginária via "imag".

III.14.4 LatLon: coordenadas como se escrevem numa carta

Uma latitude tem duas representações naturais, e elas não se parecem: a que o código quer ($-23,1667$) e a que a carta imprime ($23^{\circ} 10' S$). LatLon existe para deixar o arquivo de configuração usar a segunda sem que nada no código precise saber disso.

A classe deriva de `Number` e tem quatro *slots*:

```

1  // A forma legível: 23 graus, 10 minutos, 12 segundos, Sul
2  latitude: ( LatLon direction: s degrees: 23 minutes: 10 seconds: 12.0 )
3  longitude: ( LatLon direction: w degrees: 46 minutes: 20 )
4
5  // Equivale exatamente a escrever, em graus decimais com sinal:
6  latitude: -23.17
7  longitude: -46.3333

```

Listing 114: LatLon: graus, minutos, segundos e hemisfério.

O mecanismo é o do slot de sempre: cada *setter* (`direction`, `degrees`, `minutes`, `seconds`) chama `computeVal()`, que consolida grau/minuto/segundo mais o hemisfério no campo `val` herdado de `Number` — em graus decimais, com sinal. A direção é um dos identificadores `n`, `s`, `e` ou `w`, e é ela que decide o sinal.

Por que — por que LatLon deriva de Number

Porque assim ela é aceita em **qualquer** *slot* que espere um `Number`. `Player::setSlotInitLatitude()` recebe um `base::Number*` e chama `getReal()` — não sabe, nem precisa saber, se o autor do cenário escreveu $-23,17$ ou a forma sexagesimal. É o mesmo princípio das famílias de unidades da Seção III.5: o tipo carrega a interpretação, e o consumidor recebe sempre o valor canônico.

III.14.5 Operators: aritmética declarativa em EDL

As classes `Add`, `Subtract`, `Multiply` e `Divide` (declaradas em `numeric/Operators.hpp`) tomam uma direção diferente: em vez de uma função C++ que calcula um valor, cada uma é um `Number` configurável por *slots*. Os operandos são fornecidos em EDL; o resultado — também um `Number` — pode alimentar o *slot* de outro objeto.

Duas particularidades precisam ser conhecidas antes de escrever a primeira expressão. A primeira é que os **nomes de fábrica são os próprios símbolos aritméticos**, não os nomes das classes C++:

```
1 IMPLEMENT_SUBCLASS(Add, "+")
2 IMPLEMENT_SUBCLASS(Subtract, "-")
3 IMPLEMENT_SUBCLASS(Multiply, "*")
4 IMPLEMENT_SUBCLASS(Divide, "/")
```

Listing 115: Os nomes de fábrica dos operadores (`src/base/numeric/Operators.cpp`).

Escrever (`Divide ...`) em EDL produz, portanto, o erro `undefined factory name: Divide` — a forma correta é (`/ ...`). Note também que `Subtract`, `Multiply` e `Divide` herdam de `Add` e apenas sobrescrevem `operation()`; toda a maquinaria de operandos vive em `Add`.

A segunda é a forma dos operandos. `Add` declara os *slots* `n2...n10` — o **primeiro** operando não é um deles: é o *slot value* herdado de `Number`. Um operador aceita, assim, de dois a dez operandos:

```
1 BEGIN_SLOTTABLE(Add)
2   "n2",      // 2nd number (first number is from Number)
3   "n3",      // 3rd number
4   // ... ate ...
5   "n10",     // 10th number
6 END_SLOTTABLE(Add)
```

Listing 116: A tabela de *slots* de `Add`: o 1º operando vem de `Number`, os demais são `n2...n10`.

Na prática, escreve-se tudo posicionalmente (Seção III.3.3): o índice global 1 é o *value* de `Number`, o 2 é `n2`, e assim por diante — o que dá à notação um aspecto agradavelmente prefixado:

```
1 -- velocidade = distancia / tempo, calculada em tempo de montagem da arvore
2 --   600.0 -> slot 1 ("value", de Number); 3.6 -> slot 2 ("n2", de Add)
3 velocidade: ( / 600.0 3.6 )
4 -- o slot "velocidade" recebe um Number com valor 600/3.6 = 166.67
5
6 -- a forma nomeada equivalente, mais verbosa:
7 velocidade: ( / value: 600.0 n2: 3.6 )
8
9 -- operadores aninham, e aceitam ate dez operandos
10 resultado: ( + ( * 2.0 3.0 ) -- 6.0
11              4.0 )          -- + 4.0
12 -- resultado = 10.0
13
14 -- soma de cinco parcelas numa unica forma
15 total: ( + 1.0 2.0 3.0 4.0 5.0 ) -- 15.0
```

Listing 117: Operadores como objetos: aritmética declarativa sem código C++.

O efeito é uma forma limitada de expressão aritmética declarativa: um parâmetro pode ser definido em função de outros, calculado no momento em que a árvore é montada, sem que nenhuma linha de C++ precise ser escrita ou recompilada — a mesma promessa de reconfiguração sem recompilação da Seção I.1, estendida ao valor dos próprios parâmetros.

III.15 Comunicação em rede

A camada **network** fornece a infraestrutura de comunicação usada pela interoperabilidade em rede (DIS, HLA): *sockets* UDP (unicast, *broadcast*, *multicast*) e TCP (cliente e servidor). A decisão de projeto repete um padrão já familiar: uma interface abstrata (**NetHandler**) define o contrato, uma implementação concreta (**PosixHandler**) cumpre esse contrato sobre *sockets* POSIX/Berkeley, e subclasses especializam o tipo de *socket*.

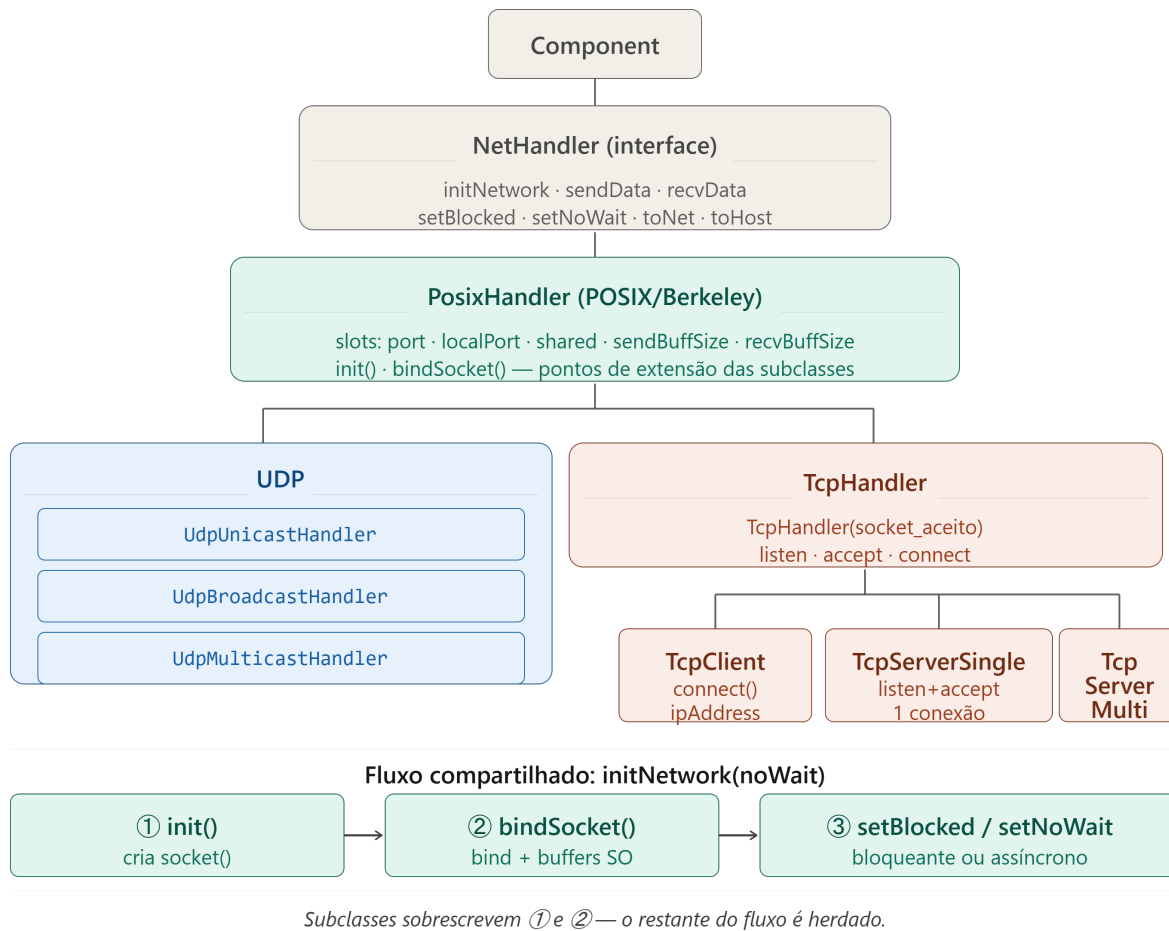


Figure 20: Hierarquia de rede: `NetHandler` define o contrato; `PosixHandler` implementa sobre POSIX; subclasses UDP e TCP especializam o tipo de *socket*. O fluxo de `initNetwork()` é compartilhado, com `init()` e `bindSocket()` como pontos de extensão.

III.15.1 NetHandler: o contrato

`NetHandler` herda de `Component` e define o contrato de transporte. Nem tudo ali é virtual puro — `initNetwork()` e `init()` têm implementação na base, e é a subclasse que as estende:

```

1 class NetHandler : public Component {
2 public:
3     // Inicializa o socket (noWaitFlag = modo nao-bloqueante)
4     virtual bool initNetwork(const bool noWaitFlag) = 0;
5
6     virtual bool isConnected() const = 0;
7     virtual bool closeConnection() = 0;
8
9     // Envia e recebe dados raw
10    virtual bool sendData(const char* const packet, const int size) = 0;
11    virtual unsigned int rcvData(char* const packet, const int maxSize) = 0;
12
13    // Alterna modo bloqueante / nao-bloqueante apos inicializacao
14    virtual bool setBlocked() = 0;
15    virtual bool setNoWait() = 0;
16
17    // Conversao de byte order em BLOCO (host <-> network, big-endian).
18    // ATENCAO ao contrato: o buffer tem de ser 'nl' palavras de 4 bytes
19    // SEGUIDAS de 'ns' palavras de 2 bytes. Devolvem void, nao bool.
20    static void toNet (const void* const hostData, void* const netData,
  
```



```

21         const int nl, const int ns);
22     static void toHost(const void* const netData, void* const hostData,
23                       const int nl, const int ns);
24
25     // Conversao campo a campo -- 16 sobrecargas (int16..uint64, float, double).
26     // Sao ESTAS que o DIS usa ao montar um PDU, e viram no-op quando o
27     // host ja e big-endian.
28     static void toNetOrder(int16_t* const, const int16_t);
29     // ... e as demais ...
30 };

```

Listing 118: NetHandler: interface mínima de qualquer transporte de rede (resumo de include/mixr/base/network/NetHandler.hpp).

Os utilitários de ordem de *bytes* encapsulam `htonl/htons` e seus inversos, e existem em duas formas com propósitos distintos. `toNet()` e `toHost()` convertem um **bloco** de uma vez, e por isso impõem um leiaute rígido: n_l palavras de 4 *bytes* seguidas de n_s palavras de 2. A família `toNetOrder()/fromNetOrder()`, com dezesseis sobrecargas, converte **campo a campo** — e é essa que o `interop/` de fato usa ao montar um PDU, porque um *Entity State* mistura `doubles`, `floats` e inteiros de vários tamanhos numa ordem que nenhum leiaute em bloco descreveria.

Por que — a conversão é um não-fazer-nada em máquinas *big-endian*

Todas essas funções consultam um *flag* estático, inicializado uma vez por `checkByteOrder()` — que simplesmente testa se `htons(1) == 1`. Num *host* que já seja *big-endian*, converter seria **inverter indevidamente**, e por isso as funções devolvem o valor intacto. É a razão de o código de serialização do DIS chamar `swapBytes()` sob a condição `isNotNetworkByteOrder()`, e não incondicionalmente (Seção X.4).

III.15.2 PosixHandler: implementação e fluxo de inicialização

`PosixHandler` implementa o contrato sobre a API POSIX/Berkeley. Os *slots* comuns a todas as especializações são:

```

1  ( UdpMulticastHandler      -- qualquer subclasse herda estes slots
2    port:      2010          -- porta de destino
3    localPort: 2011          -- porta local (necessaria para receber)
4    shared:    1             -- SO_REUSEADDR
5    sendBuffSizeKb: 32       -- buffer de envio (KB, padrao 32)
6    recvBuffSizeKb: 128      -- buffer de recepcao (KB, padrao 128)
7    localIpAddress: "192.168.1.10" -- interface especifica (opcional)
8    ignoreSourcePort: 2011   -- descarta o que vier desta porta de origem
9  )

```

Listing 119: `PosixHandler`: slots compartilhados por todas as subclasses.

Armadilha — `port` e `localPort` não são “destino” e “origem”

É a leitura natural, e está errada. Os dois *slots* mudam de papel conforme o *handler* esteja enviando ou recebendo:

	Enviando	Recebendo
<code>port</code>	porta de <i>destino</i>	porta em que se escuta
<code>localPort</code>	porta de <i>origem</i>	porta em que se escuta

E há o *fallback*: se `localPort` não for declarado, o `socket` liga-se a `port` — `if (getLocalPort())`

```
!= 0) addr.sin_port = htons(getLocalPort()); else addr.sin_port = htons(getPort());, no  
bindSocket() de cada subclasse UDP.
```

Só `port` é obrigatório. Declarar apenas ele dá um *handler* que envia e recebe na mesma porta — o que costuma ser o que se quer em *multicast*, e quase nunca o que se quer em *unicast* entre dois processos na mesma máquina.

`ignoreSourcePort` existe por causa exatamente desse último caso. Em *multicast* ou *broadcast*, tudo o que o processo envia volta para ele próprio, e a aplicação passa a receber os seus próprios pacotes como se fossem de outro simulador. O tratamento está dentro do laço de recepção do `PosixHandler`: se a porta de origem do datagrama recebido for igual a `ignoreSourcePort`, ele não é entregue — e o laço **tenta de novo**, em vez de devolver zero *bytes*. Descartar não é o mesmo que “nada chegou”.

Depois de cada recepção bem-sucedida, `getLastFromAddr()` e `getLastFromPort()` devolvem quem enviou o último pacote. É como uma aplicação distingue vários emissores no mesmo grupo *multicast* sem que o protocolo carregue essa informação.

Regra — um *slot* que existe em uma única subclasse: backlog

`backlog` não pertence ao `PosixHandler` — é declarado só em `TcpServerMultiple`, e é o segundo argumento de `listen()`: o número máximo de conexões pendentes na fila de aceitação. O padrão é **1**, o que significa que um servidor TCP MIXR recusa a segunda conexão simultânea a menos que se declare outra coisa.

O fluxo de `initNetwork()` é fixo e compartilhado, com dois pontos de extensão virtuais para as subclasses:

```
1  bool PosixHandler::initNetwork(const bool noWaitFlag)
2  {
3      // Ponto de extensao 1: cria o socket e configura opcoes especificas
4      bool ok { init() };
5
6      if (ok) {
7          // Ponto de extensao 2: bind() + configura buffers de SO
8          ok = bindSocket();
9
10         if (ok) {
11             // Modo bloqueante ou nao-bloqueante
12             if (noWaitFlag) ok = setNoWait();
13             else           ok = setBlocked();
14         }
15     }
16
17     initialized = ok;
18     return ok;
19 }
```

Listing 120: `PosixHandler::initNetwork()`: fluxo real com pontos de extensão (`src/base/network/PosixHandler.cpp`).

Cada subclasse concreta sobrescreve `init()` e `bindSocket()` para configurar o tipo de *socket* adequado (`SOCK_DGRAM` vs. `SOCK_STREAM`, opções de *multicast*, etc.) — o restante do fluxo é herdado sem modificação.

III.15.3 Os tipos de *socket* concretos

UDP. Três especializações cobrem os padrões de envio UDP:

```
1  -- Unicast: envia para um destino especifico
2  rede: ( UdpUnicastHandler
```

```

3 |     ipAddress: "192.168.1.20" -- destino
4 |     port: 3000
5 | )
6 |
7 | -- Broadcast: envia para toda a sub-rede.
8 | -- O SO_BROADCAST e ligado sempre, em init() -- nao ha slot para ele.
9 | -- O UNICO slot proprio da classe e networkMask, e ele e obrigatorio:
10 | -- e dele que sai o endereco de broadcast.
11 | rede: ( UdpBroadcastHandler
12 |     localIpAddress: "192.168.1.10"
13 |     networkMask: "255.255.255.0"
14 |     port: 3001
15 |     shared: 1
16 | )
17 |
18 | -- Multicast: modo preferido pelo DIS (grupos 224.x.x.x - 239.x.x.x)
19 | rede: ( UdpMulticastHandler
20 |     multicastGroup: "224.0.0.251" -- grupo
21 |     port: 2010
22 |     ttl: 4 -- max 4 roteadores
23 |     loopback: 1 -- receber o proprio envio (testes)
24 | )

```

Listing 121: Os três tipos de socket UDP em EDL.

`UdpMulticastHandler` acrescenta `joinTheGroup()` ao fluxo de `initNetwork()`: após a inicialização base, o *socket* se inscreve no grupo multicast via `IP_ADD_MEMBERSHIP`. O TTL controla quantos roteadores o pacote pode atravessar antes de ser descartado — valor 1 limita à sub-rede local.

TCP. `TcpHandler` é a base TCP e tem um construtor especial que recebe um *socket* já conectado (`TcpHandler(LcSocket sn)`): é assim que um servidor cria um *handler* para cada cliente aceito sem precisar inicializar do zero.

As três especializações TCP cobrem os papéis de cliente e servidor:

```

1 | // CLIENTE: inicia a conexao no initNetwork()
2 | // slot ipAddress = IP do servidor
3 | class TcpClient : public TcpHandler {
4 |     bool initNetwork(const bool noWait) override {
5 |         // ... BaseClass::initNetwork() ...
6 |         ::connect(socketNum, ...); // <-- diferencial do cliente
7 |     }
8 | };
9 |
10 | // SERVIDOR SIMPLES: aceita uma conexao, socket de escuta eh substituido
11 | class TcpServerSingle : public TcpHandler {
12 |     bool initNetwork(const bool noWait) override {
13 |         // ... listen() + accept() ...
14 |         // socket aceito substitui socketNum
15 |     }
16 | };
17 |
18 | // SERVIDOR MULTIPLO: mantem socket de escuta ativo, cria filho por cliente
19 | class TcpServerMultiple : public TcpHandler {
20 | public:
21 |     // Chame em loop para aceitar cada novo cliente
22 |     TcpHandler* acceptConnection(); // retorna TcpHandler pre-conectado
23 | };

```

Listing 122: Os três papéis TCP e como cada um inicializa.

O uso típico de `TcpServerMultiple` na camada *interop*:

```

1 | // setup: inicia o socket de escuta
2 | server->initNetwork(true); // noWait = true

```

```

3
4 // a cada frame ou em thread dedicada:
5 TcpHandler* client { server->acceptConnection() };
6 if ( client != nullptr ) {
7     // NAO chame ref() aqui: acceptConnection() faz 'new TcpHandler(...)',
8     // portanto o objeto ja vem com contador 1 e a posse e sua.
9     // Um ref() extra tornaria o unref() abaixo inocuo e vazaria o handler.
10
11     // usa o cliente -- sendData()/recvData()
12     // ...
13     client->unref(); // unico unref(): destroi quando terminar
14 }

```

Listing 123: Padrão de uso de `TcpServerMultiple`: servidor que aceita múltiplos clientes.

Esses *handlers* são consumidos diretamente pela camada `interop`: um `NetIO` de `DIS`, por exemplo, recebe um `UdpMulticastHandler` configurado via EDL e o usa para `recvData()/sendData()` de *Entity State PDUs* a cada *frame*.

III.16 Interfaces abstratas de E/S (*linkage*)

O `concepts/linkage` define, em três classes puramente abstratas, o contrato de entrada/saída entre a simulação e dispositivos de *hardware* — *joysticks*, controles de voo, painéis de *cockpit* físico. A decisão de projeto é separar **interface** de **implementação**: `base/concepts/linkage` define apenas o contrato; as implementações concretas residem no módulo `linkage`, construído sobre essas interfaces.

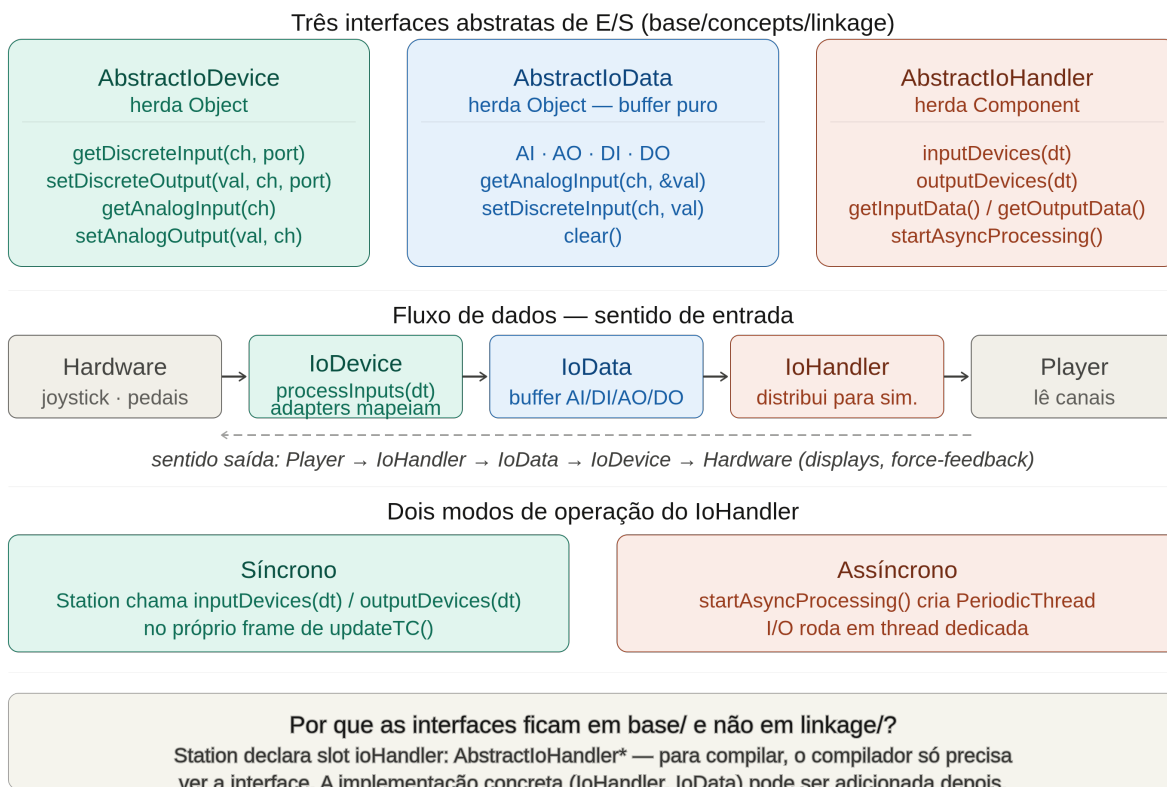


Figure 21: As três interfaces abstratas de E/S (topo), o fluxo de dados no sentido entrada e saída (centro) e os dois modos de operação do `IoHandler` — síncrono e assíncrono (baixo). A nota explica por que as interfaces ficam em `base/` e não em `linkage/`.

III.16.1 AbstractIoDevice: interface com o hardware

AbstractIoDevice herda de `Object` e define o contrato com o dispositivo físico em termos de quatro famílias de canais:

```
1 class AbstractIoDevice : public Object {
2 public:
3     virtual void reset() = 0;    // abre o dispositivo
4
5     // Entradas discretas (bool): botoes, switches
6     virtual int  getNumDiscreteInputChannels() const = 0;
7     virtual int  getNumDiscreteInputPorts()   const = 0;
8     virtual bool getDiscreteInput(bool* const value,
9                                     const int channel,
10                                    const int port) const = 0;
11
12     // Saidas discretas
13     virtual bool setDiscreteOutput(const bool value,
14                                    const int channel,
15                                    const int port) = 0;
16
17     // Entradas analogicas (double): eixos, pedais
18     virtual int  getNumAnalogInputs() const = 0;
19     virtual bool getAnalogInput(double* const value,
20                                 const int channel) const = 0;
21
22     // Saidas analogicas
23     virtual int  getNumAnalogOutputs() const = 0;
24     virtual bool setAnalogOutput(const double value,
25                                   const int channel) = 0;
26
27     // Leitura/escrita em batch (chamados pelo IoHandler)
28     void processInputs(const double dt,
29                        AbstractIoData* const inData);
30     void processOutputs(const double dt,
31                         const AbstractIoData* const outData);
32 };
```

Listing 124: AbstractIoDevice: interface completa com o hardware (resumo de `include/mixr/base/concepts/linkage/AbstractIoDevice.hpp`).

Canal e porta são endereçados por índice a partir de zero. O número de canais varia por dispositivo; `getNumAnalogInputs()` etc. permitem que o código consumidor itere sem conhecer o hardware específico.

III.16.2 AbstractIoData: o *buffer* de valores

AbstractIoData herda de `Object` e é um *buffer* puro — armazena os valores lidos/escritos, sem decidir como organizá-los internamente:

```
1 class AbstractIoData : public Object {
2 public:
3     // Numero de canais disponiveis
4     virtual int  getNumAnalogInputChannels() const = 0;
5     virtual int  getNumDiscreteInputChannels() const = 0;
6     virtual int  getNumAnalogOutputChannels() const = 0;
7     virtual int  getNumDiscreteOutputChannels()const = 0;
8
9     // Leitores e escritores por canal
10    virtual bool getAnalogInput( const int ch, double* const v) const = 0;
11    virtual bool getDiscreteInput(const int ch, bool* const v) const = 0;
12    virtual bool setAnalogInput( const int ch, const double v)      = 0;
13    virtual bool setDiscreteInput(const int ch, const bool v)       = 0;
14
15    virtual bool getAnalogOutput( const int ch, double* const v) const = 0;
```

```

16     virtual bool getDiscreteOutput(const int ch, bool* const v) const = 0;
17     virtual bool setAnalogOutput( const int ch, const double v)      = 0;
18     virtual bool setDiscreteOutput(const int ch, const bool  v)      = 0;
19
20     virtual void clear() = 0; // zera todos os canais
21 };

```

Listing 125: AbstractIoData: buffer com quatro famílias de canais (resumo de include/mixr/base/concepts/linkage/AbstractIoData.hpp).

A implementação concreta (IoData no módulo linkage) decide como armazenar esses canais — tipicamente, `std::vector<double>` para AI/AO e `std::vector<bool>` para DI/DO.

III.16.3 AbstractIoHandler: o orquestrador

AbstractIoHandler herda de `Component` e orquestra o conjunto: percorre a lista de dispositivos, chama `processInputs/processOutputs` em cada um e expõe os buffers para a simulação:

```

1  class AbstractIoHandler : public Component {
2  public:
3      // Leitura de todos os dispositivos registrados
4      // (ignorado se processamento assíncrono estiver ativo)
5      void inputDevices(const double dt);
6      void outputDevices(const double dt);
7
8      // Acesso ao buffer de entrada e saída
9      AbstractIoData* getInputData();
10     const AbstractIoData* getInputData() const;
11     AbstractIoData* getOutputData();
12     const AbstractIoData* getOutputData() const;
13
14     // Inicia processamento assíncrono (opcional)
15     // Cria uma PeriodicThread interna -- I/O roda desacoplado do frame
16     void startAsyncProcessing();
17
18 private:
19     virtual bool async() = 0;
20     virtual void inputDevicesImpl(const double dt) = 0;
21     virtual void outputDevicesImpl(const double dt) = 0;
22     virtual AbstractIoData* getInputDataImpl() = 0;
23     virtual AbstractIoData* getOutputDataImpl() = 0;
24     virtual void startAsyncProcessingImpl() = 0;
25 };

```

Listing 126: AbstractIoHandler: interface do orquestrador (resumo de include/mixr/base/concepts/linkage/AbstractIoHandler.hpp).

O padrão *Non-Virtual Interface* (NVI) é visível aqui: os métodos públicos (`inputDevices`, `getInputData` etc.) verificam o estado assíncrono antes de delegar para os métodos virtuais privados. Isso garante que, no modo assíncrono, chamadas síncronas vindas do laço principal sejam silenciosamente ignoradas — evitando condição de corrida sem que o chamador precise saber do modo.

Modo síncrono vs. assíncrono. No modo síncrono (padrão), `Station` chama `inputDevices(dt)` e `outputDevices(dt)` diretamente no seu `updateTC()`, dentro do *frame* de tempo crítico. No modo assíncrono, `startAsyncProcessing()` cria uma `PeriodicThread` interna (vide Seção III.8.2) que corre a uma taxa configurável, independente do *frame* de simulação — adequado para dispositivos de E/S com latência ou taxa própria.

Por que as interfaces ficam em `base/`? `Station` declara o `slot ioHandler:` do tipo `AbstractIoHandler*`. Para que `Station` compile, o compilador precisa apenas *ver* essa declaração de interface — a implementação concreta (`IoHandler`, `IoData`, `UsbJoystick`) pode residir no módulo `linkage` e ser adicionada depois, sem alterar nem recompilar `Station`. É a mesma separação interface/implementação que mantém a camada `base` compilável de forma autônoma.

No cenário da interceptação

Nada em `base` aparece por nome no arquivo de configuração — e é justamente esse o ponto. Quando o cenário do Seção II.6 declara

```
initLatitude: (LatLon direction: s degrees: 23 minutes: 30)
```

já entraram em ação: o *parser* EDL, que reconheceu a forma (Seção III.3); a fábrica, que criou um `LatLon` pelo nome (Seção III.3.1); o sistema de *slots*, que resolveu `direction`, `degrees` e `minutes` para índices globais (Seção III.1.5); a família `Number`, que entrega o resultado como graus decimais (Seção III.14.4); e a contagem de referências, que manterá o objeto vivo enquanto o *player* existir (Seção II.3). A árvore inteira que essas linhas constroem é uma árvore de `Component` (Seção III.2).

Ler `base` é aprender a ler todo o resto: as camadas acima não introduzem mecanismos novos, apenas classes novas sobre estes.

Grandeza	Referência	Subclasses	Domínio
Time	Seconds	MilliSeconds, MicroSeconds, NanoSeconds, Minutes, Hours, Days	temporização geral
Angle	Semicircles	Degrees, Radians	navegação, dinâmica
Distance	Meters	CentiMeters, MicroMeters, Microns, KiloMeters, Inches, Feet, NauticalMiles, StatuteMiles	altitude, alcance
Mass	KiloGrams	Grams, Slugs	dinâmica FPS
Force	Newtons	KiloNewtons, PoundForces, Poundals	empuxo
Power	Watts	MilliWatts, KiloWatts, Horsepower, DecibelWatts, DecibelMilliWatts	radar, RF
Frequency	Hertz	KiloHertz, MegaHertz, GigaHertz, TeraHertz	frequência de RF
Energy	Joules	KiloWattHours, BTUs, Calories, FootPounds	termodinâmica
Area	SquareMeters	SquareFeet, SquareInches, SquareYards, SquareMiles, SquareCentiMeters, SquareMilliMeters, SquareKiloMeters, DecibelSquareMeters	RCS em dBsm
Volume	CubicMeters	CubicFeet, CubicInches, Liters	tanques
<i>Sem famílias — classes isoladas derivadas diretamente de Number:</i>			
LinearVelocity	m/s	(composta: <i>slots distance</i> + time)	velocidade
AngularVelocity	rad/s	(composta: <i>slots angle</i> + time)	motores, antenas
Density	kg/m ³	(composta)	ar, combustível
FlowRate	m ³ /s	(composta)	consumo

Table 5: Famílias de unidades, referência interna e domínio de uso típico.

priority	Linux	Windows (HIGH_PRIORITY_CLASS)
1.0	SCHED_FIFO (max FIFO)	TIME_CRITICAL (15)
[0.9, 1.0)	SCHED_FIFO (90% FIFO)	HIGHEST (2)
[0.5, 0.6)	SCHED_FIFO (50% FIFO)	NORMAL (0)
0.0	SCHED_OTHER (normal)	IDLE (−15)

Table 6: Mapeamento da prioridade portátil [0, 1] para os agendadores de Linux e Windows.

Nível	Interface	Quando usar	Custo por chamada
Baixo	<code>lfi_1D–lfi_5D</code>	laços numéricos, sem EDL	$O(1)$ com cache, $O(n)$ sem
Alto	<code>Table1–Table5</code>	configuração via EDL	idem + overhead de <code>Object</code>

Table 7: Resumo dos dois níveis de LFI: interface, caso de uso e custo.

Classe	Espaço	Uso típico
Rgb / Rgba	RGB(A)	componentes diretos; Rgba acrescenta <i>alpha</i>
Hsv / Hsva	HSV(A)	escolha perceptual de matiz; mapas de cor
Hls	HLS	variante de HSV com <i>lightness</i>
Cmy	CMY	subtrativo, orientado a impressão
Cie	CIE	colorimetria absoluta, com <i>white point</i>
Yiq	YIQ	luminância/crominância (vídeo NTSC)

Table 8: As subclasses de Color em base/colors.

Constante (prefixo MIXR_CONFIG_)	Padrão	Dimensiona
MAX_INTERVAL_TIMERS	500	array estático de Timer
MAX_PLAYERS_OF_INTEREST	4000	Gimbal::MAX_PLAYERS e os arrays do Tdb
RF_MAX_EMISSIONS	800	filas de Emission em RfSystem
MAX_TRACKS	200	lista de Track do TrackManager
MAX_REPORTS	200	relatórios de detecção por quadro
MAX_NETIO_ENTITIES	5000	entidades de rede em NetIO
MAX_NETIO_ENTITY_TYPES	1000	tabela de tipos do EntityManager
MAX_NETIO_NEW_OUTGOING	150	novos <i>players</i> publicados por quadro

Table 9: Os parâmetros globais de include/mixr/config.hpp.

Arquivo genérico	Linux (compilado)	Windows (não compilado)
AbstractThread.cpp	AbstractThread_linux.cpp	AbstractThread_msvc.cpp
PeriodicThread.cpp	PeriodicThread_linux.cpp	PeriodicThread_msvc.cpp
SyncThread.cpp	SyncThread_linux.cpp	SyncThread_msvc.cpp
util/system_utils.cpp	util/platform/system_linux.cpp	system_msvc.cpp
linkage/IODevice.cpp	linkage/platform/UsbJoystick_linux.cpp	UsbJoystick_msvc.cpp

Table 10: Arquivos pareados: o genérico declara a interface; o de plataforma implementa. Neste *fork*, apenas a coluna Linux entra no meson.build.

IV A camada `simulation`

IV.1 Visão geral: a espinha dorsal de execução

A camada `base` fornece as ferramentas; a camada `models` define *o quê* está sendo simulado. Entre as duas, `simulation` define *como* a simulação é executada: o laço de tempo, a sincronização entre *threads*, os pontos de extensão para rede, gravação de dados e geração de imagem. É uma biblioteca autônoma (`libmixr_simulation.so`) que depende apenas de `mixr_base` — nenhuma das 13 classes deste módulo precisa saber o que é um `Player`, um `Radar` ou um `Aircraft`.

Os 14 arquivos de fonte (13 classes mais a fábrica) organizam-se em cinco grupos:

1. **Interfaces abstratas de infraestrutura** — `AbstractRecorderComponent`, `AbstractDataRecorder`, `AbstractIgHost`, `AbstractNetIO` e `AbstractNib`: os contratos que `Station` usa para conversar com gravadores, geradores de imagem e redes sem conhecer suas implementações concretas.
2. **Entidade simulada** — `AbstractPlayer`: a interface que toda entidade simulada deve implementar, vista por `Simulation` e pelas *threads* do *pool*.
3. **O par *executive*** — `Simulation` e `Station`: o núcleo de execução. `Simulation` contém os *players* e gerencia o tempo; `Station` contém `Simulation` e cria as *threads*.
4. **Threads concretas** — `StationTcPeriodicThread`, `StationBgPeriodicThread`, `StationNetPeriodicThread` (criadas por `Station`) e `SimulationTcSyncThread[]`, `SimulationBgSyncThread[]` (criadas por `Simulation`): materializam o paralelismo descrito na Seção II.5.
5. **Fábrica** — `factory.cpp`: registra todos os nomes de fábrica do módulo.

A relação entre os dois componentes centrais é de contenção: `Station` contém `Simulation` no *slot* `simulation`: e é o *container* raiz de toda a árvore. `Simulation`, por sua vez, contém a lista de *players* (acessível via `getPlayers()`) e o estado global de execução (tempo, *freeze*, área de jogo).

O fluxo de controle a cada *frame* começa na `StationTcPeriodicThread`, que acorda na taxa configurada em `tcRate`: (padrão 50 Hz) e chama `Station::processTimeCriticalTasks(dt)`. Este método itera `fastForwardRate` vezes sobre `tcFrame(dt)`, que por sua vez aciona o laço de tempo crítico de toda a árvore. O Quadro 11 resume as *threads* e suas responsabilidades.

Thread	Criada por	Chama
<code>StationTcPeriodicThread</code>	<code>Station</code>	<code>processTimeCriticalTasks(dt)</code>
<code>StationBgPeriodicThread</code>	<code>Station</code>	<code>processBackgroundTasks(dt)</code>
<code>StationNetPeriodicThread</code>	<code>Station</code>	<code>processNetworkTasks(dt)</code>
<code>SimulationTcSyncThread[]</code>	<code>Simulation</code>	<code>updateTC(dt)</code> (fatia de players)
<code>SimulationBgSyncThread[]</code>	<code>Simulation</code>	<code>updateData(dt)</code> (fatia de players)

Table 11: As cinco *threads* do módulo `simulation` e suas responsabilidades.

O restante desta seção segue a ordem de dependência: primeiro as interfaces abstratas de infraestrutura (Seção IV.2), depois `AbstractPlayer` (Seção IV.3), o par *executive* (Seções IV.4 e IV.5), as *threads* concretas (Seção IV.6) e a fábrica (Seção IV.7).

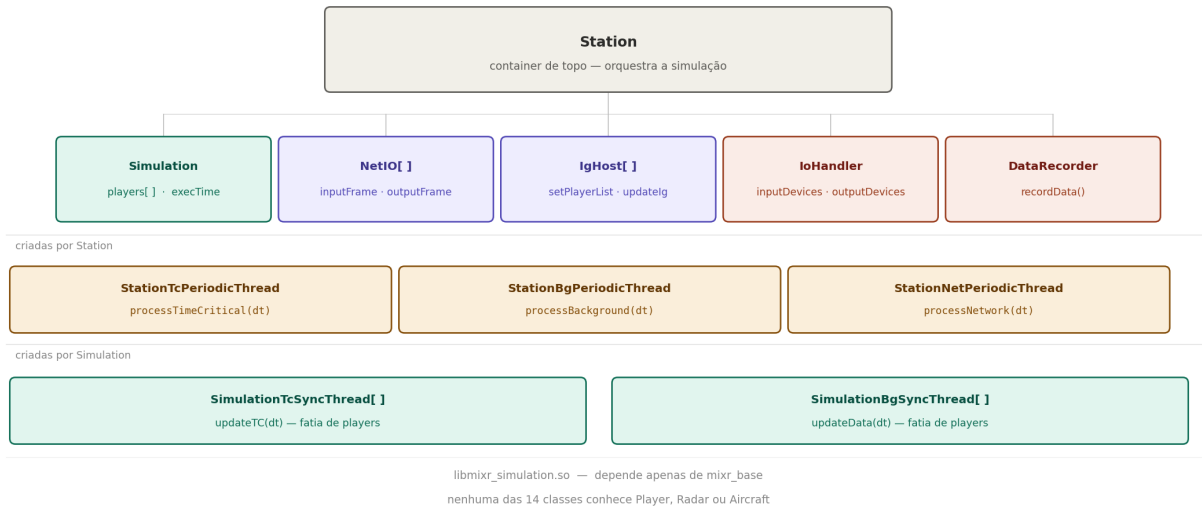


Figure 22: Arquitetura do módulo `simulation`: `Station` no topo com seus cinco *slots* principais (esquerda); as *threads* periódicas criadas por `Station` e as *threads* síncronas criadas por `Simulation` (direita). Setas tracejadas indicam qual componente cria cada *thread*. Os cinco grupos do módulo aparecem na faixa inferior.

IV.2 Interfaces abstratas de infraestrutura

`Station` precisa de um protocolo fixo para conversar com subsistemas que ela não conhece concretamente — um gravador de dados, um gerador de imagem, uma rede DIS ou HLA. A decisão é a mesma de `base/concepts/linkage` (Seção III.16): definir a interface aqui, e deixar a implementação para módulos posteriores (`recorder`, `interop`, e a aplicação do usuário no caso de `AbstractIgHost`).

IV.2.1 `AbstractRecorderComponent` e `AbstractDataRecorder`

Em uma simulação com centenas de eventos por segundo, gravar tudo é proibitivo. `AbstractRecorderComponent` resolve isso com dois filtros mutuamente exclusivos, configuráveis por *slot*:

```

1 class AbstractRecorderComponent : public base::Component {
2     // slot "enabledList": grava APENAS os IDs listados
3     // slot "disabledList": grava TUDO EXCETO os IDs listados
4     // (os dois sao mutuamente exclusivos; enabledList tem prioridade)
5
6     bool isDataEnabled(const unsigned int id) const;
7
8 private:
9     unsigned int* enabledList {}; // IDs habilitados (nullptr = todos)
10    unsigned int numEnabled {};
11    unsigned int* disabledList {}; // IDs desabilitados (nullptr = nenhum)
12    unsigned int numDisabled {};
  
```

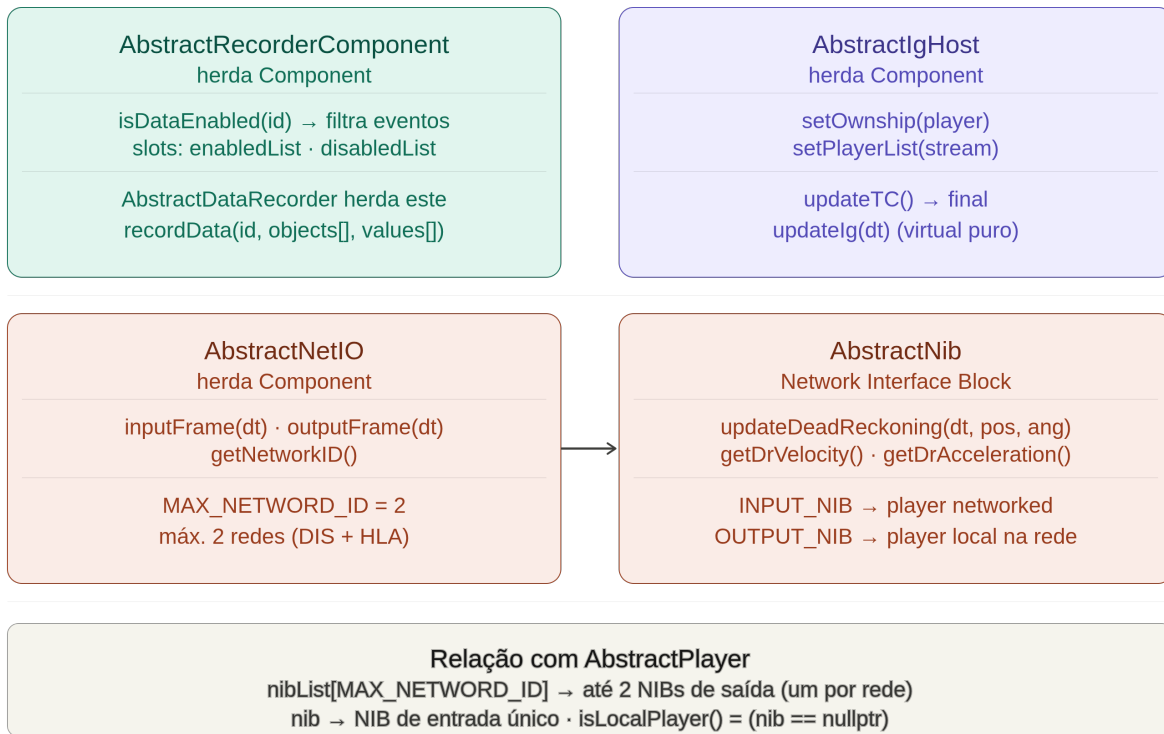


Figure 23: As quatro interfaces abstratas de infraestrutura. A seta de `AbstractNetIO` para `AbstractNib` indica que o `NetIO` é o criador e dono dos NIBs. A caixa inferior mostra como `AbstractPlayer` carrega até `MAX_NETWORK_ID` NIBs de saída e no máximo um NIB de entrada.

```

13 | };
14 |
15 | // Implementacao inline -- zero overhead quando evento esta filtrado
16 | inline bool AbstractRecorderComponent::isEnabled(const unsigned int id) const
17 | {
18 |     bool ok { true }; // padrao: habilitado
19 |     if (id != REID_END_OF_DATA) {
20 |         if (numEnabled > 0 && enabledList != nullptr) {
21 |             ok = false;
22 |             for (unsigned int i = 0; !ok && i < numEnabled; i++)
23 |                 ok = (id == enabledList[i]);
24 |         } else if (numDisabled > 0 && disabledList != nullptr) {
25 |             for (unsigned int i = 0; ok && i < numDisabled; i++)
26 |                 ok = (id != disabledList[i]);
27 |         }
28 |     }
29 |     return ok;
30 | }
  
```

Listing 127: `AbstractRecorderComponent`: filtros de evento e a função de consulta `isEnabled()` real — *inline* para custo zero quando o evento está desativado (resumo de `include/mixr/simulation/AbstractRecorderComponent.hpp`).

Os identificadores de evento são constantes `REID_*` definidas em `dataRecorderTokens.hpp` — `REID_PLAYER_DATA`, `REID_WEAPON_RELEASED`, `REID_TRACK_DATA` etc. Esse vocabulário é o contrato entre `models`, `simulation` e `recorder`.

`AbstractDataRecorder` herda `AbstractRecorderComponent` e define o ponto de entrada de gravação. O código de simulação nunca o chama diretamente — usa macros que expandem para uma chamada condi-

cionada:

```

1  class AbstractDataRecorder : public AbstractRecorderComponent {
2  public:
3      Station*    getStation();    // container Station (nao ref()'d)
4      Simulation* getSimulation();  // Simulation aninhada (nao ref()'d)
5
6      // Ponto de entrada de gravacao (chamado pelas macros)
7      bool recordData(
8          const unsigned int    id,          // REID_*
9          const base::Object*    pObjects[4], // players/objetos associados
10         const double           values[4]    // valores numericos associados
11     );
12
13     // Descarga de registros acumulados -- chamado no background por Station
14     virtual void processRecords();
15
16 protected:
17     // Implementacao real do despacho (sobrescrita por DataRecorder)
18     virtual bool recordDataImp(const unsigned int id,
19                               const base::Object* pObjects[4],
20                               const double values[4]);
21     virtual bool processUnhandledId(const unsigned int id) = 0;
22 };
23
24 // recordData() aplica o filtro antes de chamar a implementacao
25 inline bool AbstractDataRecorder::recordData(
26     const unsigned int id, const base::Object* pObjects[4], const double values[4])
27 {
28     bool recorded {};
29     if (isDataEnabled(id)) { // zero overhead se filtrado
30         recorded = recordDataImp(id, pObjects, values);
31         if (!recorded) processUnhandledId(id);
32     }
33     return recorded;
34 }

```

Listing 128: AbstractDataRecorder: interface e recordData() real (include/mixr/simulation/AbstractDataRecorder.hpp).

O par de macros BEGIN_RECORD_DATA_SAMPLE/ END_RECORD_DATA_SAMPLE expande para:

```

1  // Uso tipico em models/Player.cpp:
2  BEGIN_RECORD_DATA_SAMPLE( getWorldModel()->getDataRecorder(), REID_PLAYER_DATA )
3      SAMPLE_1_OBJECT( this )
4  END_RECORD_DATA_SAMPLE()
5
6  // Expande aproximadamente para:
7  {
8      AbstractDataRecorder* r { getWorldModel()->getDataRecorder() };
9      if (r != nullptr && r->isDataEnabled(REID_PLAYER_DATA)) {
10         const base::Object* objs[4] { this, nullptr, nullptr, nullptr };
11         const double vals[4] {};
12         r->recordData(REID_PLAYER_DATA, objs, vals);
13     }
14 }

```

Listing 129: Expansão real das macros de gravação — custo zero quando sem gravador ou evento filtrado (resumo de include/mixr/simulation/recorder_macros.hpp).

IV.2.2 AbstractIgHost

Interface para um *Image Generator* — um sistema externo de visualização 3D que recebe a lista de *players* e gera imagem. O contrato é mínimo e deliberadamente assim:

```

1 class AbstractIgHost : public base::Component {
2 public:
3     // Chamados por Station::updateTC() antes de tcFrame()
4     virtual void setOwnship(AbstractPlayer* const) = 0;
5     virtual void setPlayerList(base::PairStream* const) = 0;
6
7     // updateTC() e final: sempre chama BaseClass::updateTC(dt) ANTES de updateIg()
8     // Subclasses NAO podem sobrescrever updateTC() -- apenas updateIg()
9     void updateTC(const double dt = 0.0) final {
10         BaseClass::updateTC(dt); // inicializacao obrigatoria da base
11         updateIg(dt);           // ponto de extensao da subclasse
12     }
13
14 private:
15     virtual void updateIg(const double dt = 0.0) = 0; // unico ponto de extensao
16 };

```

Listing 130: AbstractIgHost: interface completa — apenas três métodos, com `updateTC()` marcado `final` (include/mixr/simulation/AbstractIgHost.hpp).

A decisão de marcar `updateTC()` como `final` é a garantia que `AbstractIgHost` oferece: não há como uma subclasse pular `BaseClass::updateTC(dt)` por engano — o compilador rejeita qualquer tentativa de sobrescrever `updateTC()` em classes derivadas.

IV.2.3 AbstractNetIO

A interface mínima que `Station` exige de qualquer sistema de rede:

```

1 class AbstractNetIO : public base::Component {
2 public:
3     // No maximo 2 redes simultaneas (dimensiona nibList em AbstractPlayer)
4     static const unsigned int MAX_NETWORK_ID { 2 };
5
6     // Chamados por StationNetPeriodicThread a cada frame de rede
7     virtual void inputFrame(const double dt) = 0; // processa entidades recebidas
8     virtual void outputFrame(const double dt) = 0; // envia estado dos players locais
9
10     virtual unsigned short getNetworkID() const = 0;
11 };

```

Listing 131: AbstractNetIO: contrato real com a constante `MAX_NETWORK_ID` (include/mixr/simulation/AbstractNetIO.hpp).

A separação entre `inputFrame` e `outputFrame` — em vez de um único `updateTC` — permite que cada protocolo escolha sua estratégia de *threading*: DIS pode processar entrada e saída em *threads* distintas; HLA pode exigir que ambas corram na mesma. `MAX_NETWORK_ID = 2` não é apenas um limite de `Station`: dimensiona o array `nibList` de cada `AbstractPlayer`, como veremos na Seção IV.3.

IV.2.4 AbstractNib: o bloco de interface de rede

NIB (*Network Interface Block*) representa a relação entre um *player* local e sua contraparte na rede. A distinção é direcional:

```

1 class AbstractNib : public base::Component {
2 public:
3     virtual unsigned short getPlayerID() const = 0;
4     virtual AbstractNetIO* getNetIO() = 0;
5 };

```

```

6 // Dead reckoning: extrapola posicao e orientacao entre updates de rede
7 virtual bool updateDeadReckoning(
8     const double dt, // delta de tempo (s)
9     base::Vec3d* const pNewPos, // posicao extrapolada (saida)
10    base::Vec3d* const pNewAngles // orientacao extrapolada (saida)
11 ) = 0;
12
13 // Vetores de estado no instante T0 (base do dead reckoning)
14 virtual const base::Vec3d& getDrVelocity() const = 0;
15 virtual const base::Vec3d& getDrAcceleration() const = 0;
16 virtual const base::Vec3d& getDrAngularVelocities() const = 0;
17 };

```

Listing 132: AbstractNib: interface com *dead reckoning* virtual puro (include/mixr/simulation/AbstractNib.hpp).

Um **NIB de entrada** (INPUT_NIB) representa uma entidade recebida da rede: o AbstractNetIO cria o NIB e um *player networked* correspondente, sem fisica local. Um **NIB de saída** (OUTPUT_NIB) representa um *player* local sendo anunciado para a rede.

updateDeadReckoning() é virtual puro: cada protocolo implementa o algoritmo que melhor lhe convém sobre as funções de nav_utils (Seção III.7). *Dead reckoning* é o que permite a uma entidade remota continuar se movendo de forma plausível entre dois *updates* de rede — evitando que cada *player* precise transmitir seu estado a 50 Hz.

IV.3 AbstractPlayer: a interface de toda entidade simulada

AbstractPlayer é o contrato mínimo que Simulation exige de qualquer entidade — aeronave, míssil, veículo terrestre. Não é instanciada diretamente: Player, em models (Seção VI.3), é a implementação concreta.

```

1 class AbstractPlayer : public base::Component {
2     // Slots proprios (apenas dois)
3     // slot "id": identificador unico [1..65535]
4     // slot "mode": modo inicial como string ("INACTIVE", "ACTIVE", "DEAD")
5 public:
6     // enum SIMPLES (nao 'enum class'): os valores sao usados sem qualificacao,
7     // como em isMode(ACTIVE) ou setMode(DELETE_REQUEST).
8     enum Mode {
9         INACTIVE, // existe na arvore, mas nao e atualizado nem transmitido
10        ACTIVE, // participa normalmente da simulacao
11        KILLED, // destruido (condicao de "morto")
12        CRASHED, // colidiu com o solo (condicao de "morto")
13        DETONATED, // arma detonou (condicao de "morto")
14        PRE_RELEASE, // arma criada mas ainda nao liberada (apenas flyouts)
15        LAUNCHED, // arma inicial ja lancada (apenas a arma "original")
16        DELETE_REQUEST // sinaliza intencao de sair da lista ativa
17    };
18
19    bool isLocalPlayer() const { return (nib == nullptr); }
20    bool isNetworkedPlayer() const { return (nib != nullptr); }
21
22    // NIBs de saida: um por rede (maximo MAX_NETWORK_ID = 2)
23    AbstractNib* getLocalNib(const unsigned int netId);
24
25    // NIB de entrada: no maximo um (identifica player networked)
26    AbstractNib* getNib() { return nib; }
27
28    void reset() override; // local: restaura initMode; networked: aguarda rede
29
30 private:
31     Mode mode {ACTIVE}; // ATENCAO: o padrao e ACTIVE, nao INACTIVE
32     Mode initMode {ACTIVE};

```

```

33 |
34 | // Array de NIBs de saída -- alocado no construtor
35 | AbstractNib** nibList {}; // nibList[MAX_NETWORK_ID]
36 | AbstractNib* nib {}; // NIB de entrada (nullptr = player local)
37 | unsigned int netID {}; // ID da rede do nib de entrada
38 | };

```

Listing 133: AbstractPlayer: slots, ciclo de vida e NIBs (resumo de include/mixr/simulation/AbstractPlayer.hpp).

Dois detalhes desse bloco costumam surpreender. O primeiro é que **Mode** é um **enum** simples, não um **enum class**: no código do *framework* os valores aparecem sem qualificação (`isMode(PRE_RELEASE)`, `setMode(DETONATED)`), e é assim que os demais capítulos deste manual os escrevem. O segundo é que o modo inicial padrão é **ACTIVE** — um *player* declarado em EDL sem o *slot mode* entra participando da simulação. Quem quiser um *player* dormente precisa declará-lo explicitamente.

O ciclo de vida. Os oito modos não formam uma cadeia única: há dois caminhos distintos, e a distinção entre **LAUNCHED** e **PRE_RELEASE** é exatamente a distinção entre a arma *original* (a que fica presa ao cabide) e o *flyout* (o clone que voa), detalhada na Seção VI.13.

Entidade comum: **INACTIVE** / **ACTIVE** $\xrightarrow{\text{dano}}$ **KILLED** / **CRASHED** $\rightarrow$ **DELETE_REQUEST**

Arma original: **ACTIVE** $\xrightarrow{\text{release()}}$ **LAUNCHED** *Flyout:* **PRE_RELEASE** $\xrightarrow{\text{fase 0}}$ **ACTIVE** $\xrightarrow{\text{impacto}}$
DETONATED $\rightarrow$ **DELETE_REQUEST**

DELETE_REQUEST é a única saída: um *player* nunca é destruído imediatamente (contagem de referências, Seção II.3). Ele sinaliza a intenção, e `Simulation::updatePlayerList()` o remove da lista ativa no próximo ciclo — garantindo que nenhuma referência pendente de outro *player* seja invalidada no meio de um quadro.

NIBs e a distinção local vs. networked. O array `nibList` é alocado no construtor com `MAX_NETWORK_ID = 2` posições e liberado em `deleteData()`. `copyData()` deliberadamente *não* copia esses ponteiros: NIBs são recriados pelo `AbstractNetIO` a cada `reset()`, pois representam relações de rede transitórias.

A pergunta `isLocalPlayer()` — decidida pela presença de um NIB de entrada (`nib != nullptr`) — governa *quem* atualiza posição e orientação a cada quadro: um *player* local é movido pelo seu `DynamicsModel`; um *player networked* é movido pelo *dead reckoning* do seu NIB. O mesmo flag governa `reset()`: locais reiniciam para `initMode`; *networked* aguardam o próximo *update* de rede.

IV.4 Simulation: o executive

Se **Station** organiza os subsistemas, **Simulation** governa *o tempo* e *a lista de players* — é onde os princípios de atualização determinística e faseada (Seção II) ganham implementação concreta em C++.

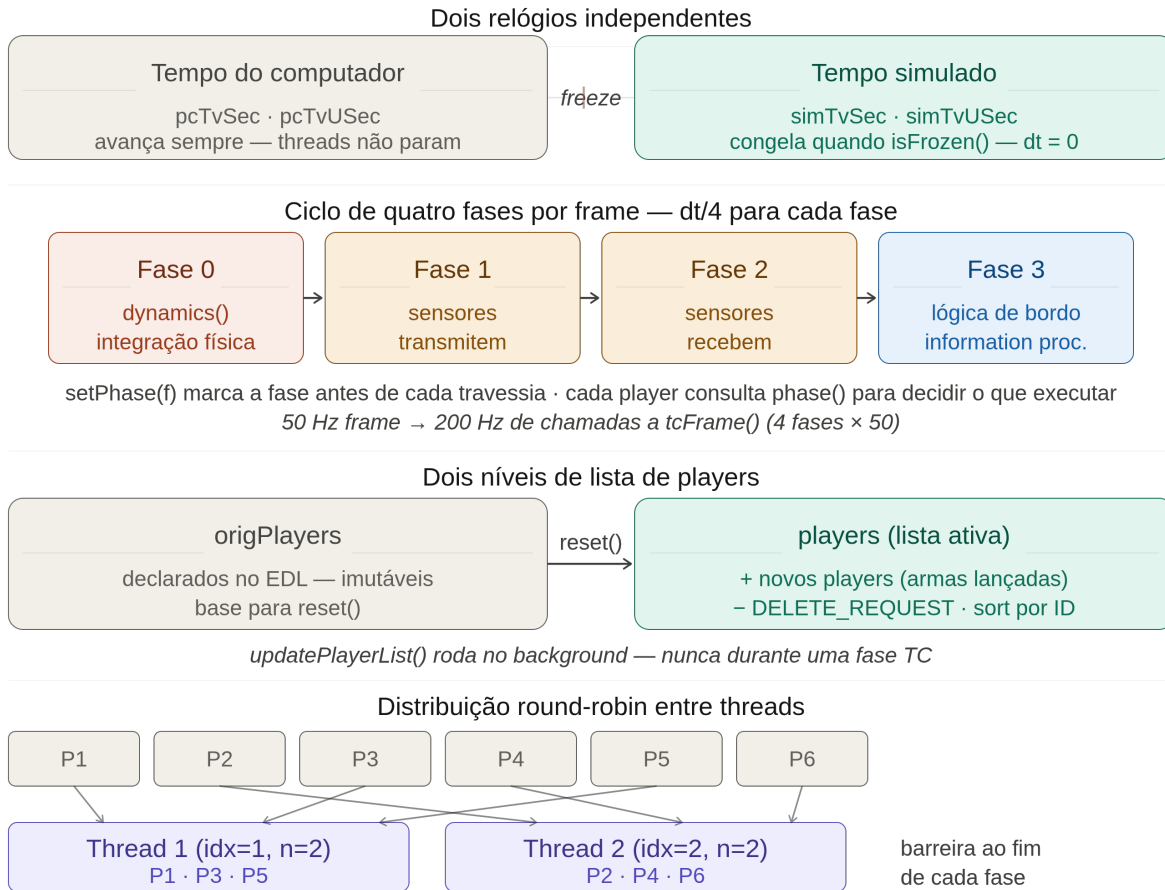


Figure 24: Simulation: os dois relógios independentes (topo), o ciclo de quatro fases por *frame* com a progressão de fase (centro), os dois níveis de lista de *players* e a relação entre *origPlayers* e *players* (baixo-esq), e a distribuição *round-robin* entre *threads* (baixo-dir).

IV.4.1 Dois relógios: computador e simulado

Simulation mantém dois pares de contadores de tempo separados — um para o computador, outro para o mundo simulado:

```

1 // Tempo do computador (UTC real -- nao e congelado)
2 unsigned long pcTvSec {}; // segundos desde 1970-01-01
3 unsigned long pcTvUSec {}; // microssegundos fracionarios
4
5 // Tempo simulado (pode ser congelado)
6 unsigned long simTvSec {};
7 unsigned long simTvUSec {};
8
9 // Inicio de updateTC():
10 double dt0 { dt };
11 if (isFrozen()) dt0 = 0.0; // dt0 = 0 congela a simulacao...
12
13 // ... mas o tempo do computador avanca normalmente:
14 // pcTvSec e pcTvUSec sao atualizados independentemente de isFrozen()
15
16 // O tempo simulado usa dt0 -- MAS so quando nao esta escravizado:
17 double deltaTime { dt0 };
18 if (simTimeSlaved) deltaTime = dt; // escravo => NAO congela
19
20 unsigned long newSimTvUSec {
21     simTvUSec + static_cast<unsigned long>(deltaTime * 1000000.0 + 0.5)

```

```

22 };
23 while (newSimTvUsec >= 1000000) {
24     newSimTvUsec -= 1000000;
25     simTvSec++;
26 }
27 simTvUsec = newSimTvUsec;

```

Listing 134: Os dois relógios em `Simulation`: campos reais e lógica de congelamento (`src/simulation/Simulation.cpp`).

A consequência desta separação: quando `isFrozen()` está ativo, `dt0 = 0` e nenhum *player* recebe avanço de tempo — a física para. Mas o relógio do computador e as *threads* de sincronização continuam correndo normalmente. A simulação pode ser pausada e retomada sem que os mecanismos de `SyncThread` (Seção III.8.3) precisem saber do congelamento.

Armadilha — por padrão o relógio *simulado* não congela

O parágrafo acima vale para os *players*, não para o relógio simulado. Quem decide é `simTimeSlaved`, e ele nasce `true`: com o relógio escravizado ao do computador, `deltaTime` recebe `dt` — e não `dt0` — de modo que `simTvSec/simTvUsec` continuam correndo mesmo com a simulação congelada.

Ele só passa a `false` se o arquivo EDL fixar um instante inicial, isto é, se declarar qualquer um dos *slots* `simulationTime`, `day`, `month` ou `year`; é o que faz a linha `simTimeSlaved = (simTime0 < 0) && (simDay0 == 0) && ...` no `reset()`. Só então `freeze()` para também o relógio simulado.

Na prática: uma `Simulation` declarada sem data nem hora congela a *física* e não congela o *relógio*. Quem grava dados com marca de tempo simulado precisa saber disso.

IV.4.2 O ciclo de quatro fases

A cada chamada de `updateTC(dt)`, `Simulation` percorre a lista de *players* **quatro vezes** — uma por fase. Cada fase passa `dt0/4` para os *players*, de modo que a soma das quatro fases é sempre `dt0`:

```

1  // Captura a lista de players para este frame (ref() implícito pelo safe_ptr)
2  base::safe_ptr<base::PairStream> currentPlayerList = players;
3
4  for (unsigned int f = 0; f < 4; f++) {
5
6      setPhase(f); // marca a fase -- players consultam phase() em tcFrame()
7
8      if (reqTcThreads == 1) {
9          // Thread unica: processa todos os players em sequencia
10         updateTcPlayerList(currentPlayerList, (dt0 / 4.0), 1, 1);
11
12     } else if (numTcThreads > 0) {
13         // Pool de threads: distribui round-robin
14         for (unsigned short i = 0; i < numTcThreads; i++) {
15             unsigned int idx { static_cast<unsigned int>(i + 1) };
16             tcThreads[i]—>start0(currentPlayerList, (dt0/4.0), idx, reqTcThreads);
17         }
18         // Thread principal processa o ultimo subconjunto
19         updateTcPlayerList(currentPlayerList, (dt0/4.0), reqTcThreads, reqTcThreads);
20
21         // Barreira: espera todas as threads concluirem antes da proxima fase
22         base::SyncThread** pp { reinterpret_cast<base::SyncThread*>(&tcThreads[0]) };
23         base::SyncThread::waitAllCompleted(pp, numTcThreads);
24     }
25 }
26
27 // Atualiza contadores de ciclo/frame
28 int cframe { static_cast<int>(frame() + 1) };
29 if (cframe >= 16) { incCycle(); cframe = 0; }
30 setFrame(cframe);

```

```
31 | setPhase(0); // fase volta a 0 apos o frame
```

Listing 135: Laço de quatro fases real de `Simulation::updateTC()` (`src/simulation/Simulation.cpp`).

As quatro fases têm papéis definidos — o mesmo ciclo descrito na Seção II.4, agora com o código que o implementa:

Fase	O que os players executam	Por que nesta ordem
0	<code>dynamics()</code> — integração física	posição atualizada antes dos sensores
1	sensores transmitem (emissões, interrogações)	transmitem após mover
2	sensores recebem (detecções, respostas)	recebem o que foi transmitido na fase 1
3	lógica de bordo (<i>information processing</i>)	decide com dados já atualizados

Table 12: As quatro fases de `Simulation::updateTC()` e seus papéis.

A frequência efetiva de chamada a `tcFrame()` é quatro vezes a taxa do *frame*: a 50 Hz, cada *player* recebe 200 chamadas de `tcFrame(dt/4)` por segundo. Quem consulta `phase()` para decidir o que executar é o `updateTC()` de cada componente. O exemplo canônico não está em `Player`, e sim em `System` — a base de todo subsistema embarcado — porque é ali que os quatro papéis de fase têm métodos dedicados:

```
1 void System::updateTC(const double dt0)
2 {
3     if (ownship == nullptr && getOwnship() == nullptr) return;
4
5     double dt{dt0};
6     if (isFrozen()) dt = 0.0;
7
8     // Delta time for methods that are running every fourth phase
9     double dt4{dt * 4.0}; // <-- recompoe o dt INTEGRAL do frame
10
11     WorldModel* sim{ownship->getWorldModel()};
12     if (sim == nullptr) return;
13
14     switch (sim->phase()) {
15         case 0 : dynamics(dt4); break; // Fase 0: fisica
16         case 1 : transmit(dt4); break; // Fase 1: emitir (RF, IR, laser)
17         case 2 : receive(dt4); break; // Fase 2: receber (ecos, interrogacoes)
18         case 3 : process(dt4); break; // Fase 3: logica de bordo
19     }
20
21     // Propaga aos subcomponentes com 'dt' (o dt/4 original) --
22     // cada um refaz a mesma composicao para a sua propria fase.
23     BaseClass::updateTC(dt);
24 }
```

Listing 136: `System::updateTC()` real: recomposição do Δt e despacho por fase (`src/models/system/System.cpp`).

Três observações fecham o mecanismo. Primeira, o `dt4 = dt * 4.0`: cada método de fase roda uma única vez por *frame* e por isso recebe o Δt integral, não um quarto dele (Seção II.4). Segunda, a consulta é a `getWorldModel()->phase()` — a `WorldModel` de `models` (Seção VI.2), que é a subclasse concreta de `Simulation` efetivamente instanciada. Terceira, os quatro nomes `dynamics()`, `transmit()`, `receive()` e `process()` são o vocabulário que se reencontra em toda a camada `models`: um radar implementa `transmit()` e `receive()`, um autopiloto implementa `process()` (Seção VI.10.1).

O `Player` em si tem o mesmo `switch`, porém com apenas o `case 0` preenchido — dinâmica, gravação de dados e atualização de assinaturas. Os `cases 1 a 3` são vazios: o trabalho das demais fases é feito pelos subsistemas filhos, alcançados pela chamada a `BaseClass::updateTC()` ao final (Seção VI.3.3).

Ciclos e frames. 16 *frames* formam um ciclo. Os contadores `frame()` (0–15) e `cycle()` são acessíveis por todos os *players* e usados para escalonar tarefas de baixa frequência — por exemplo, um radar que atualiza sua matriz de alvos a cada 4 *frames* verifica (`frame() % 4 == 0`).

IV.4.3 Dois níveis de lista de *players*

`Simulation` mantém duas listas de *players* com propósitos distintos:

```
1 // Lista original: declarada no EDL, nunca modificada durante a execucao
2 base::safe_ptr<base::PairStream> origPlayers;
3
4 // Lista ativa: copia sincronizada, modificada no background
5 base::safe_ptr<base::PairStream> players;
```

Listing 137: As duas listas de *players* em `Simulation`: campos reais (resumo de `include/mixr/simulation/Simulation.hpp`).

`origPlayers` é populada no `reset()` a partir do *slot* `players`: do EDL e nunca é modificada durante a execução.

Por que — duas listas em vez de uma lista com histórico

A separação resolve o problema do *reset* de graça. Reiniciar uma simulação significa desfazer tudo o que aconteceu: armas lançadas, entidades vindas da rede, *players* destruídos. Com uma lista só, isso exigiria registrar cada alteração para poder revertê-la.

Com duas, o *reset* é uma reconstrução: `players` é montada de novo a partir de `origPlayers`. Um míssil lançado no minuto anterior nunca esteve em `origPlayers` — e portanto **desaparece sem que exista nenhuma linha de código para removê-lo**. A lógica de limpeza não foi escrita porque não é necessária.

`players` é a lista ativa, sincronizada por `updatePlayerList()` no *background*:

```
1 void Simulation::updatePlayerList()
2 {
3     // Verifica se ha motivo para reconstruir a lista
4     bool yes { newPlayerQueue.isEmpty() }; // novos players?
5
6     if (!yes) {
7         // Verifica DELETE_REQUEST
8         base::safe_ptr<base::PairStream> pl = players;
9         base::List::Item* item { pl->getFirstItem() };
10        while (yes && item != nullptr) {
11            const auto pair = static_cast<base::Pair*>(item->getValue());
12            const auto p = static_cast<AbstractPlayer*>(pair->object());
13            yes = p->isMode(AbstractPlayer::DELETE_REQUEST);
14            item = item->getNext();
15        }
16    }
17
18    if (yes) {
19        // Reconstrói: copia origPlayers + networked players, descarta DELETE_REQUEST
20        // e insere novos players da fila -- em ordem crescente de ID
21        base::safe_ptr<base::PairStream> newList( new base::PairStream() );
22        // ... copia origPlayers ...
23        // ... copia players networked ativos ...
24        // ... insere newPlayerQueue ...
25        players = newList; // swap atomico via safe_ptr
26    }
27 }
```

Listing 138: `updatePlayerList()`: remove `DELETE_REQUEST`, adiciona novos e reordena (`src/simulation/Simulation.cpp`).

Regra — a lista é substituída, nunca editada

Este é o padrão que torna seguro modificar a lista de *players* enquanto a *thread* de tempo crítico a percorre, e ele tem duas metades que precisam existir juntas:

1. `updatePlayerList()` constrói uma **lista nova** e a instala com uma única atribuição a um `safe_ptr`. A lista antiga nunca é alterada no lugar.
2. `updateTC()` captura a lista **uma vez**, no início do *frame* (`safe_ptr<> currentPlayerList = players`), e segura essa referência pelas quatro fases.

O resultado é que uma troca ocorrida no meio do *frame* não é vista por ele: a *thread* de tempo crítico termina o quadro sobre a lista com que o começou, e só enxerga a nova no quadro seguinte. Um *player* nunca aparece ou some entre a fase 1 e a fase 2 — o que é exatamente a garantia de consistência que o modelo de fases da Seção II.4 exige.

É o mesmo mecanismo de `Component::processComponents()` (Seção III.2.1), aplicado à lista de *players* em vez da de componentes.

Adição de *players* em tempo de execução. Armas recém-lançadas (`Weapon::release()`) são inseridas na fila `newPlayerQueue` e incorporadas por `updatePlayerList()`. `addNewPlayer(name, player)` é o ponto de entrada:

```
1 // Sobrecarga 1: recebe o Pair já montado -- e a que enfileira de fato
2 bool Simulation::addNewPlayer(base::Pair* const player)
3 {
4     if (player == nullptr) return false;
5     player->ref();           // a fila torna-se co-proprietaria
6     newPlayerQueue.put(player); // safe_queue -- thread-safe
7     return true;
8 }
9
10 // Sobrecarga 2: conveniência -- monta o Pair a partir de um nome C
11 bool Simulation::addNewPlayer(const char* const playerName,
12                               AbstractPlayer* const player)
13 {
14     if (playerName == nullptr || player == nullptr) return false;
15     const auto pair = new base::Pair(playerName, player);
16     bool ok{addNewPlayer(pair)};
17     pair->unref();           // a fila já tem a sua referencia
18     return ok;
19 }
20 // Na proxima chamada a updatePlayerList() (background), o player aparece
21 // na lista ativa e começa a receber tcFrame() / updateData()
```

Listing 139: As duas sobrecargas reais de `addNewPlayer()` (`src/simulation/Simulation.cpp`).

O `ref()` na primeira sobrecarga e o `unref()` na segunda são o mesmo idioma de transferência de posse visto em `List` (Seção III.4.1): quem enfileira entrega a posse à fila e larga a sua própria referência.

Identificadores únicos. `Simulation` é a fonte de IDs únicos para correlação entre módulos:

```
1 // IDs para eventos gerais (explosões, impactos, etc.)
2 unsigned short getNewEventID() { return ++eventID; }
3
4 // IDs para eventos de arma (correlação weapon release <-> detonation)
5 unsigned short getNewWeaponEventID() { return ++eventWpnID; }
6
7 // IDs para armas lançadas (identificação única na lista de players)
8 unsigned short getNewReleasedWeaponID() { return ++relWpnId; }
```

Listing 140: Geração de IDs únicos: eventos, armas e lançamentos.

Esses IDs aparecem tanto nos *players* de *models* (para identificar eventos de combate) quanto nos registros do *recorder* e nos PDUs do *interop/dis* — garantindo correlação consistente entre os três módulos sem que nenhum deles precise coordenar entre si.

IV.4.4 Distribuição *round-robin* entre *threads*

O algoritmo de distribuição de *players* entre *threads* é ***round-robin* por índice**: a *thread* de índice *idx* (1-based) processa o 1^o , $(1 + n)^o$, $(1 + 2n)^o$, ... *players* — onde *n* é o número total de *threads*. Com 2 *threads* e 6 *players*: *thread* 1 processa P1, P3, P5; *thread* 2 processa P2, P4, P6.

```
1 void Simulation::updateTcPlayerList(  
2     base::PairStream* const playerList,  
3     const double dt,  
4     const unsigned int idx, // indice desta thread (1-based)  
5     const unsigned int n) // total de threads  
6 {  
7     unsigned int index { idx }; // proximo player a processar  
8     unsigned int count {};  
9  
10    base::List::Item* item { playerList->getFirstItem() };  
11    while (item != nullptr) {  
12        count++;  
13        if (count == index) {  
14            base::Pair* pair { static_cast<base::Pair*>(item->getValue()) };  
15            AbstractPlayer* ip { static_cast<AbstractPlayer*>(pair->object()) };  
16            ip->tcFrame(dt); // processa este player  
17            index += n; // pula para o proximo da fatia  
18        }  
19        item = item->getNext();  
20    }  
21 }
```

Listing 141: `updateTcPlayerList()`: distribuição *round-robin* real (`src/simulation/Simulation.cpp`).

A *thread* principal executa `updateTcPlayerList(..., reqTcThreads, reqTcThreads)` — o último subconjunto. As *threads* do *pool* executam os subconjuntos anteriores em paralelo. Ao final de cada fase, a *thread* principal espera todas com `SyncThread::waitAllCompleted()` (Seção III.8.3) — a barreira que garante que nenhuma fase comece antes que todas as *threads* tenham concluído a fase anterior.

Limite de *threads*. A regra é imposta no próprio *setter* do *slot*, e não em tempo de execução:

```
1 // Max threads is the number of processors assigned to this  
2 // process minus one, or minimum of one.  
3 const int np{base::AbstractThread::getNumProcessors()};  
4 int maxT{1};  
5 if (np > 1) maxT = np - 1;  
6  
7 const int v{msg->getInt()};  
8 if (v >= 1 && v <= maxT) {  
9     reqTcThreads = v;  
10    ok = true;  
11 } else {  
12     std::cerr << "Simulation::setSlotNumTcThreads(): invalid number of threads: " << v;  
13     std::cerr << "; number of processors = " << np;  
14     std::cerr << "; use [ 1 ... " << maxT << " ];" << std::endl;  
15 }
```

Listing 142: `Simulation::setSlotNumTcThreads()` (`src/simulation/Simulation.cpp`).

Armadilha — o número de *threads* depende da máquina que carrega o arquivo

O teto não é uma constante: é `getNumProcessors() - 1`, avaliado no momento em que o *slot* é preenchido. Um cenário com `numTcThreads: 8`, escrito e testado numa estação de 16 núcleos, **falha ao carregar** numa máquina de 4 — e a falha é a de qualquer *slot* rejeitado: o *parser* conta mais um erro, o valor fica no padrão de 1, e a simulação sobe monothread.

É por isso que vale conferir `num_errors` depois de carregar (Seção III.3): sem essa conferência, a diferença entre a máquina de desenvolvimento e a de execução aparece como uma queda de desempenho sem explicação.

Por que — por que reservar um núcleo

As *threads* de tempo crítico rodam em `SCHED_FIFO` quando a prioridade é maior que zero (Seção III.8.1), e uma *thread* `SCHED_FIFO` não é preemptada por nada de prioridade menor. Ocupar *todos* os núcleos com elas deixaria o sistema operacional — e as próprias *threads* de fundo e de rede do MIXR — sem onde rodar. O núcleo reservado não é folga: é o que mantém a máquina responsiva.

Com esse limite e a distribuição *round-robin*, o ganho é aproximadamente linear com o número de CPUs — até o ponto em que o custo de sincronização por fase supera o ganho, que depende da complexidade de cada *player*.

IV.5 Station: a raiz da simulação

`Station` é o `Component` de nível mais alto de toda simulação MIXR — a raiz da árvore que o EDL constrói. Ela não simula nada por si só; organiza e dá ritmo aos subsistemas que simulam: o *executive* (`Simulation`), as redes (`AbstractNetIO`), os geradores de imagem (`AbstractIgHost`), o *hardware* de E/S (`AbstractIoHandler`) e o gravador (`AbstractDataRecorder`).

IV.5.1 Configuração via EDL

`Station` expõe seus subsistemas e parâmetros de *threading* como *slots*:

```
1 ( Station
2   simulation:      ( Simulation ... )    -- executive obrigatorio
3   networks:       { net1: ( DisNetIO ... ) } -- lista de redes (opcional)
4   igHosts:        { ig1: ( MyIgHost ) }  -- geradores de imagem (opcional)
5   ioHandler:      ( MeuIoHandler ... )   -- handler de E/S da aplicacao (opcional)
6   dataRecorder:   ( DataRecorder ... )   -- gravador (opcional)
7
8   -- taxas das threads (Hz); 0 = sem thread, roda sincronamente
9   tcRate:         50                     -- thread TC a 50 Hz (padrao)
10  tcPriority:      0.9                    -- SCHED_FIFO no Linux
11  bgRate:         20                     -- thread background a 20 Hz
12  netRate:        50                     -- thread de rede a 50 Hz
13
14  fastForwardRate: 1                     -- 1 = tempo real; 4 = 4x mais rapido
15  ownship:        "ownship"              -- nome do player principal
16
17  enableUpdateTimers: true                -- chama Timer::updateTimers() no TC
18  startupResetTimer: ( Seconds 0.1 )     -- envia RESET_EVENT apos 0.1s
19 )
```

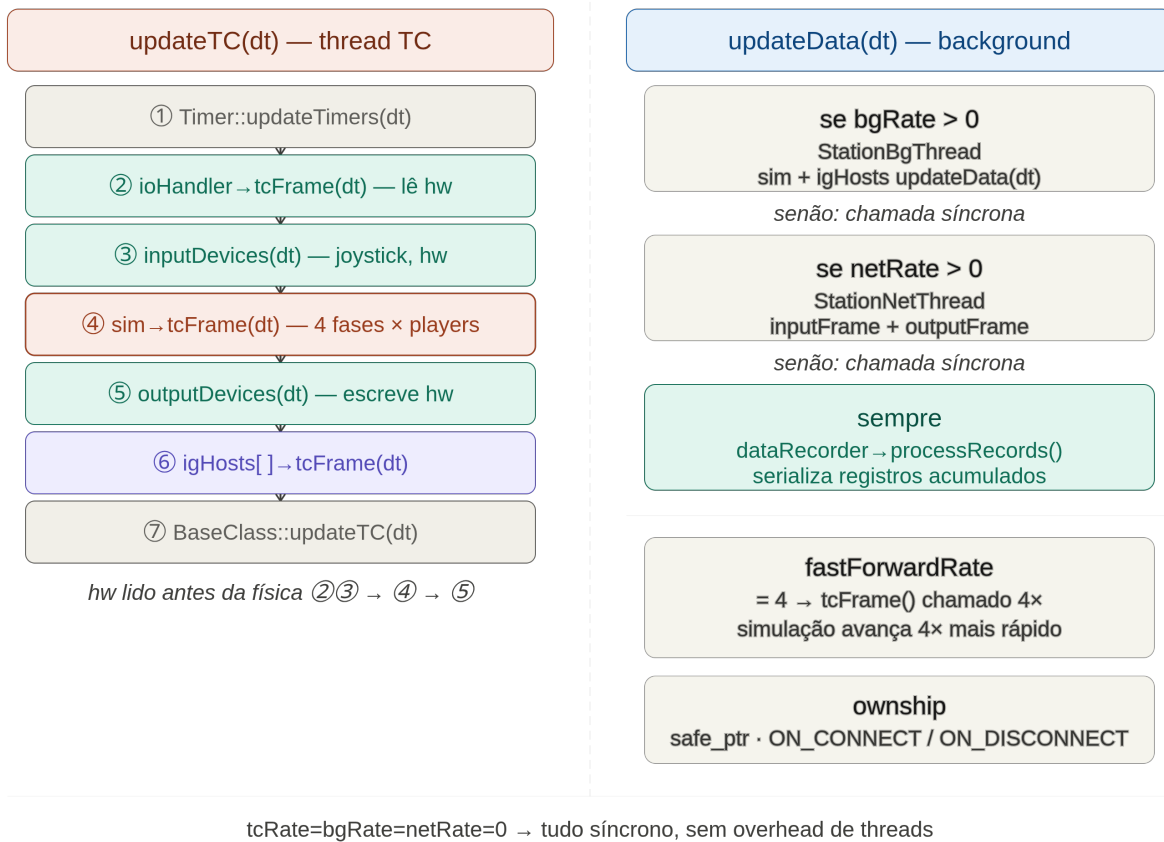



Figure 25: Os dois quadros de **Station**: **updateTC()** com a ordem fixa de sete passos (esquerda) e **updateData()** com os três ramos condicionais e a chamada incondicional ao gravador (direita). A faixa inferior mostra **fastForwardRate** e **ownship**.

Listing 143: Declaração típica de **Station** em EDL com os slots mais importantes.

Uma taxa igual a zero (**bgRate**: 0) significa que o processamento correspondente roda sincronamente na *thread* que chama **updateData()** — útil para depuração ou quando não há CPUs extras disponíveis.

IV.5.2 O quadro **updateTC(dt)**: sete passos em ordem fixa

A ordem de execução de **Station::updateTC()** não é arbitrária — cada passo depende do anterior:

```

1 void Station::updateTC(const double dt)
2 {
3     // ① Avança todos os Timers antes de qualquer logica
4     // (garante que isExpired() retorna valores corretos no frame atual)
5     if (isUpdateTimersEnabled()) {
6         base::Timer::updateTimers(dt);
7     }
8
9     // ② Le o hardware de E/S (joystick, pedais, painéis)
10    if (ioHandler != nullptr) {
11        ioHandler->tcFrame(dt);
12    }
13

```



```

14 // ③ Copia os valores lidos para os canais de entrada
15 inputDevices(dt);
16
17 // ④ FÍSICA: percorre todos os players em 4 fases
18 // (o passo mais custoso -- ver Secao sec:sim-simulation)
19 if (sim != nullptr) sim->tcFrame(dt);
20
21 // ⑤ Copia os valores de saida para o hardware
22 outputDevices(dt);
23
24 // ⑥ Atualiza os geradores de imagem (IG hosts)
25 // Passa ownship e lista de players para cada IG antes de tcFrame
26 if (sim != nullptr && igHosts != nullptr) {
27     base::PairStream* playerList { sim->getPlayers() };
28     base::List::Item* item { igHosts->getFirstItem() };
29     while (item != nullptr) {
30         const auto pair = static_cast<base::Pair*>(item->getValue());
31         const auto p = static_cast<AbstractIgHost*>(pair->object());
32         p->setOwnship(ownship);
33         p->setPlayerList(playerList);
34         p->tcFrame(dt);
35         item = item->getNext();
36     }
37     if (playerList != nullptr) playerList->unref();
38 }
39
40 // ⑦ Startup RESET: envia RESET_EVENT apos startupResetTime segundos
41 if (startupResetTimer >= 0) {
42     startupResetTimer -= dt;
43     if (startupResetTimer < 0) this->event(RESET_EVENT);
44 }
45
46 BaseClass::updateTC(dt);
47 }

```

Listing 144: Station::updateTC() real: sete passos em ordem fixa (src/simulation/Station.cpp).

A ordem dos passos ②③ (leitura de *hardware*) antes de ④ (física) e ⑤ (escrita) depois de ④ é o mesmo princípio de consistência por fase da Seção II.4, estendido ao mundo externo: todo o estado físico externo é lido em uma única passagem antes de qualquer lógica rodar, e atualizado em uma única passagem depois.

O passo ⑥ merece atenção: `setOwnship()` e `setPlayerList()` são chamados a cada *frame* — não apenas no `reset()`. Isso permite que o *ownship* seja trocado dinamicamente (o jogador “salta” para outro *player*) sem que o IG precise saber sobre a troca.

IV.5.3 O quadro `updateData(dt)`: ramos condicionais e gravador

Station::updateData() tem três ramos condicionais e uma chamada incondicional:

```

1 void Station::updateData(const double dt)
2 {
3     // Ramo 1: cria a thread de background, se pedida e ainda inexistente
4     if (getBackgroundRate() > 0 && !doWeHaveTheBgThread()) {
5         createBackgroundProcess();
6     }
7
8     // Ramo 2: sem thread de background --> roda sincronamente aqui
9     if (getBackgroundRate() == 0 && !doWeHaveTheBgThread()) {
10        processBackgroundTasks(dt);
11    }
12
13    // Ramo 3: cria a thread de rede -- note a guarda 'networks != nullptr':
14    // sem redes configuradas, nenhuma thread de rede e criada
15    if (getNetworkRate() > 0 && networks != nullptr && !doWeHaveTheNetThread()) {
16        createNetworkProcess();

```

```

17     }
18
19     // Ramo 4: sem thread de rede -> roda sincronamente aqui
20     if (getNetworkRate() == 0 && networks != nullptr && !doWeHaveTheNetThread()) {
21         processNetworkInputTasks(dt);
22         processNetworkOutputTasks(dt);
23     }
24
25     // Incondicional: descarga do gravador de dados (sempre no background)
26     if (dataRecorder != nullptr) dataRecorder->processRecords();
27
28     BaseClass::updateData(dt);
29 }

```

Listing 145: `Station::updateData()`: criação lazy das threads e descarga incondicional do gravador (`src/simulation/Station.cpp`).

Duas *threads* — a de *background* e a de rede — são criadas *lazily*, na primeira chamada de `updateData()` em que a taxa correspondente seja maior que zero. Isso permite que a configuração seja modificada antes dessa primeira chamada, por exemplo via `setSlotBgRate()` logo após o EDL ser carregado. A guarda `networks != nullptr` no ramo de rede é significativa: uma *Station* sem o *slot networks* não paga por nenhuma *thread* de rede nem executa `processNetworkInputTasks()` — os quatro ramos simplesmente não se aplicam.

A *thread* de tempo crítico é a exceção. `StationTcPeriodicThread` não é criada aqui, e essa assimetria é deliberada. Quem decide se existe uma *thread* de tempo crítico é a aplicação, chamando `createTimeCriticalProcess()` explicitamente durante a inicialização. A alternativa — não chamá-la — deixa a aplicação livre para acionar `station->updateTC(dt)` a partir do seu próprio laço, de um *timer* do sistema operacional ou de uma interrupção de *hardware* externo, que é justamente o caso de uso de simuladores acoplados a bancadas reais. O cabeçalho de `Station.hpp` documenta as três funções (`createTimeCriticalProcess()`, `createNetworkProcess()` e `createBackgroundProcess()`) como virtuais, previstas para serem sobrescritas por uma *Station* derivada que queira outra política.

Implementação de `processBackgroundTasks()`.

```

1 void Station::processBackgroundTasks(const double dt)
2 {
3     // E/S em background (sem bloqueio)
4     if (ioHandler != nullptr) ioHandler->updateData(dt);
5
6     // Simulation executive: updatePlayerList() e updateData() dos players
7     if (sim != nullptr) sim->updateData(dt);
8
9     // IG hosts (background do gerador de imagem)
10    if (igHosts != nullptr) {
11        base::List::Item* item { igHosts->getFirstItem() };
12        while (item != nullptr) {
13            const auto pair = static_cast<base::Pair*>(item->getValue());
14            const auto p = static_cast<AbstractIgHost*>(pair->object());
15            p->updateData(dt);
16            item = item->getNext();
17        }
18    }
19 }

```

Listing 146: `processBackgroundTasks()`: o que a thread de background executa (`src/simulation/Station.cpp`).

IV.5.4 Fast-forward e ownship

Fast-forward. `fastForwardRate` controla quantas vezes `tcFrame(dt)` é chamado por *tick* da `StationTcPeriodicThread`:

```
1 void Station::processTimeCriticalTasks(const double dt)
2 {
3     // fastForwardRate = 1 : tempo real (padrao)
4     // fastForwardRate = 4 : 4x mais rapido que o tempo real
5     for (unsigned int jj = 0; jj < getFastForwardRate(); jj++) {
6         tcFrame(dt); // cada chamada avanca dt segundos de fisica
7     }
8     // O dt nao e dividido: cada iteracao e um frame completo de dt segundos.
9     // A thread corre a tcRate Hz, entao o tempo simulado avanca a
10    // fastForwardRate * tcRate Hz.
11 }
```

Listing 147: `processTimeCriticalTasks()`: o loop de fast-forward real.

Com `tcRate = 50 Hz` e `fastForwardRate = 4`, a simulação avança a 200 Hz de tempo simulado por cada segundo real — quatro vezes mais rápido que o tempo real. Útil para missões longas ou para *burn-in* de modelos de IA antes de testes.

Ownship. `Station` mantém uma referência `safe_ptr` ao *player* principal, localizado por nome no `reset()`:

```
1 bool Station::setOwnshipByName(const char* const newOS)
2 {
3     base::PairStream* pl { (sim != nullptr) ? sim->getPlayers() : nullptr };
4     if (pl == nullptr) return false;
5
6     base::Pair* p { pl->findByName(newOS) };
7     if (p != nullptr) {
8         const auto newOwnship = static_cast<AbstractPlayer*>(p->object());
9         if (newOwnship != ownship) {
10             setOwnshipPlayer(newOwnship);
11         }
12     }
13     pl->unref();
14     return (ownship != nullptr);
15 }
16
17 bool Station::setOwnshipPlayer(AbstractPlayer* const newOS)
18 {
19     // Desconecta o ownship anterior
20     if (ownship != nullptr) ownship->event(ON_OWNESHIP_DISCONNECT);
21     ownship = newOS;
22     // Conecta o novo ownship
23     if (ownship != nullptr) ownship->event(ON_OWNESHIP_CONNECT);
24     return true;
25 }
```

Listing 148: `setOwnshipByName()`: localiza o player pelo nome e envia eventos de conexão (`src/simulation/Station.cpp`).

Os eventos `ON_OWNESHIP_CONNECT` e `ON_OWNESHIP_DISCONNECT` permitem que *displays* e subsistemas de controle e visualização (C&D) saibam quando o foco mudou — um *cockpit* virtual pode, por exemplo, reprogramar suas telas ao receber `ON_OWNESHIP_CONNECT` em um novo *player*. A troca é segura porque `ownship` é um `safe_ptr`: o *player* anterior não é destruído até que ninguém mais o referencie.

IV.6 As *threads* do simulation

Esta seção mostra como os conceitos abstratos de `PeriodicThread` (Seção III.8.2) e `SyncThread` (Seção III.8.3) ganham corpo em `simulation`: cinco subclasses concretas, em dois papéis distintos.

IV.6.1 As três `PeriodicThread` de `Station`

`StationTcPeriodicThread`, `StationBgPeriodicThread` e `StationNetPeriodicThread` são subclasses triviais de `base::PeriodicThread`: cada uma implementa apenas `userFunc(dt)`, delegando imediatamente para o método correspondente de `Station`. A implementação completa de cada uma cabe em seis linhas:

```
1  // Tc: chama processTimeCriticalTasks() -- inclui o fastForwardRate
2  unsigned long StationTcPeriodicThread::userFunc(const double dt)
3  {
4      Station* station { static_cast<Station*>(getParent()) };
5      station->processTimeCriticalTasks(dt);
6      return 0;
7  }
8
9  // Bg: chama processBackgroundTasks()
10 unsigned long StationBgPeriodicThread::userFunc(const double dt)
11 {
12     Station* station { static_cast<Station*>(getParent()) };
13     station->processBackgroundTasks(dt);
14     return 0;
15 }
16
17 // Net: chama processNetworkInputTasks() + processNetworkOutputTasks()
18 unsigned long StationNetPeriodicThread::userFunc(const double dt)
19 {
20     Station* station { static_cast<Station*>(getParent()) };
21     station->processNetworkInputTasks(dt);
22     station->processNetworkOutputTasks(dt);
23     return 0;
24 }
```

Listing 149: As três `PeriodicThread` são subclasses mínimas — delegam e saem (`src/simulation/StationTcPeriodicThread.cpp` e `StationBgPeriodicThread.cpp`).

Essas *threads* só existem se a taxa correspondente for maior que zero. Com `tcRate = 0`, por exemplo, nenhuma `StationTcPeriodicThread` é criada: a aplicação chama `station->updateTC(dt)` diretamente no laço principal — útil para depuração ou para sincronizar com um *interrupt* externo de hardware, como detalhado no cabeçalho de `Station.hpp`.

`StationTcPeriodicThread` é criada por `createTimeCriticalProcess()`, chamada pela aplicação durante a inicialização (e não a partir de `updateData()`, ao contrário das *threads* de *background* e de rede):

```
1  void Station::createTimeCriticalProcess()
2  {
3      if (tcThread == nullptr) {
4          tcThread = new StationTcPeriodicThread(this, getTimeCriticalRate());
5          tcThread->unref(); // safe_ptr<> tem a posse
6
7          if (tcStackSize > 0) tcThread->setStackSize(tcStackSize);
8
9          bool ok { tcThread->start(getTimeCriticalPriority()) };
10         // prioridade [0,1] → SCHED_FIFO no Linux (ver Sec. sec:base-threading)
11         if (!ok) {
12             tcThread = nullptr;
13             // registra erro -- nao lanca execucao
14         }
15     }
```

```
16 | }
```

Listing 150: `Station::createTimeCriticalProcess()`: criação da thread TC (`src/simulation/Station.cpp`).

IV.6.2 O *pool* SyncThread de Simulation

`SimulationTcSyncThread` e `SimulationBgSyncThread` materializam o *pool* com barreira por fase da Seção II.5. A interface que cada uma expõe é mínima: um método `start0()` que carrega os parâmetros de trabalho e dispara a execução.

```
1 // start0(): carrega parametros e acorda o worker (signalStart)
2 void SimulationTcSyncThread::start0(
3     base::PairStream* const pl1, // lista de players (capturada no frame)
4     const double dt1, // dt/4 para esta fase
5     const unsigned int idx1, // indice desta thread (1-based)
6     const unsigned int n1) // total de threads no pool
7 {
8     pl0 = pl1; // parametros copiados para membros privados
9     dt0 = dt1;
10    idx0 = idx1;
11    n0 = n1;
12
13    signalStart(); // acorda o worker que dorme em waitForStart()
14 }
15
16 // userFunc(): executa quando acordado por signalStart()
17 unsigned long SimulationTcSyncThread::userFunc()
18 {
19     if (pl0 != nullptr && idx0 > 0 && idx0 <= n0) {
20         Simulation* sim { static_cast<Simulation*>(getParent()) };
21         sim->updateTcPlayerList(pl0, dt0, idx0, n0);
22         // ao retornar, SyncThread chama signalCompleted() automaticamente
23     }
24     return 0;
25 }
```

Listing 151: `SimulationTcSyncThread`: a classe completa — carrega parâmetros, dispara e processa sua fatia (`src/simulation/SimulationTcSyncThread.cpp`).

`SimulationBgSyncThread` é idêntica, exceto que chama `updateBgPlayerList()` em vez de `updateTcPlayerList()` — o mecanismo de despacho é exatamente o mesmo.

IV.6.3 O protocolo de despacho por fase

A cada uma das quatro fases do quadro TC (Seção IV.4), o padrão de despacho é:

```
1 // Dentro do loop for (f = 0; f < 4; f++):
2
3 setPhase(f); // marca a fase corrente para todos os players
4
5 // 1. Dispara os workers (idx 1 .. reqTcThreads-1)
6 for (unsigned short i = 0; i < numTcThreads; i++) {
7     unsigned int idx { static_cast<unsigned int>(i + 1) };
8     tcThreads[i]->start0(currentPlayerList, dt0/4.0, idx, reqTcThreads);
9     // start0() copia parametros e chama signalStart()
10    // o worker acorda e começa updateTcPlayerList() em paralelo
11 }
12
13 // 2. Thread principal processa o ultimo subconjunto (idx = reqTcThreads)
14 updateTcPlayerList(currentPlayerList, dt0/4.0, reqTcThreads, reqTcThreads);
15
```

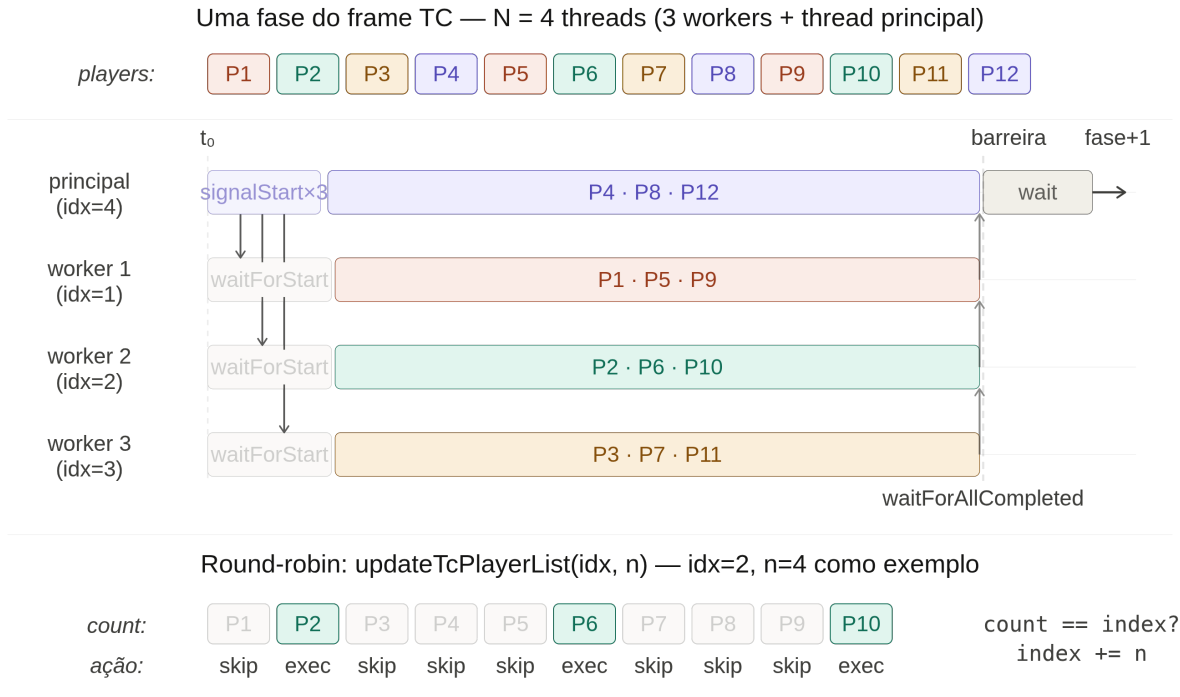
```

16 // 3. Barreira: espera todos os workers sinalizarem conclusao
17 base::SyncThread** pp { reinterpret_cast<base::SyncThread*>(&tcThreads[0]) };
18 base::SyncThread::waitAllCompleted(pp, numTcThreads);
19
20 // 4. Somente aqui a proxima fase pode comecar

```

Listing 152: Despacho real de uma fase em `Simulation::updateTC(): start0()` nos workers, processamento local, barreira (`src/simulation/Simulation.cpp`).

O diagrama abaixo mostra uma fase com 4 *threads* (3 workers + thread principal) e 12 *players*: cada *thread* recebe exatamente 3 *players* por distribuição *round-robin*.



Workers nunca são recriados entre fases — dormem em `waitForStart()` após cada conclusão.
Custo de despacho por fase: N-1 chamadas `signalStart()` + 1 `waitAllCompleted()`.

Figure 26: Uma fase do quadro TC com $N = 4$ threads e 12 players. A thread principal chama `start0()` nos três workers (que dormiam em `waitForStart()`) e imediatamente processa sua própria fatia em paralelo. A barreira `waitAllCompleted()` bloqueia até todos concluírem, antes da fase seguinte. A lista de players (baixo) mostra a distribuição *round-robin*: Worker 1 processa P1, P5, P9; Worker 2 processa P2, P6, P10; Worker 3 processa P3, P7, P11; e a thread principal (`idx = 4`) processa P4, P8, P12.

IV.6.4 A distribuição *round-robin*

O algoritmo em `updateTcPlayerList(idx, n)` é direto: percorre a lista sequencialmente, processa os players nas posições `idx`, `idx+n`, `idx+2n`... e pula os demais.

```

1 void Simulation::updateTcPlayerList(
2     base::PairStream* const playerList,
3     const double      dt,
4     const unsigned int idx, // esta thread processa o idx-esimo player
5     const unsigned int n)  // passo: pula n-1 players entre cada um
6 {
7     unsigned int index { idx }; // proximo numero de ordem a processar
8     unsigned int count {};      // contador de posicao na lista

```

```

9
10 base::List::Item* item { playerList->getFirstItem() };
11 while (item != nullptr) {
12     count++;
13     if (count == index) {
14         base::Pair* pair { static_cast<base::Pair*>(item->getValue()) };
15         AbstractPlayer* ip { static_cast<AbstractPlayer*>(pair->object()) };
16         ip->tcFrame(dt); // processa este player
17         index += n;     // proximo player desta thread
18     }
19     item = item->getNext();
20 }
21 }

```

Listing 153: `updateTcPlayerList()`: distribuição *round-robin* sem filas, sem alocação (`src/simulation/Simulation.cpp`).

Com $N = 4$ *threads* e 12 *players*, cada uma processa exatamente 3 — sem coordenação adicional, sem filas, sem alocação durante o quadro. O custo de despacho é um `signalStart()` e um `waitForCompleted()`, ambos implementados como operações de *mutex* pré-bloqueado (Seção III.8.3), com custo de microssegundos.

Os *workers* nunca são destruídos e recriados entre quadros: são criados no `reset()` de `Simulation` e permanecem em `waitForStart()` entre fases, consumindo zero CPU enquanto dormem.

IV.7 Fábrica do módulo

Como em `base/factory.cpp` (Seção II.3), `simulation/factory.cpp` registra os nomes de fábrica deste módulo:

```

1 base::Object* factory(const std::string& name)
2 {
3     base::Object* obj {};
4
5     if ( name == Simulation::getFactoryName() ) { obj = new Simulation(); }
6     else if ( name == Station::getFactoryName() ) { obj = new Station(); }
7     // Apenas duas classes: as demais do modulo sao abstratas.
8     // AbstractNib e AbstractNetIO nao sao instanciados diretamente --
9     // suas implementacoes concretas vem de interop/ (Sec. sec:interop)
10
11     return obj;
12 }

```

Listing 154: `simulation/factory.cpp`: nomes de fábrica do módulo.

O padrão de encadeamento de fábricas é o mesmo já estabelecido em `base`: a aplicação combina as fábricas de `base`, `simulation`, `models` e demais bibliotecas na ordem de precedência desejada. Nenhum mecanismo novo é introduzido aqui — apenas mais dois nomes na mesma cadeia.

Com `simulation` coberto, a próxima camada de dependência é `terrain` (Seção V), seguida de `models` (Seção VI) — onde `Player`, `DynamicsModel`, `Radar` e os demais componentes que preenchem a lista de *players* de `Simulation` são finalmente definidos.

No cenário da interceptação

No cenário do Seção II.6, esta camada é o que faz o tempo passar.

A `Station` declarada com `tcRate: 50` cria a *thread* de tempo crítico e passa a chamar `tcFrame()` cinquenta vezes por segundo. Cada chamada percorre os dois *players* quatro vezes — uma por fase — de modo que `falcao1` e `alvo1` recebem duzentas visitas por segundo, sempre na mesma ordem e sempre com todas as emissões antes de todas as recepções.

O *slot ownship*: "`falcao1`" é resolvido no *reset*, e é o que faz a aplicação ter um ponto de vista. E os dois *players* do arquivo vão para `origPlayers`: o míssil que o `falcao1` lançar entrará apenas em `players`, e desaparecerá no próximo *reset* sem que ninguém precise removê-lo.

V A camada terrain

V.1 Visão geral e hierarquia

terrain resolve um problema único: dar acesso a dados de elevação do terreno por uma **interface uniforme**, independentemente do formato do arquivo de origem. A decisão de projeto separa o **contrato** (Terrain) das **implementações concretas** (DataFile e suas subclasses por formato) e acrescenta um **agregador** (QuadMap) que combina vários arquivos numa única consulta.

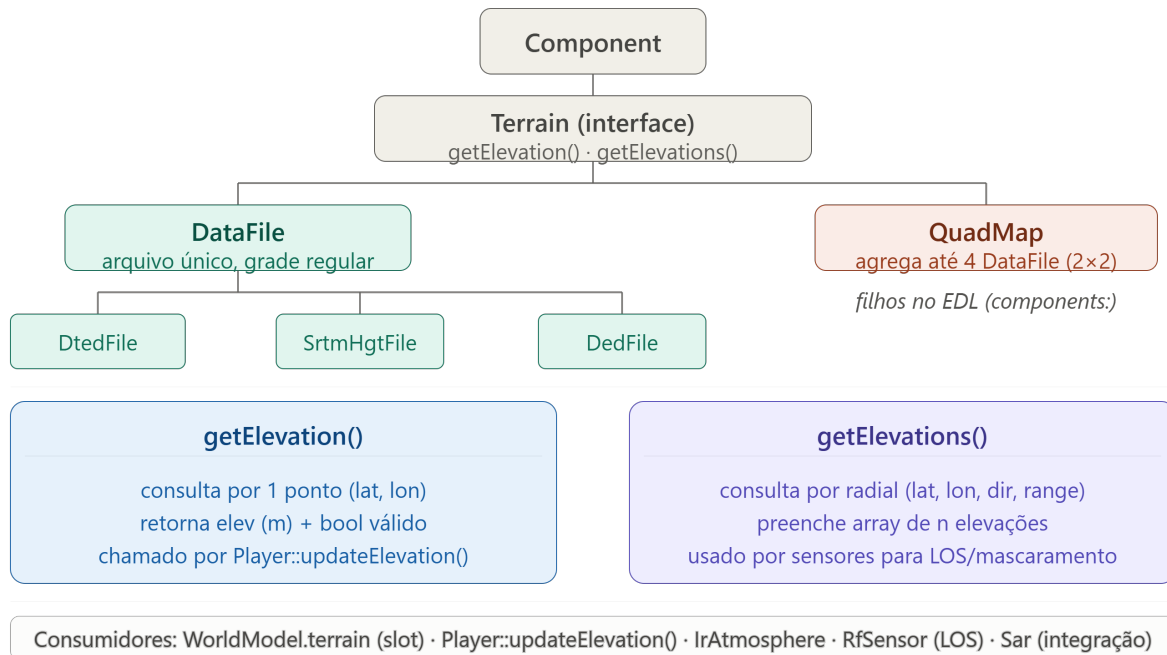


Figure 27: Hierarquia do módulo terrain: Terrain define o contrato com dois métodos; DataFile implementa a leitura de arquivo único em grade regular com três subclasses de formato; QuadMap agrega até quatro arquivos em grade 2 × 2. Os dois métodos de consulta e seus consumidores são resumidos nas faixas inferiores.

terrain depende apenas de mixr_base — mas é, ele próprio, uma dependência de models: WorldModel (Seção VI.2) tem um slot terrain, e Player::updateElevation() (Seção VI.3) consulta a elevação sob cada entidade a cada quadro. A mesma interface é consultada por sensores — um radar precisa saber se uma montanha bloqueia a linha de visão até o alvo.

V.2 Terrain: a interface

Terrain é a classe abstrata que define o contrato. O estado comum a qualquer banco de dados de elevação é a sua **célula de cobertura**: os quatro cantos geográficos (latitudeSW/longitudeSW/latitudeNE/longitudeNE) e os extremos de elevação. É a partir desses cantos que QuadMap (Seção V.5) decide qual sub-banco contém um ponto.

Armadilha — os *slots* são *path* e *file*, não *pathname* e *filename*

Terrain tem exatamente dois *slots*, e os nomes não acompanham os dos acessores. Os métodos chamam-se `setPathname()` e `setFilename()`, mas em EDL escreve-se *path* e *file*.

É a mesma classe de divergência dos nomes de fábrica (Seção III.3.1), agora no nível do *slot*: o nome C++ e o nome EDL são declarados em lugares diferentes, e nada obriga a coincidirem. Um *pathname*: num arquivo de configuração produz `slot not found` e o *tile* simplesmente não carrega.

Duas consultas formam o contrato dinâmico:

```
1  class Terrain : public base::Component {
2  public:
3      // Consulta por UM ponto: devolve true se dentro da cobertura,
4      // preenchendo 'elev' em metros.
5      // Chamado por Player::updateElevation() a cada fase 0.
6      virtual bool getElevation(
7          double* const elev,
8          const double lat, const double lon,
9          const bool interp = false) const = 0;
10
11     // Consulta por RADIAL: preenche 'n' elevacoes ao longo de uma linha reta
12     // a partir de (lat,lon), na 'direction' (graus verdadeiros), ate 'maxRng' m.
13     // Retorna quantos pontos foram encontrados dentro da cobertura.
14     // Usado por sensores para calcular linha de visao e mascaramento.
15     virtual unsigned int getElevations(
16         double* const elevations, bool* const validFlags,
17         const unsigned int n,
18         const double lat, const double lon,
19         const double direction, const double maxRng,
20         const bool interp = false) const = 0;
21
22     // --
23     // Utilitarios  estaticos  de pos-processamento sobre o array de elevacoes:
24     // --
25
26     // vbwShadowChecker(): determina quais pontos estao em sombra de terreno.
27     // Para cada ponto i: maskFlags[i]=true se o ponto esta oculto
28     // (tanLookAngle < tanLower, onde tanLower e o maximo historico ao longo
29     // do radial -- simula o efeito de "horizonte" progressivo).
30     static bool vbwShadowChecker(
31         bool* const maskFlags,
32         const double* const elevations,
33         const bool* const validFlags,
34         const unsigned int n,
35         const double range, const double refAlt,
36         const double beamAngle = 0, // centro do feixe (graus)
37         const double beamWidth = 180); // largura total (graus)
38
39     // aac(): calcula o cosseno do angulo de aspecto do terreno em cada ponto.
40     // Vetor m1 = direcao do feixe ao ponto; vetor m2 = normal a inclinacao.
41     // cos(aspecto) = m1 . m2 usado por modelos de clutter radar para
42     // estimar a intensidade do retorno do solo.
43     static bool aac(
44         double* const aacData,
45         const double* const elevData,
46         const bool* const maskFlags,
47         const unsigned int n,
48         const double range, const double refAlt);
49
50     // targetOcculting(): high-level que chama getElevations + occultCheck
51     bool targetOcculting(
52         const double refLat, const double refLon, const double refAlt,
53         const double tgtLat, const double tgtLon, const double tgtAlt) const;
54 };
```

Listing 155: Terrain: os dois métodos virtuais puros e as funções estáticas de pós-processamento (resumo de `include/mixr/terrain/Terrain.hpp`).

O `vbwShadowChecker()` combina dois testes numa única passagem. O primeiro é o algoritmo de *horizonte crescente*: percorre o radial do observador ao alvo acumulando o maior ângulo de visada encontrado (`tanLower`); qualquer ponto com ângulo abaixo do máximo histórico está em sombra — é o mesmo algoritmo de *shadow ray* de *ray casting*, aplicado ao perfil 2D de elevação. O segundo é o recorte pelo feixe: pontos acima da borda superior (`tanUpper`) também são mascarados, por estarem fora do campo de visada mesmo sem obstrução.

```

1  // check-fontes: esquematico
2  // Bordas do feixe, saturadas para evitar a singularidade de tan(90 graus)
3  double beamUpper { beamAngle + beamWidth/2.0 };
4  if (beamUpper > 89.9999) beamUpper = 89.9999;
5  double beamLower { beamAngle - beamWidth/2.0 };
6  if (beamLower < -89.9999) beamLower = -89.9999;
7
8  const double tanUpper { std::tan(beamUpper * base::angle::D2RCC) };
9  double      tanLower { std::tan(beamLower * base::angle::D2RCC) };
10
11 const double deltaRng { range / (n - 1) };
12 double currentRange {};
13
14 for (unsigned int i = 0; i < n; i++) {
15     if (validFlags == nullptr || validFlags[i]) {
16         double tanLookAngle {};
17         if (currentRange > 0)
18             tanLookAngle = (elevations[i] - refAlt) / currentRange;
19         else
20             tanLookAngle = (elevations[i] - refAlt) / 1.0;
21
22         // Visível exige AS DUAS condições:
23         // >= tanLower : acima do horizonte acumulado E da borda inferior
24         // <= tanUpper : abaixo da borda superior do feixe
25         if (tanLookAngle >= tanLower && tanLookAngle <= tanUpper) {
26             maskFlags[i] = false; // visível
27             tanLower = tanLookAngle; // eleva o horizonte
28         } else {
29             maskFlags[i] = true; // em sombra ou fora do feixe
30         }
31     } else {
32         maskFlags[i] = true; // dado inválido -> mascarado
33     }
34     currentRange += deltaRng;
35 }

```

Listing 156: `vbwShadowChecker()`: horizonte crescente e recorte pelas duas bordas do feixe. O fonte (`src/terrain/Terrain.cpp`) repete o laço em dois ramos quase idênticos, com e sem `validFlags`; aqui eles estão fundidos num só.

Note que `tanLower` acumula duas responsabilidades: nasce como a borda inferior do feixe e passa a representar o horizonte crescente à medida que a varredura avança. `tanUpper`, ao contrário, é constante — o feixe não "encolhe" por cima. A saturação em $\pm 89,9999^\circ$ evita a singularidade de $\tan(90^\circ)$ para feixes muito largos, e é o motivo de o valor padrão `beamWidth = 180` não produzir infinito.

V.3 DataFile: um arquivo de elevação em grade regular

`DataFile` implementa `Terrain` para o caso comum: um único arquivo cobrindo uma grade regular de pontos (*posts*) em latitude e longitude. Os dados ficam em memória como uma matriz `columns[lon_idx][lat_idx]` de `short`:

```

1 class DataFile : public Terrain {
2 protected:
3     short**    columns {}; // [lon_idx][lat_idx] -- em metros
4     double     latSpacing {}; // espacamento entre posts em lat (graus)

```

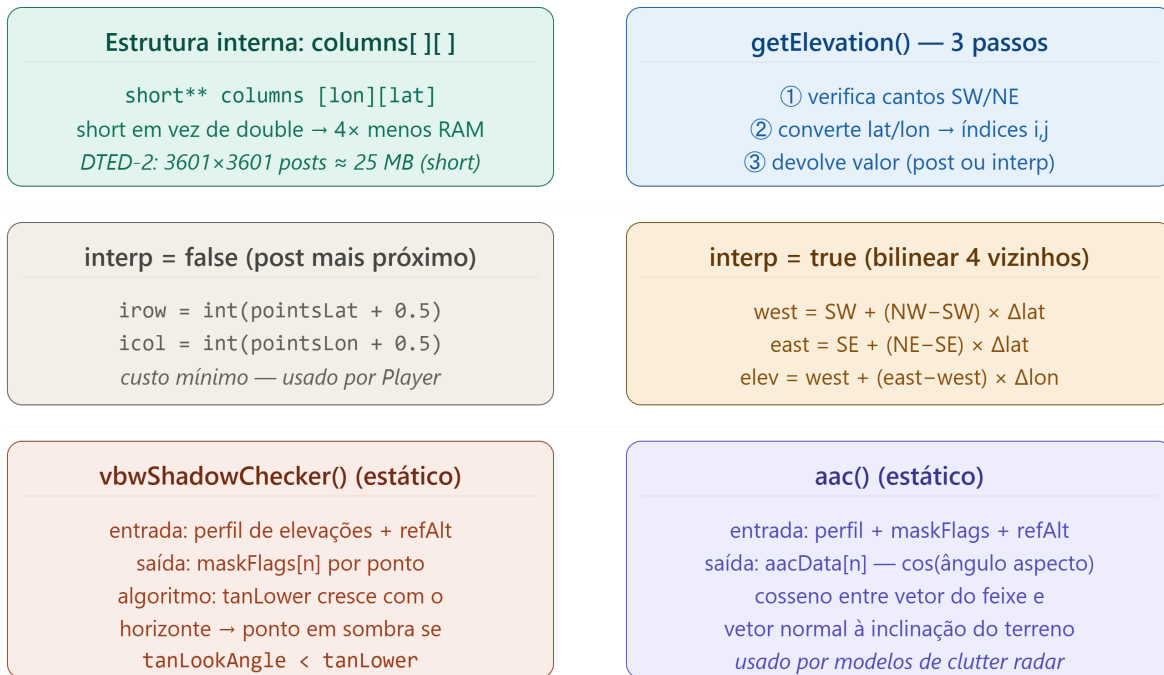


Figure 28: Estrutura interna de DataFile: matriz `columns[lon][lat]` de `short` (topo-esq); os três passos de `getElevation()` (topo-dir); comparação entre `interp=false` (post mais próximo) e `interp=true` (bilinear de quatro vizinhos) (centro); utilitários estáticos `vbwShadowChecker()` e `aac()` (baixo).

```

5 | double lonSpacing {}; // espacamento entre posts em lon (graus)
6 | unsigned int nptlat {}; // numero de posts em latitude (linhas)
7 | unsigned int nptlong {}; // numero de posts em longitude (colunas)
8 | short voidValue {-32767}; // valor de post invalido/vazio
9 | };
10 |
11 | // Porque short e nao double?
12 | // DTED-2: 3601 x 3601 posts por tile de 1deg x 1deg
13 | // short: 3601 * 3601 * 2 bytes = ~25 MB por tile
14 | // double: 3601 * 3601 * 8 bytes = ~100 MB por tile
15 | // Carregando 4 tiles no QuadMap: 100 MB vs 400 MB
16 | // Elevacoes em metros raramente excedem +/-32767 m (short e suficiente)

```

Listing 157: DataFile: campos protegidos reais — indexação e escolha de `short` (resumo de `include/mixr/terrain/DataFile.hpp`).

O carregamento é **preguiçoso**, e acontece um nível acima do que se esperaria: quem o dispara é `Terrain::reset()`, não `DataFile`. Nada é lido do disco antes do primeiro `reset`:

```

1 | void Terrain::reset ()
2 | {
3 |     if ( !isDataLoaded() ) {
4 |         loadData();
5 |     }
6 |
7 |     BaseClass::reset ();
8 | }

```

Listing 158: `Terrain::reset()` na íntegra (`src/terrain/Terrain.cpp`).

São seis linhas, e cada detalhe conta. `loadData()` é virtual puro — é o único método que cada formato precisa implementar. A guarda `isDataLoaded()` torna a operação idempotente: um segundo `reset` da

simulação não relê gigabytes de disco. E `BaseClass::reset()` vem **depois** do carregamento, de modo que, quando os componentes filhos forem reiniciados, os dados já estão em memória.

Essa última ordem é o que faz o `QuadMap` funcionar:

```

1 void QuadMap::reset()
2 {
3     // Resetting our base class will reset our components,
4     // which will load the elevation data files
5     BaseClass::reset();
6
7     // Now find our files
8     findDataFiles();
9 }

```

Listing 159: `QuadMap::reset()`: por que a ordem importa (`src/terrain/QuadMap.cpp`).

Por que — o agregador só pode se montar depois dos agregados

`findDataFiles()` varre os componentes filhos e aceita apenas os que já têm dados carregados (Seção V.5). Se ele rodasse antes de `BaseClass::reset()`, encontraria quatro *tiles* vazios e montaria um `QuadMap` sem nenhum arquivo — silenciosamente, porque um `QuadMap` com zero *tiles* é um objeto válido que apenas responde **false** a toda consulta. A propagação do *reset* pela árvore de componentes (Seção III.2.5) é o que garante a ordem correta.

`getElevation()` segue três passos; com `interp=true`, o passo ③ interpola bilinearmente entre os quatro *posts* vizinhos ao ponto solicitado:

```

1 double value{};           // the elevation (meters)
2
3 // Early out tests
4 if ( !isDataLoaded() ||   // Not loaded or
5     (lat < getLatitudeSW() ||
6     lat > getLatitudeNE()) || // wrong latitude or
7     (lon < getLongitudeSW() ||
8     lon > getLongitudeNE()) // wrong longitude
9 ) return false;
10
11 // Compute the lat and lon points
12 double pointsLat {(lat - getLatitudeSW()) / latSpacing};
13 if (pointsLat < 0) pointsLat = 0;
14
15 double pointsLon {(lon - getLongitudeSW()) / lonSpacing};
16 if (pointsLon < 0) pointsLon = 0;
17
18 if (interp) {
19     // South-west corner post is [icol][irow]
20     unsigned int irow {static_cast<unsigned int>(pointsLat)};
21     unsigned int icol {static_cast<unsigned int>(pointsLon)};
22     if (irow > (nptlat-2)) irow = (nptlat-2);
23     if (icol > (nptlong-2)) icol = (nptlong-2);
24
25     // delta from s-w corner post
26     double deltaLat {static_cast<double>(pointsLat - static_cast<double>(irow))};
27     double deltaLon {static_cast<double>(pointsLon - static_cast<double>(icol))};
28
29     // Get the elevations at each corner
30     double elevSW {static_cast<double>(columns[icol][irow])};
31     double elevNW {static_cast<double>(columns[icol][irow+1])};
32     double elevSE {static_cast<double>(columns[icol+1][irow])};
33     double elevNE {static_cast<double>(columns[icol+1][irow+1])};
34
35     // Interpolate the west point, then the east point
36     double westPoint {elevSW + (elevNW - elevSW) * deltaLat};
37     double eastPoint {elevSE + (elevNE - elevSE) * deltaLat};
38
39     // Interpolate between the west and east points

```

```

40     value = westPoint + (eastPoint - westPoint) * deltaLon;
41 }
42 else {
43     // No -- just use the nearest post
44     unsigned int irow {static_cast<unsigned int>(pointsLat + 0.5f)};
45     unsigned int icol {static_cast<unsigned int>(pointsLon + 0.5f)};
46     if (irow >= nptlat) irow = (nptlat-1);
47     if (icol >= nptlong) icol = (nptlong-1);
48
49     value = static_cast<double>(columns[icol][irow]);
50 }
51
52 *elev = value;
53 return true;
54 }

```

Listing 160: `DataFile::getElevation()`: os dois modos de retorno (`src/terrain/DataFile.cpp`). O cabeçalho da função foi omitido; o corpo é integral.

Vale destrinchar a aritmética de índice, porque ela concentra três decisões numa linha cada.

`pointsLat` e `pointsLon` são posições **fracionárias** na grade: quantos espaçamentos cabem entre o canto sudoeste e o ponto pedido. A parte inteira é o índice do *post*, a fracionária é onde se está entre dois deles.

Na interpolação, o truncamento para `unsigned int` dá o canto sudoeste do quadrado, e a saturação em `nptlat-2` garante que `irow+1` ainda exista — é isso que impede o acesso fora do *array* exatamente na borda norte ou leste do *tile*. Já no modo do *post* mais próximo, o `+0.5` antes do truncamento é o arredondamento, e a saturação é em `nptlat-1`, porque aí não há vizinho a consultar.

Armadilha — a ordem dos índices é `columns[lon][lat]`

O *array* é indexado primeiro por **longitude** e só depois por latitude — `columns[icol][irow]`, e não o contrário. O nome ajuda a lembrar: o dado é organizado em *colunas* de longitude, cada uma contendo todas as elevações daquela coluna em latitude. Trocar os dois índices não gera erro de compilação e devolve elevações de outro lugar do mundo.

A escolha entre os dois modos não é estilística: para a dinâmica de voo o *post* mais próximo é suficiente (custo mínimo a 50 Hz); para um sensor que precisa de um perfil contínuo ao longo de um radial, a interpolação evita descontinuidades artificiais na linha de visão.

V.4 Formatos de arquivo: DTED, SRTM e DED

Três subclasses de `DataFile` implementam `loadData()` para três formatos distintos — a interface de consulta é idêntica para as três; o que muda é apenas como `columns[][]` é preenchido.

DTED (*Digital Terrain Elevation Data*). Padrão militar (MIL-PRF-89020B), em três níveis de resolução:

Nível	Espaçamento	<i>Posts</i> /célula
DTED-0	30 arc-sec (~900 m)	121 × 121
DTED-1	3 arc-sec (~90 m)	1201 × 1201
DTED-2	1 arc-sec (~30 m)	3601 × 3601

Table 13: Níveis de resolução do formato DTED.

O arquivo contém três cabeçalhos binários fixos (UHL, DSI, ACC) seguidos pelos dados de elevação em colunas de longitude. Um detalhe de codificação crítico: os valores são armazenados em *big-endian* e *sinal-magnitude*, não em complemento de dois:

```
1 short DtedFile::readValue(const unsigned char hbyte, const unsigned char lbyte)
2 {
3     // The data is stored as 2 byte characters (sign and magnitude)
4     // with high byte first. The high bit is the sign bit. Check for
5     // sign bit and then turn it off and set SIGN_VAL accordingly.
6     short height{};
7     short sign_val{1};
8     unsigned char nhbyte{hbyte};
9
10    if (hbyte & ~0177) {
11        // sign bit set
12        nhbyte = hbyte & 0177;
13        sign_val = -1;
14    }
15    height = (256 * static_cast<short>(nhbyte) + static_cast<short>(lbyte)) * sign_val;
16
17    return height;
18 }
```

Listing 161: `DtedFile::readValue()`: conversão de sinal-magnitude (`src/terrain/dted/DtedFile.cpp`).

As constantes estão em octal, o que esconde o que elas fazem: `0177` é `0x7F`, os sete bits baixos; `~0177` é a máscara do bit de sinal. E a multiplicação por 256 é o deslocamento de oito bits que recompõe o valor de dois *bytes* — escrito como aritmética porque o resultado precisa ser `short` com sinal, e não um padrão de bits.

Por que — por que sinal-magnitude, e não complemento de dois

O DTED é um formato dos anos 1970, e sinal-magnitude era a convenção de troca de dados científicos da época — independente da arquitetura da máquina, que naquele momento nem sempre usava complemento de dois. A consequência para quem lê o arquivo hoje é que **não se pode simplesmente reinterpretar dois *bytes* como um `int16_t`**: uma elevação de -100m ficaria com o valor errado. Daí a existência desta função.

Há uma segunda sobrecarga de quatro *bytes*, com a mesma lógica, usada para ler a soma de verificação do rodapé de cada coluna — conferida apenas quando o *slot* de verificação está ligado.

SRTM (*Shuttle Radar Topography Mission*). Dados gratuitos da NASA/USGS cobrindo $\sim 80\%$ da superfície terrestre. Não há cabeçalhos — todo o metadado vem do nome do arquivo e do tamanho:

```
1 bool SrtmHgtFile::determineSrtmInfo(const std::string& srtmFilename,
2                                     std::streamoff size)
3 {
4     // Extract the number of latitude and longitude lines
5     unsigned int num_lat{};
6     unsigned int num_lon{};
7     // Extract the data intervals for latitude and longitude
8     switch (size) {
9         case 2884802: // SRTM3: 3 arc-sec
10         latSpacing = 3.0 / 3600.0;
11         lonSpacing = 3.0 / 3600.0;
12         num_lat = 1201;
13         num_lon = 1201;
14         break;
15         case 25934402: // SRTM1: 1 arc-sec
16         latSpacing = 1.0 / 3600.0;
17         lonSpacing = 1.0 / 3600.0;
18         num_lat = 3601;
```



```

19     num_lon = 3601;
20     break;
21     default:
22         return false;
23 }
24
25 // valid SRTM file extensions
26 if (srtmFilename.substr(7, 4) != ".hgt" && srtmFilename.substr(7, 4) != ".HGT") {
27     return false;
28 }
29 // ... daqui em diante, o nome fornece hemisferios e canto sudoeste ...
30 }

```

Listing 162: `SrtmHgtFile::determineSrtmInfo()`: a resolução vem do tamanho do arquivo (`src/terrain/srtm/SrtmHgtFile.cpp`).

O nome do arquivo é lido por **posição fixa**: o método recebe os últimos onze caracteres (`N38W077.hgt`) e indexa diretamente — os caracteres 7 a 10 são a extensão, e os anteriores dão hemisfério e coordenada do canto sudoeste.

Armadilha — um único *byte* a mais e o arquivo é rejeitado

O `switch` é sobre o tamanho **exato** em *bytes*. Não há tolerância, e o `default` devolve `false` sem diagnóstico específico: a mensagem que o usuário vê é apenas “*ERROR in determining SRTM type*”. Um arquivo truncado no *download*, descompactado com conversão de fim de linha, ou de uma variante de resolução diferente das duas previstas, falha exatamente da mesma forma — e a causa mais provável, na prática, é a primeira.

A codificação dos dados é a mesma do DTED — *big-endian*, sinal-magnitude — portanto o mesmo `readValue()` se aplica.

DedFile (*Digital Elevation Data*). Formato proprietário do MIXR — mais simples que os dois anteriores, sem cabeçalhos complexos, usado tipicamente para dados sintéticos ou convertidos de outras fontes durante o desenvolvimento.

V.5 QuadMap: agregação em grade 2×2

Uma simulação raramente cabe num único *tile* de $1^\circ \times 1^\circ$. **QuadMap** gerencia até `MAX_DATA_FILES = 4` arquivos em grade 2×2 , descobertos automaticamente como filhos no EDL:

```

1 terrain: ( QuadMap
2     components: {
3         sw: ( DtedFile path: "/data/terrain" file: "n38w077.dt2" )
4         se: ( DtedFile path: "/data/terrain" file: "n38w076.dt2" )
5         nw: ( DtedFile path: "/data/terrain" file: "n39w077.dt2" )
6         ne: ( DtedFile path: "/data/terrain" file: "n39w076.dt2" )
7     }
8 )

```

Listing 163: QuadMap em EDL: os quatro tiles como componentes filhos.

Armadilha — quatro é um limite rígido, e o excedente some em silêncio

`MAX_DATA_FILES` vale exatamente 4 — o comentário no cabeçalho é lacônico a respeito: “*Only 4 files (as in Quad!)*”. O nome da classe é o limite.

O laço de descoberta para em `count < MAX_DATA_FILES`, de modo que um quinto *tile* declarado em

components é **ignorado sem aviso**: o EDL carrega, o QuadMap funciona, e há um buraco no mapa exatamente onde estava aquele arquivo. Cobrir uma área maior exige aninhar QuadMaps — um QuadMap é um Terrain como qualquer outro, e portanto pode ser filho de outro QuadMap.

A descoberta repete o padrão de `Player::updateSystemPointers()` (Seção VI.3): varre os componentes filhos por tipo:

```
1 void QuadMap::findDataFiles()
2 {
3     clearData();
4     base::PairStream* sub { getComponents() };
5     if (sub == nullptr) return;
6
7     unsigned int count {};
8     base::List::Item* item { sub->getFirstItem() };
9     while (item != nullptr && count < MAX_DATA_FILES) {
10         const auto pair = static_cast<base::Pair*>(item->getValue());
11         const auto df = dynamic_cast<Terrain*>(pair->object());
12         // Aceita qualquer Terrain já carregado (DtedFile, SrtmHgtFile, DedFile)
13         if (df != nullptr && df->isDataLoaded()) {
14             df->ref();
15             dataFiles[count++] = df;
16         }
17         item = item->getNext();
18     }
19     numDataFiles = count;
20     sub->unref();
21
22     // Calcula o envelope de cobertura como uniao dos quatro cantos
23     if (numDataFiles > 0) {
24         double elevMin{999999.0}, elevMax{-999999.0};
25         double lowerLat{90.0}, lowerLon{180.0};
26         double upperLat{-90.0}, upperLon{-180.0};
27         for (unsigned int i = 0; i < numDataFiles; i++) {
28             elevMin = std::min(elevMin, dataFiles[i]->getMinElevation());
29             elevMax = std::max(elevMax, dataFiles[i]->getMaxElevation());
30             lowerLat = std::min(lowerLat, dataFiles[i]->getLatitudeSW());
31             lowerLon = std::min(lowerLon, dataFiles[i]->getLongitudeSW());
32             upperLat = std::max(upperLat, dataFiles[i]->getLatitudeNE());
33             upperLon = std::max(upperLon, dataFiles[i]->getLongitudeNE());
34         }
35         setMinElevation(elevMin); setMaxElevation(elevMax);
36         setLatitudeSW(lowerLat); setLongitudeSW(lowerLon);
37         setLatitudeNE(upperLat); setLongitudeNE(upperLon);
38     }
39 }
```

Listing 164: `QuadMap::findDataFiles()`: descoberta automática e cálculo do envelope de cobertura (`src/terrain/QuadMap.cpp`).

As duas consultas do contrato resolvem a agregação de maneiras diferentes, e a diferença é instrutiva.

`getElevation()` — um ponto — é uma **disjunção**: tenta cada *tile* em sequência e para no primeiro que responda. Faz sentido, porque um ponto pertence a um *tile* só.

`getElevations()` — um radial — é uma **colaboração**: todos os quatro recebem os *mesmos arrays* `elevations[]` e `validFlags[]`, e cada um preenche apenas as posições que caem na sua cobertura, marcando o *flag* correspondente. Um radial de 80 km que atravessasse a fronteira entre dois *tiles* sai com a primeira metade escrita por um e a segunda pelo outro — sem que o chamador saiba que havia uma fronteira.

Por que — o *array* de *flags* é o que torna a colaboração possível

Sem `validFlags[]`, um *tile* não teria como dizer “estas posições não são minhas” sem escrever um valor — e qualquer valor escolhido (zero, `-32767`) seria indistinguível de uma elevação legítima. O *array* de *flags* separa “elevação zero” de “sem dado”, e é isso que permite a vários *tiles* escreverem no mesmo *buffer* sem se atrapalharem. É também por isso que `vbwShadowChecker()` e `aac()` recebem esse mesmo *array*: um ponto sem dado é tratado como mascarado, e não como um vale.

```
1  bool QuadMap::getElevation(double* const elev,
2                               const double lat, const double lon,
3                               const bool interp) const
4  {
5      if (!isDataLoaded() ||
6          lat < getLatitudeSW() || lat > getLatitudeNE() ||
7          lon < getLongitudeSW() || lon > getLongitudeNE()) return false;
8
9      bool found {}; double value {};
10     for (unsigned int i = 0; i < numDataFiles && !found; i++)
11         found = dataFiles[i]—>getElevation(&value, lat, lon, interp);
12     if (found) *elev = value;
13     return found;
14 }
15
16 unsigned int QuadMap::getElevations(
17     double* const elevations, bool* const validFlags,
18     const unsigned int n, const double lat, const double lon,
19     const double direction, const double maxRng, const bool interp) const
20 {
21     unsigned int num {};
22     // Radial pode cruzar fronteiras: cada tile contribui com seus pontos
23     for (unsigned int i = 0; i < numDataFiles && num < n; i++)
24         num += dataFiles[i]—>getElevations(
25             elevations, validFlags, n, lat, lon, direction, maxRng, interp);
26     return num;
27 }
```

Listing 165: `QuadMap::getElevation()` e `getElevations()` reais (`src/terrain/QuadMap.cpp`).

A fábrica do módulo é mínima — apenas quatro classes registradas:

```
1  base::Object* factory(const std::string& name)
2  {
3      base::Object* obj {};
4      if (name == QuadMap::getFactoryName()) obj = new QuadMap();
5      else if (name == DedFile::getFactoryName()) obj = new DedFile();
6      else if (name == DtedFile::getFactoryName()) obj = new DtedFile();
7      else if (name == SrtmHgtFile::getFactoryName()) obj = new SrtmHgtFile();
8      return obj;
9  }
```

Listing 166: `terrain/factory.cpp`: as quatro classes do módulo.

Os demais utilitários estáticos de Terrain. Além de `vbwShadowChecker()` e `aac()`, detalhados acima, Terrain expõe outras funções estáticas de pós-processamento sobre o array de elevações: `occultCheck()` e `occultCheck2()` (oclusão ponto a ponto, a base sobre a qual `targetOcculting()` e `targetOcculting2()` são construídos), `cLight()` (iluminação Lambertiana do perfil, para representação visual do relevo) e `getElevationColor()` (mapeamento de elevação para cor via uma tabela de `base::Color`, Seção III.12). Todas seguem o mesmo contrato: operam sobre arrays já preenchidos por `getElevations()`, sem estado próprio e sem tocar no disco.

No cenário da interceptação

O cenário do Seção II.6 declara quatro *tiles* DTED em torno do Vale do Paraíba, num **QuadMap** no *slot terrain* da **WorldModel**. Eles são lidos do disco uma única vez, no primeiro *reset*, e a partir daí servem a dois consumidores muito diferentes.

O primeiro é barato e constante: **Player::updateElevation()** pergunta a cota sob cada *player* a cada quadro, com **interp = false** — o *post* mais próximo basta, e a 50 Hz o custo importa.

O segundo é caro e esporádico: quando o radar do **falcao1** precisa saber se o **alvo1** está atrás de uma serra, ele pede um *radial* inteiro com **getElevations()** e passa o resultado por **vbwShadowChecker()**. Aí a interpolação vale a pena, porque um perfil com degraus produziria falsos bloqueios.

É a mesma interface atendendo aos dois — e é por isso que ela tem dois métodos, e não um.

VI A camada `models`

VI.1 Visão geral: o que está sendo simulado

Se `simulation` define *como* a simulação executa, `models` define *o quê* está sendo simulado: aeronaves, mísseis, radares, sistemas de armas, navegação, ambiente. É de longe a maior biblioteca do MIXR — 107 arquivos de implementação e outros tantos cabeçalhos, compilados em `libmixr_models.so` — e depende de `mixr_base`, `mixr_simulation`, `mixr_terrain` e `jsbsim`. Para efeito de comparação, é cerca de sete vezes o tamanho de `simulation` e a única camada que depende de todas as três abaixo dela.

Esses 107 arquivos organizam-se em oito domínios funcionais:

Domínio	Conteúdo
<code>WorldModel</code>	simulação situada no globo terrestre (estende <code>Simulation</code>)
<code>Player</code>	entidade simulada concreta
<code>dynamics/</code>	física de voo (<code>RacModel</code> , <code>LaeroModel</code> , <code>JSBSimModel</code>)
<code>system/</code>	subsistemas embarcados (radar, piloto, armas, countermeasures)
<code>navigation/</code>	rotas, waypoints, INS, GPS
<code>environment/</code>	atmosfera IR (<code>AbstractAtmosphere</code>)
<code>player/</code>	tipos concretos (aeronaves, armas, veículos)
dados e mensagens	<code>Emission</code> , <code>Track</code> , <code>SensorMsg</code> , <code>Tdb</code> , ...

Table 14: Os oito domínios funcionais de `libmixr_models.so`.

Três classes de dados transversais — `SynchronizedState`, `Message` e `TargetData` — completam o módulo, ao lado de `SimAgent/MultiActorAgent` (extensões do UBF, Seção III.10) e da fábrica do módulo. A progressão das subseções segue a ordem de dependência interna: `WorldModel` e `Player` primeiro (o núcleo que tudo referencia), depois os sistemas que vivem dentro de um `Player`, e por fim os tipos concretos e os dados que circulam entre sistemas.

VI.2 `WorldModel`: a simulação situada no mundo

`WorldModel` é a subclasse concreta de `Simulation` (Seção IV.4) que qualquer simulação real instancia — não `Simulation` diretamente. Ela acrescenta ao *executive* tudo que é necessário para **situar a simulação no globo terrestre**: o ponto de referência geodésico, o modelo do elipsoide, o banco de dados de terreno e o modelo atmosférico.

VI.2.1 O ponto de referência e a *world matrix*

O conceito mais importante de `WorldModel` é o **ponto de referência** — o centro da área de jogo, definido pelos *slots* `latitude/longitude`. Quando esses *slots* são preenchidos, `setRefLatitude()` e `setRefLongitude()` disparam imediatamente o pré-cálculo de $\sin(\phi_{\text{ref}})$, $\cos(\phi_{\text{ref}})$ e da *world matrix* `wm`:

```
1 bool WorldModel::setRefLatitude(const double v)
2 {
3     bool ok { v <= 90.0 && v >= -90.0 };
4     if (ok) {
5         refLat = v;
6         const double r { base::angle::D2RCC * refLat };
```

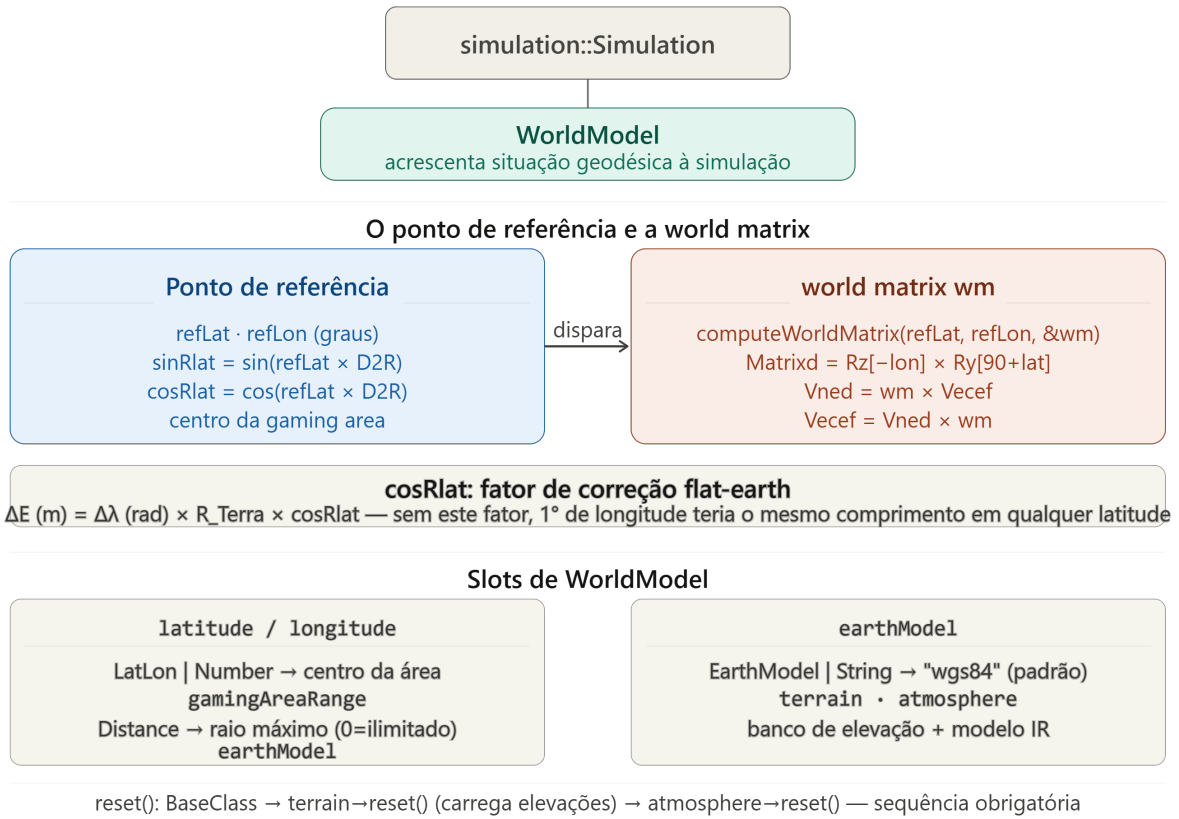


Figure 29: WorldModel estende Simulation com o ponto de referência geodésico (topo-esq) e a *world matrix* (topo-dir), mais os *slots* de configuração (centro) e a sequência obrigatória do `reset()` (rodapé).

```

7   sinRlat = std::sin(r); // pré-calculado para uso por todos os players
8   cosRlat = std::cos(r); // idem
9   base::nav::computeWorldMatrix(refLat, refLon, &wm); // Ry[−(90+lat)] * Rz[lon]
10  }
11  return ok;
12  }
13
14  bool WorldModel::setRefLongitude(const double v)
15  {
16      bool ok { v <= 180.0 && v >= −180.0 };
17      if (ok) {
18          refLon = v;
19          base::nav::computeWorldMatrix(refLat, refLon, &wm); // recalcula wm
20      }
21      return ok;
22  }

```

Listing 167: `setRefLatitude()` e `setRefLongitude()`: pré-calculam senos, cossenos e a *world matrix* a cada mudança do ponto de referência (`src/models/WorldModel.cpp`).

A *world matrix* **wm** converte vetores entre NED (*North-East-Down*, o plano tangente ao globo no ponto de referência) e ECEF (*Earth-Centered, Earth-Fixed*):

$$\mathbf{wm} = R_y(-(90^\circ + \phi_{\text{ref}})) \cdot R_z(\lambda_{\text{ref}}), \quad \mathbf{V}_{\text{ned}} = \mathbf{wm} \cdot \mathbf{V}_{\text{ecef}}, \quad \mathbf{V}_{\text{ecef}} = \mathbf{V}_{\text{ned}} \cdot \mathbf{wm} \quad (4)$$

O `cosRlat` é o fator de correção das projeções *flat-earth* que todos os *players* usam ao converter deslocamentos angulares em metros no plano local. Como a largura de um grau de longitude em metros varia com

a latitude — de ≈ 111 km no equador até zero no polo — `cosRlat` corrige essa diferença:

$$\Delta E_m = \Delta \lambda_{\text{rad}} \times R_{\oplus} \times \cos(\phi_{\text{ref}}) \quad (5)$$

O código real de `Player::setPosition()` (em `src/models/player/Player.cpp`) usa exatamente essa fórmula via `base::nav::convertLL2PosVecS()` (modo esférico) ou `convertLL2PosVecE()` (modo elipsóide):

```

1  bool Player::setPositionLLA(const double lat, const double lon,
2                               const double alt, const bool slaved)
3  {
4      WorldModel* s { getWorldModel() };
5      const double refLat { s->getRefLatitude() };
6      const double refLon { s->getRefLongitude() };
7      const double cosRlat { s->getCosRefLat() };
8
9      if (s->isGamingAreaUsingEarthModel()) {
10         // Modo elipsóide: usa EarthModel e sinRlat
11         const double sinRlat { s->getSinRefLat() };
12         base::nav::convertLL2PosVecE(
13             refLat, refLon, sinRlat, cosRlat,
14             lat, lon, alt, &posVecNED, em);
15     } else {
16         // Modo esférico: usa apenas cosRlat
17         base::nav::convertLL2PosVecS(
18             refLat, refLon, cosRlat,
19             lat, lon, alt, &posVecNED);
20     }
21
22     // Verifica limite da gaming area
23     posVecValid = (maxRefRange <= 0.0) ||
24                   (posVecNED.length2() <= maxRefRange * maxRefRange);
25     // ...
26 }
```

Listing 168: Como `Player::setPositionLLA()` usa `cosRlat` e `sinRlat` de `WorldModel` (`src/models/player/Player.cpp`).

VI.2.2 Configuração via EDL

```

1  ( WorldModel
2      -- Ponto de referência: centro da gaming area
3      latitude: ( LatLon 35.0 )      -- 35°N
4      longitude: ( LatLon -117.5 )   -- 117.5°O (deserto de Mojave)
5
6      -- Raio da gaming area (0 = ilimitado)
7      gamingAreaRange: ( KiloMeters 500.0 )
8
9      -- Modelo de Terra: WGS-84 (padrão se omitido)
10     earthModel: "wgs84"
11
12     -- Usar projeções elipsoidais (true) ou esférico simplificado (false)
13     gamingAreaUseEarthModel: true
14
15     -- Banco de dados de elevação de terreno
16     terrain: ( QuadMap
17         components: {
18             sw: ( DtedFile path: "/data/terrain" file: "n34w118.dt1" )
19             se: ( DtedFile path: "/data/terrain" file: "n34w117.dt1" )
20         }
21     )
22
23     -- Modelo atmosférico (para simulação IR)
24     atmosphere: ( IrAtmosphere ... )
25 )
```

Listing 169: Declaração típica de `WorldModel` em EDL: ponto de referência, modelo de terra, terreno e atmosfera.

O `slot earthModel` aceita tanto um objeto `EarthModel` completo quanto uma `string` de nome, resolvida por `base::EarthModel::getEarthModel(*msg)`:

```
1  bool WorldModel::setSlotEarthModel(const base::String* const msg)
2  {
3      bool ok {};
4      if (msg != nullptr && msg->len() > 0) {
5          const base::EarthModel* p { base::EarthModel::getEarthModel(*msg) };
6          if (p != nullptr) {
7              ok = setEarthModel(p); // clona o modelo
8          } else {
9              std::cerr << "WorldModel: earth model not found: "
10                 << *msg << std::endl;
11          }
12      } else {
13          ok = setEarthModel(nullptr); // volta ao WGS-84 default
14      }
15      return ok;
16  }
```

Listing 170: `setSlotEarthModel(String*)`: resolução do nome para a constante estática correspondente (`src/models/WorldModel.cpp`).

VI.2.3 O reset como ponto de inicialização

`WorldModel::reset()` encadeia a inicialização na ordem correta:

```
1  void WorldModel::reset()
2  {
3      // 1. Inicializa o executive (Simulation::reset()): reconstrói
4      // a lista de players, cria o pool de threads
5      BaseClass::reset();
6
7      // 2. Inicializa o banco de dados de terreno
8      // (primeiro reset: carrega os arquivos de elevacao do disco)
9      if (terrain != nullptr) {
10         std::cout << "Loading Terrain Data..." << std::endl;
11         terrain->reset();
12         std::cout << "Finished!" << std::endl;
13     }
14
15     // 3. Inicializa o modelo atmosferico
16     if (atmosphere != nullptr) atmosphere->reset();
17 }
```

Listing 171: `WorldModel::reset()`: sequência de inicialização real (`src/models/WorldModel.cpp`).

A ordem importa: `BaseClass::reset()` precisa ocorrer primeiro porque reconstrói a lista de *players* e cria o *pool* de *threads* que os subsistemas de terreno e atmosfera poderão usar. O carregamento do terreno é sinalizado ao operador pelas mensagens "Loading Terrain Data..."/"Finished!", pois pode levar segundos para bancos de dados grandes. O modelo atmosférico é inicializado por último, pois pode depender dos limites de altitude definidos pelo terreno.

Acesso ao WorldModel de dentro de um Player. Qualquer objeto na árvore obtém o `WorldModel` via `findContainerByType`:

```

1 // Em qualquer Player ou System:
2 WorldModel* wm { static_cast<WorldModel*>(
3     findContainerByType( typeid(WorldModel) )
4 ) };
5 if (wm != nullptr) {
6     double lat { wm->getRefLatitude() }; // graus
7     double lon { wm->getRefLongitude() };
8     double cos { wm->getCosRefLat() }; // para projecoes flat-earth
9     terrain :: Terrain* t { wm->getTerrain() }; // elevacao do terreno
10 }
11
12 // Atalho disponivel em Player:
13 WorldModel* s { getWorldModel() }; // equivalente ao findContainerByType acima

```

Listing 172: Padrão de acesso ao WorldModel de qualquer subsistema.

VI.3 Player: a entidade simulada

Player é a classe mais importante de **models** e a mais referenciada por todo o restante do módulo. Implementa **AbstractPlayer** (Seção IV.3) e é a raiz da hierarquia de entidades simuladas — aeronaves, mísseis, veículos, efeitos e qualquer outro objeto que ocupe a lista de *players* de **Simulation**.

VI.3.1 Estado cinemático completo: três representações simultâneas

Um **Player** mantém posição e orientação em **três sistemas de coordenadas simultaneamente**, todos atualizados a cada fase 0:

```

1 // Posicao em tres sistemas
2 base::Vec3d posVecNED; // [Norte, Leste, Baixo] metros -- plano local tangente
3 base::Vec3d posVecECEF; // [x, y, z] metros geocentricos
4 double latitude {}; // graus
5 double longitude {};
6 double altitude {}; // metros HAE
7
8 // Orientacao
9 base::Matrixd rm; // rotacao corpo → NED
10 base::Matrixd rmW2B; // rotacao ECEF → corpo (rm * wm)
11 base::Quat q; // quaternion de orientacao (Quat, nao "Quatd")
12
13 // Velocidades
14 base::Vec3d velVecNED; // velocidade em NED (m/s)
15 base::Vec3d velVecECEF; // velocidade em ECEF (m/s) -- enviada pela rede
16 base::Vec3d angularVel; // velocidade angular no eixo corpo (rad/s)

```

Listing 173: Campos de posição e orientação em **Player** (resumo de `include/mixr/models/player/Player.hpp`).

Manter as três representações simultaneamente não é redundância — é uma decisão de desempenho e coerência arquitetural. Cada subsistema usa o sistema que lhe é natural:

- **DynamicsModel** integra equações de movimento em NED (`posVecNED`, `velVecNED`);
- **AbstractNib** monta PDUs para a rede em ECEF (`posVecECEF`, `velVecECEF`);
- O EDL configura posição inicial em geodésicos (`initLatitude`, `initLongitude`, `initAlt`) — ou, alternativamente, em NED (`initXPos`, `initYPos`) ou em ECEF (`initGeocentric`).

A responsabilidade é dividida em dois métodos, e confundir os dois é fácil:

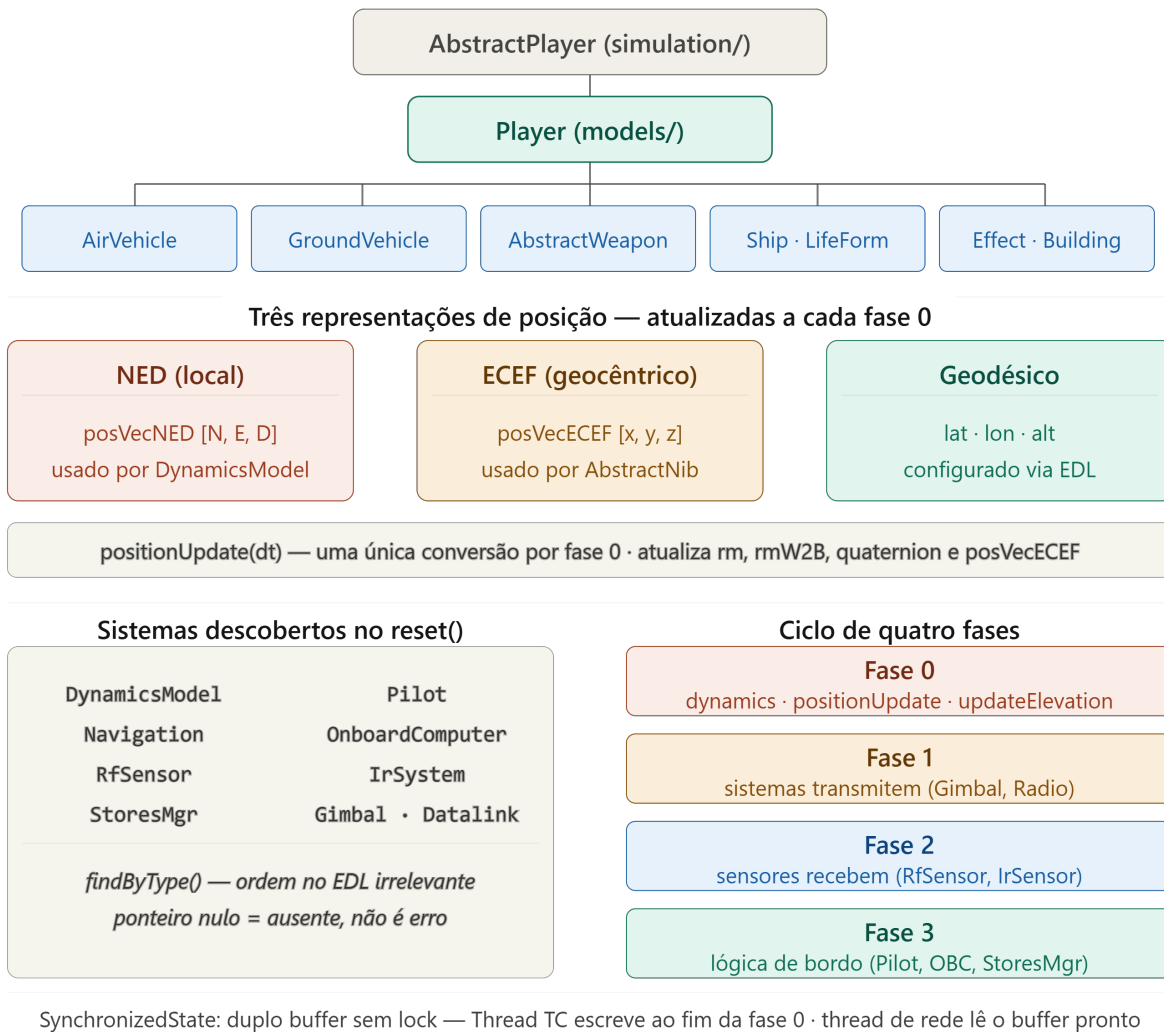


Figure 30: Player: hierarquia de subtipos (topo), três representações de posição atualizadas a cada fase 0 (centro-esq), sistemas descobertos por tipo no `reset()` (baixo-esq) e o ciclo de quatro fases por *frame* (baixo-dir).

- `positionUpdate(dt)` **integra** a posição a partir da velocidade;
- `setPosition()` / `setPositionLLA()` **sincronizam** as três representações entre si.

O primeiro chama o segundo. É no segundo que vivem as conversões `convertPosVec2ll*`, o `computeWorldMatrix()` e o `convertGeod2Ecef()` — e não em `positionUpdate()`.

```

1 void Player::positionUpdate(const double dt)
2 {
3     if ( !isLocalPlayer() ) return;    // networked: quem move e o dead reckoning
4
5     const bool pfrz{(isPositionFrozen() || isPositionSlaved())};
6     const bool afrz{(isAltitudeFrozen() || isAltitudeSlaved())};
7
8     // Clamping ao solo: so para tipos que andam no chao
9     const bool gcEnabled{tElevValid &&
10         isMajorType(GROUND_VEHICLE | SHIP | BUILDING | LIFE_FORM)};
11
12     const bool enabled{( vp > 0 && dt != 0 && (!pfrz || !afrz) )};
13     if ( enabled && useCoordSys == CS_LOCAL ) {
14
15         base::Vec3d newPosVecNED = posVecNED;

```

```

16
17     const double ue{velVecNED.x()}, ve{velVecNED.y()}, we{velVecNED.z()};
18     const double ue0{velVecN1.x()}, ve0{velVecN1.y()}, we0{velVecN1.z()};
19
20     // INTEGRACAO TRAPEZOIDAL: media entre a velocidade atual e a anterior
21     if (!pfrz) {
22         newPosVecNED[INORTH] += (ue + ue0) * 0.5 * dt;
23         newPosVecNED[IEAST] += (ve + ve0) * 0.5 * dt;
24     }
25     if (!afrz) {
26         if (gcEnabled) newPosVecNED[IDOWN] = -(tElev + tOffset); // colado ao solo
27         else newPosVecNED[IDOWN] += (we + we0) * 0.5 * dt;
28     }
29
30     setPosition(newPosVecNED); // <- e AQUI que as tres representacoes casam
31     velVecN1 = velVecNED;      // guarda para o trapezio do proximo quadro
32 }
33 }

```

Listing 174: `Player::positionUpdate()`: o ramo `CS_LOCAL` (`src/models/player/Player.cpp`). Os ramos `CS_GEOD` e `CS_WORLD` seguem a mesma forma noutros referenciais.

Três coisas que só se veem lendo o código. A integração é **trapezoidal**, não de Euler: usa a média entre a velocidade atual e a do quadro anterior, guardada em `velVecN1`. O *ground clamping* não integra a altitude — ele a *substitui* pela elevação do terreno mais um deslocamento, e só para veículos terrestres, navios, edificações e `LifeForm`. E há um *slot* de congelamento por eixo: posição e altitude podem ser congeladas independentemente.

Por que — três sistemas de coordenadas para integrar

O campo `useCoordSys` escolhe em qual referencial a integração acontece: `CS_LOCAL` (NED da *gaming area*), `CS_GEOD` (latitude/longitude/altitude) ou `CS_WORLD` (ECEF). A escolha não é estética: integrar em NED é barato e preciso perto da origem, mas acumula erro de projeção longe dela; integrar em ECEF é uniforme no globo inteiro, ao custo de números grandes. Trocar de sistema em voo é permitido — e quando isso acontece, a velocidade anterior é zerada, porque o trapézio não faria sentido entre referenciais diferentes.

VI.3.2 Descoberta automática de sistemas

`Player` não declara *slots* diretos para seus subsistemas — radar, piloto, navegação, etc. são **componentes filhos** no `PairStream`, descobertos automaticamente por tipo no `reset()` via `updateSystemPointers()`:

```

1 void Player::updateSystemPointers()
2 {
3     loadSysPtrs = false;
4     setDynamicsModel( findByType(typeid(DynamicsModel)) );
5     setDatalink(       findByType(typeid(Datalink)) );
6     setGimbal(         findByType(typeid(Gimbal)) );
7     setIrSystem(       findByType(typeid(IrSystem)) );
8     setNavigation(     findByType(typeid(Navigation)) );
9     setOnboardComputer(findByType(typeid(OnboardComputer)) );
10    setPilot(           findByType(typeid(Pilot)) );
11    setRadio(           findByType(typeid(Radio)) );
12    setSensor(          findByType(typeid(RfSensor)) );
13    setStoresMgr(       findByType(typeid(StoresMgr)) );
14 }

```

Listing 175: `updateSystemPointers()`: descobre subsistemas por tipo (`src/models/player/Player.cpp`).

Cada `set*()`, como `setNavigation()`, usa `isClassType(typeid(Navigation))` para confirmar o tipo antes de aceitar:

```

1  bool Player::setNavigation(base::Pair* const sys)
2  {
3      bool ok {};
4      if (sys == nullptr) {
5          if (nav != nullptr) nav->unref();
6          nav = nullptr;
7          ok = true;
8      } else if (sys->object()->isClassType(typeid(Navigation))) {
9          if (nav != nullptr) nav->unref();
10         nav = sys;
11         nav->ref();
12         ok = true;
13     }
14     return ok;
15 }

```

Listing 176: `setNavigation()`: aceita somente o tipo correto (`src/models/player/Player.cpp`).

O sinalizador `loadSysPtrs` é usado por `processComponents()` para forçar uma nova varredura quando a lista de componentes é modificada em tempo de execução — sem que o usuário precise chamar `reset()` manualmente:

```

1  void Player::processComponents(base::PairStream* const list,
2      const std::type_info& filter,
3      base::Pair* const add, base::Component* const remove)
4  {
5      base::Component::processComponents(list, filter, add, remove);
6      loadSysPtrs = true;  // re-varre os ponteiros no proximo updateTC()
7  }

```

Listing 177: `processComponents()`: dispara nova descoberta de sistemas.

A consequência: a ordem dos filhos no EDL é irrelevante, e um `Player` sem `DynamicsModel` simplesmente não tem física — não é um erro fatal, e o código não precisa verificar; cada chamada a `getDynamicsModel()` testa o ponteiro antes de usar.

VI.3.3 O ciclo de quatro fases

`Player::updateTC(dt)` chama `BaseClass::updateTC(dt)`, que distribui o `tcFrame(dt)` para todos os filhos (subsistemas). A diferenciação por fase é feita com `getWorldModel()->phase()`:

```

1  void Player::updateTC(const double dt0)
2  {
3      // Garante que os ponteiros de sistema estao atualizados
4      if (loadSysPtrs) {
5          updateSystemPointers();
6          loadSysPtrs = false;
7      }
8
9      if (mode == ACTIVE || mode == PRE_RELEASE) {
10         double dt { isFrozen() ? 0.0 : dt0 };
11         double dt4 { dt * 4.0 };  // dt total do frame (= dt/4 * 4)
12
13         switch (getWorldModel()->phase()) {
14
15             case 0:  // Fase 0: DINAMICA
16                 // Integra equacoes de movimento
17                 dynamics(dt4);  // delega para DynamicsModel
18
19                 // Logging de dados (gravador)

```

```

20         if (dataLogTime > 0.0) {
21             dataLogTimer -= dt4;
22             if (dataLogTimer <= 0.0) {
23                 BEGIN_RECORD_DATA_SAMPLE(
24                     getWorldModel()->getDataRecorder(), REID_PLAYER_DATA)
25                 SAMPLE_1_OBJECT(this)
26                 END_RECORD_DATA_SAMPLE()
27                 dataLogTimer = dataLogTime;
28             }
29         }
30
31         // Atualiza assinaturas RF e IR apos mover
32         if (signature != nullptr) signature->updateTC(dt4);
33         if (irSignature != nullptr) irSignature->updateTC(dt4);
34         break;
35
36     case 1: // Fase 1: sistemas TRANSMITEM
37         break; // tratado pelos subsistemas em BaseClass::updateTC()
38
39     case 2: // Fase 2: sensores RECEBEM
40         break;
41
42     case 3: // Fase 3: LOGICA de bordo
43         break;
44     }
45
46     // BaseClass::updateTC() atualiza todos os subsistemas filhos
47     // (Pilot, Navigation, RfSensor, etc.) com a mesma fase
48 }
49 }

```

Listing 178: Player::updateTC(): o switch real de fases (src/models/player/Player.cpp).

Note que as fases 1, 2 e 3 têm os case vazios no Player — os subsistemas filhos recebem tcFrame(dt) via BaseClass::updateTC() e cada um implementa seu comportamento de fase internamente. A separação garante a consistência da Seção II.4: quando o radar processa retornos na fase 2, todos os alvos já se moveram (fase 0) e os sinais já foram transmitidos (fase 1).

VI.3.4 SynchronizedState: o snapshot seguro entre threads

O AbstractNetIO lê posição e velocidade de cada player local na sua própria thread — concorrentemente com a thread TC que atualiza esses valores. Sem proteção, a thread de rede poderia ler um estado parcialmente escrito. SynchronizedState resolve isso com um mecanismo de **duplo buffer sem lock**: dois snapshots alternados, syncState1 e syncState2, escritos ao final da fase 0:

```

1 // Ao final da fase 0, apos positionUpdate():
2 if (!syncState1Ready) {
3     // Escreve no buffer 1
4     syncState1.setGeocPosition(getGeocPosition());
5     syncState1.setGeocVelocity(getGeocVelocity());
6     syncState1.setGeocAcceleration(getGeocAcceleration());
7     syncState1.setGeocEulerAngles(getGeocEulerAngles());
8     syncState1.setAngularVelocities(getAngularVelocities());
9     syncState1.setTimeExec(getWorldModel()->getTimeSec());
10    syncState1.setTimeUtc(getWorldModel()->getTimeOfDay());
11    syncState1.setValid(true);
12    syncState1Ready = true; // buffer 1 pronto para leitura
13    syncState2Ready = false; // buffer 2 sera o proximo a ser escrito
14 } else {
15     // Escreve no buffer 2
16     syncState2.setGeocPosition(getGeocPosition());
17     // ... idem ...
18     syncState2Ready = true;
19     syncState1Ready = false;

```

```
20 | }
```

Listing 179: `Player` alterna entre `syncState1` e `syncState2` ao final de cada fase 0 (`src/models/player/Player.cpp`).

A *thread* de rede lê via `getSynchronizedState()`, que devolve o buffer marcado como pronto:

```
1 inline const SynchronizedState& Player::getSynchronizedState() const
2 {
3     if (syncState1Ready)
4         return syncState1;
5     else
6         return syncState2;
7 }
```

Listing 180: `getSynchronizedState()`: sempre retorna o buffer completo (`include/mixr/models/player/Player.inl`).

O mecanismo funciona porque apenas a *thread* TC **escreve** nos buffers (alternando entre eles), e apenas a *thread* de rede **lê** (o buffer marcado como pronto). Não há condição de corrida: a *thread* TC nunca escreve no buffer que a *thread* de rede está lendo, pois `syncState1Ready/syncState2Ready` garantem que apenas um deles está disponível para leitura a cada instante.

`SynchronizedState` carrega em ECEF: posição, velocidade, aceleração, ângulos de Euler geocêntricos e velocidade angular — exatamente o conjunto que um PDU DIS necessita para o campo de *dead reckoning*.

VI.4 Dynamics: modelos de física de voo

O `DynamicsModel` é o subsistema responsável por integrar as equações de movimento de um `Player` a cada fase 0 do quadro de tempo crítico (Seção VI.3). A decisão de projeto é a mesma de toda a arquitetura do MIXR: uma **interface abstrata** define o contrato, e subclasses concretas implementam modelos de fidelidade crescente.

VI.4.1 DynamicsModel: o contrato

`DynamicsModel` define três grupos de métodos virtuais, todos com implementações *default* que não fazem nada (ou retornam `false/0`):

```
1 class DynamicsModel : public base::Component {
2 public:
3     // — GRUPO 1: integração (chamado pelo Player na fase 0) —
4     virtual void dynamics(const double dt); // default: não faz nada
5     virtual void atReleaseInit(); // arma: init no lançamento
6
7     // — GRUPO 2: comandos do Autopilot —
8     // Todos retornam false por default: o Autopilot interpreta isso como
9     // "modo não suportado por este modelo" e usa entrada de stick em vez disso.
10    virtual bool setHeadingHoldOn(const bool);
11    virtual bool setCommandedHeadingD(const double h,
12                                     const double hDps = 0,
13                                     const double maxBank = 0);
14    virtual bool setVelocityHoldOn(const bool);
15    virtual bool setCommandedVelocityKts(const double v, const double vNps = 0);
16    virtual bool setAltitudeHoldOn(const bool);
17    virtual bool setCommandedAltitude(const double a,
18                                     const double aMps = 0,
19                                     const double maxPitch = 0);
```

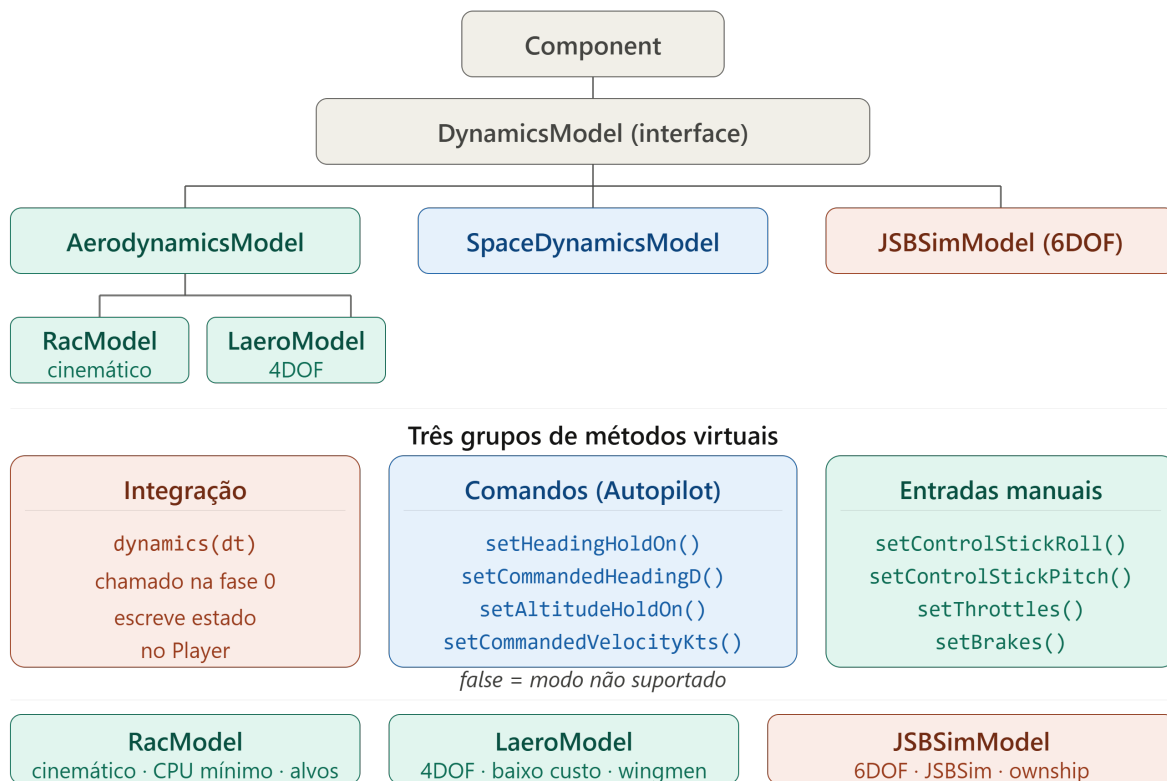


Figure 31: Hierarquia de DynamicsModel (topo): AerodynamicsModel especializa com métricas aerodinâmicas e ramifica em RacModel, LaeroModel e JSBSimModel — os três derivam dela. Os três grupos de métodos (centro) definem o contrato: integração, comandos do autopilot e entradas manuais. A faixa inferior compara os modelos concretos.

```

20
21 // — GRUPO 3: entradas manuais (joystick) —————
22 // roll: -1.0 (max esq) → 0.0 (centro) → +1.0 (max dir)
23 virtual void setControlStickRollInput(const double roll);
24 // pitch: -1.0 (nariz desce) → 0.0 (centro) → +1.0 (nariz sobe)
25 virtual void setControlStickPitchInput(const double pitch);
26 // throttle: <0=corte, 0=idle, 1=MIL, 2=afterburner
27 virtual int setThrottles(const double* const pos, const int num);
28 virtual void setBrakes(const double left, const double right);
29 };

```

Listing 181: DynamicsModel: os três grupos de interface (resumo de include/mixr/models/dynamics/DynamicsModel.hpp).

O padrão de `false` nesses métodos *parece* desenhado para permitir que o chamador detecte modos não suportados e recue para outro mecanismo. Seria um bom desenho. Não é o que acontece:

Armadilha — o valor de retorno dos comandos de autopiloto é descartado

Autopilot::headingController(), altitudeController() e velocityController() chamam setCommandedHeadingD(), setHeadingHoldOn() e congêneres **ignorando o bool devolvido**. A escolha entre modo *hold* e comando de manche é feita por isHeadingHoldOn() || isNavModeOn() — estado do próprio autopiloto — e não pela resposta do modelo de dinâmica. Uma busca em toda a árvore não encontra um único ponto que teste esses retornos.

Na prática: acoplar um Autopilot a um DynamicsModel que não implemente *heading hold* não produz

recuo automático para o manche. Produz uma aeronave que não vira, sem nenhum diagnóstico. Os `false` existem como valor de retorno plausível, não como protocolo em uso.

`AerodynamicsModel` estende o contrato com métricas aerodinâmicas, todas com *default* retornando 0.0:

```

1 class AerodynamicsModel : public DynamicsModel {
2 public:
3     virtual double getGload() const; // G's (+up, 1.0 em voo nivel)
4     virtual double getMach() const; // numero de Mach
5     virtual double getAngleOfAttack() const; // rad (getAngleOfAttackD() em graus)
6     virtual double getSideSlip() const; // rad
7     virtual double getFlightPath() const; // rad
8     virtual double getCalibratedAirspeed() const; // kts
9     virtual double getAmbientPressureRatio() const;
10    virtual double getWingSweepAngle() const;
11    // entradas de trim (alem do stick)
12    virtual void setTrimSwitchRollInput(const double rollTrim);
13    virtual void setTrimSwitchPitchInput(const double pitchTrim);
14    virtual void setRudderPedalInput(const double pedal);
15 };

```

Listing 182: `AerodynamicsModel:` métricas aerodinâmicas (resumo de `include/mixr/models/dynamics/AerodynamicsModel.hpp`).

VI.4.2 `RacModel`: comportamento sem aerodinâmica

Rate-and-Control simula o movimento como **resposta de primeira ordem** aos comandos — sem equações de sustentação, arrasto ou momento. É o modelo adequado para alvos, ameaças simuladas e *wingmen* de baixa prioridade.

```

1 class RacModel final : public AerodynamicsModel {
2     // Slots:
3     // minSpeed <Number> -- velocidade minima (kts)
4     // speedMaxG <Number> -- velocidade para Gmax (kts)
5     // maxg <Number> -- G's maximos (default: 4.0)
6     // maxAccel <Number> -- aceleracao maxima (m/s/s, default: 10.0)
7     // cmdAltitude <Distance>
8     // cmdHeading <Angle>
9     // cmdSpeed <Number> -- velocidade comandada (kts)
10
11 private:
12     double vpMin {}; // velocidade minima (m/s)
13     double vpMaxG {250.0}; // velocidade para G maximo (m/s)
14     double gMax {4.0}; // G maximo
15     double maxAccel {10.0}; // aceleracao maxima longitudinal (m/s/s)
16     double cmdAltitude {-9999.0};
17     double cmdHeading {-9999.0};
18     double cmdVelocity {-9999.0};
19
20     void updateRAC(const double dt); // logica interna de convergencia
21 };

```

Listing 183: `RacModel:` campos de performance e *slots* (resumo de `include/mixr/models/dynamics/RacModel.hpp`).

A cada `dynamics(dt)`, `updateRAC()` calcula o erro entre o rumo atual e o comandado, vira a uma taxa proporcional limitada por $G_{\max}$ e v_{pMaxG} ; sobe ou desce em direção à altitude comandada limitado por `maxAccel`; e acelera em direção à velocidade comandada dentro do mesmo limite. Não há matriz de rotação nem quaternion — o modelo opera diretamente em ângulos de Euler e velocidade escalar.

VI.4.3 LaeroModel: quatro graus de liberdade

Modelo linear de quatro graus de liberdade (4DOF): translação (x, y, z) mais *yaw* (ψ). *Roll* e *pitch* são determinados pelos comandos, não por equações de momento. A integração usa Adams-Bashforth de segunda ordem:

$$u_k = u_{k-1} + \frac{1}{2}(3\dot{u}_{k-1} - \dot{u}_{k-2}) \Delta t \quad (6)$$

Mais estável que Euler simples com o mesmo custo, pois usa dois passos anteriores para extrapolação. As velocidades no referencial do corpo são convertidas para NED pela matriz de cossenos diretores calculada dos ângulos de Euler atuais via `base::nav::computeRotationalMatrix()` (Seção III.7).

VI.4.4 JSBSimModel: fidelidade completa (6DOF)

Integra a biblioteca JSBSim, carregando um arquivo XML de definição de aeronave:

```
1 dynamicsModel: ( JSBSimModel
2   model:  "f16"      -- carrega <rootDir>/aircraft/f16/f16.xml
3   rootDir: "/data/jsbsim"
4 )
```

Listing 184: JSBSimModel em EDL: nome do modelo e diretório raiz.

A cada `dynamics(dt)`, o ciclo é:

```
1 void JSBSimModel::dynamics(const double dt)
2 {
3   // 1. Autopilot escreveu comandos via setCommandedHeadingD() etc.
4   //   -- JSBSimModel os traduz para posicoes de superficie FCS
5   //   (aileron, elevator, rudder, throttle) via PropertyManager
6
7   // 2. Executa um passo do simulador
8   fdm->Run();
9
10  // 3. Extrai resultados e escreve no Player
11  Player* p { static_cast<Player*>(findContainerByType(typeid(Player))) };
12
13  // posicao geodesica
14  p->setPositionLLA(
15    fdm->GetPropagate()->GetLatitudeDeg(),
16    fdm->GetPropagate()->GetLongitudeDeg(),
17    fdm->GetPropagate()->GetAltitudeASLmeters());
18
19  // velocidade NED
20  const double vNorth { fdm->GetPropagate()->GetVel(JSBSim::FGJSBBase::eNorth) };
21  // ...
22
23  // angulos de Euler
24  p->setEulerAngles(
25    fdm->GetPropagate()->GetEuler(JSBSim::FGJSBBase::ePhi),
26    fdm->GetPropagate()->GetEuler(JSBSim::FGJSBBase::eTht),
27    fdm->GetPropagate()->GetEuler(JSBSim::FGJSBBase::ePsi));
28 }
```

Listing 185: Ciclo de integração de `JSBSimModel::dynamics()` (simplificado).

`atReleaseInit()` garante que uma arma inicializa seu `JSBSimModel` com as condições do lançador no instante do lançamento — posição geodésica, velocidade NED e ângulos de Euler são copiados do `Player` pai antes de `fdm->Run()` ser chamado pela primeira vez.

Interface do Autopilot com o modelo de dinâmica. O controlador de rumo é curto e vale ler inteiro, porque é o padrão que os três controladores seguem:

```

1  bool Autopilot::headingController()
2  {
3      // Re-trava o modo (o autopiloto e a fonte da verdade, nao o modelo)
4      setHeadingHoldMode( isHeadingHoldOn() );
5
6      Player* pv{getOwnship()};
7      if (pv != nullptr) {
8          DynamicsModel* md{pv->getDynamicsModel()};
9          if (md != nullptr) {
10             if ( isHeadingHoldOn() || isNavModeOn() ) {
11                 // Quantiza o comando a 0.1 grau antes de enviar
12                 const int ihdg10{static_cast<int>( getCommandedHeadingD() * 10.0f )};
13                 const double hdg{static_cast<double>(ihdg10) / 10.0};
14                 md->setCommandedHeadingD(hdg, maxTurnRateDps, maxBankAngleDegs);
15                 md->setHeadingHoldOn( true );    // <- retorno IGNORADO
16             } else {
17                 md->setHeadingHoldOn( false );    // <- retorno IGNORADO
18                 md->setControlStickRollInput( getControlStickRollInput() );
19             }
20         }
21     }
22     return true;    // sempre true
23 }

```

Listing 186: Autopilot::headingController() transcrito (src/models/system/Autopilot.cpp).

Dois detalhes que só se veem lendo o código. O primeiro é a **quantização a 0,1°**: o comando é truncado antes de descer ao modelo, o que evita que ruído numérico no cálculo de rumo produza microcorreções contínuas. O segundo é que a decisão entre modo *hold* e manche depende exclusivamente do estado do autopiloto — `isHeadingHoldOn()` ou `isNavModeOn()` — e nunca do que o modelo respondeu.

Modelo	Fidelidade	CPU	DOF	Uso típico
RacModel	Baixa	Mínimo	Cinemático	Alvos, ameaças
LaeroModel	Média	Baixo	4DOF	Wingmen, UAVs
JSBSimModel	Alta	Significativo	6DOF	Ownship

Table 15: Comparação dos modelos de dinâmica de voo do MIXR.

VI.5 Sistemas embarcados: base e cadeia de RF

Quase todo subsistema embarcado de um `Player` herda de `System`, que por sua vez herda de `Component`. As exceções valem ser conhecidas: `DynamicsModel` deriva diretamente de `Component`, e o mesmo vale para `Route` e `Steerpoint` — nenhum dos três participa do despacho por fase de `System`. A extensão central que `System` acrescenta é o acesso preguiçoso (*lazy*) ao `Player` dono e ao `WorldModel` da simulação — via `findContainerByType()`, na primeira chamada, com resultado em *cache*:

```

1  Player* System::getOwnship()
2  {
3      // Primeira chamada: findOwnship() sobe a arvore de containers ate
4      // encontrar o Player dono e guarda o ponteiro em 'ownship'.
5      // Chamadas seguintes: o teste contra nullptr e todo o custo.
6      if (ownship == nullptr) findOwnship();
7      return ownship;
8  }
9
10 // findOwnship() faz a busca propriamente dita:
11 //   ownship = static_cast<Player*>(findContainerByType(typeid(Player)));
12 // idem para getWorldModel(), que busca a WorldModel na arvore

```

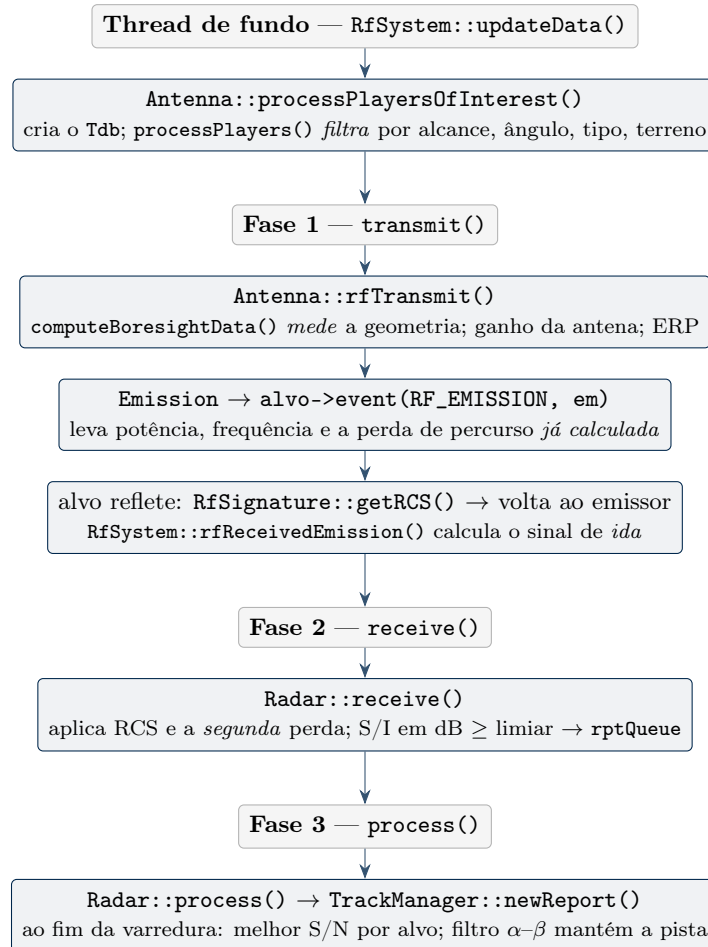
Listing 187: `System::getOwnship()`: busca preguiçosa com cache — custo $O(n)$ apenas na primeira chamada, $O(1)$ depois (`src/models/system/System.cpp`).

A segunda extensão de `System` é a que dá ritmo a tudo: o despacho por fase. Vale lembrá-lo aqui, porque cada método citado no resto desta seção é um dos quatro (Seção II.4):

Fase	Método de System	Quem o implementa na cadeia RF
0	<code>dynamics(dt4)</code>	<code>Stt</code> (aponta a antena para a pista)
1	<code>transmit(dt4)</code>	<code>RfSensor</code> → <code>Antenna::rfTransmit()</code>
2	<code>receive(dt4)</code>	<code>Radar</code> (equação do radar, limiar)
3	<code>process(dt4)</code>	<code>Radar</code> (consolida e alimenta o <code>TrackManager</code>)

VI.5.1 O percurso completo de uma detecção

As subseções seguintes descrevem cada peça da cadeia RF isoladamente. Antes disso, vale ver o caminho inteiro de uma única detecção — do momento em que o radar decide que um alvo é interessante até a pista aparecer no `TrackManager`. É esse percurso que dá sentido a cada peça, e ele atravessa **duas threads e três fases**:



Três propriedades do desenho merecem ser fixadas antes de descer aos detalhes, porque cada uma contraria uma suposição natural:

1. **A filtragem e a medição estão em lugares diferentes.** Escolher *quais* alvos considerar é caro e tolera defasagem, então roda na *thread* de fundo. Medir a geometria *daqueles* alvos é barato e precisa ser atual, então roda a cada transmissão.
2. **A entrega é por evento, não por chamada.** A Emission chega ao alvo como RF_EMISSION (Seção III.2.4) — o transmissor não conhece a interface do receptor.
3. **A detecção não vira pista na mesma fase.** Entre o eco (fase 2) e a pista (fase 3) há uma fila, e a consolidação por alvo acontece apenas ao fim de uma varredura completa da antena.

VI.5.2 Gimbal e Tdb: apontamento e preenchimento

Gimbal é a base de tudo que aponta — antenas, óticas, buscadores IR. O seu papel na cadeia é montar, uma vez por passagem, a lista de alvos que vale a pena considerar: o Tdb.

Armadilha — o Tdb é montado na *thread* de fundo, não na fase 1

É natural supor que a seleção de alvos aconteça na fase 1, junto com a transmissão. Não acontece. Quem chama `processPlayersOfInterest()` é `RfSystem::updateData()` — e `updateData()` roda na ***thread de fundo***, a uma taxa própria e não sincronizada com o quadro de tempo crítico (o caminho IR é análogo, via `IrSystem::updateData()`). Gimbal sequer sobrescreve `updateTC()`. A consequência de projeto é boa: a filtragem de centenas de *players* — cara e tolerante a alguns milissegundos de defasagem — sai do caminho crítico. A consequência prática é que o Tdb que a fase 1 encontra pode ter sido montado há um ou dois quadros.

O dimensionamento vem do *slot* `maxPlayersOfInterest`, cujo padrão é **200**. A constante `Gimbal::MAX_PLAYERS` — *alias* de `MIXR_CONFIG_MAX_PLAYERS_OF_INTEREST`, padrão 4000 — é outra coisa: dimensiona os *arrays* temporários dentro de `Antenna::rfTransmit()` e do `IrSeeker`. E ela **não limita** o *slot*: `setMaxPlayersOfInterest()` atribui sem verificar, de modo que declarar `maxPlayersOfInterest` acima de 4000 é aceito sem reclamação.

O que acontece nesse caso não é um estouro de *buffer* — e vale dizê-lo, porque é a conclusão natural e ela está errada. Os *arrays* do Tdb são alocados dinamicamente pelo valor do *slot*, e os consumidores saturam a contagem antes de indexar: tanto `Antenna::rfTransmit()` quanto o `IrSeeker` abrem com `if (ntgts > MAX_PLAYERS) ntgts = MAX_PLAYERS;`. O efeito real é mais discreto e por isso mais traiçoeiro: os alvos além do quatro-milésimo são **descartados em silêncio** na transmissão, embora o Tdb os tenha recolhido. Um cenário com mais de 4000 *players* de interesse não quebra — apenas deixa de iluminar parte deles.

```
1 unsigned int Gimbal::processPlayersOfInterest(base::PairStream* const poi)
2 {
3     // Cria novo Tdb para este frame
4     const auto tdb0 = new Tdb(maxPlayers, this);
5
6     // Preenche: LOS, ranges, rngRates, vetores az/el em coord. gimbal
7     unsigned int ntgts { tdb0->processPlayers(poi) };
8
9     setCurrentTdb(tdb0); // armazena para uso por Antenna e RfSensor
10    tdb0->unref();
11    return ntgts;
12 }
```

Listing 188: `Gimbal::processPlayersOfInterest()`: cria o `Tdb` e dispara `processPlayers()` que computa a geometria (`src/models/system/Gimbal.cpp`).

Aqui é preciso desfazer um segundo mal-entendido, complementar ao anterior: `Tdb::processPlayers()` **não calcula geometria**. Ele apenas *filtra* a lista de *players* — por alcance máximo, por ângulo, por máscara de tipo, por oclusão de terreno e pelo *flag* de “apenas locais”. O resultado é uma lista de candidatos, não um conjunto de vetores.

A geometria propriamente dita — LOS, alcances, taxas de aproximação, transformação para o referencial do *gimbal*, ângulos fora do *boresight* — é calculada por `Tdb::computeBoresightData()`, e essa é chamada **a cada transmissão**, de dentro de `Antenna::rfTransmit()` e do `IrSeeker`. Ou seja: a divisão de custo é “filtrar raramente, medir sempre”, e não o contrário:

```
1   for (unsigned int i = 0; i < numTgts; i++) {
2
3       // Target vectors (ECEF or local gaming area NED)
4       base::Vec3d pt; // Position Vector
5       base::Vec3d vt; // Velocity vector [ x y z ] (meters/second)
6       // ... escolha entre ECEF e NED conforme 'usingEcefFlg' ...
7
8       // Target LOS vector
9       base::Vec3d los{pt - p0};
10
11      // Normalized and compute length [unit vector and range(meters)]
12      ranges[i] = los.normalize();
13
14      // Computer range rate (meters/sec)
15      rngRates[i] = ( vt - v0 ) * los );
16
17      // Save the LOS vectors (own to tgt) and (tgt back to own)
18      if (usingEcefFlg) {
19          // Rotate the LOS vectors into their local tangent planes
20          losO2T[i] = wm * los;
21          losT2O[i] = targets[i]->getWorldMat() * (-los);
22      } else {
23          losO2T[i] = los;
24          losT2O[i] = -los;
25      }
26  }
27
28  // Start with the body to gimbal matrix
29  base::Matrixd mm{gimbal->getRotMat()};
30
31  // Post multi by the inertial (NED) to body matrix
32  if (ownHdgOnly) {
33      // Using heading only, ignore ownship roll and pitch
34      base::Matrixd rr;
35      rr.makeRotate( ownship->getHeading(), 0, 0, 1);
36      mm *= rr;
37  } else {
38      // Using ownship roll, pitch and heading
39      mm *= ownship->getRotMat();
40  }
41  // ... losG = mm * losO2T, e dai os angulos fora do boresight ...
```

Listing 189: `Tdb::computeBoresightData()`: o núcleo geométrico — LOS, alcance, taxa de variação e transformação para o referencial do *gimbal* (`src/models/Tdb.cpp`).

Repare no *flag* `ownHdgOnly`, que vem de `Gimbal::isUsingHeadingOnly()` e é ligado por padrão: com ele, apenas a proa do *ownship* entra na matriz de transformação, e rolagem e arfagem são ignoradas. O efeito é estabilizar o *gimbal* em relação ao horizonte — é o que se quer de um radar de busca, que não deve varrer o céu e o chão alternadamente quando a aeronave inclina. Desligá-lo faz o feixe acompanhar rigidamente a

atitude do corpo.

`ScanGimbal` acrescenta padrões de varredura programáticos. Os modos reais, do `enum` de `ScanGimbal.hpp`, são sete:

Modo	Padrão
MANUAL_SCAN	sem varredura: o apontamento é comandado de fora
HORIZONTAL_BAR_SCAN	varredura em barras horizontais (busca típica de radar)
VERTICAL_BAR_SCAN	idem, em barras verticais
CONICAL_SCAN	cone em torno do eixo — usado no rastreo de alvo único
CIRCULAR_SCAN	círculo completo (vigilância)
PSEUDO_RANDOM_SCAN	sequência pseudoaleatória de posições
SPIRAL_SCAN	espiral, do centro para fora

Table 16: Modos de varredura de `ScanGimbal`. Não existem `BAR_SCAN` nem `SECTOR_SCAN`.

`StabilizingGimbal` adiciona estabilização inercial que desacopla o movimento da antena do movimento da aeronave, mantendo o feixe no alvo durante manobras.

VI.5.3 Antenna: transmissão e cálculo de ERP

`Antenna` herda de `ScanGimbal` — e não de `Gimbal` diretamente, o que importa porque `Stt` e `Sar` comandam-na pela API de varredura. Na fase 1, `rfTransmit()` itera sobre os alvos no `Tdb`, calcula a Potência Efetivamente Irradiada (ERP) na direção de cada alvo usando o padrão de ganho da antena, e cria um objeto `Emission` por alvo:

```

1  // A geometria e (re)calculada AQUI, a cada transmissao
2  unsigned int ntgts{tdb->computeBoresightData()};
3  if (ntgts > MAX_PLAYERS) ntgts = MAX_PLAYERS;
4
5  // Consulta o padrao de ganho: tabela 2D (az x el) em DECIBEIS
6  double gainTgt0[MAX_PLAYERS]{};
7  if (gainPatternDeg) {
8      for (unsigned int i1 = 0; i1 < ntgts; i1++) {
9          gainTgt0[i1] = gainFunc2->f( (aazr[i1] * base::angle::R2DCC),
10                                     (aelr[i1] * base::angle::R2DCC) )/10.0;
11      }
12  } else {
13      for (unsigned int i1 = 0; i1 < ntgts; i1++) {
14          gainTgt0[i1] = gainFunc2->f( aazr[i1], aelr[i1] )/10.0f;
15      }
16  }
17  base::pow10Array(gainTgt0, gainTgt, ntgts);    // 10^(dB/10) -> razao de POTENCIA
18
19  // Ganho efetivo = padrao x ganho nominal da antena
20  double aeGain[MAX_PLAYERS]{};
21  base::multArrayConst(gainTgt, getGain(), aeGain, ntgts);
22
23  // Compute Effective Radiated Power (watts) (Equation 2-1)
24  double erp[MAX_PLAYERS]{};
25  base::multArrayConst(aeGain, xmit->getPower(), erp, ntgts);
26
27  for (unsigned int i = 0; i < ntgts; i++) {
28      // Only of power exceeds an optional threshold
29      if (erp[i] > threshold) {
30          // ... obten uma Emission (reciclada da pilha, ou clonada) ...
31          em->setRange( ranges[i] );    // <- e setRange() que calcula lossRng
32          em->setPower( erp[i] );

```

```

33     em->setGain( aeGain[i] );
34     em->setTarget( targets[i] );
35     // ...
36     targets[i]->event(RF_EMISSION, em); // entrega POR EVENTO
37 }
38 }

```

Listing 190: `Antenna::rfTransmit()`: do padrão de ganho à ERP e à entrega por evento (`src/models/system/Antenna.cpp`).

Dois detalhes desse trecho merecem ser fixados, porque cada um corrige uma suposição natural.

Armadilha — o ganho é dividido por dez antes da exponenciação

A tabela de padrão de antena guarda **decibéis**, e a conversão para razão linear é $10^{dB/10}$ — daí o `/10.0` antes do `pow10Array()`. Esquecê-lo, e calcular 10^{dB} , produz ganhos com ordens de grandeza a mais. O expoente dividido por dez é o que faz da razão uma razão de *potência*, e não de amplitude.

É `Emission::setRange()` — não o `Tdb` — que calcula e guarda a perda de percurso $1/(4\pi r^2)$. O receptor nunca precisa conhecer a distância: multiplica pelo campo já pronto (Seção VI.15.1).

VI.5.4 RfSystem, RfSensor e Radar

Na fase 2, cada `Emission` refletida retorna ao transmissor original. O cálculo do sinal, porém, **não acontece todo em um lugar** — ele é partido em dois, e entender essa partição é entender o desenho da cadeia RF.

Primeira metade: o trecho de ida, em `RfSystem`. Todo sistema RF que recebe uma emissão calcula primeiro o sinal de *um caminho só*, antes de saber se é um eco ou uma emissão direta:

```

1  double losses{getRfSignalProcessLoss() * em->getAtmosphericAttenuationLoss()
2                * em->getTransmitLoss()};
3  if ( losses < 1.0) losses = 1.0;
4
5  const double rl{em->getRangeLoss()};      // 1/(4*pi*R^2), ja pronto
6
7  // Sinal de UM caminho
8  const double signal{em->getPower() * rl * raGain / losses};

```

Listing 191: `RfSystem::rfReceivedEmission()`: o sinal de ida (`src/models/system/RfSystem.cpp`).

Segunda metade: o trecho de volta, em `Radar`. Só quem sabe que aquilo é um eco de radar aplica a RCS e a *segunda* perda de percurso:

```

1  // Interferencia = ruido termico + jamming; ruido = so o termico.
2  // (equacao S/I de Hannen: I = N + J)
3  const double interference{(getRfRecvNoise() + jamSignal) * getRfReceiveLoss()};
4  const double noise{getRfRecvNoise() * getRfReceiveLoss()};
5
6  // Para cada Emission da fila (protegida por packetLock):
7  double rcs{em->getRCS()};
8  double rl {em->getRangeLoss()};
9
10 signal *= (rcs * rl);      // <- RCS e a SEGUNDA perda de percurso
11 signal *= rflGain;        // ganho de integracao (slot 'igain')
12
13 // Razao sinal/interferencia, em dB
14 // ... comparada contra o limiar, tambem em dB:

```

```

15 |     if (signalToInterferenceRatioDbl >= getRfThreshold()
16 |         && em->getRange() <= (maxRng * 1.25)
17 |         && rptQueue.isNotFull()) {
18 |         // enfileira o relatório -- NAO chama o TrackManager aqui
19 |     }

```

Listing 192: `Radar::receive()`: o trecho de volta e o limiar (`src/models/system/Radar.cpp`, essência).

Armadilha — a equação do radar do MIXR não é a do livro-texto

A forma clássica $P_r = \frac{P_t G_t G_r \lambda^2 \sigma}{(4\pi)^3 R^4}$ tem um λ^2 e um $(4\pi)^3$. **Nenhum dos dois aparece no código.** O que o MIXR calcula é

$$S = P_t \cdot G_r \cdot r_\ell^2 \cdot \sigma \cdot G_{\text{int}} / L, \quad r_\ell = \frac{1}{4\pi R^2},$$

com o quadrado de r_ℓ vindo de r_ℓ ter sido aplicado uma vez na ida e outra na volta. O comprimento de onda existe na **Emission** e é usado para Doppler, não para o balanço de potência. Quem comparar resultados do MIXR com uma planilha de equação de radar padrão vai encontrar uma diferença constante — e ela está aqui.

Por que — por que a perda de percurso é pré-calculada na emissão

O detalhe elegante do desenho é o campo `lossRng` da **Emission**: o transmissor, que já conhece a distância ao alvo, calcula $1/(4\pi R^2)$ uma vez e o embarca na mensagem. O receptor **nunca precisa saber a que distância está o emissor** — basta multiplicar. É o mesmo valor aplicado duas vezes que produz o R^4 do eco, sem que ninguém eleve nada à quarta potência.

Note ainda dois pontos que a leitura apressada inverte. O limiar `getRfThreshold()` é comparado em **dB**, não em razão linear. E `receive()` **não** entrega nada ao **TrackManager**: ele enfileira relatórios em `rptQueue`, e é `Radar::process()` — na fase 3, ao fim da varredura — que consolida o melhor sinal por alvo e só então chama `newReport()`.

Jammer herda de **RfSensor** e transmite ruído que eleva o nível de interferência nos receptores inimigos. Não há distinção entre banda larga e *spot*: `Jammer::transmit()` marca toda emissão como **Emission::ECM_NOISE**, e é o **Radar** receptor que soma esse sinal ao seu termo de interferência. **Rwr** (*Radar Warning Receiver*) recebe **Emissions** sem transmitir e alimenta o **RwrTrkMgr** com as ameaças detectadas.

Resta uma quarta subclasse, e ela é peculiar: **SensorMgr** tem `EMPTY_SLOTTABLE`, `EMPTY_COPYDATA`, `EMPTY_DELETEDATA` e um construtor vazio — o arquivo inteiro tem doze linhas úteis. Não acrescenta comportamento nenhum a **RfSensor**.

Por que — uma classe vazia que existe para agrupar

Não é código morto; é um recipiente. **RfSensor** herda de **System**, e portanto é um **Component** com filhos — pendurar vários radares sob um **SensorMgr** os faz atualizar juntos, na ordem da lista, sob um único nó nomeado da árvore.

Ela existe pela mesma razão que existe um **Component** genérico: para dar um *nó* onde só se quer estrutura. O ganho está em EDL — um `sensor: ( SensorMgr { ( Radar ... ) ( Rwr ... ) } )` dá ao *player* um conjunto de sensores em vez de um; e como **SensorMgr** é um **RfSensor**, todo o código que espera um sensor continua funcionando sem saber que recebeu um grupo.

É o padrão *Composite* clássico, com o mínimo de código que ele exige.

VI.6 Track managers

O `TrackManager` é o elo entre o sensor (que detecta um alvo num único quadro) e o `OnboardComputer` (que precisa de histórico para calcular ameaças e selecionar armas). Mantém a lista de `Tracks`, filtra associações entre relatórios e *tracks*, e propaga a cinemática entre quadros.

```
1  class TrackManager : public System {
2      // Slots:
3      //   maxTracks    <Number>  -- max de tracks (default: MAX_TRKS)
4      //   maxTrackAge  <Time>    -- tempo maximo de vida de um track (default: 3s)
5      //   firstTrackId <Integer> -- primeiro ID de track (default: 1000)
6      //   alpha        <Number>  -- parametro alpha do filtro (default: 1.0)
7      //   beta         <Number>  -- beta (default: 0.0)
8      //   gamma        <Number>  -- gamma (default: 0.0)
9
10     Track*    tracks[MAX_TRKS] {}; // lista de tracks ativos
11     unsigned int nTrks {};         // numero de tracks ativos
12     mutable long trkListLock {};   // semaforo que protege a lista
13
14     // Filas de entrada (de Emission + SNR)
15     base::safe_queue<Emission*> emQueue { MAX_TRKS };
16     base::safe_queue<double> snQueue { MAX_TRKS };
17
18     // Matriz de predicao A para o filtro alfa-beta
19     double A[3][3] {}; // computada em makeMatrixA(dt)
20     double alpha {1.0};
21     double beta {};
22     double gamma {};
23
24     double maxTrackAge {3.0}; // segundos
25 };
```

Listing 193: `TrackManager`: campos da classe e parâmetros do filtro α - β - γ (resumo de `include/mixr/models/system/trackmanager/TrackManager.hpp`).

O estado de um `Track` combina cinemática (posição, velocidade, aceleração em NED relativas ao *own-ship*), qualidade do sinal (`snDbl`), identificação (`iffCode`, `trackSide`) e temporização (`age`). Quando `trkPlayer != nullptr`, a cinemática é atualizada diretamente da posição do player local — sem correlação de sinal. Para players *networked* ou não resolvidos, `processTrackList()` aplica o filtro α - β - γ :

```
1  // X(k+1) = A * X(k) + B * U(k)
2  // onde U(k) = (posicao_observada - posicao_predita)
3  if (haveU[i]) {
4      double b0 { alpha }; // correcao de posicao
5      double b1 {};        // correcao de velocidade (= beta / dt)
6      double b2 {};        // correcao de aceleracao (= gamma / 2dt^2)
7      tracks[i]->setPosition(
8          (tpos*A[0][0] + tvel*A[0][1] + tacc*A[0][2]) + (u[i]*b0));
9      tracks[i]->setVelocity(
10         (tpos*A[1][0] + tvel*A[1][1] + tacc*A[1][2]) + (u[i]*b1));
11     tracks[i]->setAcceleration(
12         (tpos*A[2][0] + tvel*A[2][1] + tacc*A[2][2]) + (u[i]*b2));
13 } else {
14     // Sem observacao neste frame: apenas prediz com o modelo cinematico
15     tracks[i]->setPosition(tpos*A[0][0] + tvel*A[0][1] + tacc*A[0][2]);
16     // ...
17 }
```

Listing 194: Filtro α - β - γ em `RwrTrkMgr::processTrackList()`: equação de estado discreta (`src/models/system/trackmanager/RwrTrkMgr.cpp`).

Armadilha — β é usado por dois gerentes; γ por nenhum

Os três *slots* alpha, beta e gamma sugerem um filtro α - β - γ completo. O código conta outra história:

- Em `RwrTrkMgr`, os ganhos de velocidade e aceleração são declarados como `double b1{}; double b2{};` — literalmente zero. **Nem β nem γ têm qualquer efeito ali.**
- Em `AirTrkMgr` e `GmtiTrkMgr`, β é usado, na forma $b_1 = \beta/\text{idade}$ — dividido pela idade da *pista*, não pelo Δt do quadro.
- A linha que calcularia b_2 a partir de γ está **comentada** nos dois. Uma busca por **gamma** em todo o diretório dos gerentes mostra que ele é lido para um membro e nunca mais consultado.

Na prática: ajustar **gamma**: num arquivo EDL não produz efeito nenhum, e não gera aviso. O filtro efetivo é α - β em dois dos quatro gerentes, e α puro nos demais.

`AirAngleOnlyTrkMgr` rastreia alvos usando apenas medições angulares (azimute e elevação), sem alcance — o modo de operação de buscadores IR passivos e *seekers* de mísseis antes do *endgame*. Entre ele e `TrackManager` existe ainda `AngleOnlyTrackManager`, que acrescenta os *slots* `azimuthBin` e `elevationBin` — o tamanho da célula de resolução angular usada para associar relatórios a pistas.

A variante `AirAngleOnlyTrkMgrPT` não é *predictive tracking*: o PT é de *perceived truth*. Ela existe para consumir os relatórios do `MergingIrSensor`, que funde em um único retorno os alvos caídos dentro da mesma célula angular. A “verdade percebida” é justamente esse centroide de dois ou mais alvos — e o *slot* `usePerceivedPosVel` decide se posição e velocidade fundidas também são aproveitadas, ou apenas os ângulos.

VI.7 Sensores especializados: IR, SAR e variantes de radar

VI.7.1 Cadeia IR: `IrSeeker` e `IrQueryMsg`

O equivalente IR da cadeia RF opera com o mesmo ciclo de fases, mas o *protocolo* é de *query/response* em vez de emissão livre. Na fase 1, o `IrSeeker` constrói uma `IrQueryMsg` por alvo e a envia ao *player* alvo via `Player::irReceivedQuery()`:

```
1 unsigned int ntgts { tdb0->computeBoresightData() };
2
3 for (unsigned int i = 0; i < ntgts; i++) {
4     // Filtra pelo alcance maximo do sensor
5     if (maximumRange > 0.0 && ranges[i] > maximumRange) continue;
6
7     // Obtém um pacote livre do pool (evita alocação por frame)
8     IrQueryMsg* query { freeQueryStack.pop() };
9     if (query == nullptr) query = new IrQueryMsg();
10
11     // Preenche o pacote com os parametros do sensor e do alvo
12     query->setRange(ranges[i]);
13     query->setAngleAspect(anglesOffBoresight[i]);
14     query->setSendingSensor(this);
15     query->setGimbal(this);
16
17     // Envia ao alvo: alvo consultara seu IrSignature e preencherá o sinal
18     targets[i]->irReceivedQuery(query, this);
19 }
```

`IrSeeker::irRequestSignature()`: envia `IrQueryMsg` a cada alvo no `Tdb` (`src/models/system/IrSeeker.cpp`, simplificado).

Na fase 2, o alvo preenche a potência do sinal por banda espectral de volta na `IrQueryMsg`, e `WorldModel::getAtmosphere()` atenua o sinal pela transmitância atmosférica. O `IrSensor` calcula o SNR — razão entre o sinal atenuado e a irradiância equivalente de ruído (NEI) multiplicada pelo IFOV — e, se acima do limiar, gera um relatório para o `AngleOnlyTrackManager`.

VI.7.2 Variantes de radar: `Tws`, `Stt` e `Gmti`

`Tws` (*Track While Scan*). Semanticamente é uma subclasse de `Radar` que mantém a varredura contínua enquanto rastreia múltiplos alvos. Toda a lógica TWS vive no `AirTrkMgr` associado — o `Tws` funciona como uma etiqueta de intenção que configura o `TrackManager` correto.

`Stt` (*Single Target Track*). Implementa `dynamics(dt)` com lógica real: aponta a antena para o primeiro *track* do `TrackManager` usando varredura cônica (`CONICAL_SCAN`), estimando o erro de apontamento pelo desequilíbrio de potência entre os lados do cone. Na ausência de *track*, reverte para varredura horizontal de busca.

`Gmti` (*Ground Moving Target Indicator*). Aponta a antena para um *Point of Interest* (vetor NED, *slot poi*) e alimenta o `GmtiTrkMgr` com detecções de alvos terrestres em movimento.

VI.7.3 `Sar`: imagem sintética de abertura

`Sar` herda de `Radar` — não de `Gimbal` — e é o componente que atende a um pedido de imageamento. O fluxo é disparado por um `ActionImagingSar` (Seção VI.16.2): a ação chama `setStarePoint()` com latitude, longitude e elevação do ponto de interesse, depois `requestImage()` com largura, altura e resolução. O `Sar` aponta a antena, marca `isImagingInProgress()` e, decorrido o tempo de integração, produz um objeto `Image`.

O único *slot* da classe é `chipSize`; o tempo de integração é uma constante de arquivo de dez segundos, não configurável.

Armadilha — o `Sar` não usa o terreno e não gera uma imagem real

Duas expectativas naturais que o código não cumpre:

- **Não há terreno.** `Sar.cpp` não inclui nem menciona `terrain::Terrain`; não há chamada a `getElevations()`, nenhum cálculo de ângulo de incidência, nenhuma geometria de visada.
- **A imagem é um padrão de teste.** Ao concluir, o código faz literalmente `p->testImage(width, height)` — uma figura sintética gerada proceduralmente, sem relação alguma com o cenário.

E não existe `SAR_COMPLETE_EVENT`: uma busca por esse identificador em toda a árvore não devolve nada. A conclusão é **consultada**, não notificada — `ActionImagingSar::process()` chama `sar->isImagingInProgress()` a cada quadro até que devolva falso.

O que está pronto é o *protocolo*: pedido, apontamento, espera, entrega de um `Image`. O conteúdo da imagem é o ponto de extensão deixado em aberto.

VI.8 Comunicações e gerenciamento de armazenamento

VI.8.1 Cadeia de comunicações: `Radio`, `CommRadio` e `Datalink`

`Radio` é a base — e note que ela deriva de `RfSystem`, não de `System` diretamente. Essa escolha de herança é a chave de todo o mecanismo: **no MIXR, uma mensagem de rádio é uma emissão de RF como qualquer outra.**

```
1 bool CommRadio::transmitDataMessage(base::Object* const msg)
2 {
3     bool sent{};
4     if (getOwnship() == nullptr) return sent;
5
6     if (msg != nullptr && isTransmitterEnabled() && getAntenna() != nullptr) {
7         // A mensagem viaja DENTRO de uma Emission
8         const auto em = new Emission();
9         em->setDataMessage(msg);           // <- a carga util
10        em->setFrequency(getFrequency());
11        em->setBandwidth(getBandwidth());
12        em->setPower(getPeakPower());
13        em->setTransmitLoss(getRfTransmitLoss());
14        em->setMaxRangeNM(getMaxDetectRange());
15        em->setTransmitter(this);
16        em->setReturnRequest(false);        // nao queremos eco
17        getAntenna()->rftTransmit(em);      // mesma cadeia RF de sempre
18        em->unref();
19        sent = true;
20    }
21    return sent;
22 }
```

Listing 196: `CommRadio::transmitDataMessage()` transcrito (`src/models/system/CommRadio.cpp`).

Por que — rádio reusa a cadeia de RF inteira

Não há varredura da lista de *players*, não há comparação de canal, não há entrega direta. A mensagem entra na mesma *Antenna* e percorre o mesmo caminho de *Emission* descrito na Seção VI.5.3. Isso significa que a comunicação herda, de graça, tudo o que a cadeia RF já modela: alcance limitado pela potência, perda de percurso, direcionalidade da antena, e a possibilidade de ser interferida por um *Jammer*. Um rádio no MIXR não é uma abstração de mensagem — é um transmissor.

Na recepção, o `CommRadio` extrai a carga útil e a entrega ao `Datalink`, não ao *player*:

```
1 void CommRadio::receivedEmissionReport(Emission* const em)
2 {
3     if (em != nullptr && datalink != nullptr) {
4         base::Object* msg{em->getDataMessage()};
5         if (msg != nullptr) datalink->event(DATALINK_MESSAGE, msg);
6     }
7 }
```

Listing 197: `CommRadio::receivedEmissionReport()` transcrito.

Repare no nome do *token*: `DATALINK_MESSAGE`, sem sufixo `_EVENT`. E note que a carga é um `base::Object*` qualquer — o *framework* não impõe formato de mensagem.

Datalink estende System (e não Radio) e é o destinatário dessas mensagens; é ele que as enfileira e as apresenta ao OnboardComputer. A separação faz sentido: o rádio é o meio físico, o *datalink* é o protocolo.

VI.8.2 Iff: identificação amigo ou inimigo

Iff mantém os códigos de transponder e os *flags* que dizem quais modos estão ligados:

```
1  iff1 : ( Iff
2      mode1:  45    // 5 bits octal
3      mode2: 1234  // 12 bits
4      mode3a: 7777 // squawk (codigo ATC) -- o slot e "mode3a"
5      // mode4 e mode5: criptografados (omitidos em EDL)
6  )
```

Listing 198: Iff em EDL: modos 1–5 com códigos.

Os *slots* reais são onze: os códigos mode1, mode2, mode3a, mode4a e mode4b; os interruptores enableMode1, enableMode2, enableMode3a, enableMode4 e enableModeC; e o seletor whichMode4, que escolhe entre os códigos 4A e 4B.

Armadilha — não há modo 5, e o TrackManager não consulta o Iff

Duas correções sobre o que se costuma supor. Primeira: **não existe** mode5 — o modo 5 (Link 16) não é modelado. O que existe e passa despercebido é o **modo C**, o de altitude, com o seu próprio interruptor. Segunda: nenhum dos quatro gerentes de pista consulta o Iff de nada — uma busca por Iff no diretório dos *track managers* não devolve nada. O campo iffCode de um Track existe, mas quem o preenche tem de ser código da aplicação.

VI.8.3 StoresMgr: gerenciamento de armas

StoresMgr é o equivalente simulado do SMS (*Stores Management System*) de uma aeronave real. O método central, releaseWeapon(), executa o lançamento completo:

```
1  AbstractWeapon* Stores::releaseWeapon(AbstractWeapon* const wpn)
2  {
3      AbstractWeapon* flyout {};
4      Player* own { getOwnship() };
5
6      if (wpn != nullptr && own != nullptr) {
7          // Seta o lançador e executa o release da arma
8          wpn->setLaunchVehicle(own); // herda pos/vel/hdg do lançador
9          flyout = wpn->release();    // muda modo para ACTIVE, cria flyout
10     }
11     return flyout; // pre-ref()'d -- caller deve unref()
12 }
```

Listing 199: Stores::releaseWeapon(): lançamento real — herda estado do lançador e adiciona à simulação (src/models/system/Stores.cpp).

Após o retorno, wpn->release() já adicionou a arma à lista ativa da Simulation via addNewPlayer(): a partir desse momento a arma tem física própria e é processada como qualquer outra entidade no ciclo de quatro fases.

SimpleStoresMgr implementa a seleção de arma por modo de emprego (A2A ou A2G) sem lógica de MFCD ou armamento condicional:

```

1 AbstractWeapon* SimpleStoresMgr::getCurrentWeapon()
2 {
3     AbstractWeapon* wpn{};
4     if ( isWeaponDeliveryMode(A2A) ) {
5         wpn = getNextMissile(); // We need a missile
6     } else {
7         wpn = getNextBomb(); // We need a bomb
8     }
9     return wpn;
10 }

```

Listing 200: SimpleStoresMgr::getCurrentWeapon(): seleciona míssil ou bomba conforme modo de entrega (src/models/system/SimpleStoresMgr.cpp).

O *slot stores*: de ambas as classes aceita um PairStream de AbstractWeapon:

```

1 components: {
2     sms: ( StoresMgr // <- nome de fabrica "StoresMgr" = SimpleStoresMgr!
3         stores: {
4             1: ( Missile // estacao 1
5                 type: "AIM-120C" // slot 'type', herdado de Player
6                 maxTOF: ( Seconds 60.0 )
7                 tsg: ( Seconds 1.5 ) // so guia apos 1.5 s de voo
8             )
9             3: ( Missile
10                 type: "AIM-120C"
11                 maxTOF: ( Seconds 60.0 )
12                 tsg: ( Seconds 1.5 )
13             )
14             5: ( Bomb // estacao 5
15                 type: "MK-82"
16                 fuzeTime: ( Seconds 4.0 ) // e 'fuzeTime', nao 'maxFuseTime'
17             )
18             7: ( Chaff maxTOF: ( Seconds 8.0 ) )
19         }
20     )
21 }

```

Listing 201: StoresMgr em EDL: lista de armas por estação.

Três correções escondidas nesse bloco, e todas são pegadinhas reais:

Armadilha — (StoresMgr) não instancia StoresMgr

O nome de fábrica "StoresMgr" pertence a SimpleStoresMgr. A classe StoresMgr registra-se como "BaseStoresMgr" — e nem sequer está registrada em models/factory.cpp. Escrever (StoresMgr ...) num arquivo EDL constrói, portanto, um SimpleStoresMgr, com a sua seleção de arma por modo A2A/A2G. É o caso mais consequente da divergência entre nome de classe e nome de fábrica (Seção III.3.1).

Além disso: armas **não têm** um *slot model*. A identidade de uma arma vem do *slot type*, herdado de Player, e é essa *string* que o Ntm usa para casar tipos DIS (Seção X.5.1). E o *slot* de espoleta da Bomb chama-se *fuzeTime*, não *maxFuseTime*.

Gun: o canhão como um store. A classe chama-se Gun, no singular, e a sua herança é a chave para entender o módulo: ela deriva de ExternalStore, exatamente como Stores — e não de System nem de StoresMgr.

Por que — ExternalStore é o elo que falta

`ExternalStore` é a classe que representa *qualquer coisa pendurada numa estação*: tem os *slots* `type` e `jettisonable` e mais nada. Dela derivam `Stores` (um cabide que contém outros *stores*), `Gun` (um canhão), `FuelTank` (com `fuelWt` e `capacity`) e `AvionicsPod`. É essa herança comum que permite tratar uniformemente “o que está pendurado” — inclusive para `jettison` — e é por isso que um canhão ocupa uma estação como se fosse uma bomba.

O disparo é governado por `Gun::process()`, que mantém um `burstFrameTimer` e usa `computeBulletRatePerSecond()` para decidir quantos projéteis cabem no intervalo; `burstFrame()` então os emite. Os *slots* são `bulletType`, `rounds`, `unlimited`, `rate`, `burstRate` e a orientação da boca (`position`, `roll`, `pitch`, `yaw`). Cada `Bullet` é um *player* com trajetória balística própria. Não existe `StoresMgr::releaseOneBullet()`.

VI.9 Navegação: rotas, *steerpoints* e sistemas inerciais/GPS

VI.9.1 Navigation: o sistema base

`Navigation` herda de `System` e mantém o estado de navegação completo. A cada `updateData(dt)`, consulta a `Route` ativa e produz os dados de pilotagem que o `Autopilot` consome:

```
1 // Produzidos por Route.computeSteerpointData() e lidos pelo Autopilot
2 double tbrg {}; // True bearing to next steerpoint (deg)
3 double mbrg {}; // Magnetic bearing (deg)
4 double dst {}; // Distance to next steerpoint (NM)
5 double ttg {}; // Time-To-Go (sec)
6 double eta {}; // Estimated Time of Arrival (sec UTC)
7 double xte {}; // Cross-Track Error (NM) negativo = esq.
8 double tcrs {}; // True Course of current leg (deg)
9 double mcrcs {}; // Magnetic Course of current leg (deg)
```

Listing 202: Navigation: campos de saída produzidos a cada `updateData(dt)` (resumo de `include/mixr/models/navigation/Navigation.hpp`).

Armadilha — `Ins` e `Gps` são recipientes de dados, não modelos

É tentador supor que `Ins` simule deriva inercial e que `Gps` simule latência de posicionamento. **Nenhum dos dois simula coisa alguma.**

Ambos são `EMPTY_SLOTTABLE` — não têm um único *slot* — e nenhum dos dois sobrescreve `updateData()`, `process()`, `dynamics()` ou `reset()`. `Ins` tem 80 linhas de *getters* e *setters* para `gyroBias`, `accelBias`, `wander`, `alignMode`, `alignTTG` e `quality`; `Gps` tem 83 linhas de enumerações de estado e armazenamento de chaves criptográficas. Não há integração de acelerações, não há deriva acumulada, não há atualização a 1 Hz — não há nenhum código que *execute*.

São estruturas para uma aplicação preencher e ler: o esqueleto de um modelo de navegação, deixado para quem precisar dele. Um cenário que declare (`Ins`) obtém um componente inerte que se comporta exatamente como a `Navigation` base — lendo posição verdadeira do *player*.

Onde a navegação de fato acontece, portanto, é na própria `Navigation` e na `Route` que ela consulta — e é para lá que vai a próxima subseção.

VI.9.2 Steerpoint e Route

Um **Steerpoint** carrega posição geodésica, parâmetros de voo e uma **Action** opcional a executar ao chegar. O campo **next** (identificador) nomeia o próximo *steerpoint*, permitindo rotas não lineares:

O método de cálculo é `compute(nav, from)` — o segundo argumento é o *steerpoint* anterior, e é ele que permite distinguir o cálculo “direto para” (sem **from**) do cálculo “ao longo da perna” (com), que é o que produz o *cross-track error*.

```
1 // Campos calculados pelo metodo compute(nav) a cada updateData()
2 double tbrg {}; // True bearing direct-to (deg)
3 double dst {}; // Distance direct-to (NM)
4 double ttg {}; // Time-To-Go (sec)
5 double xte {}; // Cross-Track Error (NM)
6 double eta {}; // Est. Time of Arrival (sec UTC)
7 double elt {}; // PTA Early/Late Time (sec)
8
9 // Configurados via slots no EDL
10 double cmdAlt {}; // Commanded altitude (m)
11 double cmdAirspeed {}; // Commanded airspeed (kts)
12 double pta {}; // Planned Time of Arrival (sec UTC)
```

Listing 203: **Steerpoint:** campos de estado calculados a cada `compute()` (resumo de `include/mixr/models/navigation/Steerpoint.hpp`).

Route agrega a sequência de **Steerpoint** e implementa o sequenciamento automático:

```
1 void Route::updateData(const double dt)
2 {
3     BaseClass::updateData(dt);
4
5     const Navigation* nav {
6         static_cast<const Navigation*>(
7             findContainerByType(typeid(Navigation))) };
8
9     if (nav != nullptr) {
10         // 1. Computa tbrg, dst, ttg, xte para cada steerpoint da rota
11         computeSteerpointData(dt, nav);
12
13         // 2. Verifica se chegou ao steerpoint corrente e avança
14         autoSequencer(dt, nav);
15     }
16 }
```

Listing 204: `Route::updateData()`: chama `computeSteerpointData()` e `autoSequencer()` a cada *frame* (`src/models/navigation/Route.cpp`).

O sequenciamento automático exige **três** condições simultâneas, e a terceira costuma passar despercebida: o *slot* `autoSequence` tem de estar ligado (padrão: sim); a distância ao *steerpoint* tem de ser menor que `autoSeqDistNM` (padrão 2 NM); e o rumo ao *steerpoint* tem de diferir do rumo atual em pelo menos 90°. Essa última condição significa “o ponto já ficou para trás” — sem ela, uma aeronave que apenas passasse perto de um *waypoint* o consumiria indevidamente. Ao capturar, a rota dispara a **Action** do *steerpoint* (se houver) e avança com `incStpt()`. Os métodos `directTo(nome)` e `directTo(índice)` permitem ao **OnboardComputer** desviar da sequência em tempo de execução:

```
1 // Desvio para waypoint por nome (chamado pelo OnboardComputer)
2 route->directTo("wp3"); // vai direto para wp3
3
4 // Desvio por índice
5 route->directTo(3); // vai direto para o 3o steerpoint
6
7 // O steerpoint atual reporta automaticamente tbrg/dst ao Navigation
8 // no proximo updateData()
```

Listing 205: Desvio de rota em tempo de execução via `directTo()`.

`Bullseye` é uma subclasse trivial de `Steerpoint` — semanticamente um ponto fixo de referência tático, não um waypoint de rota. O *slot* `bullseye:` da `Navigation` aceita um `Bullseye` para coordenação entre pilotos no formato “090/30 do *bullseye*” (30 NM a leste do ponto de referência).

VI.10 Piloto, autopiloto e computador de bordo

VI.10.1 Pilot e Autopilot

`Pilot` é um *stub* — herda de `System` sem lógica adicional, servindo de base para subclasses de pilotagem autônoma. `Autopilot` é a subclasse concreta: seis modos independentes, ativáveis em qualquer combinação, todos processados no laço TC:

```
1 void Autopilot::process(const double dt)
2 {
3     modeManager();           // re-trava os modos ativos
4     headingController();     // produz cmdHdg para o DynamicsModel
5     altitudeController();    // produz cmdAlt
6     velocityController();    // produz cmdSpd
7     BaseClass::process(dt);
8 }
```

Listing 206: `Autopilot::process()`: dispatcher dos controladores — chamado a cada fase 3 do frame TC (`src/models/system/Autopilot.cpp`).

O modo `navModeOn` ativa os três holds simultaneamente e direciona o rumo pelo `Navigation`:

```
1 bool Autopilot::setNavMode(const bool flag)
2 {
3     bool navModeOn1 { navModeOn };
4
5     // navModeOn requer que os dois SAS estejam ativos
6     navModeOn = flag && isRollSasOn() && isPitchSasOn();
7
8     if (navModeOn) {
9         // Liga os tres holds juntos
10        setHeadingHoldMode(true);
11        setAltitudeHoldMode(true);
12        setVelocityHoldMode(true);
13    }
14
15    // Ao desligar: captura valores correntes do player para evitar
16    // transiente brusco na retomada do controle manual
17    if (!navModeOn && navModeOn1) {
18        Player* pv { getOwnship() };
19        if (pv != nullptr) {
20            setCommandedHeadingD( pv->getHeadingD() );
21            setCommandedAltitudeFt( pv->getAltitudeFt() );
22            setCommandedVelocityKts(pv->getTotalVelocityKts() );
23        }
24    }
25    return (flag == navModeOn);
26 }
```

Listing 207: `Autopilot::setNavMode()`: habilita heading/alt/spd hold e captura valores correntes ao desligar (`src/models/system/Autopilot.cpp`).

O algoritmo `flyCRS()`. O cerne do modo NAV é um algoritmo de interceptação de curso que adapta o ângulo de interceptação à magnitude do *cross-track error*:

```

1  bool Autopilot::flyCRS(const double latDeg, const double lonDeg, const double crsDeg)
2  {
3      Player* pPlr { getOwnship() };
4      const double MAX_BANK_RAD { maxBankAngleDeps * base::angle::D2RCC };
5      const double velMps { pPlr->getTotalVelocity() };
6
7      double brgDeg {}, distNM {};
8      base::nav::fl2bd(pPlr->getLatitude(), pPlr->getLongitude(),
9                      latDeg, lonDeg, &brgDeg, &distNM);
10
11     // Erro de posicao relativo ao curso comandado
12     const double posErrDeg { base::angle::aepcdDeg(brgDeg - crsDeg) };
13     const double posErrRad { posErrDeg * base::angle::D2RCC };
14
15     // Raio de curvatura para o bank maximo configurado
16     const double rocMtr { velMps * velMps / base::ETHGM / std::tan(MAX_BANK_RAD) };
17
18     // XTE em unidades de raio de curvatura
19     const double xtRngMtr { std::fabs(distNM * sin(posErrRad)) * base::distance::NM2M };
20     const double xtRngRoc { xtRngMtr / rocMtr };
21
22     double hdgCmdDeg {};
23     if (xtRngRoc >= 1.2) {
24         // XTE grande: intercepta perpendicularmente (90 graus da pista)
25         hdgCmdDeg = base::sign(posErrDeg) * 90.0 + crsDeg;
26     } else {
27         // XTE pequeno: angulo proporcional -- convergencia suave
28         double x { 1.0 - xtRngRoc };
29         if (x > 1.0) x = 1.0;
30         if (x < -1.0) x = -1.0;
31         const double alfaDeg { std::acos(x) * base::angle::R2DCC };
32         // betaDeg e o angulo de aproximacao ao raio de curvatura
33         const double y { (rocMtr - 20.0) / rocMtr };
34         const double betaDeg { std::acos(y) * base::angle::R2DCC };
35         const double gamaDeg { base::sign(posErrDeg) * (alfaDeg - betaDeg) };
36         hdgCmdDeg = base::angle::aepcdDeg(gamaDeg + crsDeg);
37     }
38
39     return setCommandedHeadingD(hdgCmdDeg);
40 }

```

Listing 208: `Autopilot::flyCRS()`: interceptação proporcional ao XTE (`src/models/system/Autopilot.cpp`, simplificado).

O princípio é idêntico ao LNAV de sistemas FMS reais: para erros pequenos, o ângulo de interceptação é proporcional ao XTE, garantindo convergência suave; para erros grandes, a aeronave vira perpendicular à pista para capturar rapidamente. A constante 1.2 define a fronteira entre os dois regimes em unidades do raio de curvatura máximo.

Configuração via EDL.

```

1  components: {
2      pilot1: ( Autopilot
3          -- Modos iniciais
4          navMode:      true    -- segue a rota de Navigation
5          altitudeHoldMode: true
6          velocityHoldMode: true
7          headingHoldMode: true
8
9          -- Limites de performance ("pilot limits", nao "aircraft limits").
10         -- Todos tem padrao 0.0: e OBRIGATORIO declara-los (ver quadro abaixo).
11         maxBankAngle:  30.0 -- deg
12         maxRateOfTurnDps: 3.0 -- deg/s (3 = "standard rate turn")
13         maxClimbRateMps: 10.0 -- m/s
14         maxPitchAngle:  15.0 -- deg
15         maxAcceleration: 5.0 -- m/s2

```

```

16
17 -- Valores comandados iniciais
18 holdAltitude: ( Feet 25000 )
19 holdVelocityKts: 350.0
20 holdHeading: ( Degrees 90 )
21
22 -- Follow—the—lead (formacao): tres slots escalares , nao um vetor
23 leadPlayerName: "alpha"
24 leadFollowingDistanceTrail: ( Meters 500 )
25 leadFollowingDistanceRight: ( Meters 150 )
26 leadFollowingDeltaAltitude: ( Meters 0 )
27 )
28 }

```

Listing 209: Autopilot em EDL: modos iniciais e limites de performance.

Armadilha — os limites do autopiloto nascem em zero

`maxBankAngleDegs` e `maxTurnRateDps` são membros **inicializados com zero**, não com 30° e $3^\circ/\text{s}$. A consequência é concreta: em `flyCRS()` o raio de curvatura é $r = v^2 / (g \cdot \tan(\text{bank}))$, e com *bank* igual a zero esse raio é infinito. O erro de trajetória normalizado vai a zero, o algoritmo entra permanentemente no ramo de “erro pequeno” e a aeronave nunca faz uma curva de interceptação decente. Um **Autopilot** sem esses dois *slots* declarados não é um autopiloto com valores conservadores — é um autopiloto quebrado, e em silêncio.

VI.10.2 OnboardComputer: *shoot list* e ações de missão

`OnboardComputer` herda de `System` e é responsável por duas funções: manter a *shoot list* e gerenciar as ações dos *steerpoints*.

```

1 void OnboardComputer::updateData(const double dt)
2 {
3     BaseClass::updateData(dt);
4     actionManager(dt); // gerencia Action ativa do steerpoint corrente
5     updateShootList(); // recalcula a shoot list a partir do TrackManager
6 }

```

Listing 210: `OnboardComputer::updateData()`: as duas responsabilidades em background (`src/models/system/OnboardComputer.cpp`).

`updateShootList()` copia a lista de *tracks* do `AirTrkMgr` associado e seleciona o *next-to-shoot*:

```

1 void OnboardComputer::updateShootList(const bool step)
2 {
3     const unsigned int MAX_TRKS { 20 };
4     base::safe_ptr<Track> trackList[MAX_TRKS];
5
6     // Busca o AirTrkMgr; cai para qualquer TrackManager disponivel
7     TrackManager* tm { getTrackManagerByType(typeid(AirTrkMgr)) };
8     if (tm == nullptr) tm = getTrackManagerByType(typeid(TrackManager));
9     int n { (tm != nullptr) ? tm->getTrackList(trackList, MAX_TRKS) : 0 };
10
11     if (n > 0) {
12         // Encontra o indice do nextToShoot atual na lista
13         int cNTS { -1 };
14         for (int i = 0; i < n && cNTS < 0; i++) {
15             if (nextToShoot == trackList[i]) cNTS = i;
16         }
17
18         // Se nao temos NTS ou o track sumiu: pega o mais proximo
19         if (cNTS < 0) {
20             int closest { 0 };

```

```

21     double minRange { trackList[0]->getRange() };
22     for (int i = 1; i < n; i++) {
23         if (trackList[i]->getRange() < minRange) {
24             minRange = trackList[i]->getRange();
25             closest = i;
26         }
27     }
28     setNextToShoot(trackList[closest]);
29 }
30 }
31 }

```

Listing 211: OnboardComputer::updateShootList(): cópia e seleção do alvo mais prioritário (src/models/system/OnboardComputer.cpp, simplificado).

actionManager() processa a Action ativa do steerpoint corrente:

```

1 void OnboardComputer::actionManager(const double dt)
2 {
3     if (action != nullptr) {
4         action->process(dt);           // avança o estado da action
5         if (action->isCompleted()) {
6             action = nullptr;         // descarta ao completar
7         }
8     }
9 }
10 // triggerAction() e chamado pela Route.autoSequencer() ao capturar um steerpoint

```

Listing 212: actionManager(): processa e descarta a Action ao completar (src/models/system/OnboardComputer.cpp).

VI.10.3 CollisionDetect

CollisionDetect opera em duas fases separadas: updateData() (background) filtra a lista de players; process() (TC) verifica distâncias:

```

1 cd1: ( CollisionDetect
2     collisionRange: ( Meters 4.0 )      -- distancia de colisao (padrao: 4 m)
3     maxRange2Players: ( NauticalMiles 1.0 ) -- varre players ate 1 NM
4     maxAngle2Players: ( Degrees 0.0 )    -- 0 = sem restricao angular
5     maxPlayers:      20                  -- max de players of interest
6
7     playerTypes: {                      -- tipos a verificar
8         t1: "air"
9         t2: "weapon"
10    }
11
12     localOnly:      false    -- inclui players networked
13     sendCrashEvents: true    -- envia CRASH_EVENT ao colidir
14     useWorldCoordinates: true -- usa ECEF (vs. NED local)
15 )

```

Listing 213: CollisionDetect em EDL: todos os parâmetros configuráveis.

O evento CRASH_EVENT é enviado tanto ao ownship quanto ao player com o qual colidiu. O player intercepta em Player::event(CRASH_EVENT) e muda seu modo para CRASHED, iniciando o fluxo de remoção via DELETE_REQUEST na próxima chamada de Simulation::updatePlayerList().

VI.11 Assinaturas RF e IR

Sensores detectam alvos porque alvos **refletem ou emitem energia**. As classes de assinatura encapsulam essa propriedade — separando “quão visível é este alvo” de “como o sensor calcula o que recebeu”.

VI.11.1 Assinaturas RF: RfSignature e subclasses

RfSignature define um único método virtual puro:

```
1 class RfSignature : public base::Component {
2 public:
3     // Retorna o RCS (m²) para a Emission recebida.
4     // em->getAzimuthAoi() e em->getElevationAoi() dao o angulo de chegada
5     // do radar em relacao ao alvo -- usado por SigAzEl.
6     virtual double getRCS(const Emission* const em) = 0;
7 };
```

Listing 214: RfSignature: interface mínima (include/mixr/models/Signatures.hpp).

Sete subclasses concretas cobrem os casos de uso — e note o nome de fábrica da base: RfSignature registra-se como "Signature", não como o nome da classe.

```
1 // SigConstant: RCS fixo -- aceita Number(m²), Decibel(dBsm) ou Area
2 double SigConstant::getRCS(const Emission* const)
3 {
4     return rcs; // valor pre-convertido para m² no setSlotRCS()
5 }
6 // slot "rcs": base::Number (m²) | base::Decibel (dBsm) | base::Area (qualquer unidade)
7
8 // SigSphere: RCS analitico de esfera -- invariante com o angulo de chegada
9 // computeRcs(r) = PI * r * r
10 double SigSphere::getRCS(const Emission* const)
11 {
12     return rcs; // = PI * radius * radius, calculado em setRadius()
13 }
14
15 // SigAzEl: interpola Table2(az, el) -> RCS
16 // az e el sao o angulo de chegada da Emission em relacao ao alvo (rad ou graus)
17 double SigAzEl::getRCS(const Emission* const em)
18 {
19     double iv1 { em->getAzimuthAoi() }; // rad (ou graus se inDegrees)
20     double iv2 { em->getElevationAoi() };
21     if (isOrderSwapped()) std::swap(iv1, iv2); // suporta tabelas El x Az
22     if (isInDegrees()) {
23         iv1 *= base::angle::R2DCC;
24         iv2 *= base::angle::R2DCC;
25     }
26     double rcs { tbl->lfi(iv1, iv2) }; // interpolacao bilinear (LFI)
27     if (isDecibel())
28         rcs = std::pow(10.0, rcs / 10.0); // dBsm -> m²
29     return rcs;
30 }
31
32 // SigSwitch: seleciona entre sub-assinaturas pelo camouflageType do player
33 // camouflageType=0 -> componente 1, camouflageType=1 -> componente 2, etc.
34 double SigSwitch::getRCS(const Emission* const em)
35 {
36     const Player* own { static_cast<const Player*>(
37         findContainerByType(typeid(Player))) };
38     if (own == nullptr) return 0.0;
39
40     unsigned int cam { own->getCamouflageType() + 1 }; // 1-based
41     base::Pair* pair { findByIndex(cam) };
42     if (pair != nullptr) {
43         const auto sig = dynamic_cast<RfSignature*>(pair->object());
```

```

44     if (sig != nullptr) return sig->getRCS(em);
45   }
46   return 0.0;
47 }

```

Listing 215: As subclasses de RfSignature: implementações reais (src/models/Signatures.cpp).

Além dessas, três formas analíticas fechadas, todas **independentes do ângulo de chegada**: SigPlate ($\sigma = 4\pi(ab)^2/\lambda^2$, com os *slots* a e b), SigDihedralCR ($8\pi a^4/\lambda^2$) e SigTrihedralCR ($12\pi a^4/\lambda^2$). As duas últimas são EMPTY_SLOTTABLE: herdam a e b de SigPlate e usam apenas a. Servem a calibração e *test ranges*, onde o refletor é de fato um objeto geométrico conhecido.

Vale sublinhar o que isso significa: apesar do nome, SigPlate não modela uma placa vista de vários ângulos — devolve sempre o mesmo RCS. A única subclasse com dependência angular é SigAzEl.

Em EDL, a escolha da subclasse determina a fidelidade do modelo de assinatura:

```

1  // Alvo simples: RCS constante de 5 m²
2  signature: ( SigConstant rcs: 5.0 )
3
4  // Ou em dBsm:
5  signature: ( SigConstant rcs: ( dB 7.0 ) ) // 7 dBsm ≈ 5 m²
6
7  // Esfera de raio 1 m: RCS = π × l² ≈ 3.14 m²
8  signature: ( SigSphere radius: ( Meters 1.0 ) )
9
10 // Tabela az × el (para aeronaves reais -- captura dependencia angular)
11 signature: ( SigAzEl
12   table: ( Table2
13     x: [ -3.14159 0.0 3.14159 ] // azimuth (rad)
14     y: [ -1.5708 0.0 1.5708 ] // elevacao (rad)
15     data: [ 2.0 3.5 2.0 // linha az=-pi
16             4.0 8.0 4.0 // linha az=0 (frente)
17             2.0 3.5 2.0 ] // linha az=+pi
18   )
19   inDegrees: false
20   inDecibel: false
21 )

```

Listing 216: Três formas de declarar assinatura RF em EDL: constante, esfera e tabela.

VI.11.2 Assinaturas IR: IrSignature e AircraftIrSignature

IrSignature é a base abstrata para alvos IR. O método central calcula a área projetada do alvo e preenche a IrQueryMsg:

```

1  bool IrSignature::getIrSignature(IrQueryMsg* const msg)
2  {
3    bool ok{};
4    if (msg != nullptr) {
5      double projectedAreaInFOV{getSignatureArea(msg)};
6      msg->setProjectedArea(projectedAreaInFOV);
7      // if no projectedAreaInFOV, then target was not in FOV
8      if (projectedAreaInFOV > 0.0){
9        ok = true;
10       // FAB - set simple signature value
11       msg->setSignatureAtRange(getCalculatedHeatSignature());
12       msg->setEmissivity(getEmissivity());
13     }
14   }
15   return ok;
16 }

```

Listing 217: `IrSignature::getIrSignature()` (`src/models/IrSignature.cpp`).

`AircraftIrSignature` decompõe o sinal em três componentes físicos independentes — cada um com uma tabela de *lookup* própria:

```

1 // Tabelas de lookup:
2 //   airframeSignatureTable: Table4(mach, alt_m, az_rad, el_rad) -> W/sr
3 //   plumeSignatureTable:   Table5(PLA, mach, alt_m, az_rad, el_rad) -> W/sr
4 //   hotPartsSignatureTable: Table5(PLA, vel_kt, alt_m, az_rad, el_rad) -> W/sr
5 //   *WavebandFactorTable:  Table2(centro_banda, largura) -> fator [0..1]
6 //   Nota: os fatores de banda de cada componente devem somar 1.0
7
8 // Calculo do PLA (Power Lever Angle) a partir das forcas de empuxo
9 double AircraftIrSignature::getPLA(const AirVehicle* const airModel)
10 {
11     double currentThrust {}, idleValue {}, milValue {}, maxValue {};
12     airModel->getEngThrust( &currentThrust, 1);
13     airModel->getEngThrustIdle( &idleValue, 1);
14     airModel->getEngThrustMil( &milValue, 1);
15     airModel->getEngThrustAb( &maxValue, 1);
16
17     // PLA: 0.0 = idle, 1.0 = MIL power, 2.0 = afterburner
18     if (currentThrust < milValue) {
19         return (currentThrust - idleValue) / (milValue - idleValue);
20     } else if (currentThrust == milValue) {
21         return 1.0;
22     } else if (currentThrust < maxValue) {
23         return 1.0 + (currentThrust - milValue) / (maxValue - milValue);
24     }
25     return 2.0; // afterburner pleno
26 }

```

Listing 218: `AircraftIrSignature`: os três componentes e o cálculo do PLA (`src/models/AircraftIrSignature.cpp`).

O afterburner ($PLA = 2.0$) produz sinal IR ordens de magnitude maior que o voo de cruzeiro ($PLA \approx 0.4$ – 0.6). O sinal final por banda espectral é:

$$S_{\text{banda}} = (S_{\text{airframe}} + S_{\text{plume}} + S_{\text{hotParts}})_{\text{banda}} \times f_{\text{waveband}} \quad (7)$$

O array `signatureByWaveband` resultante é gravado na `IrQueryMsg` e consumido por `IrAtmosphere` (Seção VI.12), que atenua cada banda pela transmitância atmosférica antes de retornar o sinal ao `IrSensor`.

```

1 irSignature: ( AircraftIrSignature
2
3 // Airframe: calor por atrito -- funcao de mach, altitude, az, el
4 airframeSignatureTable: ( Table4
5     // x=mach y=altitude(m) z=az(rad) za=el(rad) -> W/sr
6     ...
7 )
8 airframeWavebandFactorTable: ( Table2
9     // x=centro_banda(um) y=largura(um) -> fator
10    x: [ 3.0  5.0 ] // bandas MWIR (3–5 um)
11    y: [ 2.0  2.0 ]
12    data: [ 0.6  0.4 ] // 60% na banda 3–5, 40% na banda 5–7
13 )
14
15 // Plume: escapamento -- fortemente dependente do PLA
16 plumeSignatureTable: ( Table5
17     // x=PLA(0..2) y=mach z=alt(m) za=az(rad) z2=el(rad) -> W/sr
18     ...
19 )
20 plumeWavebandFactorTable: ( Table2 ... )

```

```

21
22 // Hot parts: bocal + turbina -- sempre quentes
23 hotPartsSignatureTable: ( Table5 ... )
24 hotPartsWavebandFactorTable: ( Table2 ... )
25 )

```

Listing 219: AircraftIrSignature em EDL com as seis tabelas.

VI.12 Ambiente: atmosfera e propagação IR

A atmosfera atenua a radiação IR de forma seletiva, e essa atenuação determina se um sensor consegue detectar um alvo a uma dada distância e altitude. **AbstractAtmosphere** define o contrato e duas implementações concretas cobrem fidelidades diferentes.

VI.12.1 IrAtmosphere: modelo simples

IrAtmosphere aplica uma transmitância exponencial por banda espectral:

```

1 // transmissivityTable1: Table1 onde x = centro da banda (μm)
2 // e data = coeficiente de absorção α (por km)
3 // τ = exp( -α × range × 0.001 ) (range em metros → km)
4
5 double IrAtmosphere::getTransmissivity(const double wavebandCenter,
6                                       const double range) const
7 {
8     double trans { 1.0 };
9     if (transmissivityTable1 != nullptr) {
10         trans = transmissivityTable1->Ifi(wavebandCenter); // α para esta banda
11         trans = std::exp(trans * -0.001 * range);           // τ = e^(-α × r/1000)
12     }
13     return trans; // [0.0 = bloqueado, 1.0 = sem perda]
14 }

```

Listing 220: IrAtmosphere::getTransmissivity(): transmitância exponencial por banda (src/models/environment/IrAtmosphere.cpp).

A radiância de fundo é decidida pela geometria sensor-alvo (acima ou abaixo do horizonte):

```

1 // -- 1. Para onde o sensor esta olhando (angulo de arfagem, NED) --
2 const base::Matrixd mm{msg->getGimbal()->getRotMat() * msg->getOwnship()->getRotMat()};
3 base::Vec3d angles;
4 base::nav::computeEulerAngles(mm, &angles);
5 currentViewAngle = angles[Player::IPITCH];
6
7 // -- 2. Onde esta o horizonte, dada a altitude do ownship --
8 const double er{base::nav::ERAD60 * base::distance::NM2M}; // raio da Terra
9 const double distEC{msg->getOwnship()->getAltitudeM() + er}; // dist ao centro
10 const double dh2{(distEC * distEC) - (er * er)};
11 hDist = std::sqrt(dh2); // distancia ao horizonte
12 hTanAng = ( hDist / er );
13 viewAngleToHorizon = std::atan(hTanAng);
14
15 // -- 3. Que fracao do campo de visada e ceu e que fracao e terra --
16 const double angleToHorizon{currentViewAngle + viewAngleToHorizon};
17 const double fovtheta{msg->getSendingSensor()->getIFOVTheta()};
18
19 if (angleToHorizon - fovtheta >= 0) {
20     backgroundRadiance = getSkyRadiance(); // so ceu
21 } else if (fovtheta - angleToHorizon >= 2*fovtheta) {
22     backgroundRadiance = getEarthRadiance(); // so terra
23 } else {

```

```

24 // olhando para os dois: mistura proporcional
25 const double ratio{0.5 + 0.5 * angleToHorizon / fovtheta};
26 backgroundRadiance = ratio * getSkyRadiance() + (1.0-ratio) * getEarthRadiance();
27 }

```

Listing 221: `IrAtmosphere::calculateAtmosphereContribution()`: como a radiancia de fundo é decidida (`src/models/environment/IrAtmosphere.cpp`).

O cálculo do fundo é geométrico, e não uma aproximação por altitude: o *framework* descobre para onde o sensor aponta compondo a matriz do *gimbal* com a do *ownship*, calcula o ângulo ao horizonte a partir da altitude e do raio da Terra, e compara os dois contra o **IFOV do sensor**. Daí saem três casos — todo céu, toda terra, ou a mistura linear entre eles.

Por que — o fundo depende do sensor, não só da geometria

O termo decisivo é `getIFOVTheta()`: um sensor de campo estreito apontado logo acima do horizonte vê só céu, enquanto um de campo largo na mesma direção vê os dois. Modelar o fundo sem o campo de visada daria a mesma resposta para os dois — e é justamente contra o fundo que o sinal do alvo precisa se destacar.

A soma por banda, em seguida, é ponderada duas vezes: pela fração que aquela banda ocupa da faixa atmosférica total, e pelo *overlap* entre a banda e os limites espectrais do sensor. Só então cada parcela é atenuada pela transmitância:

```

1 const double fractionOfBandToTotal{(upperBandBound - lowerBandBound) / totalWavelengthRange};
2 const double overlapRatio{(upperOverlap - lowerOverlap) / (upperBandBound - lowerBandBound)};
3
4 const double backgroundRadianceInBand{backgroundRadiance * fractionOfBandToTotal * overlapRatio};
5
6 *totalSignal += radiantIntensityInBin * getTransmissivity(i, range2D);

```

Listing 222: O laço por banda espectral (`src/models/environment/IrAtmosphere.cpp`).

A configuração em EDL é direta:

```

1 atmosphere: ( IrAtmosphere
2   waveBands: ( Table1
3     x: [ 3.200 3.775 4.325 4.750 ] // centros das bandas (μm)
4     data: [ 0.4 0.75 0.35 0.5 ] // larguras das bandas (μm)
5   )
6   transmissivityTable1: ( Table1
7     x: [ 3.200 3.775 4.325 4.750 ] // centros (igual ao waveBands)
8     data: [ 0.011 0.004 0.280 0.017 ] // α (coef. absorção+espalhamento / km)
9   )
10   skyRadiance: 11.2 // W/sr.m² -- fundo celeste
11   earthRadiance: 1.0 // W/sr.m² -- fundo terrestre
12 )

```

Listing 223: `IrAtmosphere` em EDL com coeficientes de absorção por banda (exemplo do cabeçalho).

VI.12.2 IrAtmosphere1: modelo de alta fidelidade

`IrAtmosphere1` substitui a transmitância 1D e o fundo constante por três tabelas multidimensionais:

```

1 class IrAtmosphere1 : public IrAtmosphere {
2 private:
3   // Table2(banda, alt_alvo) -> W/sr
4   // Radiação solar refletida pelo alvo: varia com a altitude
5   // (densidade atmosférica menor em altitude = mais UV chega ao alvo)
6   const base::Table2* solarRadiationTable {};

```



```

7
8 // Table3(banda, alt_sensor, angulo_visada) -> W/sr
9 // Radiancia de fundo: sensor a 45° abaixo mistura solo quente e céu frio
10 // conforme a altitude do sensor e o angulo de visada (0=down, PI=up)
11 const base::Table3* backgroundRadiationTable {};
12
13 // Table4(banda, alt_sensor, alt_alvo, range) -> fração [0..1]
14 // Transmissividade: funcao completa da geometria do caminho
15 // A densidade do ar varia com a altitude -- a absorcao e diferente
16 // em cada trecho do trajeto sensor-alvo
17 const base::Table4* transmissivityTable {};
18 };

```

Listing 224: IrAtmosphere1: campos privados com as três tabelas e suas dimensões (include/mixr/models/environment/IrAtmosphere1.hpp).

A transmitância em IrAtmosphere1 é consultada com quatro coordenadas, em vez de uma:

```

1 double IrAtmosphere1::getTransmissivity(
2     const double wavebandCenter,
3     const double seekerAltitude,
4     const double targetAltitude,
5     const double range) const
6 {
7     double trans{1.0};
8     if (transmissivityTable != nullptr){
9         trans = transmissivityTable->lfi(wavebandCenter,seekerAltitude,targetAltitude,range);
10    } else {
11        // use base class' simple transmissivity if no table
12        trans = BaseClass::getTransmissivity(wavebandCenter,range);
13    }
14    return trans;
15 }

```

Listing 225: IrAtmosphere1::getTransmissivity(): quatro coordenadas em vez de uma — e o recuo para o modelo simples quando não há tabela (src/models/environment/IrAtmosphere1.cpp).

Note o **else**: sem a tabela de quatro dimensões, IrAtmosphere1 não devolve transmitância unitária — ele recua para o modelo exponencial simples da classe base. Isso o torna seguro de declarar em EDL antes de ter os dados em mãos: a fidelidade cai para a do modelo simples, mas nada deixa de funcionar. Há ainda uma segunda sobrecarga, de cinco argumentos, que recebe os limites inferior e superior da banda em vez do centro e o calcula ela mesma.

O cálculo do ângulo de visada em calculateAtmosphereContribution() usa a curvatura terrestre para decidir quanto do campo de visada cobre radiancia de fundo celeste vs. terrestre:

```

1 // tang do angulo ao alvo corrigido pelo raio da Terra (12 756 776 m = diametro)
2 const double tanPhi { (target->getAltitudeM() - ownship->getAltitudeM())
3     / range2D };
4 const double tanPhiPrime { tanPhi - (range2D / 12756776.0) };
5
6 double viewingAngle { std::atan(tanPhiPrime) };
7 // Converte para [0, PI]: 0 = apontado para baixo (terra), PI = para cima (céu)
8 viewingAngle += base::PI / 2.0;
9
10 // Consulta a tabela de fundo com (banda, alt_sensor, viewingAngle)
11 double bkg { backgroundRadiationTable->lfi(center, seekerAlt, viewingAngle) };

```

Listing 226: Cálculo do ângulo de visada com correção pela curvatura terrestre (src/models/environment/IrAtmosphere1.cpp).

Quando usar cada modelo. IrAtmosphere é suficiente para simulações táticas de médio alcance onde a fidelidade absoluta de detecção IR não é o objetivo. IrAtmosphere1 é necessário quando a probabilidade

de detecção e de *lock* de *seekers* IR precisa ser comparada com dados de ensaio — qualificação de sistemas, análise de eficácia de contramedidas IR e definição de envelopes de emprego.

VI.13 Armas e efeitos

Armas e efeitos são **Player** — participam do mesmo ciclo de quatro fases, têm física própria e são processados pela **Simulation** como qualquer aeronave ou veículo. O que os distingue é o ciclo de vida: nascem presos a um lançador (**PRE_RELEASE**), tornam-se autônomos ao ser lançados (**ACTIVE**) e encerram sua existência por impacto, autodestruição ou expiração de combustível (**DETONATED** → **DELETE_REQUEST**).

VI.13.1 AbstractWeapon: o ciclo de vida de uma arma

AbstractWeapon define os estados e as transições. A distinção central é entre a **arma inicial** (o objeto declarado no EDL, preso ao lançador) e o **flyout** (o clone que voa de fato). **release()** cria o clone, configura os ponteiros e adiciona o flyout à lista de players:

```
1  AbstractWeapon* AbstractWeapon::release()
2  {
3      AbstractWeapon* flyout {};
4
5      if (!isReleased() && !isBlocked() && !isJettisoned()) {
6          Player* lplayer { getLaunchVehicle() };
7          WorldModel* sim { static_cast<WorldModel*>(
8              findContainerByType(typeid(WorldModel))) };
9
10         if (!getWillHang() && lplayer != nullptr && sim != nullptr) {
11
12             // Clona a arma inicial -- este clone e o flyout que ira voar
13             flyout = this→clone();
14             flyout→container(sim);
15             flyout→reset();
16
17             // Liga os ponteiros de identidade entre inicial e flyout
18             flyout→setFlyoutWeapon(flyout); // flyout aponta para si mesmo
19             flyout→setInitialWeapon(this);
20             flyout→setID(sim→getNewReleasedWeaponID());
21
22             // Herda o lançador e o lado (BLUE/RED)
23             flyout→setLaunchVehicle(lplayer);
24             flyout→setSide(lplayer→getSide());
25
26             // Flyout entra na lista em PRE_RELEASE (vai a ACTIVE na fase 0)
27             flyout→setMode(PRE_RELEASE);
28             flyout→setReleased(true);
29             flyout→setReleaseHold(false);
30
31             // Marca a arma inicial como lançada
32             setMode(Player::LAUNCHED);
33             setFlyoutWeapon(flyout);
34             setInitialWeapon(this);
35             setReleased(true);
36
37             // Adiciona o flyout ao player list da Simulation
38             char pname[32];
39             std::sprintf(pname, "W%05d", flyout→getID());
40             sim→addNewPlayer(pname, flyout);
41         }
42     }
43     return flyout; // pre-ref()'d -- caller deve unref()
44 }
```

Listing 227: `AbstractWeapon::release()`: clona a arma, configura o flyout e o adiciona à `Simulation` (`src/models/player/weapon/AbstractWeapon.cpp`, simplificado).

Na fase 0 do próximo *frame*, o flyout faz a transição de `PRE_RELEASE` para `ACTIVE` e chama `atReleaseInit()`:

```
1 void AbstractWeapon::updateTC(const double dt)
2 {
3     BaseClass::updateTC(dt);
4
5     unsigned int ph { getWorldModel()->phase() };
6
7     // Fase 0: transicao PRE_RELEASE -> ACTIVE apos dynamics() da base,
8     // garantindo que a posicao relativa ao lançador já foi calculada
9     if (ph == 0 && isMode(PRE_RELEASE) && !isReleaseHold()) {
10         atReleaseInit(); // inicializa DynamicsModel com estado do lançador
11         setMode(ACTIVE);
12     }
13
14     // Fase 3: rastreamento de alvo e atualização do TOF
15     if (ph == 3 && isActive() && isLocalPlayer() && !isJettisoned() && !isDummy()) {
16         if (posTrkEnb) positionTracking(); // atualiza tgtPos a partir do track
17         if (isMode(ACTIVE)) updateTOF(dt * 4.0);
18     }
19 }
```

Listing 228: `AbstractWeapon::updateTC()`: transição real de `PRE_RELEASE` para `ACTIVE` ao final da fase 0 (`src/models/player/weapon/AbstractWeapon.cpp`).

Enquanto está em `PRE_RELEASE`, a arma mantém sua posição relativa ao lançador a cada fase 0:

```
1 void AbstractWeapon::dynamics(const double dt)
2 {
3     if (isMode(PRE_RELEASE)) {
4         // Presa ao lançador: posicao, orientacao e velocidade sao DERIVADAS dele
5         setSide(getLaunchVehicle()->getSide());
6         // orientacao = computeRotationalMatrix(initAngles) * matriz do lancador
7         // posicao = lvM * initPos + posicao do lancador
8         // velocidade = velocidade do lancador; aceleracao = 0
9         // NOTA: BaseClass::dynamics() NAO e chamada neste ramo.
10    }
11    else {
12        // Em voo: se nao ha DynamicsModel configurado, a arma usa o SEU PROPRIO
13        if (isLocalPlayer() && !isDummy() && getDynamicsModel() == nullptr) {
14            weaponGuidance(dt); // lei de guiagem embutida
15            weaponDynamics(dt); // integracao embutida
16        }
17        BaseClass::dynamics(dt);
18    }
19 }
```

Listing 229: `AbstractWeapon::dynamics()`: dois regimes, e o modelo embutido que só aparece no segundo (`src/models/player/weapon/AbstractWeapon.cpp`, estrutura).

Por que — armas têm física própria, ao contrário dos demais *players*

Este é o ponto em que a regra “a física vem do `DynamicsModel`” tem a sua exceção, e ela é deliberada. Um míssil declarado sem `DynamicsModel` **não** fica parado: `weaponGuidance()` e `weaponDynamics()` assumem, com uma lei de guiagem proporcional e uma integração simples. Cada subtipo de arma sobrescreve os dois conforme a sua natureza — `Missile` implementa perseguição proporcional, `Effect` implementa balística com arrasto, `Bullet` implementa balística pura.

A razão é prática: uma simulação pode ter centenas de armas em voo simultaneamente, e exigir que cada uma trouxesse um modelo de dinâmica configurado seria oneroso de escrever e de executar. Configurar

um `DynamicsModel` numa arma é possível, e serve para quando se quer fidelidade — mas é a exceção, não a regra.

A detonação é gerada por dois caminhos, e cada um tem o seu método: `collisionNotification()` para o impacto em outro *player*, e `crashNotification()` para o impacto no solo — este último com o resultado `DETONATE_GROUND_IMPACT`. `AbstractWeapon` **não** sobreescreve `killedNotification()`.

```

1  bool AbstractWeapon::collisionNotification(Player* const other)
2  {
3      // Guarda: so detona se for um player local e o override nao estiver ativo
4      if (isCrashOverride() || !isLocalPlayer()) return false;
5
6      bool ok { killedNotification(other) }; // avisa a cadeia normal primeiro
7
8      setMode(DETONATED);
9      setDetonationResults(DETONATE_ENTITY_IMPACT);
10
11     // Calcula ponto de detonação no sistema de coordenadas do alvo
12     setTargetPlayer(other, false);
13     setLocationOfDetonation();
14
15     // Verifica quem foi afetado pela detonação (raio lethal e burst)
16     checkDetonationEffect();
17
18     // Grava o evento para o recorder e interop/DIS
19     BEGIN_RECORD_DATA_SAMPLE(getWorldModel()->getDataRecorder(),
20                             REID_WEAPON_DETONATION)
21         SAMPLE_3_OBJECTS(this, getLaunchVehicle(), getTargetPlayer())
22         SAMPLE_2_VALUES(DETONATE_ENTITY_IMPACT, getDetonationRange())
23     END_RECORD_DATA_SAMPLE()
24
25     return ok;
26 }

```

Listing 230: `AbstractWeapon::collisionNotification()`: detonação por impacto em outro player, com gravação de evento (`src/models/player/weapon/AbstractWeapon.cpp`).

A ordem importa: `killedNotification()` é chamado *antes* da detonação, para que a cadeia normal de notificação de abate corra com a arma ainda em voo. Só depois o modo passa a `DETONATED` e `checkDetonationEffect()` varre os *players* dentro dos raios `lethalRange` e `maxBurstRng`.

Slot	Tipo	Padrão	Significado
<code>maxTOF</code>	Time	60,0 s	tempo máximo de voo
<code>tsg</code>	Time	9999,0 s	tempo para iniciar guiagem
<code>maxBurstRng</code>	Distance	500,0 m	raio de dano
<code>lethalRange</code>	Distance	50,0 m	raio letal
<code>sobt</code>	Time	9999,0 s	início de queima
<code>eobt</code>	Time	0,0 s	fim de queima
<code>maxGimbal</code>	Angle	30,0°	ângulo máximo do buscador

Table 17: Slots de `AbstractWeapon` com os valores padrão efetivamente usados pelo código (membros de `AbstractWeapon.hpp`).

Slots principais. Duas armadilhas se escondem nesses padrões. Antes delas, um caso que *parece* uma e não é — e que só se resolve lendo o construtor. O membro é declarado `double maxGimbal {};`, o que sugere zero; a leitura apressada da tabela de *slots* pára aí. Mas `AbstractWeapon::AbstractWeapon()` chama `initData()`, e é lá que está a linha

```
setMaxGimbalAngle(30.0 * base::angle::D2RCC);
```

O padrão efetivo é, portanto, 30°, exatamente o que o comentário do cabeçalho anuncia. O inicializador vazio na declaração não é o valor final de nenhum membro que `initData()` toque — e essa é a convenção do MIXR inteiro, não uma particularidade desta classe.

A primeira armadilha de verdade são os dois valores sentinela de 9999,0 s. `tsg` e `sobt` não são “muito grandes” por acaso: como `isGuidanceEnabled()` exige `getTOF() >= tsg` e `isEngineBurnEnabled()` exige `tof >= sobt`, o padrão desliga efetivamente a guiagem e a queima de motor. Uma arma só guia e só acende o motor se o EDL disser explicitamente quando.

A segunda é o modo inicial. A Seção IV.3 observou que o padrão de `AbstractPlayer` é `ACTIVE`; o construtor de `AbstractWeapon` o sobrescreve com `setMode(INACTIVE)` e `setInitMode(INACTIVE)`. Armas, ao contrário das demais entidades, nascem dormentes — o que é a única escolha coerente para objetos que passam a maior parte da simulação pendurados num cabide.

VI.13.2 Subtipos de arma

`Missile` não contém um buscador como membro nem faz rastreo na fase 2. A sua guiagem é `Missile::weaponGuidance()`, alimentada por `calculateVectors()`, que obtém a geometria ou do *player* alvo diretamente, ou de um `Track` do `TrackManager` do lançador. A guiagem é habilitada automaticamente quando `tof >= tsg` e `isTargetPositionValid()`, verificado por `isGuidanceEnabled()`:

```
1 bool AbstractWeapon::isGuidanceEnabled() const
2 {
3     return (getTOF() >= tsg)           // passou do tempo de inicio
4         && ((getCategory() & GUIDED) != 0) // arma e do tipo guiado
5         && isTargetPositionValid();      // tem posicao de alvo valida
6 }
7
8 bool AbstractWeapon::isEngineBurnEnabled() const
9 {
10    return (tof >= sobt && tof <= eobt); // dentro da janela de queima
11 }
```

Listing 231: `AbstractWeapon::isGuidanceEnabled()`: condição real para ativar a guiagem (`src/models/player/weapon/AbstractWeapon.cpp`).

`Aam` (ar–ar), `Sam` (superfície–ar) e `Agm` (ar–superfície) especializam a lógica de *endgame*. `Bomb` consulta o `Designator` do `OnboardComputer` para o ponto de impacto terminal (JDAM). `Bullet` tem trajetória balística pura e vida útil muito curta, criado em rajadas pelo `Guns`.

VI.13.3 Contramedidas: Chaff, Flare e Decoy

Contramedidas são `Effect`, que por sua vez é subclasse de `AbstractWeapon` — e não de `Player` diretamente. Herdam portanto todo o ciclo de vida de arma da Seção VI.13.1: nascem em `PRE_RELEASE` presas ao lançador, viram `ACTIVE` ao serem soltas e expiram por `maxTOF`.

`Effect` acrescenta um único *slot*, `dragIndex`, e uma única lógica: `weaponDynamics()`, uma trajetória balística com arrasto. É isso.

Armadilha — Chaff, Flare e Decoy são o mesmo código

Os três arquivos são **byte a byte equivalentes**: vinte e seis linhas cada, com `EMPTY_SLOTTABLE`, `EMPTY_COPYDATA`, `EMPTY_DELETEDATA`, `getCategory()` devolvendo `GRAVITY`, e um construtor que apenas fixa a *string* de tipo. Nenhum dos três toca num `RfSensor`, num buscador IR, numa assinatura ou num `DynamicsModel`.

Em particular: **Chaff não** satura radar nenhum, **Flare não** tem assinatura IR elevada, e **Decoy não** é um mini-veículo com dinâmica própria. As três classes existem para dar nome e tipo DIS a um objeto balístico de vida curta — a diferença entre elas é exclusivamente semântica.

O efeito de contramedida, se desejado, tem de ser construído por composição em EDL, exatamente como para qualquer outro *player*: é o *slot signature* que dá ao **Chaff** um RCS grande, e o *irSignature* que dá ao **Flare** uma assinatura térmica. O *framework* fornece as peças; a eficácia da contramedida é uma propriedade do cenário, não da classe.

```
1 storesMgr: ( StoresMgr
2   stores: {
3     7: ( Chaff
4       maxTOF:      ( Seconds 8.0 )
5       maxBurstRng: ( Meters 200.0 )
6       signature: ( SigConstant rcs: ( dB 15.0 ) ) // 15 dBsm >> aeronave
7     )
8     8: ( Flare
9       maxTOF:      ( Seconds 4.0 )
10      irSignature: ( AircraftIrSignature
11                  // baseHeatSignature muito maior que a da aeronave real
12                  ...
13                )
14    )
15  }
16 )
```

Listing 232: Configuração típica de contramedidas em EDL, como filhos do `StoresMgr`.

VI.14 Tipos de *player*: ar, solo, mar e espaço

Cada família de tipo concreto de *player* especializa `Player` com parâmetros e comportamentos específicos do seu domínio.

Regra — a taxonomia de *players* é quase toda nominal

Antes de percorrer a hierarquia, convém calibrar a expectativa. Dos cerca de vinte tipos concretos de *player*, **apenas seis contêm comportamento**: `AirVehicle`, `GroundVehicle`, `SamVehicle`, `LifeForm`, `SpaceVehicle` e as classes de arma e efeito. Todos os demais — `Aircraft`, `Helicopter`, `UnmannedAirVehicle`, `Tank`, `Artillery`, `ArmoredVehicle`, `WheeledVehicle`, `Ship`, `Building`, `MannedSpaceVehicle`, `UnmannedSpaceVehicle`, `InfantryMan`, `Parachutist` — são arquivos de vinte linhas cujo construtor apenas fixa uma *string* de tipo padrão.

Isso não é uma deficiência: é a consequência coerente do princípio da Seção II.1. O que distingue um caça de um helicóptero no MIXR não é a classe, é o `DynamicsModel`, a assinatura e os sensores que o arquivo EDL pendura neles. A classe serve para dar um nome de fábrica, um tipo DIS e um ponto de ancoragem — e é só.

Uma ressalva importante ao que se costuma dizer sobre física: ela vem do `DynamicsModel` *quando existe um*. Armas são a exceção, e têm um modelo interno próprio (Seção VI.13.1).

VI.14.1 Veículos aéreos

`AirVehicle` é a base da família aérea. Ao contrário de outras classes que sobrescrevem `dynamics()`, `AirVehicle` apenas delega para o `AerodynamicsModel` descoberto no `reset()`:

```
1  double AirVehicle::getGload() const
2  {
3      const AerodynamicsModel* aero { getAerodynamicsModel() };
4      return (aero != nullptr) ? aero->getGload() : 0.0;
5  }
6
7  double AirVehicle::getMach() const
8  {
9      const AerodynamicsModel* aero { getAerodynamicsModel() };
10     // Se nao ha modelo aerodinamico, usa a velocidade total do Player (aproximado)
11     return (aero != nullptr) ? aero->getMach() : BaseClass::getMach();
12 }
13
14 double AirVehicle::getCalibratedAirspeed() const
15 {
16     const AerodynamicsModel* aero { getAerodynamicsModel() };
17     // Fallback: velocidade total em kts quando sem modelo aerodinamico
18     return (aero != nullptr) ? aero->getCalibratedAirspeed()
19                               : getTotalVelocityKts();
20 }
```

Listing 233: `AirVehicle`: acesso às métricas aerodinâmicas via delegação ao `AerodynamicsModel` (src/models/player/air/AirVehicle.cpp).

`Aircraft` (asa fixa), `Helicopter` (asa rotativa) e `UnmannedAirVehicle` (VANT) não acrescentam nada além da *string* de tipo padrão — nem sequer sobrescrevem `getMajorType()`, que é definido uma única vez em `AirVehicle`. Também é `AirVehicle` quem expõe `getLandingGearPosition()` e `getWeaponBayDoorPosition()`, delegando ao `AerodynamicsModel`, para alimentar o campo *appearance* do PDU DIS com as articulações corretas. O único *slot* de toda a família aérea é o `initGearPos` de `AirVehicle`.

VI.14.2 Veículos terrestres e o lançador dinâmico

`GroundVehicle` é a base dos veículos terrestres. Além de `getMajorType() = GROUND_VEHICLE`, acrescenta um lançador dinâmico com campos e lógica de movimento reais:

```
1  void GroundVehicle::launcherDynamics(const double dt)
2  {
3      if (lnchrMoveTime > 0 && cmdLnchrPos != NONE) {
4
5          // Taxa de movimento: (max - min) / tempo de deslocamento total
6          double rate { (lnchrUpAngle - lnchrDownAngle) / lnchrMoveTime };
7          double angle { lnchrAngle };
8
9          if (cmdLnchrPos == UP && lnchrAngle != lnchrUpAngle) {
10             angle = lnchrAngle + (rate * dt);
11             if (angle >= lnchrUpAngle) {
12                 angle = lnchrUpAngle;
13                 rate = 0.0; // chegou ao limite superior
14             }
15         } else if (cmdLnchrPos == DOWN && lnchrAngle != lnchrDownAngle) {
16             angle = lnchrAngle - (rate * dt);
17             if (angle <= lnchrDownAngle) {
18                 angle = lnchrDownAngle;
19                 rate = 0.0; // chegou ao limite inferior
20             }
21         }
22     }
23     lnchrAngle = angle;
```



```

24 |         InchrRate = rate;
25 |     }
26 | }
27 |
28 | // Chamado em dynamics() de cada frame:
29 | void GroundVehicle::dynamics(const double dt)
30 | {
31 |     BaseClass::dynamics(dt);
32 |     launcherDynamics(dt); // move o lançador para posicao comandada
33 | }

```

Listing 234: `GroundVehicle::launcherDynamics()`: move o lançador para a posição comandada a uma taxa calculada pelos limites configurados (`src/models/player/ground/GroundVehicle.cpp`).

Os *slots* que configuram o lançador e o EDL de um `SamVehicle`:

```

1 | sam1: ( SamVehicle
2 |     type: "SA-6"
3 |
4 |     // Posicao comandada e limites do lancador
5 |     commandedPosition: up           // 'up' ou 'down' (Identifier)
6 |     launcherUpAngle: ( Degrees 75.0 ) // angulo maximo
7 |     launcherDownAngle: ( Degrees 0.0 ) // angulo minimo
8 |     launcherMoveTime: ( Seconds 8.0 ) // tempo para ir de DOWN a UP
9 |
10 |    // Envelope de lancamento
11 |    maxLaunchRange: ( KiloMeters 25.0 )
12 |    minLaunchRange: ( KiloMeters 2.0 )
13 |
14 |    // Subistemas sao COMPONENTES, nao slots
15 |    components: {
16 |        sms: ( StoresMgr
17 |            stores: {
18 |                1: ( Sam type: "SA-6" )
19 |                2: ( Sam type: "SA-6" )
20 |            }
21 |        )
22 |    }
23 | )

```

Listing 235: Configuração do lançador dinâmico de um `SamVehicle`.

Toda a maquinaria de lançador descrita acima vive em `GroundVehicle`, e é herdada por igual. `Tank` e `Artillery` são *stubs* de vinte linhas que só fixam a *string* de tipo — não modelam canhão nem elevação de cano. A única subclasse terrestre com comportamento próprio é `SamVehicle` (cerca de 160 linhas), que acrescenta o envelope de lançamento. `GroundStation`, `GroundStationUav` e `GroundStationRadar` são postos fixos, tipicamente sem `DynamicsModel`.

VI.14.3 LifeForm: o infante simulado

`LifeForm` é o mais rico dos tipos simples. Mantém um `actionState` com 15 posturas possíveis, atualizado automaticamente pela velocidade em `setVelocity()`:

```

1 | bool LifeForm::setVelocity(const double ue, const double ve, const double we)
2 | {
3 |     bool ok { BaseClass::setVelocity(ue, ve, we) };
4 |
5 |     const double tempX { std::fabs(ue) };
6 |     const double tempY { std::fabs(ve) };
7 |
8 |     // Nao altera a postura se estiver em paraquedas
9 |     if (actionState != PARACHUTING) {
10 |         if (tempX == 0 && tempY == 0) actionState = UPRIGHT_STANDING;

```



```

11         if (tempX > 0 || tempY > 0)    actionState = UPRIGHT_WALKING;
12         if (tempX > 8 || tempY > 8)    actionState = UPRIGHT_RUNNING;
13     }
14     return ok;
15 }

```

Listing 236: LifeForm::setVelocity(): atualização automática da postura a partir da velocidade (src/models/player/LifeForm.cpp).

O actionState é mapeado diretamente para o campo *appearance* do PDU EntityState do protocolo DIS:

```

1  // Bits 16–19 do campo appearance do EntityState PDU
2  unsigned int bits { 1 }; // padrao: UPRIGHT_STANDING
3  if (lf->getActionState() == LifeForm::UPRIGHT_STANDING) bits = 1;
4  else if (lf->getActionState() == LifeForm::UPRIGHT_WALKING) bits = 2;
5  else if (lf->getActionState() == LifeForm::UPRIGHT_RUNNING) bits = 3;
6  else if (lf->getActionState() == LifeForm::KNEELING) bits = 4;
7  else if (lf->getActionState() == LifeForm::PRONE) bits = 5;
8  else if (lf->getActionState() == LifeForm::CRAWLING) bits = 6;
9  else if (lf->getActionState() == LifeForm::SWIMMING) bits = 7;
10 else if (lf->getActionState() == LifeForm::PARACHUTING) bits = 8;
11 else if (lf->getActionState() == LifeForm::JUMPING) bits = 9;
12 else if (lf->getActionState() == LifeForm::SITTING) bits = 10;
13 else if (lf->getActionState() == LifeForm::SQUATTING) bits = 11;
14 else if (lf->getActionState() == LifeForm::CROUCHING) bits = 12;
15 else if (lf->getActionState() == LifeForm::WADING) bits = 13;
16 else if (lf->getActionState() == LifeForm::SURRENDER) bits = 14;
17 else if (lf->getActionState() == LifeForm::DETAINED) bits = 15;
18
19 pdu->appearance |= (bits << 16); // bits 16–19

```

Listing 237: Mapeamento de actionState para os bits 16–19 do campo *appearance* do PDU DIS — 15 posturas cobertas (src/interop/dis/Nib_entity_state.cpp).

Simulações DIS conformes exibem o soldado na postura correta em todos os *viewers* conectados.

look(up, sdws) separa o ângulo de visada da orientação do corpo — a orientação do corpo muda o rumo, mas o ângulo de visada (lookAngle) controla a direção de mira independentemente:

```

1 void LifeForm::look(const double up, const double sdws)
2 {
3     if (getDamage() < 1 && lockMode != LOCKED) {
4         lockMode = SEARCHING;
5         // sdws roda o corpo (modifica heading)
6         double hdg { getEulerAngles().z() };
7         hdg = base::angle::aeprdRad(hdg + sdws);
8         // 'up' muda apenas lookAngle, nao o pitch da aeronave
9         lookAngle += up;
10        // aplica o novo heading sem alterar o pitch/roll
11    }
12 }
13
14 void LifeForm::fire ()
15 {
16     StoresMgr* mgr { getStoresManagement() };
17     if (mgr != nullptr) {
18         if (weaponSel == LF_GUN) {
19             // Usa o lookAngle como pitch do canhao
20             Gun* myGun { mgr->getGun() };
21             if (myGun != nullptr) {
22                 myGun->setPitch(lookAngle * base::angle::D2RCC);
23                 myGun->fireControl(true);
24             }
25         } else { // LF_MISSILE
26             Missile* msl { mgr->releaseOneMissile() };
27             if (msl != nullptr && tgtAquired && tgtPlayer != nullptr)
28                 msl->setTargetPlayer(tgtPlayer, true);
29         }
30     }
31 }

```

```

30     }
31 }

```

Listing 238: `LifeForm::look()`: separa visada de orientação do corpo (`src/models/player/LifeForm.cpp`, simplificado).

VI.14.4 SpaceVehicle e BoosterSpaceVehicle

`SpaceVehicle` retorna `getMajorType() = SPACE_VEHICLE` e encaminha comandos de empuxo e orientação ao `SpaceDynamicsModel`, localizado por `dynamic_cast` sobre o `DynamicsModel` configurado.

Armadilha — `BoosterSpaceVehicle` é um *stub* vazio

Não há `liftoffTime`, não há fase de *boost*, não há separação de estágio. O arquivo tem vinte linhas: `EMPTY_SLOTTABLE`, `EMPTY_COPYDATA`, `EMPTY_DELETEDATA` e um construtor que fixa "`GenericBoosterSpaceVehicle`". E `isEngineBurnEnabled()` é método de `AbstractWeapon` — um `SpaceVehicle` não o possui. Modelar um perfil de lançamento exige escrever o `SpaceDynamicsModel` correspondente.

VI.15 Dados e mensagens entre sistemas

Vários objetos de dados circulam entre os sistemas dentro de um *player* e, em alguns casos, entre *players* diferentes. São classes que herdam de `Object` — sem *slots*, sem ciclo de simulação; existem apenas para carregar dados de um ponto a outro com segurança de ciclo de vida.

VI.15.1 Emission: pacote de energia RF em trânsito

`Emission` carrega tudo que um receptor precisa para calcular a potência recebida sem ter acesso direto à geometria:

```

1  // Campos de RF (todos privados, acessados via getters/setters)
2  double freq {};           // Frequencia (Hz)
3  double lambda {};        // Comprimento de onda (m) = c/freq
4  double pw {};            // Largura de pulso (s)
5  double bw {};            // Largura de banda (Hz)
6  double prf {};           // Pulse Repetition Frequency (Hz)
7  unsigned int pulses {1}; // Pulsos por pacote (prf/frameRate)
8  double power {};         // Potencia Efetiva Irradiada (W)
9  double gain {};          // Ganho efetivo da antena
10 double lossRng {1.0};    // Perda de espaco livre: 1/(4π r²)
11 double lossAtmos {1.0};  // Perda atmosferica
12 double lossXmit {1.0};   // Perda do transmissor
13 double rcs {};           // Radar Cross Section (m²)
14 Antenna::Polarization polar {Antenna::NONE};
15
16 // Ponteiro RAW deliberado -- nao faz ref():
17 // Emission nunca sobrevive ao RfSystem que a criou;
18 // um ref() criaria um ciclo de referencia.
19 RfSystem* transmitter {};
20
21 //-----
22 // setRange(): atualiza lossRng para evitar recalculo no receptor
23 //-----
24 void Emission::setRange(const double r)
25 {

```

```

26     BaseClass::setRange(r); // salva o range
27     // lossRng e calculado aqui uma unica vez --
28     // o receptor usa lossRng diretamente na equacao do radar
29     if (r > 1.0)
30         lossRng = 1.0 / (4.0 * base::PI * r * r);
31     else
32         lossRng = 1.0; // proteção contra distância zero/negativa
33 }

```

Listing 239: Emission: os campos e a decisão de projeto sobre lossRng e transmitter (resumo de include/mixr/models/Emission.hpp).

O cálculo do Doppler é derivado dos campos armazenados, sem necessidade de um campo adicional:

```

1 // Doppler shift (Hz) -- equacao 1-3 de Hovanessian
2 double getDopplerShift() const
3 {
4     if (lambda > 0.0)
5         return (-2.0 * getRangeRate() / lambda);
6     return 0.0;
7 }

```

Listing 240: Emission::getDopplerShift(): Doppler calculado por demanda a partir dos campos existentes.

VI.15.2 SensorMsg e IrQueryMsg

SensorMsg é a mensagem base de qualquer sensor — carrega a geometria completa da medição:

```

1 // Par origem/destino (safe_ptr: segurança de ciclo de vida)
2 base::safe_ptr<Player> ownship; // player que enviou
3 base::safe_ptr<Player> target; // player alvo
4 Gimbal* gimbal; // gimbal que estava apontado
5
6 // Geometria da medicao
7 double maxRng {}; // Alcance maximo do sensor (NM)
8 double rng {}; // Alcance ao alvo (m)
9 double rngRate {}; // Taxa de variacao do alcance (m/s)
10 double gaz {}; // Azimute no referencial do gimbal (rad)
11 double gel {}; // Elevacao no referencial do gimbal (rad)
12 double iaz {}; // AOI azimute no referencial inercial (rad)
13 double iel {}; // AOI elevacao no referencial inercial (rad)
14 base::Vec3d losO2T; // LOS ownship→alvo (NED do ownship)
15 base::Vec3d losT2O; // LOS alvo→ownship (NED do alvo)
16 base::Vec3d ao; // Angulo de incidencia normalizado
17
18 // returnReq: distingue radar ATIVO (quer que o alvo calcule e retorne o RCS)
19 // de sensor PASSIVO tipo RWR (apenas detecta a presenca da emissao)
20 bool returnReq {};
21
22 // localOnly: se true, ignora players networked
23 bool localOnly {};

```

Listing 241: SensorMsg: campos reais com geometria e o flag returnReq (resumo de include/mixr/models/SensorMsg.hpp).

IrQueryMsg estende SensorMsg com os parâmetros IR e os campos de saída que o alvo preenche via IrSignature:

```

1 // Parametros do sensor IR (campos de entrada)
2 double lowerWavelength {}; // Limite inferior da banda espectral (μm)
3 double upperWavelength {}; // Limite superior (μm)
4 double ifov {}; // Instantaneous Field of View (sr)
5 double nei {}; // Noise Equivalent Irradiance (W/sr)
6

```

```

7 // Campos de saída (preenchidos pelo IrSignature do alvo):
8 double signatureAtRange {}; // Potencia total ao alcance (W/sr)
9 const double* signatureByWaveband {}; // Potencia por banda espectral (W/sr)
10 double projectedArea {}; // Area projetada do alvo (m²)

```

Listing 242: IrQueryMsg: extensão IR da SensorMsg com campos de entrada (sensor) e saída (assinatura do alvo).

VI.15.3 Tdb: o buffer de geometria por quadro

Tdb (*Track Data Block*) é o *buffer* preenchido pelo Gimbal na fase 1 com todos os alvos dentro do feixe. Arrays paralelos dimensionados por `maxTargets`, limitado por `Gimbal::MAX_PLAYERS` (padrão 4000, Seção VI.5.2):

```

1 Player** targets {}; // Target pointer
2 unsigned int maxTargets {}; // Max number of targets (i.e., size of the arrays)
3 unsigned int numTgts {}; // Number of targets
4
5 base::Vec3d* losG {}; // Normalized LOS vector (gimbal to target) in Gimbal coord
6 base::Vec3d* losO2T {}; // Ownship to target normalized LOS vector (ownship's NED)
7 base::Vec3d* losT2O {}; // Target to ownship normalized LOS vector (target's NED)
8
9 double* ranges {}; // Range to target (meters)
10 double* rngRates {}; // Range Rate (m/s)
11 double* aar {}; // Compute angle off antenna boresight (radians)
12 double* aazr {}; // Compute azimuth off boresight (radians)
13 double* aelr {}; // Compute elevation off boresight (radians)
14
15 double* xa {}; // termos intermediarios da conversao para
16 double* ya {}; // azimuth/elevacao, mantidos como arrays para
17 double* za {}; // permitir as operacoes vetorizadas
18 double* ra2 {};
19 double* ra {};

```

Listing 243: Os arrays paralelos do Tdb (`include/mixr/models/Tdb.hpp`).

Duas observações sobre essa lista. A primeira é que **não há um `gainTgt`**: o ganho da antena por alvo é um *array* local de `Antenna::rfTransmit()`, e não um campo do Tdb — o que faz sentido, já que o ganho é propriedade da antena, não da geometria. A segunda é o `aar`, o ângulo total fora do *boresight*: é ele, e não o par *azimute/elevação*, que o `IrSeeker` consulta para decidir se um alvo está dentro do campo de visada.

A divisão de trabalho entre os dois métodos que preenchem esses *arrays* fecha o quadro: `processPlayers()` escolhe *quais* alvos entram (preenche `targets` e `numTgts`) e roda na *thread* de fundo; `computeBoresightData()` preenche todo o resto e roda a cada transmissão.

VI.15.4 Track: o estado de um alvo rastreado

```

1 // Cinematica do alvo (NED, relativa ao ownship)
2 base::Vec3d pos; // posicao (m)
3 base::Vec3d vel; // velocidade (m/s)
4 base::Vec3d accel; // aceleracao (m/s²)
5 double age {}; // tempo desde o ultimo update (s)
6
7 // Identificacao: um enum, nao um inteiro solto
8 enum IffCode { UNKNOWN, FRIENDLY, FOE, COMMERCIAL, OTHER };
9 IffCode iffCode {UNKNOWN};
10
11 // Alvo resolvido: ponteiro CRU, nao safe_ptr
12 Player* tgt {}; // acessor: getTarget()
13
14 // Historico de sinal -- nao ha um campo 'snDbf'

```

```

15 | double lastSN[4] {}; double avgSig {}; double maxSig {}; int nSig {};
16 |
17 | // Ordenacao na shoot list (ver sec:models-abc)
18 | int shootList {};

```

Listing 244: Track: campos reais de cinemática e identificação (resumo de `include/mixr/models/Track.hpp`).

Três correções que valem para quem for ler o código: `Track` vive em `include/mixr/models/Track.hpp`, e não no cabeçalho do `TrackManager`; não existe campo `trackSide`; e o alvo resolvido é um ponteiro cru `tgt`, não um `safe_ptr`. As subclasses concretas — `RfTrack` e `IrTrack` — é que acrescentam o vínculo com a medição que originou a pista (`getLastEmission()`, `getLastQuery()`) e o *setter* de sinal.

VI.15.5 Message, TargetData, Image e Designator

As quatro classes seguintes têm em comum uma característica que vale enunciar antes de descrevê-las: **são estruturas prontas à espera de consumidores**. Estão registradas na fábrica, são configuráveis, e nenhuma delas participa de um caminho de execução do *framework*.

`Message` carrega remetente, lista de destinatários, *timestamp* e tempo de vida (`lifeSpan`, padrão 5 s), mais um código de confirmação.

Armadilha — Message e TargetData não são usadas por ninguém

Uma busca por `models::Message` fora do seu próprio `.cpp` e da fábrica não devolve nada. `CommRadio` e `Datalink` transportam um `base::Object*` *opaco* (Seção VI.8.1) — nunca um `Message`. O mesmo vale para `TargetData`: não há *slot* em `Steerpoint` que a receba, e nenhum código em `OnboardComputer` ou `StoresMgr` a lê.

As duas são vocabulário oferecido a quem for construir comunicação estruturada ou planejamento de ataque — não mecanismos em funcionamento.

Para o registro, os *slots* de `TargetData` são `enabled`, `completed`, `weaponType`, `quantity`, `manualAssign`, `stickType`, `stickDistance`, `interval`, `maxMissDistance`, `armDelay`, `angle`, `azimuth` e `velocity`.

`Image` não é uma grade de reflectividade: é um *buffer* de pixels cru (`unsigned char* data`, mais largura, altura e profundidade), acompanhado do ponto de mira geodésico, da orientação e da resolução. É criada pelo `Sar` (Seção VI.7.3) e depositada numa lista interna dele, de onde a aplicação a recolhe — não há evento de entrega.

`Designator` é um ponto de *targeting* laser, e os seus dados são **geodésicos** (latitude, longitude, elevação), não ECEF, mais frequência, potência e código. Não guarda referência ao *player* que designa.

Armadilha — nenhuma arma consulta o Designator

O único consumidor de um `DESIGNATOR_EVENT` em toda a árvore é `AbstractWeapon::onDesignatorEvent()`, e o corpo desse método é explicitamente um não-fazer-nada. O próprio comentário do fonte diz: “*no futuro, vamos querer passar isto ao nosso detector LASER. Mas ainda não temos nenhum.*” `Bomb` nem sequer inclui `Designator.hpp`. Guiagem laser não está modelada.

SynchronizedState: o *snapshot* sem lock. Já descrito na Seção VI.3: dois buffers ECEF alternados escritos pela *thread* TC ao fim da fase 0 e lidos pela *thread* de rede sem risco de inconsistência. Carrega

posição, velocidade, aceleração, ângulos de Euler geocêntricos e velocidade angular — exatamente o conjunto necessário para um PDU DIS de *dead reckoning*.

VI.16 Agentes (UBF) e fábrica do módulo

VI.16.1 SimAgent e MultiActorAgent

`SimAgent` estende `base::ubf::Agent` (Seção III.10) resolvendo dois problemas que o UBF genérico deixa em aberto: *onde* está a simulação, e *quem* é o ator.

O primeiro é um *cache* preguiçoso em duas etapas — da `Station` sobe-se à `WorldModel`:

```
1 simulation::Station* SimAgent::getStation()
2 {
3     if ( myStation==nullptr ) {
4         const auto s = dynamic_cast<simulation::Station*>(
5             findContainerByType(typeid(simulation::Station)));
6         if ( s != nullptr ) {
7             myStation = s;
8         }
9     }
10    return myStation;
11 }
12
13 WorldModel* SimAgent::getWorldModel()
14 {
15     WorldModel* sim{};
16     simulation::Station* s{getStation()};
17     if ( s != nullptr ) {
18         sim = dynamic_cast<WorldModel*>(s->getSimulation());
19     }
20    return sim;
21 }
```

Listing 245: `SimAgent`: localiza a `Station` uma vez e guarda-a (`src/models/SimAgent.cpp`).

É o mesmo idioma de `System::getOwnship()`: subir a árvore custa caro, então sobe-se uma vez. `myStation` é um ponteiro cru deliberadamente — o agente está *contido* na `Station`, e uma referência contada nessa direção criaria um ciclo.

O segundo é a resolução do ator por nome, e ela tem três ramos:

```
1 void SimAgent::initActor()
2 {
3     if ( getActor() == nullptr ) {
4         if ( actorPlayerName == nullptr ) {
5             // not correctly specified as a SimAgent, try baseClass ?
6             BaseClass::initActor();
7         } else {
8             WorldModel* sim{getWorldModel()};
9             if ( sim != nullptr ) {
10                base::Component* player{
11                    sim->findPlayerByName(actorPlayerName->getString())};
12                if ( actorComponentName == nullptr ) {
13                    // no player component specified, so the player is the actor
14                    setActor(player);
15                } else if ( player != nullptr ) {
16                    base::Pair* pair{
17                        player->findByName(actorComponentName->getString())};
18                    if ( pair != nullptr ) {
19                        setActor(dynamic_cast<base::Component*>( pair->object() ));
20                    }
21                }
22            }
23        }
24    }
25 }
```

```

21     }
22   }
23 }
24 }
25 }

```

Listing 246: `SimAgent::initActor()`: o ator pode ser o *player*, um componente dele, ou herdado da classe base (`src/models/SimAgent.cpp`).

Regra — `actorComponentName` permite agir sobre um subsistema, não sobre a aeronave inteira

Os dois *slots* de `SimAgent` compõem-se: `actorPlayerName` sozinho faz do *player* o ator; acrescentar `actorComponentName` desce a busca para *dentro* dele, via `findByName()`.

A diferença é de escopo de controle. Com `actorPlayerName: falcao1` o comportamento comanda a aeronave. Com `actorComponentName: "radar1"` acrescentado, o mesmo comportamento comanda apenas o radar — modos, alcance, varredura — sem poder tocar na trajetória.

Repare no cuidado defensivo: se o nome do componente for declarado mas não for encontrado, `setActor()` nunca é chamado e o agente fica *sem ator*, inerte. Um erro de digitação no nome não gera diagnóstico — gera silêncio.

`MultiActorAgent` controla múltiplos *players* com um único conjunto de *behaviors*. O `controller(dt)` itera sobre a lista de agentes e executa um ciclo UBF completo para cada um:

```

1 void MultiActorAgent::controller(const double dt)
2 {
3     if (getState() == nullptr || nAgents == 0) return;
4
5     // 1. Atualiza o estado global uma vez para todos os agentes
6     getState()->updateGlobalState();
7
8     // 2. Para cada par (actor, behavior):
9     for (unsigned int i = 0; i < nAgents; i++) {
10         if (agentList[i].actor == nullptr) continue;
11
12         setActor(agentList[i].actor);
13         base::ubf::AbstractBehavior* behavior { agentList[i].behavior };
14
15         // PERCEPCAO: atualiza o estado local para este ator
16         getState()->updateState(agentList[i].actor);
17
18         // DECISAO: gera a acao recomendada
19         base::ubf::AbstractAction* action {
20             behavior->genAction(getState(), dt) };
21
22         // ATUACAO: executa se houver acao (nullptr e valido)
23         if (action != nullptr) {
24             action->execute(getActor());
25             action->unref(); // Action e transitoria
26         }
27     }
28     setActor(nullptr);
29 }

```

Listing 247: `MultiActorAgent::controller()`: ciclo UBF para cada agente da lista (`src/models/MultiActorAgent.cpp`).

A configuração em EDL associa cada *player* (pelo nome) ao seu *behavior*:

```

1 // Declarado em Station.components:
2 formacao: ( MultiActorAgent
3     state: ( MinhaEstadoTatico )
4
5     // Slot 'agentList': chave = nome do player, valor = behavior
6     agentList: {

```



```

7      alpha: ( BehaviorLider   vote: 100 )
8      bravo: ( BehaviorAla    vote: 100 )
9  }
10 )

```

Listing 248: MultiActorAgent em EDL: controla uma esquadrilha de dois players com behaviors distintos.

O estado global (`updateGlobalState()`) é atualizado uma única vez antes do laço — o estado local (`updateState(actor)`) é atualizado individualmente para cada ator, permitindo que o mesmo `State` represente tanto o contexto global da simulação quanto o ponto de vista de cada *player*.

VI.16.2 Actions: integração UBF com lógica de missão

Action herda de `base::ubf::AbstractAction` e acrescenta o ciclo de vida de missão:

```

1  // Estado da action e determinado por 'manager' (safe_ptr<OnboardComputer>):
2  //  manager != nullptr => inProgress
3  //  manager == nullptr => nao iniciada ou concluida
4
5  bool Action::isReadyToStart() { return !isInProgress() && !isCompleted(); }
6  bool Action::isInProgress()   { return (manager != nullptr); }
7  bool Action::isCompleted()    { return completed; }
8
9  // trigger(): inicia a acao e registra o manager
10 bool Action::trigger(OnboardComputer* const mgr)
11 {
12     setManager(mgr); // manager != nullptr => isInProgress() = true
13     completed = false;
14     return true;
15 }
16
17 // setCompleted(): libera o manager (sai de inProgress) e marca como feito
18 void Action::setCompleted(const bool flg)
19 {
20     if (flg) setManager(nullptr); // libera o manager
21     completed = flg;
22 }
23
24 // cancel(): completa forçando -- mesmo efeito de setCompleted(true)
25 bool Action::cancel()
26 {
27     completed = true;
28     setManager(nullptr);
29     return true;
30 }
31
32 // execute() -- interface UBF: localiza o OBC e chama trigger()
33 bool Action::execute(base::Component* actor)
34 {
35     auto* obc = dynamic_cast<OnboardComputer*>(actor);
36     if (obc == nullptr) {
37         // ator pode ser o Player -- busca o OBC na arvore
38         obc = dynamic_cast<OnboardComputer*>(
39             actor->findByType(typeid(OnboardComputer)));
40     }
41     if (obc != nullptr) return trigger(obc);
42     return false;
43 }

```

Listing 249: Action: ciclo de vida real — estado determinado pelo ponteiro manager (`src/models/Actions.cpp`).

As quatro subclasses concretas cobrem os casos de missão mais comuns:

```

1  bool ActionCamouflageType::trigger(OnboardComputer* const mgr)
2  {

```



```

3   if (mgr != nullptr) {
4       Player* own { static_cast<Player*>(
5           mgr->findContainerByType(typeid(Player))) };
6       if (own != nullptr) {
7           // Muda o camouflageType do player --
8           // SigSwitch.getRCS() usara o novo tipo para selecionar
9           // a sub-assinatura correta (ex: stealth ON/OFF)
10          own->setCamouflageType(getCamouflageType());
11          BaseClass::setCompleted(true); // one-shot: completa imediatamente
12          return true;
13      }
14  }
15  return false;
16 }

```

Listing 250: ActionCamouflageType::trigger(): a ação mais simples (*one-shot*) — muda o tipo de camuflagem e completa imediatamente (src/models/Actions.cpp).

ActionImagingSar é o exemplo de ação multi-frame: inicia a captura SAR em trigger() e aguarda a conclusão em process():

```

1  bool ActionImagingSar::trigger(OnboardComputer* const mgr)
2  {
3      Player* own { static_cast<Player*>(
4          mgr->findContainerByType(typeid(Player))) };
5      if (own != nullptr) {
6          base::Pair* pair { own->getSensorByType(typeid(Sar)) };
7          if (pair != nullptr) {
8              setSarSystem(static_cast<Sar*>(pair->object()));
9          }
10     }
11     if (sar != nullptr) {
12         // Configura o ponto SAR e solicita a imagem
13         sar->setStarePoint(getSarLatitude(), getSarLongitude(),
14             getSarElevation());
15         sar->requestImage(getImageSize(), getImageSize(), getResolution());
16         setManager(mgr); // fica inProgress() == true
17         return true;
18     }
19     return false;
20 }
21
22 void ActionImagingSar::process(const double dt)
23 {
24     BaseClass::process(dt);
25     if (isInProgress() && sar != nullptr) {
26         // Verifica se o SAR terminou (imageamento nao esta mais em progresso)
27         if (!sar->isImagingInProgress()) {
28             // SAR completou -- o evento SAR_COMPLETE_EVENT ja foi enviado ao OBC
29             setCompleted(true); // sai de inProgress(), libera o manager
30         }
31     }
32 }

```

Listing 251: ActionImagingSar: trigger inicia o SAR; process aguarda a conclusão quadro a quadro (src/models/Actions.cpp).

Em EDL, as ações são declaradas no slot action: do Steerpoint correspondente:

```

1  wp3: ( Steerpoint
2      latitude: ( LatLon direction: "s" degrees: 23 minutes: 10 )
3      longitude: ( LatLon direction: "w" degrees: 46 minutes: 20 )
4      altitude: ( Feet 15000 )
5
6      // Ao capturar este waypoint, lanca uma arma da estacao 1
7      action: ( ActionWeaponRelease
8          targetLatitude: ( LatLon direction: "s" degrees: 23 minutes: 15 )
9          targetLongitude: ( LatLon direction: "w" degrees: 46 minutes: 25 )

```

```

10         targetElevation: 0.0
11         station: 1
12     )
13 )
14
15 wp4: ( Steerpoint
16     // Ao capturar wp4, inicia captura SAR -- multi-frame
17     action: ( ActionImagingSar
18         sarLatitude: ( LatLon direction: "s" degrees: 23 minutes: 20 )
19         sarLongitude: ( LatLon direction: "w" degrees: 46 minutes: 30 )
20         sarElevation: ( Feet 100.0 )
21         resolution: ( Meters 3.0 )
22         imageSize: 512
23     )
24 )

```

Listing 252: Actions declaradas em Steerpoints: `ActionWeaponRelease` e `ActionImagingSar`.

VI.16.3 Fábrica do módulo

`models/factory.cpp` registra os nomes EDL de todas as classes do módulo na mesma cadeia de fábricas de base e `simulation`:

```

1 base::Object* factory(const std::string& name)
2 {
3     base::Object* obj {};
4
5     // Actions
6     if (name == ActionImagingSar::getFactoryName()) obj = new ActionImagingSar();
7     else if (name == ActionWeaponRelease::getFactoryName()) obj = new ActionWeaponRelease();
8     else if (name == ActionDecoyRelease::getFactoryName()) obj = new ActionDecoyRelease();
9     else if (name == ActionCamouflageType::getFactoryName()) obj = new ActionCamouflageType();
10
11     // Players concretos
12     else if (name == AirVehicle::getFactoryName()) obj = new AirVehicle();
13     else if (name == Aircraft::getFactoryName()) obj = new Aircraft();
14     else if (name == Tank::getFactoryName()) obj = new Tank();
15     else if (name == SamVehicle::getFactoryName()) obj = new SamVehicle();
16     else if (name == LifeForm::getFactoryName()) obj = new LifeForm();
17     // ... ~80 classes adicionais ...
18
19     return obj;
20 }

```

Listing 253: `models/factory.cpp`: trecho real com as ações e alguns tipos de player (`src/models/factory.cpp`).

Vale desfazer aqui um mal-entendido natural: **não existe no *framework* nenhum registro global de fábricas nem função de encadeamento**. Não há `setFactory()`, não há lista de fábricas instaladas, e a ordem dos arquivos no `meson.build` é irrelevante. Cada módulo expõe apenas uma função livre `factory(const std::string&)`, e o único ponto do MIXR que consome uma fábrica é o parâmetro `factory_func` de `edl_parser()` (Seção III.3) — um ponteiro de função, um só.

O encadeamento é, portanto, **responsabilidade da aplicação**, e escreve-se à mão. O idioma é uma função que tenta cada módulo em ordem de precedência e devolve o primeiro objeto construído:

```

1 // A fabrica da aplicacao -- a ordem AQUI define a precedencia
2 base::Object* minhaFactory(const std::string& name)
3 {
4     base::Object* obj {};
5
6     // 1. Classes proprias da aplicacao vem primeiro:
7     // podem sobrepor um nome de fabrica dos modulos abaixo
8     obj = minhaApp::factory(name);

```

```

9
10 // 2. Depois as camadas do MIXR, da mais especifica para a mais geral
11 if (obj == nullptr) obj = models::factory(name);
12 if (obj == nullptr) obj = simulation::factory(name);
13 if (obj == nullptr) obj = terrain::factory(name);
14 if (obj == nullptr) obj = base::factory(name);
15
16 return obj; // nullptr -> o parser reporta "undefined factory name"
17 }
18
19 // E entregue ao parser no carregamento da configuracao:
20 int nErrs {};
21 base::Object* raiz { base::edl_parser("cenario.edl", minhaFactory, &nErrs) };

```

Listing 254: A fábrica da aplicação: o encadeamento é código explícito, não um mecanismo do *framework*.

É essa ordem escrita à mão — e nada mais — que concretiza a promessa da Seção II.3 de que “um modelo específico pode sobrepor-se a um genérico de mesmo nome”. Colocar `minhaApp::factory` antes de `models::factory` faz com que um `Aircraft` próprio da aplicação substitua o do *framework* em todos os arquivos EDL, sem que nenhuma linha do MIXR precise ser tocada.

No cenário da interceptação

Praticamente tudo o que se *vê* acontecer no cenário do Seção II.6 está nesta camada.

O `alvo1` tem apenas um `RacModel` e uma `SigConstant`: voa para onde lhe mandaram e reflete radar. É deliberadamente burro — o papel dele é ser detectado.

O `falcao1` é a cadeia inteira. Na fase 0 o `LaeroModel` integra a sua posição. Na *thread* de fundo, o `Antenna` monta o `Tdb` com os alvos de interesse. Na fase 1 transmite e a `Emission` chega ao `alvo1`, que responde com o seu RCS. Na fase 2 o `Radar` aplica a equação e enfileira o relatório; na fase 3 consolida e entrega ao `AirTrkMgr`, que passa a manter uma pista. Em paralelo, o `Autopilot` lê a rota e comanda rumo, altitude e velocidade.

Se um míssil for lançado, ele não é um objeto especial: vira mais um *player* na lista, com física própria, percorrendo as mesmas quatro fases.

Como um System alcança o ownship

`getOwnship().getWorldModel() — findContainerByType()` na 1ª chamada, *cache* depois

A equação em duas metades — uma por fase

Fase 1 — Gimbal + Antenna

`processPlayersOfInterest()` monta o Tdb
LOS · range · rangeRate · az/el por alvo

`rfTransmit()` → uma Emission por alvo
 $\text{lossRng} = 1 / (4\pi R^2)$
 $\text{ERP} = P_t \times G_t \times \text{ganho}[i]$

ida (RfSystem):
 $S = \text{ERP} \cdot \text{lossRng} \cdot G_r / \text{perdas}$

Fase 2 — RfSensor / Radar

`rfReceivedEmission()` recebe o eco

volta (Radar):
 $S \times = \sigma \cdot \text{lossRng}$
 $S \times = \text{ganhoIntegração}$

$I = (\text{ruído} + \text{jamming}) \cdot \text{perdaRx}$
 $S/I \text{ em dB} \geq \text{rfThreshold}$
→ enfileira em `rptQueue`

Não é a equação do livro-texto

A forma clássica traz λ^2 e $(4\pi)^3$. Nenhum dos dois está no código.

$S = P_t \cdot G_r \cdot \text{lossRng}^2 \cdot \sigma \cdot G_{int} / L$ com $\text{lossRng} = 1/(4\pi R^2)$

O R^4 vem de lossRng ser aplicado duas vezes — na ida e na volta. O comprimento de onda serve ao Doppler, não ao balanço de potência.

Fase 3 e a cadeia IR equivalente

Estado de um Track

posição · velocidade · aceleração (NED)
qualidade · age · iffCode · lado
 $\text{age} > \text{maxTrackAge} \rightarrow \text{removido}$
filtro α - β quando não há *player* resolvido

Cadeia IR

Fase 1: IrSeeker envia IrQueryMsg ao alvo
Fase 2: o alvo preenche potência por banda
atmosfera atenua → SNR →
AngleOnlyTrackManager

A partição em duas metades é deliberada: RfSystem calcula o trecho de **ida** sem saber se aquilo é um eco; só o Radar, que sabe, aplica a RCS e a **segunda** perda de percurso.

Figure 32: Hierarquia de sistemas embarcados (topo), fluxo da cadeia RF por fase — Fase 1: Gimbal preenche o Tdb e Antenna transmite Emissions; Fase 2: RfSensor aplica a equação do radar e alimenta o TrackManager (centro). Estado de um Track e filtro α - β - γ (baixo). Resumo da cadeia IR equivalente (rodapé).

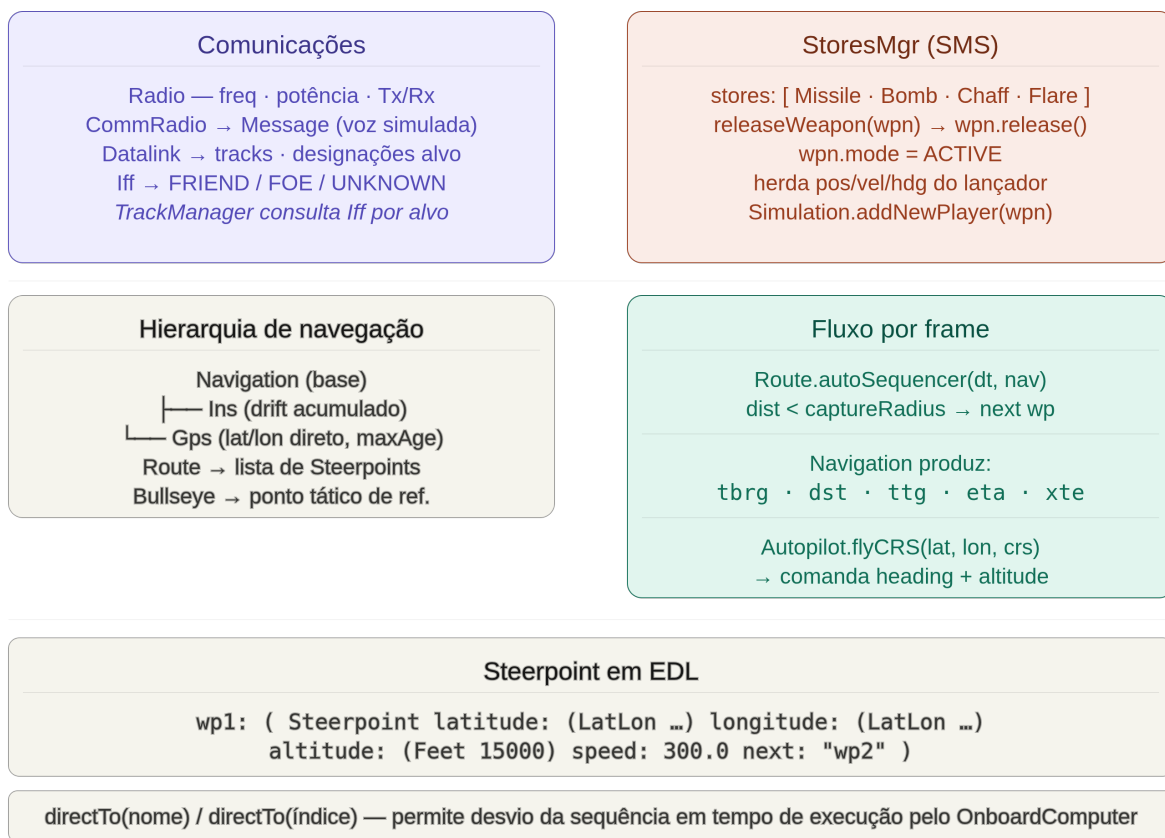


Figure 33: Sistemas de comunicação e StoresMgr (topo); hierarquia de navegação e fluxo por *frame* (centro); exemplo EDL de Steerpoint com encadeamento via *next*: e nota sobre *directTo()* (baixo).

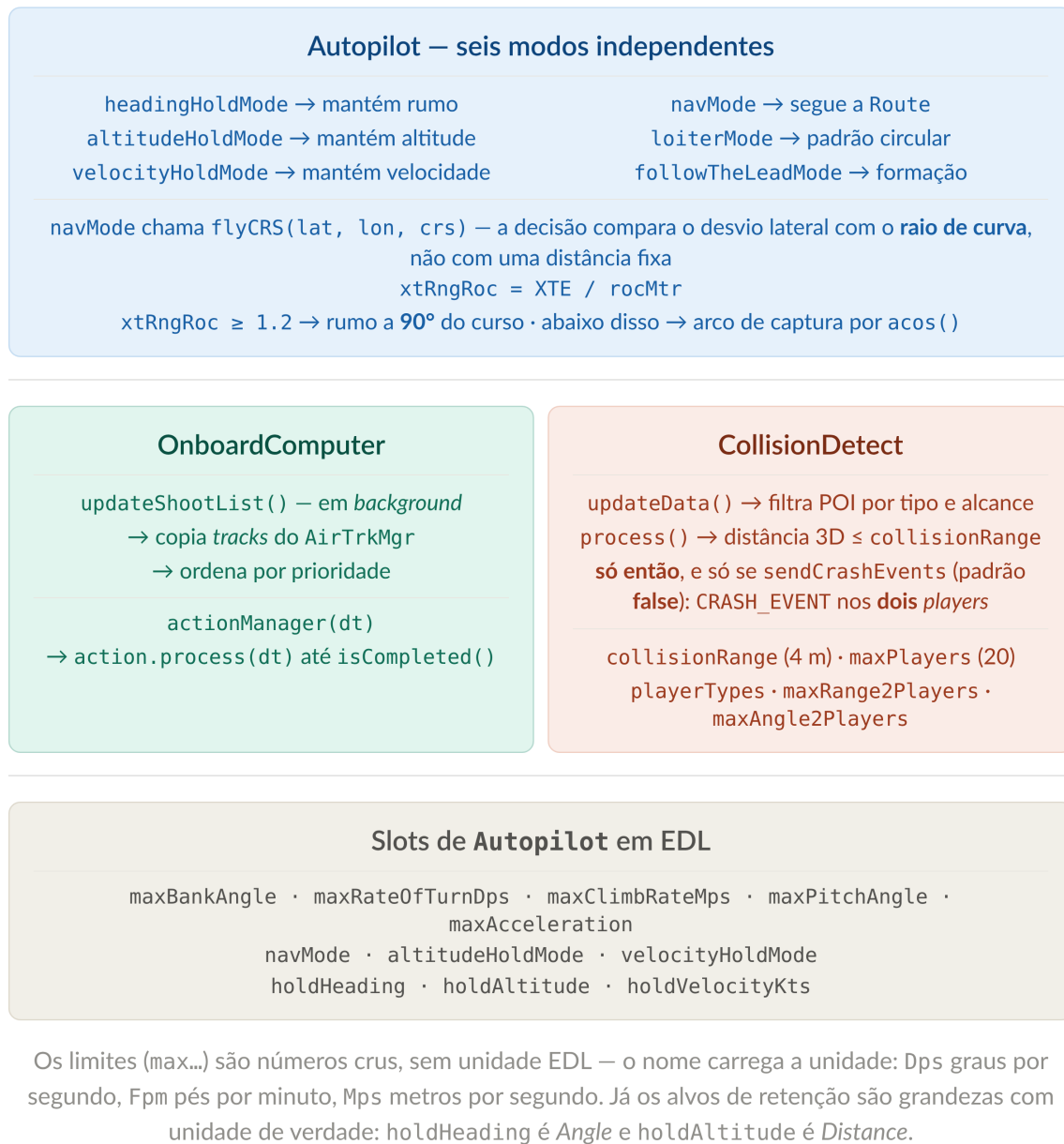


Figure 34: Os seis modos independentes do Autopilot com o algoritmo flyCRS (topo); OnboardComputer com *shoot list* e *actionManager* ao lado de CollisionDetect (centro); *slots* EDL do Autopilot (baixo).

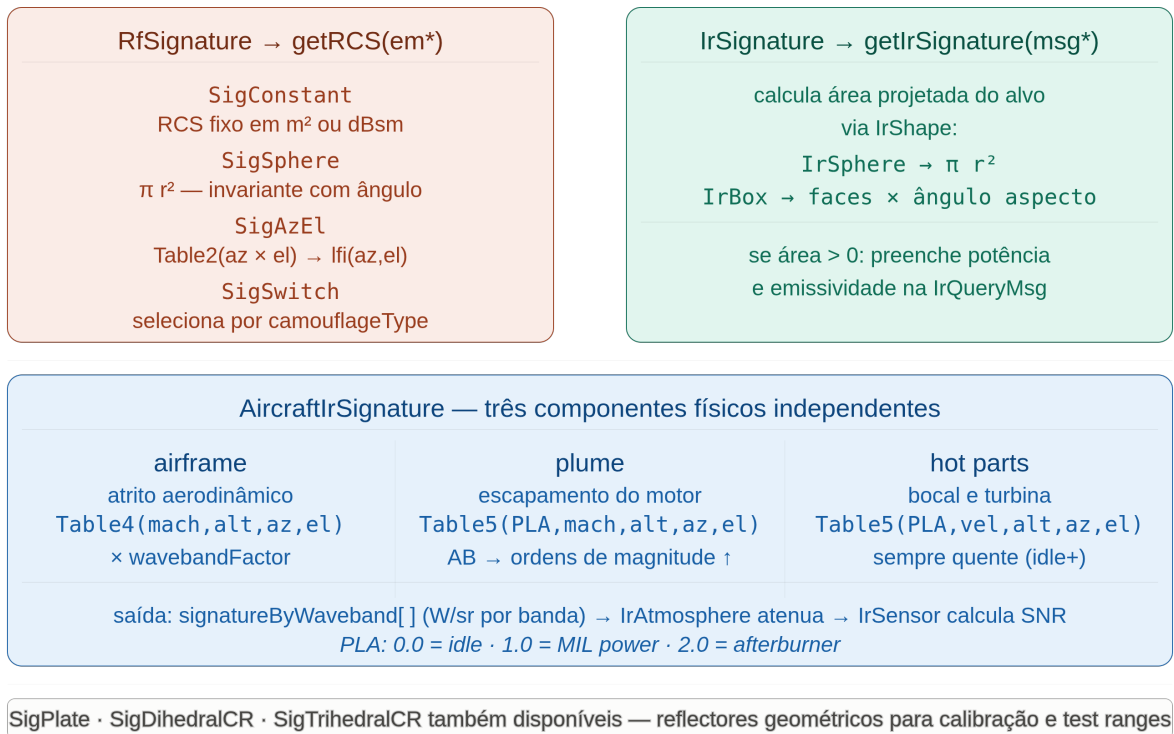


Figure 35: Quatro subclasses de RfSignature (topo-esq) e IrSignature com IrShape (topo-dir); os três componentes físicos independentes de AircraftIrSignature com suas tabelas de *lookup* (centro) e o fluxo de saída por banda espectral até o sensor (rodapé).

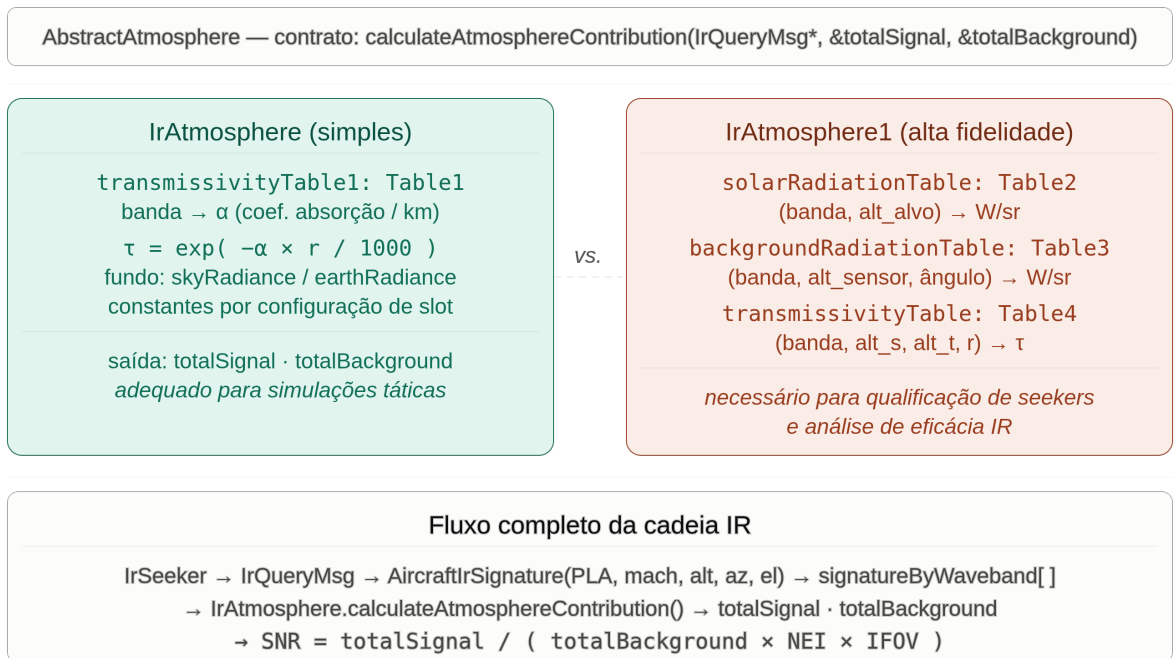
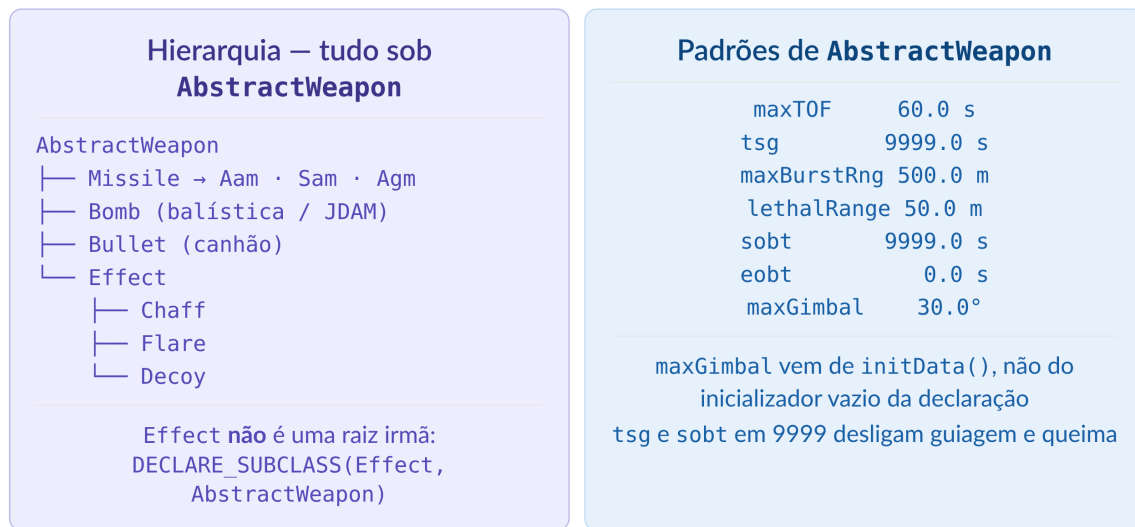
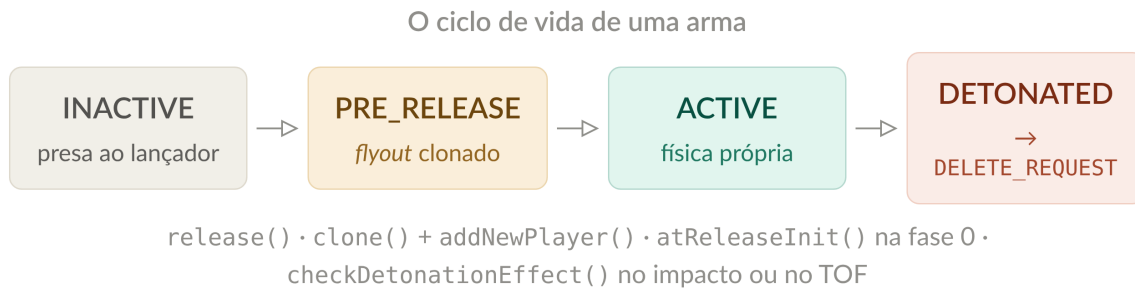


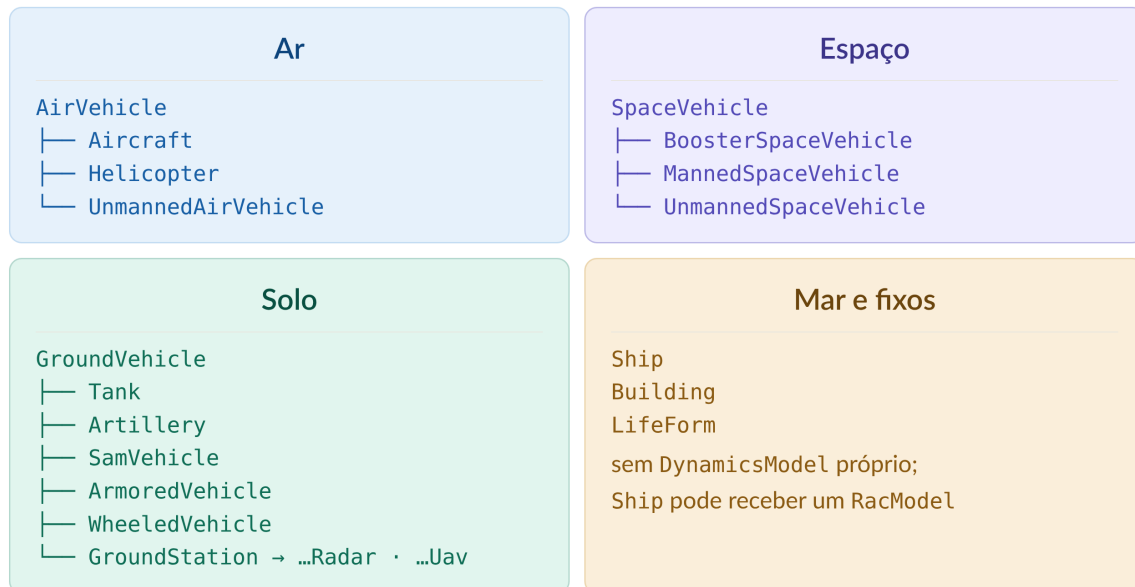
Figure 36: IrAtmosphere (simples, topo-esq) com transmissivityTable1 e fundo constante; IrAtmosphere1 (alta fidelidade, topo-dir) com três tabelas multidimensionais. O fluxo completo da cadeia IR desde o IrSeeker até o cálculo de SNR (baixo).



Categorias: GRAVITY · CLUSTER · GUIDED · MISSILE — campo de bits. isGuidanceEnabled() = tof
≥ tsg e GUIDED e alvo válido.

Figure 37: Ciclo de vida de **AbstractWeapon** com as transições reais (topo); hierarquia de subtipos e *slots* com valores padrão (centro); comparação entre as três contramedidas (baixo).

As quatro famílias de **Player** concreto



`getMajorType()` devolve o *enum* de tipo que vira classificação DIS na rede. As quatro classes de espaço são `EMPTY_SLOTTABLE` — não há `liftoffTime` nem `boost` em lugar nenhum da árvore.

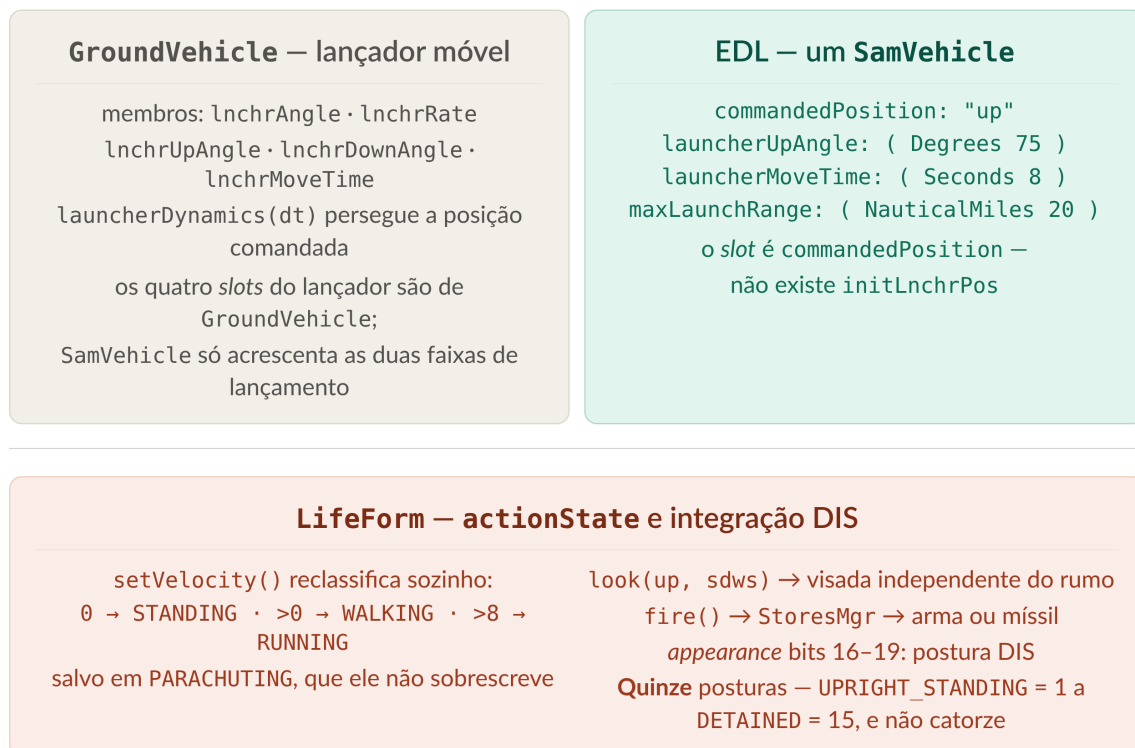


Figure 38: Os quatro domínios de *player* concreto (topo); o lançador dinâmico de `GroundVehicle` com campos reais e EDL de `SamVehicle` (centro); `LifeForm` com a atualização automática de postura e o mapeamento para o campo *appearance* do PDU DIS (baixo).



Todas herdam de Object — sem slots, sem ciclo de simulação; existem só para carregar dados com segurança de ciclo de vida.

SynchronizedState é a exceção útil: snapshot ECEF escrito pela thread TC ao fim de Player::dynamics() — a fase 0 — em dois buffers alternados, lido pela thread de rede sem lock.

Figure 39: Os seis tipos de objeto de dados: Emission e SensorMsg/IrQueryMsg (topo, cadeia de sensores); Tdb e Track (centro, geometria e rastreamento); Message/TargetData e Image/Designator (baixo, comunicação e missão). O rodapé sintetiza a invariante comum e SynchronizedState.

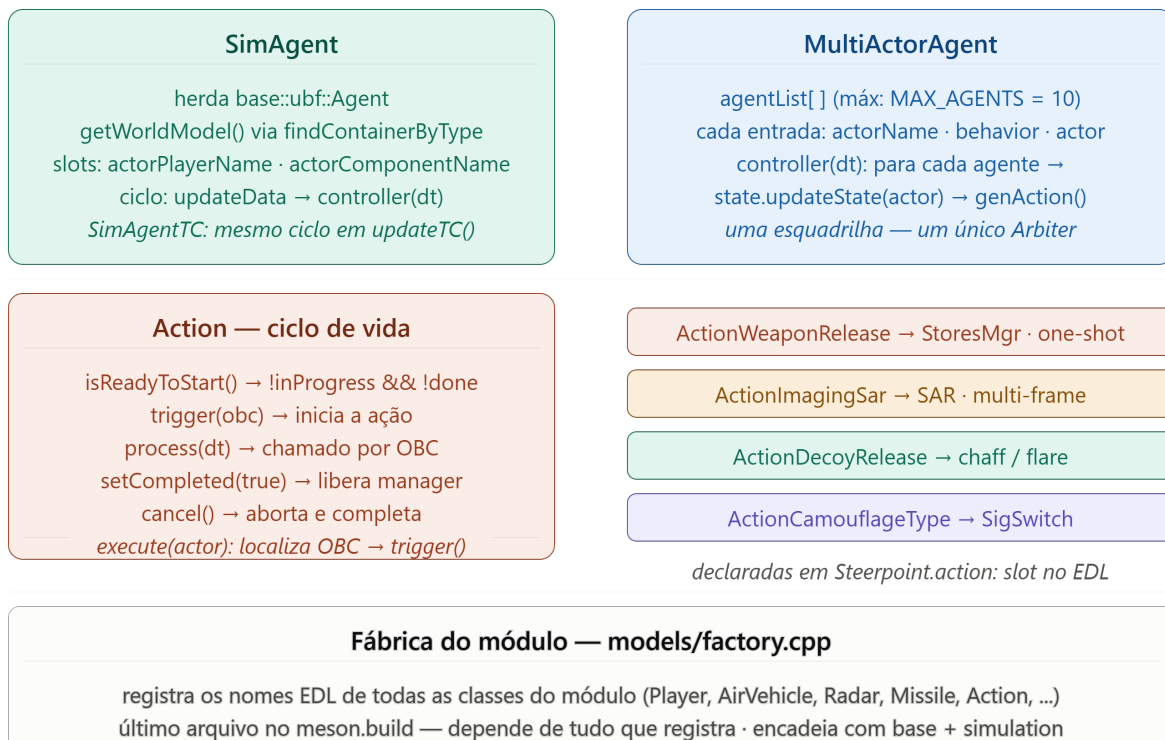


Figure 40: SimAgent e MultiActorAgent (topo): integração do UBF com o contexto da simulação. Ciclo de vida de Action com as quatro subclasses concretas (centro). Fábrica do módulo (baixo).

VII A camada linearsystem

VII.1 Visão geral: discretização de Tustin

A camada `linearsystem` fornece blocos de processamento de sinais em tempo discreto — filtros, funções de transferência, limitadores e *sample-and-hold*. Depende apenas de `mixr_base` e compila em `libmixr_linearsystem.so`.

Armadilha — esta camada não é usada por nenhuma outra, e não tem fábrica

Antes de qualquer outra coisa, convém saber em que estado o módulo se encontra neste *fork*, porque isso muda como ele deve ser lido:

- **Nenhum módulo do MIXR depende dele.** O `meson.build` de `models` lista `base`, `simulation`, `terrain` e `jsbsim` — não `linearsystem`. Uma busca por `#include "mixr/linearsystem/..."` fora do próprio diretório não devolve nada. Em particular, o Autopilot não usa `LagFilter` nem nenhum outro bloco daqui.
- **Não existe `linearsystem/factory.cpp`.** Diferentemente de todos os outros módulos, este não registra nenhuma classe. Nenhum `LagFilter`, `SaH` ou `LowpassFilter` pode ser criado a partir de um arquivo EDL sem que a aplicação escreva a sua própria fábrica.

A biblioteca é compilada, instalada e exportada pelo pacote Conan — é uma oferta ao consumidor, não uma peça em uso interno. Leia este capítulo, portanto, como a documentação de uma **caixa de ferramentas disponível**, e não de um mecanismo que já está rodando na sua simulação.

A decisão arquitetural central é a **transformação de Tustin** (também chamada bilinear): o método padrão de discretização de sistemas de controle contínuos. A transformação substitui s por:

$$s \approx \frac{2}{T} \cdot \frac{z-1}{z+1}$$

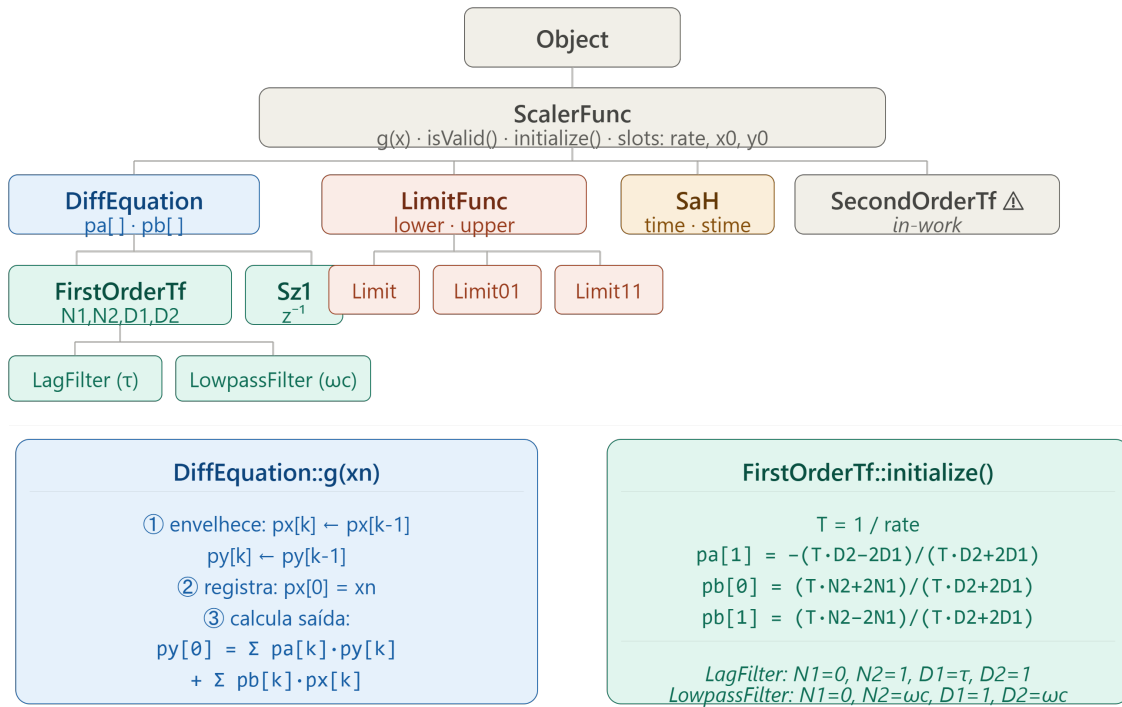
convertendo a função de transferência contínua numa equação de diferenças — uma recorrência calculável a cada *frame*. Os coeficientes são calculados uma única vez em `initialize()` e reutilizados em todas as chamadas subsequentes a `g(x)`.

Os 13 arquivos de fonte (conforme `src/linearsystem/meson.build`) organizam-se em três grupos: o motor numérico (`ScalerFunc/DiffEquation`), as funções de transferência de primeira ordem e suas especializações, e os blocos auxiliares (atrasos, *sample-and-hold* e limitadores).

VII.2 ScalerFunc e DiffEquation

`ScalerFunc` é a raiz da hierarquia e herda de `Object` — não de `Component`: um filtro não tem ciclo de simulação próprio, é chamado diretamente pelo código que o usa.

```
1 class ScalerFunc : public base::Object {
2 public:
3     // Interface pública: uma única chamada por frame
4     virtual double g(const double x) = 0; // calcula Y(n) = f(X(n))
5     bool isValid() const override;      // arrays alocados, rate > 0
6     unsigned int getRate() const { return rate; }
7
8     // Inicialização dos valores anteriores
```



iaH: time += 1/rate; se time ≥ stime → captura px[0] em py[0] e zera; initialize() seta time = stime+1 (força sample na 1ª chamada)
 imit01 = LimitFunc(0.0, 1.0) · Limit11 = LimitFunc(-1.0, 1.0) · SecondOrderTf e Sz2: ##### IN-WORK #####, não usar em produção

Figure 41: Hierarquia do módulo linearsystem: ScalerFunc como raiz (herda de Object, não de Component); DiffEquation como motor numérico com suas subclasses; LimitFunc e SaH como blocos auxiliares. As faixas inferiores detalham os três passos de DiffEquation::g() e os coeficientes Tustin calculados por FirstOrderTf::initialize().

```

9   virtual bool setX0(const double); // seta px[0..n-1] = x0
10  virtual bool setY0(const double); // seta py[0..n-1] = y0
11  virtual bool setRate(const unsigned int);
12
13  // Slots EDL: rate (Frequency ou Number), x0 (Number), y0 (Number)
14
15  protected:
16  virtual void initialize(); // calcula coeficientes (subclasses)
17  virtual void allocateMemory(unsigned int n);
18  virtual void clearMemory();
19
20  unsigned int n {}; // ordem do sistema (tamanho dos arrays)
21  double* px {}; // historico de entradas X(n), X(n-1), ...
22  double* py {}; // historico de saidas Y(n), Y(n-1), ...
23
24  unsigned int rate {}; // taxa de chamada (Hz)
25  double x0 {}; // valor inicial de X
26  double y0 {}; // valor inicial de Y
27  };

```

Listing 255: ScalerFunc: interface minimalista e campos de estado (include/mixr/linearsystem/ScalerFunc.hpp).

DiffEquation acrescenta dois arrays de coeficientes e implementa g(xn) em exatamente três passos:

```

1  double DiffEquation::g(const double xn)
2  {
3      if (!isValid()) return xn; // passthrough se invalido

```

```

4
5 // Passo 1: envelhecer o historico (shift right)
6 for (unsigned int k = (n-1); k > 0; k--) {
7     px[k] = px[k-1];
8     py[k] = py[k-1];
9 }
10
11 // Passo 2: registrar a entrada atual
12 px[0] = xn;
13
14 // Passo 3: equacao de diferencas linear
15 py[0] = 0;
16 for (unsigned int k = 1; k < n; k++)
17     py[0] += pa[k] * py[k]; // saidas passadas (feedback)
18 for (unsigned int k = 0; k < n; k++)
19     py[0] += pb[k] * px[k]; // entradas (feedforward)
20
21 return py[0];
22 }

```

Listing 256: DiffEquation::g(): o motor numérico real (src/linearsystem/DiffEquation.cpp).

Os coeficientes `pa[]` e `pb[]` são determinados pela transformação de Tustin aplicada à função de transferência contínua de cada subclasse, calculados em `initialize()`.

VII.3 Funções de transferência de primeira ordem

`FirstOrderTf` implementa a discretização de Tustin da função de transferência genérica de primeira ordem:

$$H(s) = \frac{N_1 s + N_2}{D_1 s + D_2}$$

`initialize()` deriva os coeficientes a partir dos quatro parâmetros e do período de amostragem $T = 1/\text{taxa}$:

```

1 void FirstOrderTf::initialize ()
2 {
3     BaseClass:: initialize ();
4     if (! isValid ()) return;
5
6     const double T { 1.0 / static_cast<double>(rate) };
7
8     pa[0] = 0;
9     pa[1] = -(T*d2 - 2.0*d1) / (T*d2 + 2.0*d1);
10
11     pb[0] = (T*n2 + 2.0*n1) / (T*d2 + 2.0*d1);
12     pb[1] = (T*n2 - 2.0*n1) / (T*d2 + 2.0*d1);
13
14     // Inicializa o historico com x0/y0
15     for (unsigned int k = 0; k < n; k++) {
16         px[k] = x0;
17         py[k] = y0;
18     }
19 }
20
21 // isValid(): causalidade (ordem den >= ordem num) + den != 0 + rate > 0
22 bool FirstOrderTf::isValid() const
23 {
24     unsigned int orderN { (n1 != 0) ? 1u : 0u };
25     unsigned int orderD { (d1 != 0) ? 1u : 0u };
26     bool valid { n == ORDER && rate > 0
27                 && (d1 != 0 || d2 != 0)
28                 && (orderD >= orderN) };
29     return valid && BaseClass::isValid();
30 }

```

Listing 257: FirstOrderTf::initialize(): coeficientes Tustin reais (src/linearsystem/FirstOrderTf.cpp).

Duas especializações simplificam os casos mais usados:

```

1  // LagFilter:  $H(s) = 1 / (\tau s + 1)$ 
2  //  $N1=0, N2=1, D1=\tau, D2=1$ 
3  bool LagFilter::setTau(const double v)
4  {
5      tau = v;
6      setN1(0.0); setN2(1.0);
7      setD1(tau); setD2(1.0);
8      initialize ();
9      return true;
10 }
11
12 // LowpassFilter:  $H(s) = \omega_c / (s + \omega_c)$ 
13 //  $N1=0, N2=\omega_c, D1=1, D2=\omega_c$ 
14 // slot 'wc' aceita Frequency (Hz  $\rightarrow$  rad/s via  $*2\pi$ ) ou Number (rad/s direto)
15 bool LowpassFilter::setWc(const double v)
16 {
17     wc = v;
18     setN1(0.0); setN2(wc);
19     setD1(1.0); setD2(wc);
20     initialize ();
21     return true;
22 }
23
24 bool LowpassFilter::setSlotWc(const base::Frequency* const msg)
25 {
26     // Hz  $\rightarrow$  rad/s
27     double hz { base::Hertz::convertStatic(*msg) };
28     double rps { hz * 2.0 * base::PI };
29     if (rps > 0) { setWc(rps); return true; }
30     return false;
31 }

```

Listing 258: `LagFilter::setTau()` e `LowpassFilter::setWc()`: como os parâmetros do usuário são traduzidos nos coeficientes N_1, N_2, D_1, D_2 de `FirstOrderTf`.

Uso típico no Autopilot. `LagFilter` suaviza erros de rumo e altitude antes de enviá-los como comandos ao `DynamicsModel`: quanto maior τ , mais suave — e mais atrasado. `LowpassFilter` atenua ruído de alta frequência em sinais de sensores; como o `LagFilter`, tem ganho unitário em CC ($H(0) = 1$).

SecondOrderTf: pior do que “não implementado”. `SecondOrderTf` está presente na hierarquia (e `Sz2` herda dele), mas o corpo de `initialize()` contém apenas o comentário `##### IN-WORK #####` — os coeficientes Tustin de segunda ordem nunca são calculados.

Armadilha — `SecondOrderTf` lê memória não inicializada

A consequência é mais séria do que “o filtro não filtra”. `DiffEquation::clearMemory()` zera o vetor `pa` — duas vezes, por um erro de digitação — e **nunca zera** `pb`. Em condições normais isso não importa, porque `initialize()` preenche `pb` logo em seguida. Como `SecondOrderTf::initialize()` não preenche nada, `pb` permanece com o conteúdo arbitrário do *heap*, e `DiffEquation::g()` o multiplica pelas entradas a cada quadro.

O resultado não é zero nem passagem direta: é ruído determinado pelo lixo de memória, que muda a cada execução. `SecondOrderTf` e `Sz2` não devem ser instanciados.

VII.4 Atrasos, *sample-and-hold* e limitadores

Sz1: os coeficientes crus de primeira ordem. O nome sugere um atraso de um *sample* (z^{-1}). Não é isso. Sz1 deriva de `FirstOrderTf` e o seu arquivo `.cpp` **não contém nenhum `g()`**: usa o `DiffEquation::g()` herdado. O que ele acrescenta são oito *slots* — $n1/N1$, $n2/N2$, $d1/D1$, $d2/D2$ — e mais nada.

Por que — Sz1 existe para expor coeficientes ao EDL

`FirstOrderTf` é `EMPTY_SLOTTABLE`: os seus quatro coeficientes não são configuráveis de fora. `LagFilter` e `LowpassFilter` resolvem isso expondo um parâmetro de alto nível (τ , ω_c) e derivando os coeficientes. Sz1 é a terceira alternativa: expõe N_1, N_2, D_1, D_2 diretamente, para quem já tem a função de transferência $H(s) = \frac{N_1 s + N_2}{D_1 s + D_2}$ e não quer que o *framework* a interprete. Sz2 faz o mesmo para segunda ordem — com a ressalva grave da Seção VII.3.

SaH: *sample-and-hold*. Amostra o sinal de entrada a uma taxa menor (*slot sampleRate*, em Hz) do que a taxa principal (*slot rate*) e segura o valor até a próxima amostra:

```
1 double SaH::g(const double xn)
2 {
3     if (!isValid()) return xn;
4
5     px[0] = xn;
6     time += 1.0 / rate;           // avança o contador pelo periodo da taxa mestre
7
8     if (time >= stime) {          // chegou o momento de amostrar?
9         py[0] = px[0];           // sim: captura a entrada
10        time = 0;                 // zera o contador
11    }
12    return py[0];                 // segura o ultimo valor amostrado
13 }
14
15 void SaH::initialize ()
16 {
17     BaseClass::initialize ();
18     if (!isValid()) return;
19
20     px[0] = x0;
21     py[0] = y0;
22     stime = 1.0 / sampleRate;     // periodo de amostragem (s)
23
24     // Truque: age alem do maximo para forçar um sample na 1a chamada a g()
25     time = stime + 1.0;
26 }
```

Listing 259: `SaH::g()` e `initialize()`: contador de tempo e truque de inicialização (`src/linearsystem/SaH.cpp`).

O uso típico é simular a latência de sensores que atualizam a uma taxa menor que o laço TC: um radar que atualiza a 10 Hz num laço de 50 Hz usa `SaH(rate=50, sampleRate=10)`.

Em C++, o uso é direto — construir, configurar, chamar `g()` a cada quadro:

```
1 // Em initData() ou reset() do seu componente:
2 hdgFilter = new linearsystem::LagFilter();
3 hdgFilter->setRate(50);           // a taxa em que g() sera chamado, Hz
4 hdgFilter->setTau(0.5);           // constante de tempo, segundos
5 hdgFilter->setX0(0.0);
6 hdgFilter->setY0(0.0);
7
```



```

8 | rangeHold = new linearsystem::SaH();
9 | rangeHold->setRate(50);           // taxa mestre
10 | rangeHold->setSampleRate(10);    // simula um radar a 10 Hz
11 |
12 | // A cada quadro, no updateTC() ou process():
13 | const double hdgSuave { hdgFilter->g(hdgBruto) };
14 | const double rngRetido{ rangeHold->g(rngBruto) };

```

Listing 260: Uso dos blocos a partir de C++, que é como eles podem de fato ser usados neste *fork*.

Para usá-los a partir de EDL, a aplicação precisa registrá-los na sua própria função de fábrica (Seção III.3.1):

```

1 | base::Object* minhaFactory(const std::string& name)
2 | {
3 |     base::Object* obj {};
4 |
5 |     if (name == linearsystem::LagFilter::getFactoryName())
6 |         obj = new linearsystem::LagFilter();
7 |     else if (name == linearsystem::LowpassFilter::getFactoryName())
8 |         obj = new linearsystem::LowpassFilter();
9 |     else if (name == linearsystem::SaH::getFactoryName())
10 |         obj = new linearsystem::SaH();
11 |     // ... demais blocos ...
12 |
13 |     if (obj == nullptr) obj = models::factory(name);
14 |     if (obj == nullptr) obj = base::factory(name);
15 |     return obj;
16 | }

```

Listing 261: Registrando os blocos de `linearsystem` numa fábrica da aplicação — passo que o *framework* não faz por você.

Feito isso, a forma EDL correspondente seria:

```

1 | // check-edl: fabrica-da-aplicacao
2 | ( LagFilter
3 |     rate: ( Hertz 50 )           // chamado a 50 Hz
4 |     tau: ( Seconds 0.5 )
5 |     x0:  0.0
6 |     y0:  0.0
7 | )

```

Listing 262: Os mesmos blocos em EDL, *supondo* a fábrica acima. Os nomes de fábrica coincidem com os nomes das classes neste módulo.

Limitadores. Três classes implementam o *clamp* escalar $y = \max(\text{lower}, \min(\text{upper}, x))$:

```

1 | double LimitFunc::g(const double xn)
2 | {
3 |     if (!isValid()) return xn;
4 |     px[0] = xn;
5 |     double tmp { xn };
6 |     if (tmp > upper) tmp = upper;
7 |     else if (tmp < lower) tmp = lower;
8 |     py[0] = tmp;
9 |     return py[0];
10 | }
11 |
12 | // Limit01 e Limit11 sao atalhos de construcao -- sem SLOTS proprios:
13 | Limit01::Limit01() : LimitFunc(0,1) {} // [0, 1] -- throttle
14 | Limit11::Limit11() : LimitFunc(1,1) {} // <<< veja o quadro abaixo
15 | // Limit: versao configuravel via EDL (slots 'lower' e 'upper')

```

Listing 263: `LimitFunc::g()`: implementação real do clamp (`src/linearsystem/LimitFunc.cpp`).

Limit01 limita o *throttle* em $[0, 1]$ e funciona como esperado.

Armadilha — Limit11 está quebrado no fonte

O cabeçalho de Limit11 diz “*Limits between -1 and 1*”. O construtor, em `src/linearsystem/Limit11.cpp`, escreve `LimitFunc(1,1)` — limite inferior **um**, não menos um.

O efeito não é um limite estranho: é a destruição do sinal. Com `lower == upper == 1`, `LimitFunc::g()` satura toda entrada para a constante 1,0. Um comando de superfície de controle que passe por um Limit11 deixa de ser um comando e vira deflexão máxima permanente.

Use `(Limit lower: -1.0 upper: 1.0)` enquanto o defeito não for corrigido no fonte.

Note ainda que a afirmação “sem `initialize()`” seria imprecisa: o que Limit01 e Limit11 não têm é uma *tabela de slots*. `LimitFunc::initialize()` existe e é chamada pelo construtor e por ambos os *setters*. O que os dispensa de configuração é outra coisa: `LimitFunc::isValid()` verifica apenas `lower <= upper` e **não exige rate** — ao contrário de todos os demais blocos deste módulo.

No cenário da interceptação

Esta é a única camada que **não** aparece no cenário do Seção II.6 — e a ausência é informativa. O Autopilot do `falcao1` comanda rumo e altitude sem filtro nenhum: o erro de trajetória vai direto ao modelo de dinâmica.

Suavizar esse comando é exatamente o que um `LagFilter` faria. Mas, como a abertura deste capítulo registrou, `models` não depende de `linearsystem` e não há fábrica que o instancie — então essa ligação teria de ser escrita pela aplicação. É um bom primeiro exercício de extensão: uma subclasse de `Autopilot` com um `LagFilter` membro, chamado a cada `process()`, e o resultado passado a `setCommandedHeadingD()`.

VIII A camada linkage

VIII.1 Visão geral: as quatro camadas de E/S

A camada `linkage` implementa concretamente as interfaces abstratas definidas em `base/concepts/linkage` (Seção III.16): transforma dispositivos físicos de *hardware* — joysticks, pedais, painéis de *cockpit* — em canais de dados que a simulação consome, e converte saídas da simulação em sinais para dispositivos físicos de saída. Depende apenas de `mixr_base` e compila em `libmixr_linkage.so`.

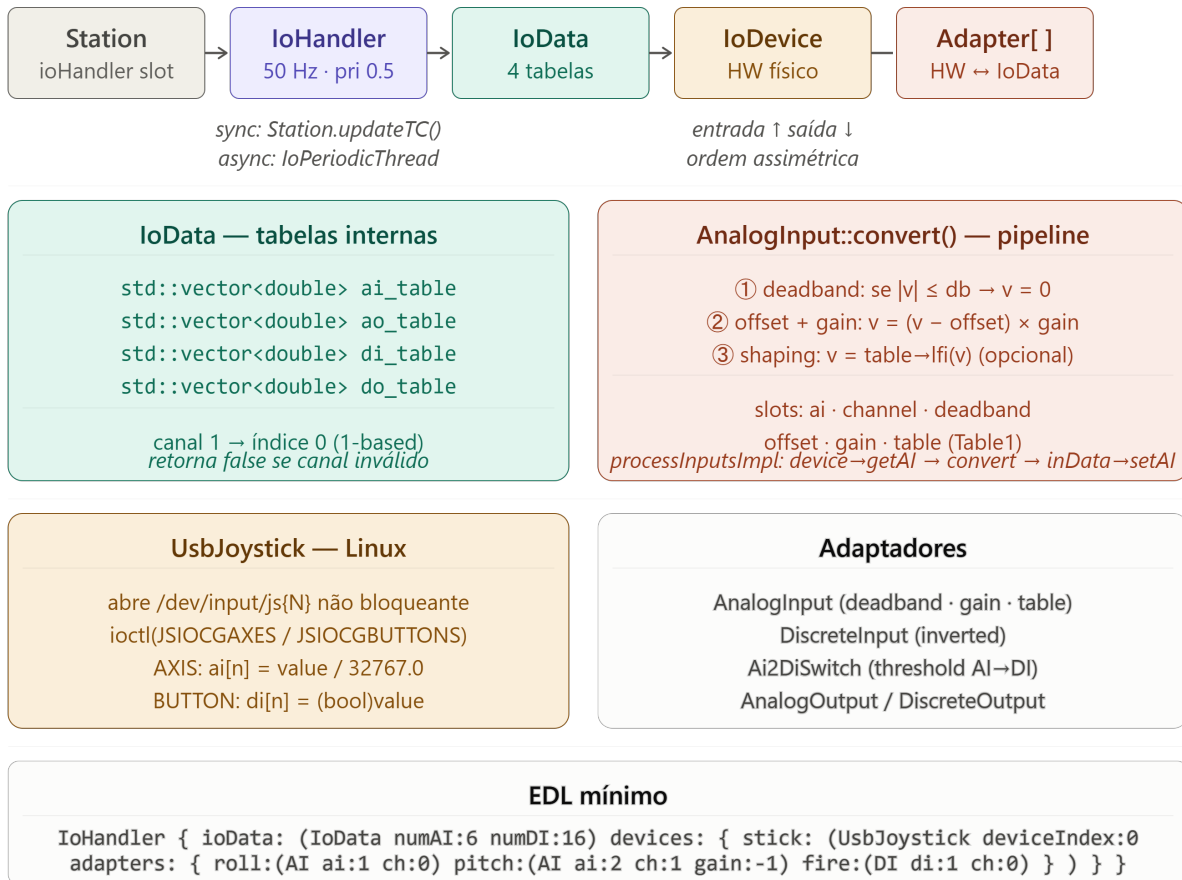


Figure 42: As quatro camadas do módulo `linkage`: fluxo estrutural de `Station` até `Adapter` (topo); tabelas internas de `IoData` e pipeline de `AnalogInput` (centro-esq e centro-dir); `UsbJoystick` e catálogo de adaptadores (baixo); EDL mínimo compondo as quatro camadas.

A arquitetura organiza-se em quatro camadas com responsabilidades estritamente separadas. O fluxo de **entrada** percorre as camadas de baixo para cima: `IoDevice` lê do hardware, cada `Adapter` transforma e copia o seu canal para o `IoData`, e o `IoHandler` disponibiliza o buffer preenchido para a simulação. O fluxo de **saída** percorre o sentido inverso. A `Station` (Seção IV.5) chama `inputDevices(dt)` antes da física e `outputDevices(dt)` depois — garantindo que o *hardware* seja lido antes do modelo de dinâmica executar e escrito depois.

VIII.2 IoData: o *buffer* central

IoData implementa `AbstractIoData` com quatro `std::vector` internos, dimensionados pelos *slots*:

```
1  class IoData : public base::AbstractIoData {
2  private:
3      std::vector<double> ai_table; // Analog Inputs (double)
4      std::vector<double> ao_table; // Analog Outputs (double)
5      std::vector<double> di_table; // Discrete Inputs (double -- bool semantics)
6      std::vector<double> do_table; // Discrete Outputs
7      // Slots: numAI, numAO, numDI, numDO (Number)
8  };
9
10 // Todos os acessores sao 1-based e retornam false sem abortar se invalido:
11 bool IoData::getAnalogInput(const int channel, double* const value) const
12 {
13     bool ok {};
14     if (value != nullptr &&
15         channel > 0 && channel <= static_cast<int>(ai_table.size())) {
16         *value = ai_table[channel-1]; // canal 1 → indice 0
17         ok = true;
18     }
19     return ok;
20 }
21
22 bool IoData::setAnalogInput(const int channel, const double value)
23 {
24     bool ok {};
25     if (channel > 0 && channel <= static_cast<int>(ai_table.size())) {
26         ai_table[channel-1] = value;
27         ok = true;
28     }
29     return ok;
30 }
31
32 // clear(): zera todas as tabelas
33 void IoData::clear()
34 {
35     std::fill(ai_table.begin(), ai_table.end(), 0.0);
36     std::fill(ao_table.begin(), ao_table.end(), 0.0);
37     std::fill(di_table.begin(), di_table.end(), false);
38     std::fill(do_table.begin(), do_table.end(), false);
39 }
40
41 // setNumAI/AO/DI/DO: redimensionam o vector
42 bool IoData::setNumAI(const int num) { ai_table.resize(num); return true; }
```

Listing 264: IoData: campos, slots e acessores (resumo de `include/mixr/linkage/IoData.hpp`).

Canais analógicos (`double`) representam eixos contínuos — joystick, pedais, potenciômetros. Canais discretos (`bool`) representam estados binários — botões, *switches*, LEDs. A separação entre `inputData` e `outputData` no `IoHandler` é puramente de **sentido de fluxo**: “entrada” é o que vem do *hardware* para a simulação, “saída” é o caminho contrário. O *slot* `ioData` aponta os dois para o *mesmo* buffer; os *slots* `inputData` e `outputData` os separam.

Armadilha — canais de dispositivo são 0-based; canais de IoData são 1-based

Esta é a fonte de erro mais comum do módulo, e as duas convenções coexistem a poucos centímetros uma da outra num arquivo EDL. O cabeçalho de `AbstractIoDevice` diz: “*All Channel (and port) numbers start with zero*”. O de `IoData` diz: “*Channel numbers are all one(1) based*”.

Num `AnalogInput`, portanto, `channel: 0` refere-se ao primeiro eixo do joystick, e `ai: 1` refere-se ao primeiro canal do `IoData`. Os dois números apontam para “o primeiro”, e são diferentes.

VIII.3 IoDevice e dispositivos concretos

IoDevice implementa `AbstractIoDevice` e é a base de todo dispositivo de *hardware*. Tem um único *slot*, *adapters*, e não contém sub-dispositivos. A assimetria do ciclo é entre *hardware* e adaptadores, e ela é a coisa mais importante do módulo:

```
1 void processInputsImpl(const double, base::AbstractIoData* const inData) final {
2     readInputs(); // 1. le do hardware para os buffers da classe
3     processInputAdapters(inData); // 2. adaptadores levam dos buffers ao IoData
4 }
5
6 void processOutputsImpl(const double, const base::AbstractIoData* const outData) final {
7     processOutputAdapters(outData); // 1. adaptadores levam do IoData aos buffers
8     writeOutputs(); // 2. escreve os buffers no hardware
9 }
```

Listing 265: IoDevice: a ordem invertida das duas direções (include/mixr/linkage/IoDevice.hpp).

Lida assim, a inversão é evidente: em ambos os casos o *hardware* fica na extremidade e o `IoData` na outra; o que muda é o sentido da travessia.

```
UsbJoystick (Linux).
1 void UsbJoystick::readInputs()
2 {
3     js_event js;
4
5     // Laco nao-bloqueante: le todos os eventos pendentes
6     while (true) {
7         const int status { static_cast<int>(<
8             read(stream, &js, sizeof(js))) };
9         if (status != sizeof(js)) break; // sem mais eventos
10
11         switch (js.type & ~JS_EVENT_INIT) {
12
13             case JS_EVENT_AXIS:
14                 // Normaliza [-32767, +32767] para [-1.0, +1.0]
15                 if (js.number < static_cast<int>(ai.size()))
16                     ai[js.number] = js.value / 32767.0;
17                 break;
18
19             case JS_EVENT_BUTTON:
20                 if (js.number < static_cast<int>(di.size()))
21                     di[js.number] = static_cast<bool>(js.value);
22                 break;
23         }
24     }
25 }
26
27 // Nao ha um open(): o dispositivo e aberto em reset(), que
28 // 1. tenta /dev/js%d (caminho antigo) -- PRIMEIRO
29 // 2. tenta /dev/input/js%d (caminho atual) -- se o primeiro falhar
30 // 3. consulta versao e nome: JSIOCGVERSION, JSIOCGNAME
31 // 4. consulta capacidades: JSIOCGAXES, JSIOCGBUTTONS
32 // (ambas devolvem unsigned char, nao int)
33 // 5. ai.resize(numAxes); di.resize(numButtons);
```

Listing 266: `UsbJoystick::readInputs()`: leitura de eventos do kernel Linux (src/linkage/platform/UsbJoystick_linux.cpp).

MockDevice. Dispositivo simulado sem acesso a hardware real, para desenvolvimento sem bancada.

Apesar do nome e do papel, MockDevice deriva diretamente de `base::AbstractIoDevice`, saltando `IoDevice`. A consequência prática é que ele **não tem o slot** `adapters`: o seu único *slot* é `generators`, e o conteúdo é verificado contra `typeid(AbstractGenerator)`. Colocar um `AnalogInput` ali é rejeitado com uma mensagem no `cerr`.

VIII.4 Adaptadores: mapeamento canal a canal

Cada adaptador mapeia **um canal** do dispositivo físico para **um canal** do `IoData`, com transformações opcionais. Todos herdam de `AbstractAdapter` e implementam `processInputsImpl()` e `processOutputsImpl()`.

```

1 AnalogInput.
2 double AnalogInput::convert(const double vin)
3 {
4     // Estagio 1: zona morta
5     double v1 { vin };
6     if (deadband != 0 && vin < deadband && vin > -deadband)
7         v1 = 0;
8
9     // Estagio 2: offset e ganho
10    const double v2 { (v1 - offset) * gain };
11
12    // Estagio 3: shaping (opcional)
13    // table e uma Table1 (LFI -- Seção sec:base-lfi)
14    double v3 { v2 };
15    if (table != nullptr) v3 = table->lfi(v2);
16
17    return v3;
18 }
19
20 void AnalogInput::processInputsImpl(
21     const base::AbstractIoDevice* const device,
22     base::AbstractIoData* const inData)
23 {
24     double vin {};
25     if (device != nullptr)
26         device->getAnalogInput(&vin, channel); // lê do hardware
27
28     double vout { convert(vin) };
29
30     if (inData != nullptr)
31         inData->setAnalogInput(location, vout); // escreve no IoData
32 }
33
34 // Slots: ai (location no IoData), channel (canal no hardware),
35 //         deadband [0,1], offset, gain, table (Table1 opcional)

```

Listing 267: `AnalogInput::convert()` e `processInputsImpl()`: o pipeline real em três estágios (`src/linkage/adapters/AnalogInput.cpp`).

DiscreteInput. Lê um botão do dispositivo (*slots* `channel` e `port`) e escreve num canal discreto do `IoData` (*slot* `di`). O *slot* `inverted` inverte o valor antes de escrever — útil para *switches* normalmente fechados.

Ai2DiSwitch. Converte um canal analógico em discreto. O *slot* de comparação chama-se `level` (não `threshold`) e o teste é `vin >= level` — inclusive. Os demais *slots* são `di`, `channel` e `inverted`. Útil para gatilhos e pedais de freio que fisicamente são analógicos mas logicamente devem se comportar como botões.

AnalogOutput e DiscreteOutput. O inverso dos adaptadores de entrada: lêem do `IoData` e escrevem no hardware. Usados para *displays* físicos, LEDs e solenóides.

Armadilha — AnalogOutput não é o espelho de AnalogInput

A transformação é a inversa, e não a mesma: $vout = (value / gain) + offset$ — o ganho **divide** e o *offset* é somado *depois*. Não há zona morta.

E há um defeito: `AnalogOutput` declara um *slot table*, mas `processOutputsImpl()` **nunca o aplica**. Uma tabela de conformação declarada na saída é aceita pelo *parser* e ignorada em silêncio.

VIII.5 Geradores de sinal sintético

Para desenvolvimento sem hardware físico há três **geradores**, que injetam valores sintéticos diretamente no `IoData`.

Regra — geradores não são adaptadores

`AbstractGenerator` é uma hierarquia **paralela** a `AbstractAdapter`, não uma subclasse dela. As assinaturas diferem: um adaptador recebe o dispositivo *e* o `IoData`; um gerador recebe `dt` e o `IoData`, e nunca toca num dispositivo. Por isso vão no *slot generators* de um `MockDevice`, e não no *adapters*.

`AnalogInputFixed` injeta um valor constante num canal AI (*slots* `ai` e `value`). `DiscreteInputFixed` injeta um estado fixo num canal DI — e os seus *slots* são `di` e `signal`, este último recebendo os identificadores ON ou OFF, não um booleano.

`AnalogSignalGen` gera sinais periódicos. As formas de onda são **quatro**: `sine`, `cosine`, `square` e `saw` — não existem "sawtooth" nem "pulse". Os demais *slots* são `ai`, `frequency` (uma `base::Frequency`, não um número solto) e `phase` (um `base::Angle`). **Não há slot de amplitude**: a saída é sempre limitada a $[-1, +1]$.

O avanço é uma integração do `dt` recebido: `time += dt`, depois $\alpha = 2\pi ft + \phi$, normalizado para $(-\pi, \pi]$ por `aepcdRad()`. O *slot* `phase` é um deslocamento fixo, não um acumulador, e `reset()` zera o tempo.

VIII.6 IoHandler: o orquestrador

`IoHandler` é o único ponto de contato entre a `Station` e toda a infraestrutura de E/S. Mantém dois `IoData` (`inData` e `outData`) e uma lista de `IoDevice`:

```
1 class IoHandler : public base::AbstractIoHandler {
2   private:
3     // Buffers de I/O
4     base::safe_ptr<base::AbstractIoData> inData; // recebido do hardware
5     base::safe_ptr<base::AbstractIoData> outData; // enviado ao hardware
6     base::safe_ptr<base::PairStream> devices; // lista de IoDevice
7
8     // Thread assíncrono (opcional)
9     double rate {50}; // Hz (slot: rate <Frequency>)
10    double pri {0.5}; // [0..1] (slot: priority <Number>)
11    base::safe_ptr<IoPeriodicThread> periodicThread;
12
13    // Slot 'ioData' seta inData = outData = mesmo buffer (combinado)
14    // Slots 'inputData' e 'outputData' setam buffers separados
15 };
16
```

```

17 // readDeviceInputs(): percorre devices e chama processInputs(dt, inData)
18 void IoHandler::readDeviceInputs(const double dt)
19 {
20     if (devices == nullptr) return;
21     base::List::Item* item { devices->getFirstItem() };
22     while (item != nullptr) {
23         const auto pair = static_cast<base::Pair*>(item->getValue());
24         const auto p = static_cast<base::AbstractIoDevice*>(pair->object());
25         p->processInputs(dt, inData);
26         item = item->getNext();
27     }
28 }
29
30 // startAsyncProcessingImpl(): cria IoPeriodicThread (subclasse de PeriodicThread)
31 void IoHandler::startAsyncProcessingImpl()
32 {
33     if (periodicThread == nullptr) {
34         periodicThread = new IoPeriodicThread(this, getRate());
35         periodicThread->unref(); // safe_ptr mantém a referencia
36         bool ok { periodicThread->start(getPriority()) };
37         if (!ok) periodicThread = nullptr;
38     }
39 }

```

Listing 268: IoHandler: campos e slots (resumo de include/mixr/linkage/IoHandler.hpp).

No modo **síncrono** (padrão), Station chama `inputDevices(dt)` e `outputDevices(dt)` diretamente no seu `updateTC()`, dentro do *frame* de tempo crítico. O mecanismo que implementa a alternativa assíncrona é um portão em `AbstractIoHandler`:

```

1 void inputDevices(const double dt) { if (!async()) inputDevicesImpl(dt); }
2 void outputDevices(const double dt) { if (!async()) outputDevicesImpl(dt); }

```

Listing 269: O portão `async()` — padrão *Non-Virtual Interface* (include/mixr/base/concepts/linkage/AbstractIoHandler.hpp).

E `IoHandler::async()` devolve simplesmente `periodicThread != nullptr`. Ou seja: assim que `startAsyncProcessing()` tiver sucesso, as chamadas que a Station faz a cada quadro **tornam-se inócuas** — o que é exatamente a intenção, já que quem passa a fazer o trabalho é a *thread*.

Armadilha — o modo assíncrono, tal como está, não executa nada

Aqui o mecanismo se fecha sobre si mesmo. A `IoPeriodicThread` chama, no seu laço, `ioHandler->inputDevices(dt)` e `ioHandler->outputDevices(dt)` — **os mesmos métodos guardados pelo portão**. Como o portão está fechado justamente por a *thread* existir, as chamadas dela também não fazem nada.

O resultado é que ligar o processamento assíncrono **desliga a E/S**. Uma implementação correta faria a *thread* chamar `inputDevicesImpl()` e `outputDevicesImpl()` diretamente. Como o módulo exige de todo modo um `IoHandler` concreto escrito pela aplicação, essa é uma das coisas que convém corrigir ao escrevê-lo.

Note ainda que não há nenhum travamento sobre o `IoData`: se a E/S de fato rodasse numa *thread* separada, a leitura pela *thread* de tempo crítico seria uma corrida.

Armadilha — não existe um `IoHandler` concreto no *framework*

Este é o ponto que decide se o módulo é utilizável tal como vem, e a resposta é não — falta uma peça, e é preciso saber disso antes de escrever a primeira linha de EDL. Três fatos que se somam:

1. O nome de fábrica de `linkage::IoHandler` é `"BaseIoHandler"`, não `"IoHandler"` (o comentário

no cabeçalho que diz o contrário está desatualizado).

2. Ele **não está registrado** em `linkage/factory.cpp`, que conhece apenas `IoData`, os cinco adaptadores, os três geradores, `MockDevice` e `UsbJoystick`.
3. Ele é genuinamente abstrato: `AbstractIoHandler` declara `inputDevicesImpl()` e `outputDevicesImpl()` como virtuais puros, e **nenhuma classe em toda a árvore os implementa**. `linkage::IoHandler` oferece apenas os auxiliares protegidos `readDeviceInputs()` e `writeDeviceOutputs()`.

Concluir daí que o módulo é inútil seria errado: a divisão é intencional. O *framework* entrega os dispositivos, os adaptadores e o *buffer*; **a aplicação entrega a política** — em que ordem ler, quando escrever, o que fazer quando um dispositivo falha. O que falta é uma classe de dez linhas que herde de `IoHandler`, implemente os dois métodos chamando os auxiliares prontos, e seja registrada na fábrica da aplicação.

O EDL abaixo, portanto, pressupõe que a aplicação tenha definido e registrado um `MeuIoHandler` nesses termos:

```
1 class MeuIoHandler final : public linkage::IoHandler {
2     DECLARE_SUBCLASS(MeuIoHandler, linkage::IoHandler)
3 public:
4     MeuIoHandler() { STANDARD_CONSTRUCTOR() }
5 private:
6     void inputDevicesImpl(const double dt) final { readDeviceInputs(dt); }
7     void outputDevicesImpl(const double dt) final { writeDeviceOutputs(dt); }
8 };
9 IMPLEMENT_SUBCLASS(MeuIoHandler, "MeuIoHandler")
10 EMPTY_SLOTTABLE(MeuIoHandler)
11 EMPTY_COPYDATA(MeuIoHandler)
12 EMPTY_DELETEDATA(MeuIoHandler)
```

Listing 270: O elo que falta: um `IoHandler` concreto da aplicação.

```
1 ioHandler: ( MeuIoHandler
2     ioData: ( IoData numAI: 6 numDI: 16 )
3
4     devices: {
5         stick: ( UsbJoystick
6             deviceIndex: 0    // /dev/input/js0
7
8         adapters: {
9             // Canal 0 do joystick → IoData.ai[1] (roll)
10            roll: ( AnalogInput
11                ai: 1
12                channel: 0
13                deadband: 0.05
14                gain: 1.0
15            )
16
17            // Canal 1 do joystick → IoData.ai[2] (pitch, invertido)
18            pitch: ( AnalogInput
19                ai: 2
20                channel: 1
21                gain: -1.0
22                // Shaping: mais sensível no centro do eixo
23                table: ( Table1
24                    x: [ -1.0 -0.5 0.0 0.5 1.0 ]
25                    data: [ -1.0 -0.15 0.0 0.15 1.0 ]
26                )
27            )
28
29            // Botao 0 → IoData.di[1] (trigger)
30            fire: ( DiscreteInput
```

```

31         di:      1
32         channel: 0
33     )
34 }
35 )
36 }
37 )

```

Listing 271: Configuração EDL completa, com o *handler* da aplicação, *UsbJoystick* e três adaptadores.

A *Station* lê `IoData.ai[1]` para *roll* e `ai[2]` para *pitch* (invertido pelo `gain: -1.0` e suavizado pela tabela de *shaping*), sem saber que o hardware subjacente é um joystick USB — a zona morta, a inversão de sinal e a curva não-linear são responsabilidade dos adaptadores, não do código de simulação.

No cenário da interceptação

O cenário do Seção II.6 roda sem *hardware*: o *falcao1* voa sob autopiloto, e o *slot ioHandler* da *Station* fica vazio.

Pôr um piloto humano no laço é acrescentar quatro linhas de EDL — um *IoData*, um *UsbJoystick* e dois *AnalogInput* — mais a classe concreta de *IoHandler* que a aplicação precisa fornecer. Feito isso, a *Station* passa a ler o manche antes da física e escrever as saídas depois, dentro do mesmo quadro de 50 Hz, e o *Autopilot* cede lugar ao `setControlStickRollInput()`.

A troca entre voo autônomo e pilotado, portanto, não recompila nada: é uma mudança de arquivo de configuração — que é precisamente a promessa da Seção I.1.

IX A camada recorder

IX.1 Visão geral e Protocol Buffers

A camada **recorder** implementa um sistema completo de gravação e reprodução de dados de simulação. Captura eventos em tempo real durante a execução, serializa com **Protocol Buffers** (protobuf) e permite analisar, imprimir ou retransmitir via rede depois. Depende de `mixr_base`, `mixr_simulation`, `mixr_models` e da biblioteca `protobuf`, compilando em `libmixr_recorder.so`.

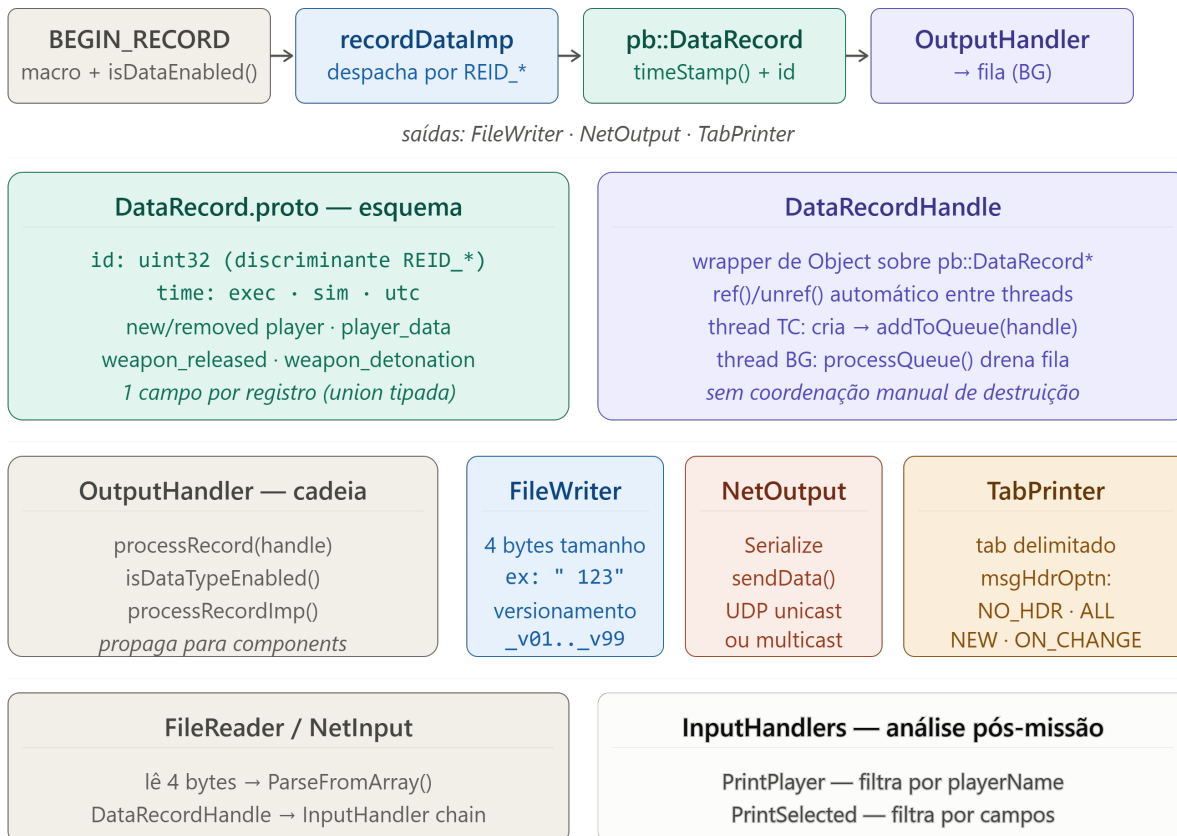


Figure 43: Fluxo do módulo **recorder**: pipeline de gravação em tempo real (topo); esquema `DataRecord.proto` e `DataRecordHandle` como wrapper de ciclo de vida entre threads (centro); cadeia de `OutputHandlers` com os três destinos concretos (baixo-esq); leitura e análise pós-missão (baixo-dir).

O `protobuf` é a única dependência externa nova deste módulo e motiva uma particularidade no *build*: antes de compilar qualquer `.cpp`, o Meson chama `protoc` sobre `DataRecord.proto` para gerar `DataRecord.pb.h` e `DataRecord.pb.cc`. Esse passo está encapsulado num `custom_target` no `proto/meson.build` do módulo — os arquivos gerados são incluídos pelos demais `.cpp` como qualquer cabeçalho ordinário.

IX.2 DataRecord.proto e DataRecordHandle

`DataRecord.proto` define o esquema de todos os eventos possíveis. A mensagem raiz tem dois campos obrigatórios e uma série de campos opcionais:

```

1 syntax = "proto2";           // 'required'/'optional' sao proto2
2
3 package mixr.recorder.pb;
4
5 message DataRecord {
6     required Time   time = 1;    // instante da amostra
7     required uint32 id   = 2;    // REID_* -- discriminante do evento
8
9     // Mensagens do gravador e da simulacao
10    optional FileIdMsg      file_id_msg      = 11;
11    optional UnknownIdMsg   unknown_id_msg   = 13;
12    optional MarkerMsg      marker_msg       = 14;
13    optional InputDeviceMsg input_device_msg = 15;
14
15    // Mensagens de player
16    optional NewPlayerEventMsg new_player_event_msg = 31;
17    optional PlayerRemovedEventMsg player_removed_event_msg = 32;
18    optional PlayerDataMsg      player_data_msg      = 33;
19    optional PlayerDamagedEventMsg player_damaged_event_msg = 34;
20    optional PlayerCollisionEventMsg player_collision_event_msg = 35;
21    optional PlayerCrashEventMsg  player_crash_event_msg  = 36;
22    optional PlayerKilledEventMsg  player_killed_event_msg  = 37;
23
24    // Mensagens de arma, track, etc.
25    // ...
26 }

```

Listing 272: DataRecord.proto: estrutura real, com os números de campo do arquivo (src/recorder/proto/DataRecord.proto).

O campo `id` é o discriminante — o código `REID_*` que identifica o tipo de evento. Apenas o campo opcional correspondente é preenchido, implementando o padrão protobuf de *union* tipada; do lado da leitura, o próprio cabeçalho do `.proto` recomenda usar `id` combinado com as funções `hasXXX()` geradas para saber qual sub-mensagem está presente.

Duas observações de ordem prática. A primeira é que o esquema é declarado em `proto2`, e não em `proto3` — daí as palavras-chave `required` e `optional`, que `proto3` eliminou. A segunda é que a numeração dos campos deixa lacunas deliberadas entre os blocos temáticos (11–15 para `recorder`, 31–37 para *players*, e assim por diante): é espaço reservado para extensões, e o cabeçalho do arquivo documenta explicitamente que aplicações devem usar o mecanismo de *extensions* do protobuf para acrescentar mensagens e códigos `REID_*` próprios, sem alterar o esquema do *framework*.

`DataRecordHandle` é um *wrapper* de ciclo de vida sobre `pb::DataRecord*`: herda de `base::Object` e participa do sistema de contagem de referências. A razão de existir é a segurança entre *threads*: o `DataRecorder` cria registros no *thread* TC (a 50 Hz) e os enfileira com `addToQueue()`; os `OutputHandlers` os processam no *background* com `processQueue()`. O `DataRecordHandle` carrega o `ref()` entre os dois contextos sem que nenhum dos lados coordene destruição explicitamente.

IX.3 DataRecorder

`DataRecorder` é a implementação concreta de `AbstractDataRecorder` e o objeto que a `Station` recebe pelo *slot* `dataRecorder`.

O vocabulário: `dataRecorderTokens.hpp`. Antes de qualquer código, convém conhecer o dicionário. Todo evento gravável do MIXR tem um número — um *token* `REID_*` — e todos estão num único cabeçalho, `include/mixr/simulation/dataRecorderTokens.hpp`. É esse número que a macro de gravação recebe, que

`recordDataImp()` despacha e que o leitor usa para reconhecer a mensagem. Os valores não são arbitrários: são agrupados por faixa.

Valor	Token	Significado e cargas úteis
<i>Controle de arquivo e sessão</i>		
0	REID_END_OF_DATA	fim do arquivo — não pode ser desabilitado
1	REID_FILE_ID	cabeçalho com os metadados da sessão
2	REID_UNHANDLED_ID_TOKEN	<i>token</i> desconhecido ou não tratado
3	REID_RESET_EVENT	a simulação foi reiniciada
<i>Entrada do operador (faixa 20)</i>		
21	REID_MARKER	marcador manual: V1 = id, V2 = fonte
22	REID_DI_EVENT	entrada discreta (chave): V3 = valor
23	REID_AI_EVENT	entrada analógica (manche): V3 = valor
<i>Ciclo de vida do player (faixa 40)</i>		
41	REID_NEW_PLAYER	P1 = o <i>player</i> criado
42	REID_PLAYER_REMOVED	P1 = o <i>player</i>
43	REID_PLAYER_DATA	amostra periódica; P1 = o <i>player</i>
44	REID_PLAYER_DAMAGED	P1 = alvo, P2 = arma
45	REID_PLAYER_COLLISION	P1, P2 = os dois <i>players</i>
46	REID_PLAYER_CRASH	P1 = o <i>player</i>
47	REID_PLAYER_KILLED	P1 = alvo, P2 = atirador
<i>Armamento (faixa 60)</i>		
61	REID_WEAPON_RELEASED	P1 = arma, P2 = atirador, P3 = alvo
62	REID_WEAPON_HUNG	mesma tripla — a arma não saiu do cabide
63	REID_WEAPON_DETONATION	mesma tripla
64	REID_GUN_FIRED	P1 = atirador, V1 = n. ^o de tiros
<i>Rastreo (faixa 80)</i>		
81	REID_NEW_TRACK	P1 = <i>player</i> , P2 = pista
82	REID_TRACK_REMOVED	idem
83	REID_TRACK_DATA	idem, periódico
<i>Faixas reservadas</i>		
500–999	reservado	uso futuro do <i>framework</i>
1000–9999	eventos do usuário	livres para a aplicação

Regra — P1..P4 são os `objs[]`; V1..V4 são os `values[]`

A notação dos comentários do cabeçalho mapeia diretamente na assinatura de `recordData(token, objs[4], values[4])`: P_n é `objs[n - 1]` — um `Object*` contado por referência — e V_n é `values[n - 1]`, um `double` cru.

Ambos os vetores têm exatamente **quatro** posições, sempre, para qualquer evento. Um evento que use um objeto e nenhum valor recebe três `nullptr` e quatro zeros — e é por isso que a assinatura é uniforme e a tabela de despacho pode ser um `switch` simples.

Ao definir um evento próprio, escolha um número entre **1000** e **9999**. Abaixo disso o número pode colidir com um *token* que uma versão futura do MIXR venha a definir.

Os slots: metadados da sessão. Nove dos dez *slots* do `DataRecorder` não afetam o que se grava — apenas identificam *qual* gravação é esta. `eventName`, `application`, `caseNum`, `missionNum`, `subjectNum`, `runNum`, `day`, `month` e `year` são copiados uma única vez para o registro `REID_FILE_ID` no início do arquivo. O décimo, `outputHandler`, é o único com efeito operacional: é o destino dos dados.

Não parece muito, e é decisivo na prática: sem esses campos, uma pasta com duzentos arquivos `.rcd` de uma campanha de testes é indistinguível por inspeção. Com eles, cada arquivo se autoidentifica.

Macros de gravação. O código de simulação nunca chama `recordData()` diretamente — usa as macros `BEGIN_RECORD_DATA_SAMPLE/END_RECORD_DATA_SAMPLE`:

```
1 BEGIN_RECORD_DATA_SAMPLE(getWorldModel()->getDataRecorder(),
2                             REID_WEAPON_DETONATION)
3     // objs []: players e armas relevantes para este evento
4     SAMPLE_3_OBJECTS(this, getLaunchVehicle(), getTargetPlayer())
5     // values []: campos numericos (tipo de detonacao e alcance)
6     SAMPLE_2_VALUES(DETONATE_ENTITY_IMPACT, getDetonationRange())
7 END_RECORD_DATA_SAMPLE()
```

Listing 273: Uso típico das macros de gravação (`src/models/player/weapon/AbstractWeapon.cpp`).

A expansão real é mais simples do que se imagina, e a diferença importa:

```
1 #define BEGIN_RECORD_DATA_SAMPLE(pRecorder, token) \
2 { ::mixr::simulation::AbstractDataRecorder* _p = pRecorder; \
3   if (pRecorder != nullptr) { /* <- UNICA guarda */ \
4     unsigned int _token = token; \
5     const ::mixr::base::Object* _obj[4] = { nullptr, ... }; \
6     double _val[4] = { 0.0, 0.0, 0.0, 0.0 }; \
7   } \
8 #define END_RECORD_DATA_SAMPLE() \
9   _p->recordData(_token, _obj, _val); }
```

Listing 274: As macros transcritas (resumo de `include/mixr/simulation/recorder_macros.hpp`).

Armadilha — o filtro não é “custo zero”

A única guarda dentro da macro é `pRecorder != nullptr`. O teste `isDataEnabled()` acontece **um nível abaixo**, já dentro de `AbstractDataRecorder::recordData()`.

A consequência: quando um evento está filtrado mas existe um gravador, **os argumentos são todos avaliados assim mesmo**. No exemplo acima, `getLaunchVehicle()`, `getTargetPlayer()` e `getDetonationRange()` são chamados, e só então o registro é descartado. Custo zero de verdade existe apenas quando *não há gravador nenhum*.

Isso é relevante para quem instrumenta um laço quente: se a coleta dos argumentos for cara, ela precisa ser guardada explicitamente por um `isDataEnabled()` escrito à mão, antes da macro.

recordDataImp(): despacho e serialização. A tabela de despacho mapeia cada `REID_*` para um `record*()` específico. Cada função cria um `pb::DataRecord`, chama `timeStamp()` e preenche a submensagem correspondente:

```
1 void DataRecorder::timeStamp(pb::DataRecord* const msg)
2 {
3     if (msg == nullptr) return;
4     simulation::Simulation* sim { getSimulation() };
5     pb::Time* time { msg->mutable_time() };
6     if (sim != nullptr) {
7         time->set_exec_time( sim->getExecTimeSec() );
8         time->set_sim_time( sim->getSimTimeOfDay() );
```

```

9         time->set_utc_time ( sim->getSysTimeOfDay() );
10     } else {
11         time->set_exec_time(-1.0);
12         time->set_sim_time (-1.0);
13         time->set_utc_time (-1.0);
14     }
15 }
16
17 bool DataRecorder::recordNewPlayer(const base::Object* objs[4],
18                                   const double values[4])
19 {
20     const auto player = dynamic_cast<const models::Player*>(objs[0]);
21     if (player == nullptr) return false;
22
23     const auto msg = new pb::DataRecord();
24     timeStamp(msg);
25     msg->set_id(REID_NEW_PLAYER);
26
27     pb::NewPlayerEventMsg* newPlayerMsg { msg->mutable_new_player_event_msg() };
28     genPlayerId( newPlayerMsg->mutable_id(), player ); // id + nome + tipo
29     genPlayerState( newPlayerMsg->mutable_state(), player ); // pos + vel + euler
30
31     sendDataRecord(msg); // empacota em DataRecordHandle e chama outputHandler
32     return true;
33 }

```

Listing 275: DataRecorder::timeStamp() e recordNewPlayer(): padrão real de serialização (src/recorder/DataRecorder.cpp).

sendDataRecord() e processRecords(). sendDataRecord() chama outputHandler->addToQueue(handle) na thread TC — enfileira sem bloquear. Na primeira chamada da sessão, envia antes um registro REID_FILE_ID com metadados (eventName, caseNum, missionNum etc.) configurados pelos slots do DataRecorder. processRecords() — chamado por Station::updateData() — drena a fila no background via outputHandler->processQueue(), compatível com a latência variável de disco e rede.

IX.4 Handlers de saída

Os OutputHandlers formam uma cadeia de responsabilidade via components: processRecord(handle) aplica isDataTypeEnabled(), chama processRecordImp() e propaga para cada componente filho. Um único DataRecordHandle pode ser gravado em arquivo e enviado via rede e impresso, numa única chamada ao topo da cadeia:

```

1 void OutputHandler::processRecord(const DataRecordHandle* const dataRecord)
2 {
3     if (dataRecord != nullptr && isDataTypeEnabled(dataRecord)) {
4         processRecordImp(dataRecord); // implementacao concreta (virtual)
5
6         // Propaga para subcomponentes (cadeia de responsabilidade)
7         base::PairStream* subs { getComponents() };
8         if (subs != nullptr) {
9             for (auto* item = subs->getFirstItem();
10                  item != nullptr; item = item->getNext()) {
11                 auto* pair = static_cast<base::Pair*>(item->getValue());
12                 auto* sc = static_cast<OutputHandler*>(pair->object());
13                 sc->processRecord(dataRecord);
14             }
15             subs->unref();
16         }
17     }
18 }
19
20 void OutputHandler::processQueue()

```

```

21 {
22     base::lock(semaphore);
23     const DataRecordHandle* dr {
24         static cast<const DataRecordHandle*>(queue.get()) };
25     base::unlock(semaphore);
26
27     while (dr != nullptr) {
28         processRecord(dr);
29         dr->unref();
30
31         base::lock(semaphore);
32         dr = static cast<const DataRecordHandle*>(queue.get());
33         base::unlock(semaphore);
34     }
35 }

```

Listing 276: `OutputHandler::processRecord()` e `processQueue()` reais (`src/recorder/OutputHandler.cpp`).

FileWriter. Serializa cada `DataRecord` num formato auto-descritivo. O enquadramento é simples o bastante para ser lido a olho num *hex dump*: quatro bytes ASCII com o tamanho do registro, seguidos pelos bytes serializados.

```

1 // registro de 123 bytes:  ' ' '1' '2' '3' <123 bytes de protobuf>
2 // registro de 12 bytes:  ' ' ' ' '1' '2' <12 bytes de protobuf>
3 //
4 // sprintf(nbuff, "%04d", n) e depois os '0' iniciais viram ' '

```

Listing 277: O enquadramento de `FileWriter`: quatro dígitos ASCII, com zeros à esquerda trocados por espaços.

O arquivo é encerrado por um registro `REID_END_OF_DATA` — que é um `DataRecord` normal, enquadrado com o seu tamanho real como qualquer outro. **Não existe um quadro de tamanho " 0"** marcando o fim: quem lê detecta o fim pelo *end-of-file* do próprio *stream*.

O versionamento automático evita sobrescrever arquivos, mas o sufixo é anexado ao **nome inteiro**, extensão inclusive: `output.rcd` vira `output.rcd_v01`, depois `output.rcd_v02`, até `_v99`.

NetOutput. Serializa com `SerializeToString(&wireFormat)` e envia via `netHandler->sendData()`, passando o ponteiro e o comprimento da *string* resultante. O *slot* `netHandler` aceita qualquer subclasse de `NetHandler` — tipicamente `UdpUnicastHandler` para um servidor centralizado ou `UdpMulticastHandler` para múltiplos clientes simultâneos.

TabPrinter. Decodifica cada `DataRecord` e imprime campos em formato tabular. O *slot* `msgHdrOptn` controla os cabeçalhos, e os dois valores do meio são fáceis de trocar:

Valor	Quando o cabeçalho é impresso
NO_HDR	nunca
ALL_MSGS	antes de todo registro
ON_CHANGE	quando o tipo do evento muda em relação ao anterior
NEW_MSG	uma única vez por tipo de evento, na primeira ocorrência

Table 18: Os valores de `msgHdrOptn`. `ON_CHANGE` é o que reage à troca de tipo; `NEW_MSG` usa um *flag* por tipo de mensagem, reiniciado a cada arquivo.

Sem o *slot filename*, a saída vai para *stdout*.

```
1 dataRecorder: ( DataRecorder
2   eventName: "Missao_ALFA"
3   caseNum: 1
4   missionNum: 1
5
6   // Cadeia de OutputHandlers como components:
7   outputHandler: ( RecorderFileWriter
8     filename: "missao_alfa"
9     pathname: "/data/gravacoes"
10
11   // Filho na cadeia: além do arquivo, imprime no terminal
12   components: {
13     printer: ( TabPrinter
14       msgHdrOptn: "NEW_MSG"
15     )
16   }
17 )
18 )
```

Listing 278: Configuração EDL de uma cadeia de gravação completa.

IX.5 *Handlers* de entrada: leitura e análise

Armadilha — *InputHandler* não é o espelho de *OutputHandler*

A simetria entre os dois lados é bem menor do que os nomes sugerem. *InputHandler* tem, na íntegra, dois métodos: *readRecord()* e o virtual puro *readRecordImp()*. **Não há cadeia de responsabilidade:** sem propagação a componentes filhos, sem fila, sem semáforo, sem filtro de tipo em *processComponents()*. A única lógica de *readRecord()* é repetir *readRecordImp()* descartando os registros que *isDataEnabled()* rejeitar.

A cadeia — essa sim, com propagação e com a restrição de tipo que a torna segura — existe apenas do lado da saída.

Os *InputHandlers* recebem *DataRecordHandle* de um *FileReader* ou *NetInput*, em vez da simulação em execução.

FileReader. A cada *readRecordImp()*: lê os 4 bytes de tamanho e os converte com *atoi()*; se o resultado for zero, pula o registro; caso contrário lê os bytes e desserializa com *ParseFromString()*. O fim é detectado por *eof()/fail()* do *stream*, não por um valor sentinela. O *DataRecordHandle* montado é **devolvido ao chamador** — não há um *handler* configurado a jusante para o qual empurrá-lo.

NetInput. Equivalente em rede do *FileReader*: recebe bytes serializados via *NetHandler*, desserializa e distribui para a cadeia de *InputHandlers*. Permite análise em tempo real — um processo separado, em outra máquina, recebe eventos via *UdpMulticastHandler* enquanto a simulação roda.

PrintHandler: a base comum dos impressores. *TabPrinter*, *PrintPlayer* e *PrintSelected* não herdam diretamente de *OutputHandler*, mas de *PrintHandler*, que concentra o que os três têm em comum: o *slot filename* (com *pathname* opcional), a abertura do arquivo com o mesmo versionamento automático de *FileWriter*, e o *fallback* para *stdout* quando nenhum arquivo é configurado. Uma subclasse precisa implementar apenas o *processRecordImp()* que formata o registro.

PrintPlayer e **PrintSelected**. **PrintPlayer** filtra registros de um *player* específico (*slot playerName*) e imprime apenas os eventos daquela entidade — útil para depurar o comportamento individual numa gravação com centenas de *players*. **PrintSelected**, apesar do nome, não seleciona *campos*: seleciona **registros**. Ele compara um campo nomeado (*fieldName*) de um tipo de mensagem (*messageToken*) contra um valor de referência (*compareToValS*, *compareToValI* ou *compareToValD*) usando uma *condition* (EQ, LT ou GT), e imprime apenas os registros que satisfizerem o teste. É um filtro predicado — útil, por exemplo, para listar só as detonações a menos de 10 m do alvo.

No cenário da interceptação

O cenário do Seção II.6 grava em `/data/gravacoes/intercepcao.rcd`, e o que se acumula ali conta a missão inteira: o **REID_NEW_PLAYER** de cada aeronave ao ser criada, o **REID_PLAYER_DATA** periódico com posição e atitude, o **REID_NEW_TRACK** quando o radar do `falcao1` fixa a pista, e — se houver disparo — o par lançamento/detonação.

Repare em quem faz cada coisa. Os registros *nascem* na *thread* de tempo crítico, dentro dos mesmos quadros que movem as aeronaves; mas quem os escreve em disco é **Station::updateData()**, na *thread* de fundo. A fila do **DataRecordHandle** é a fronteira entre as duas — e é ela que impede a latência do disco de entrar no laço de 50 Hz.

X A camada interop

X.1 Visão geral: DIS e HLA

A camada `interop` conecta a simulação MIXR a outras simulações via protocolos padrão da indústria de simulação militar. A árvore de fontes contempla dois protocolos — **DIS** (*Distributed Interactive Simulation*, IEEE 1278) e **HLA** (*High Level Architecture*, IEEE 1516, com o *object model* RPR-FOM) — mas apenas o primeiro é efetivamente construído neste *fork*. O módulo compila em duas bibliotecas:

- `libmixr_interop_common.so` — código compartilhado: gerência de NIBs e lógica de *dead reckoning*
- `libmixr_interop_dis.so` — implementação DIS completa

HLA e RPR-FOM não são compilados. Essa é uma limitação corrente do repositório, e não uma escolha de arquitetura — convém conhecê-la antes de planejar uma integração HLA. O `src/interop/meson.build` do módulo é curto e explícito quanto ao motivo:

```
1  subdir('common')
2  subdir('dis')
3
4  # TODO: figure out how to compile this
5  # subdir('hla')    <= error: ISO C++17 does not allow dynamic exception specifications
6  # subdir('rprfom') <= error: unimplemented virtual method
```

Listing 279: `src/interop/meson.build` na íntegra: dois subdiretórios ativos e dois comentados, com a causa registrada.

Os diretórios `src/interop/hla` (Ambassador, `hla::NetIO`, `hla::Nib`) e `src/interop/rprfom` estão presentes e completos no repositório; o que falta é adequá-los ao padrão C++ usado no *build* — as especificações dinâmicas de exceção (`throw(...)` em assinaturas), removidas em C++17, vêm da API do *RTI*. A dependência `openrti` continua declarada no `conanfile.py` e resolvida pelo Conan, ainda que nenhuma fonte que a utilize entre no *build* atual.

O restante desta seção descreve, portanto, o que está operacional: o núcleo comum e a implementação DIS. As referências a HLA são mantidas onde ajudam a explicar *por que* a fronteira entre `common` e o protocolo concreto foi traçada como foi.

A divisão entre `interop_common` e `interop_dis` reflete uma decisão arquitetural: os mecanismos de gerência de NIBs e o *executive* de rede são idênticos entre DIS e HLA — apenas o formato dos pacotes e o protocolo de transporte diferem. O ponto de integração com a *Station* é o *slot networks*, que aceita uma `PairStream` de `AbstractNetIO`. A *Station* chama `inputFrame(dt)` e `outputFrame(dt)` em cada rede a cada quadro — sem saber se é DIS ou HLA. A separação entre os dois métodos existe porque DIS pode processar entrada e saída em *threads* distintas, enquanto HLA exige que ambas corram no mesmo *thread* (restrição da API RTI).

X.2 interop/common: Nib e dead reckoning

`Nib` é a implementação concreta de `AbstractNib` compartilhada por DIS e HLA. Carrega o estado cinemático de uma entidade de rede no instante do último *update* recebido e extrapola a posição e orientação para os quadros seguintes.

Estado interno e algoritmos de DR.

```

1  // resetDeadReckoning(): chamado ao receber um novo update da rede
2  bool Nib::resetDeadReckoning(
3      const unsigned char dr, // algoritmo: DRM_RPW, DRM_RVW, etc.
4      const base::Vec3d& p, // posicao ECEF ao instante T0 (m)
5      const base::Vec3d& v, // velocidade ECEF / corpo (m/s)
6      const base::Vec3d& a, // aceleracao ECEF / corpo (m/s2)
7      const base::Vec3d& rpy, // angulos de Euler [phi, theta, psi] (rad)
8      const base::Vec3d& av, // velocidade angular (rad/s)
9      const double time // tempo inicial (exec time, s)
10 )
11 {
12     drP0 = p; drV0 = v; drA0 = a;
13     drRPY0 = rpy; drAV0 = av;
14
15     // Pre-computa matrizes para os algoritmos body-referenced (DRM_RPB, DRM_RVB)
16     drComputeMatrixR0(drRPY0, &drR0); // R0: rotacao mundo→corpo em T0
17     drComputeMatrixWwT(drAV0, &drWwT); // wwT: outer product de av
18     drComputeMatrixOmega(drAV0, &drOmega); // Omega: matrix anti-simetrica
19
20     // Suavizacao: se a nova posicao esta a < 1km, interpola sobre 2s
21     // para evitar descontinuidade visual na rececao do update
22     ...
23     drTime = time; // zera o contador de DR
24     return true;
25 }
26
27 // updateDeadReckoning(): chamado a cada frame pelo Player::deadReckonPosition()
28 bool Nib::updateDeadReckoning(const double dt,
29                               base::Vec3d* const pNewPos,
30                               base::Vec3d* const pNewAngles)
31 {
32     double time { updateDrTime(dt) }; // incrementa drTime por dt
33
34     mainDeadReckoning(time, &drPos, &drAngles); // extrapola posicao/orientacao
35
36     // Suavizacao: corrige lentamente o salto de posicao
37     if (smoothTime > 0.0) {
38         drPos += (smoothVel * smoothTime);
39         smoothTime -= dt;
40     }
41
42     *pNewPos = drPos;
43     *pNewAngles = drAngles;
44     return true;
45 }

```

Listing 280: `Nib::resetDeadReckoning()` e `updateDeadReckoning()`: os campos de estado e o fluxo de extrapolação reais (`src/interop/common/Nib.cpp`).

O `enum DeadReckoning` de `Nib.hpp` enumera os oito algoritmos padronizados pelo DIS mais dois valores de controle. Atenção ao nome: o sufixo é `_DRM`, não o prefixo — escreve-se `RVW_DRM`, não `DRM_RVW`. As duas dimensões que os distinguem são o *referencial* (mundo/ECEF ou corpo) e a *ordem* de cada extrapolação:

O ponto que a nomenclatura esconde e vale explicitar: `FPW_DRM` não significa “posição fixa”. O *F* refere-se à rotação (que não é extrapolada), enquanto *P* indica que a extrapolação de translação é de primeira ordem — ou seja, a entidade continua se movendo em linha reta a velocidade constante. Quem quer de fato congelar a entidade usa `STATIC_DRM`, que é o valor padrão de `drNum` e o único que não extrapola nada.

As fórmulas de translação seguem diretamente dessa leitura. Para os algoritmos de 1ª ordem (`FPW_DRM`, `RPW_DRM`): $\mathbf{p}_{\text{novo}} = \mathbf{p}_{\text{DR}} + \mathbf{v}_{\text{DR}} \cdot t_{\text{DR}}$. Para os de 2ª ordem (`RVW_DRM`, `FVW_DRM`): $\mathbf{p}_{\text{novo}} = \mathbf{p}_{\text{DR}} + \mathbf{v}_{\text{DR}} \cdot t_{\text{DR}} + \frac{1}{2} \mathbf{a}_{\text{DR}} \cdot t_{\text{DR}}^2$. Os quatro algoritmos *body-referenced* (sufixo `B_DRM`) exigem transformação adicional via as matrizes `drR0`, `drWwT` e `drOmega` pré-computadas no `resetDeadReckoning()` — neles, os vetores **v** e **a** chegam expressos no referencial do corpo da entidade, não em ECEF, e precisam ser rotacionados a cada passo.

Constante	Referencial	Rotação	Translação
OTHER_DRM	—	definido pelo usuário (<code>dynamicsOther()</code>)	—
STATIC_DRM	—	sem <i>dead reckoning</i> — estado congelado	
FPW_DRM	mundo	nenhuma	1ª ordem (velocidade constante)
RPW_DRM	mundo	1ª ordem	1ª ordem (velocidade constante)
RVW_DRM	mundo	1ª ordem	2ª ordem (aceleração constante)
FVW_DRM	mundo	nenhuma	2ª ordem (aceleração constante)
FPB_DRM	corpo	nenhuma	1ª ordem
RPB_DRM	corpo	1ª ordem	1ª ordem
RVB_DRM	corpo	1ª ordem	2ª ordem
FVB_DRM	corpo	nenhuma	2ª ordem

Table 19: O enum `DeadReckoning` de `include/mixr/interop/common/Nib.hpp`. As letras da sigla são, em ordem: *Fixed/Rate* de rotação, *Position/Velocity* de translação, e *World/Body* de referencial.

A implementação de todos eles é o método `Nib::mainDeadReckoning()`, em `src/interop/common/Nib.cpp` — um `switch` sobre `drNum` com um *case* por constante da tabela acima, usando as matrizes `drR0`, `drWwT` e `drOmega` pré-computadas no `resetDeadReckoning()`.

Armadilha — `navDR_utils` existe, mas é código morto

Há em `src/base/util/navDR_utils.cpp` uma função livre `base::navDR::deadReckoning()` que implementa exatamente os mesmos oito algoritmos, com o mesmo `switch`. É natural supor que seja ela a usada — o padrão de “funções livres para transformações sem estado” da Seção III.7 sugere isso. Não é: uma busca por `navDR::` em toda a árvore não encontra nenhum chamador fora do próprio arquivo. O *dead reckoning* que roda é o de `Nib`; `navDR_utils` é duplicação herdada que ninguém removeu.

Quando um *update* é enviado. O método que decide chama-se `Nib::isPlayerStateUpdateRequired()`, e recebe o tempo de execução corrente. A sua lógica é maior do que “o erro passou do limite”: ele responde *sim* em qualquer destes casos:

1. o *player* mudou de modo, ou pediu remoção (`DELETE_REQUEST`);
2. o *flag* de congelamento mudou;
3. a aparência mudou — dano, fumaça, chamadas ou camuflagem;
4. alguma parte articulada mudou (trem de pouso, portas, elevação de lançador);
5. o erro de posição passou de `maxPositionError`;
6. o erro de orientação passou de `maxOrientationError`;
7. passou-se `maxTimeDR` desde o último envio (o *heartbeat*).

Os três limiares são *slots* do `NetIO`, com estes padrões:

O ganho é fácil de estimar com esses números. Uma aeronave em voo reto e nivelado a 250 m/s acumula 3 m de erro de posição em cerca de 12 ms se o receptor não extrapolasse — mas com `RVW_DRM` o receptor *também* propaga velocidade e aceleração, de modo que o erro só cresce com o que a extrapolação não prevê. Em voo estável, o gatilho que dispara não é o erro: é o *heartbeat* de 5 s. Frente a um envio a 50 Hz, isso

Slot	Padrão	Papel
maxPositionError	3,0 m	erro de posição tolerado
maxOrientationError	3,0°	erro de orientação tolerado
maxTimeDR	5,0 s	<i>heartbeat</i> : envio mesmo sem erro
maxAge	12,5 s	sem receber, o NIB é descartado

Table 20: Os limiares de *dead reckoning* e seus padrões (`src/interop/common/NetIO.cpp`).

é uma redução de $250\times$ — e é por isso que os limiares são o parâmetro de escala mais importante de um exercício DIS grande.

Armadilha — esses *slots* aceitam também uma lista

Os cinco *slots* de limiar são registrados **duas vezes** em `dis::NetIO`: uma vez para um número escalar e outra para uma `PairStream`. A segunda forma permite valores distintos por *kind/domain* de entidade — um míssil pode exigir precisão bem maior que um navio. É um recurso útil e que passa despercebido porque as duas declarações têm o mesmo nome.

X.3 interop/common: NetIO

`NetIO` contém a lógica de gerência de NIBs comum a DIS e HLA. As subclasses concretas completam a implementação com o protocolo de transporte específico.

O repositório de NIBs. `NetIO` mantém duas tabelas, `inputList` e `outputList`, ambas mantidas **ordenadas por chave** e consultadas com `std::bsearch` — portanto $O(\log n)$, não $O(1)$. A chave não é o trio (`site`, `aplicação`, `player`) que se poderia esperar, e sim o par:

```

1 struct NibKey {
2     unsigned short    id;           // ID do player
3     base::safe_ptr<const String> fName; // nome do federado
4 };

```

Listing 281: A chave de busca de um NIB (resumo de `include/mixr/interop/common/NetIO.hpp`).

`Site` e `aplicação` estão dentro do *nome do federado*: `dis::NetIO` codifica os dois numa *string* no formato "`SnnAmm`" através de `makeFederateName()`, e a decodifica com `parseFederateName()`. Esse detalhe parece burocrático e não é — é justamente ele que permite ao código comum de `interop` indexar NIBs sem conhecer o esquema de endereçamento do DIS. O mesmo vale para `makeFederationName()`, que transforma o *exercise ID* em "`Ennn`". É a materialização concreta da fronteira `common/protocolo` que abriu esta seção.

As tabelas têm tamanho fixo, definido em tempo de compilação por `MIXR_CONFIG_MAX_NETIO_ENTITIES` (padrão 5000, Tabela 9) — daí não haver alocação durante o laço.

inputFrame(dt): entidades recebidas. Três passos, e os nomes reais importam porque é por eles que se estende o *framework*:

```

1 void NetIO::inputFrame(const double)
2 {
3     if (isNetworkInitialized()) {
4         netInputHandler(); // virtual puro: a subclasse lê e despacha os PDUs
5         processInputList(); // virtual puro: aplica o estado aos IPlayers

```

```

6 |     cleanupInputList();    // descarta NIBs vencidos
7 | }
8 | }

```

Listing 282: NetIO::inputFrame() transcrito (src/interop/common/NetIO.cpp).

cleanupInputList() remove um NIB quando o tempo desde o seu último *update* passa de *maxAge*, ou quando o *player* associado já está em DELETE_REQUEST. A causalidade é essa: o DELETE_REQUEST é uma *entrada* da decisão de remover, não uma consequência dela.

outputFrame(dt): **players locais.** Também dois passos — updateOutputList() e processOutputList() — e é o segundo que, para cada NIB de saída, aciona em ordem os três fabricantes de mensagem: munitionDetonationMsgFactory(), entityStateManager() e weaponFireMsgFactory(). O estado que eles serializam vem do SynchronizedState do *player* (Seção VI.3.4), e não de uma leitura direta — é isso que permite à *thread* de rede trabalhar sem correr atrás da *thread* de tempo crítico.

X.4 interop/dis: NetIO e tipos de PDU

dis::NetIO estende interop::NetIO com a implementação concreta do protocolo DIS. O seu nome de fábrica é "DisNetIO" — não NetIO — e ele acrescenta a identificação do exercício mais os *handlers* de rede:

```

1 | ( DisNetIO
2 |     siteID: 1           // padrao: 1
3 |     applicationID: 1    // padrao: 1
4 |     exerciseID: 1       // padrao: 1; ZERO funciona como curinga
5 |
6 |     // Dois handlers separados: DIS envia e recebe por sockets distintos
7 |     netInput: ( UdpMulticastHandler multicastGroup: "225.0.0.1" port: 3000
8 |                 localPort: 3000 )
9 |     netOutput: ( UdpMulticastHandler multicastGroup: "225.0.0.1" port: 3000 )
10 | )

```

Listing 283: DisNetIO em EDL: identificação e transporte. Note que são **dois handlers**, não um.

O *slot version* aceita a faixa 0–7, e não apenas “5, 6 ou 7”: os valores nomeiam revisões do padrão (VERSION_1278, VERSION_1278_1, VERSION_1278_1A, VERSION_7...), com padrão 6 (VERSION_1278_1A).

Armadilha — Designator e Receiver não são tratados

O PDU de *Designator* (24) tem uma **struct** declarada em pdu.hpp, mas **nenhum caso de despacho** — ao chegar, cai em processUserPDU(). E o PDU de *Receiver* (27) não tem sequer **struct**: não existe no fonte. Quem precisar de qualquer um dos dois tem de implementá-lo.

Entity State PDU.

```

1 | // Posicao: ECEF, double 64 bits -- precisao centimetrica em qualquer ponto.
2 | // geocPos vem do SynchronizedState do player, nao de uma leitura direta.
3 | pdu->entityLocation.X_coord = geocPos[base::nav::IX];
4 | pdu->entityLocation.Y_coord = geocPos[base::nav::IY];
5 | pdu->entityLocation.Z_coord = geocPos[base::nav::IZ];
6 |
7 | // Orientacao: angulos de Euler, float 32 bits (~0.003 graus de precisao).
8 | // ATENCAO a ordem: o indice 0 e PHI (rolagem), nao PSI.
9 | pdu->entityOrientation.phi = static_cast<float>(geocAngles[base::nav::IPHI]);
10 | pdu->entityOrientation.theta = static_cast<float>(geocAngles[base::nav::ITHETA]);
11 | pdu->entityOrientation.psi = static_cast<float>(geocAngles[base::nav::IPSI]);
12 |

```


Tipo	PDU	Conteúdo
1	Entity State	posição, velocidade, orientação, aparência
2	Fire	arma disparada (origem, destino, munição)
3	Detonation	detonação (posição, resultado)
13	Start/Resume	
14	Stop/Freeze	
15	Acknowledge	gerência de exercício: iniciar, parar, confirmar, requisitar ação e responder
16	Action Request	
18	Data Query	
20	Data	troca de dados numerados
22	Comment	texto livre
23	Electromagnetic Emission	emissão de radar/ <i>jammer</i>
25	Transmitter	rádio transmitindo
26	Signal	conteúdo de áudio/dados do rádio
56	Action Request-R	variantes confiáveis (<i>reliable</i>) das
57	Action Response-R	mensagens de ação

Table 21: Tipos de PDU efetivamente despachados por `dis::NetIO::netInputHandler()`. Há ainda o tratamento de IFF/ATC-NAVAIDS, e qualquer outro tipo cai em `processUserPDU()`, o ponto de extensão da aplicação.

```

13 // appearance: bitfield de 32 bits, montado direto no PDU
14 pdu->appearance = 0x0;
15 // bits 3-4: dano (0=intacto, 1=leve, 2=moderado, 3=destruido)
16 pdu->appearance |= (damageState & 0x3) << 3;
17 // bits 5-6: fumaca
18 pdu->appearance |= (smokeState & 0x3) << 5;
19 // bit 15: em chamas (FLAMES_BIT = 0x00008000)
20 if (isOnFire) pdu->appearance |= FLAMES_BIT;
21 // bits 16-19: postura (LifeForm::ActionState, Secao sec:models-lifeform)
22 if (lf != nullptr) {
23     unsigned int bits {1}; // padrao: UPRIGHT_STANDING
24     if (lf->getActionState() == LifeForm::UPRIGHT_RUNNING) bits = 3;
25     else if (lf->getActionState() == LifeForm::KNEELING) bits = 4;
26     // ... (15 posturas mapeadas, bits 1 a 15)
27     pdu->appearance |= (bits << 16);
28 }
29
30 // Byte swap CONDICIONAL: so quando o host nao e big-endian.
31 // Num host big-endian a conversao seria uma inversao indevida.
32 if (base::NetHandler::isNotNetworkByteOrder()) pdu->swapBytes();

```

Listing 284: Serialização do Entity State PDU: posição em ECEF double, orientação em float, e o campo `appearance` com bits de estado (`src/interop/dis/Nib_entity_state.cpp`, simplificado).

`netInputHandler()` — este é o nome real, com a coquilha do próprio fonte — lê os PDUs do *socket* em lotes e aplica **dois** filtros antes de despachar por `PDUType`. O primeiro é o exercício: `exerciseID` zero funciona como curinga e aceita tudo; qualquer outro valor descarta os PDUs de exercícios diferentes. O segundo é mais importante e menos óbvio:

Regra — o filtro de auto-eco

Cada caso de despacho descarta os PDUs cujo par *site+aplicação* seja o **seu próprio**. Sem isso, um *host* em *multicast* receberia de volta tudo o que enviou e criaria cópias fantasmas dos seus próprios *players*. É uma única linha por caso, e é o que faz o *multicast* ser utilizável.

Para um *Entity State PDU* de entidade desconhecida, `processEntityStatePDU()` cria o NIB de entrada

com `createNewInputNib()` e o insere com `addNib2InputList()`. O *player* correspondente não nasce aí: quem o cria é `NetIO::createIPlayer()`, mais adiante, clonando o protótipo do `Ntm` que casou (Seção X.5.1).

Fire e Detonation PDUs. Quando um *player* local lança uma arma, `dis::Nib` gera um *Fire PDU* com o ID da arma, do lançador e do alvo. Quando a arma detona (Seção VI.13), um *Detonation PDU* é gerado com posição de impacto e código de resultado. Receptores usam esses PDUs para criar e destruir entidades no seu próprio espaço de simulação — garantindo que um lançamento visto por um *host* DIS seja reproduzido fielmente em todos os participantes do exercício.

O “código de resultado” é o enumerado `AbstractWeapon::Detonation`, e ele distingue sete desfechos — o que importa porque só dois deles significam “acertou”:

Valor	Constante	Significado
0	DETONATE_OTHER	desfecho não classificado
1	DETONATE_ENTITY_IMPACT	impacto direto na entidade
2	DETONATE_ENTITY_PROXIMATE_DETONATION	espoleta de proximidade junto ao alvo
3	DETONATE_GROUND_IMPACT	impacto no solo
4	DETONATE_GROUND_PROXIMATE_DETONATION	detonação próxima ao solo
5	DETONATE_DETONATION	autodestruição / fim de tempo de voo
6	DETONATE_NONE	ainda não detonou — é o valor inicial

O valor de partida de `results` é `DETONATE_NONE`, e não `DETONATE_OTHER`: uma arma em voo tem um resultado que significa explicitamente “ainda não há resultado”. Os códigos 1 e 2 são os que a lógica de dano trata como acerto; 3, 4 e 5 são desfechos sem vítima.

Electromagnetic Emission PDU e o `EmissionPduHandler`. O PDU 23 aparece na Tabela 21 como despachado, mas quem o constrói e o interpreta não é o `Nib` nem o `NetIO`: é uma classe configurável em EDL, listada no `slot emissionPduHandlers` do `dis::NetIO`.

A razão é que não existe uma tradução única entre um radar MIXR e um sistema emissor DIS. O padrão identifica emissores por um *emitter name* numérico — um código de catálogo — e cada exercício adota a sua própria convenção. Um `EmissionPduHandler` é essa convenção, escrita como objeto:

```

1  emissionPduHandlers: {
2    ( EmissionPduHandler
3      emitterName: 1105          -- codigo DIS deste emissor
4      emitterFunction: 1        -- funcao (busca/aquisicao/rastreo ...)
5      sensor: ( Radar typeId: "APG-68" ) -- modelo LOCAL correspondente
6      antenna: ( Antenna )      -- antena a clonar na entrada
7      defaultOut: true          -- usado quando nada mais casa (saida)
8    )
9    ( EmissionPduHandler
10     emitterName: 0
11     defaultIn: true           -- captura toda emissao nao reconhecida
12   )
13 }
```

Listing 285: Um `EmissionPduHandler` liga um modelo de sensor local a um código de emissor DIS, nos dois sentidos.

A busca do *handler* certo é o ponto didático, porque é assimétrica — duas sobrecargas de `findEmissionPduHandler()`, cada uma com um critério de casamento diferente:

Sentido	Recebe	Casa por
Saída (local → rede)	<code>models::RfSensor*</code>	<code>typeId</code> igual ao do sensor-modelo
Entrada (rede → local)	<code>EmissionSystem*</code>	<code>emitterName</code> igual ao do PDU

Faz sentido: ao publicar, sabe-se qual é o radar local e procura-se o código; ao receber, tem-se o código e procura-se o radar. Nos dois casos, se nenhum *handler* casar, faz-se uma segunda varredura à procura de um marcado com `defaultOut` ou `defaultIn`; se nem isso houver, a função devolve `nullptr` e a emissão simplesmente não atravessa a rede.

Armadilha — o casamento de saída é por `typeId`, não por classe

`isMatchingRfSystemType()` compara `std::strcmp` entre o `getTypeId()` do sensor-modelo declarado no *slot* `sensor` e o do radar do *player*. Não há `dynamic_cast`, não há comparação de nome de classe — é uma comparação de *strings*.

Duas consequências práticas. Primeira: o `typeId` do `sensor` no *handler* precisa ser idêntico, caractere a caractere, ao do radar configurado no *player*; qualquer divergência faz a busca cair no `defaultOut`. Segunda: sem nenhum *handler* com `defaultOut`, um radar cujo `typeId` ninguém previu emite localmente sem nunca aparecer na rede — silenciosamente.

Casado o *handler*, os *slots* `sensor` e `antenna` ganham o segundo papel: na entrada, eles são **protótipos a clonar**. Ao receber uma emissão de uma entidade remota, o *handler* instancia cópias desses modelos e as pendura no *player* de entrada — que passa a ter um radar plausível, com diagrama de antena e tudo, embora quem o simula esteja noutra máquina. É o mesmo idioma de protótipo do `Ntm` (Seção X.5.1), aplicado a subsistemas em vez de a *players*.

X.5 interop/dis: EntityType e o mapeamento por Ntm

`dis::Nib` estende `interop::Nib` com os campos específicos do DIS. O tipo de entidade é codificado em sete campos hierárquicos:

```

1 struct EntityType {
2     uint8_t kind;           // plataforma, municao, lifeform, ...
3     uint8_t domain;        // terra, ar, mar, subsuperficie, espaco
4     uint16_t country;      // codigo de pais SISO
5     uint8_t category;      // depende de kind+domain (caca, ataque, ...)
6     uint8_t subcategory;
7     uint8_t specific;
8     uint8_t extra;         // configuracao especifica
9
10    // operator==: zero funciona como CURINGA em qualquer dos quatro
11    // campos finais -- os tres primeiros tem de bater exatamente.
12    bool operator==(const EntityType& a) const {
13        return kind == a.kind && domain == a.domain && country == a.country
14            && (category == a.category || category == 0 || a.category == 0)
15            && (subcategory == a.subcategory || subcategory == 0 || a.subcategory == 0)
16            && (specific == a.specific || specific == 0 || a.specific == 0)
17            && (extra == a.extra || extra == 0 || a.extra == 0);
18    }
19 };

```

Listing

286:

`EntityType`: os sete campos e a comparação com curingas (resumo de `include/mixr/interop/dis/structs.hpp`). Os valores citados nos comentários vêm do padrão SISO, não do cabeçalho.

X.5.1 Ntm: a correspondência entre tipos de rede e *players*

Falta o elo que liga os sete números do `EntityType` a uma classe de *player* do MIXR. Esse elo é o `Ntm` — *Network Type Mapper* — e o seu desenho é bem mais interessante do que uma tabela de tradução.

Um `Ntm` é um protótipo, não uma linha de tabela. A ideia central é que cada mapeamento carrega um *player* inteiro, configurado, no *slot template*:

```
1 class Ntm : public base::Object {
2     DECLARE_SUBCLASS(Ntm, base::Object)
3 public:
4     // slot "template" <Player>
5     const models::Player* getTemplatePlayer() const { return tPlayer; }
6 };
```

Listing 287: `interop::Ntm`: um *slot*, e é o que importa (`include/mixr/interop/common/Ntm.hpp`).

Ao chegar uma entidade nova da rede, o *framework* localiza o `Ntm` correspondente e **clona o seu *player* protótipo** para criar o *IPlayer* (*Interoperability Player*) que entra na lista da simulação. O que se configura, portanto, não é “que classe instanciar”, e sim “qual aeronave, já pronta, copiar” — com assinatura, sensores, dinâmica e tudo mais que o protótipo tiver.

Por que — protótipo em vez de nome de classe

Uma entidade DIS remota precisa de mais do que uma classe: precisa de um RCS plausível, de um modelo de dinâmica para o *dead reckoning* e de um tipo declarado. Clonar um protótipo declarado em EDL resolve tudo isso de uma vez, e mantém a decisão nas mãos de quem escreve o cenário em vez das de quem escreveu o *framework*.

Duas listas, não um mapa bidirecional. O `NetIO` mantém **duas** listas independentes, alimentadas por dois *slots* distintos — `inputEntityTypes` e `outputEntityTypes`. Não há um mapa único percorrido nos dois sentidos, e a razão é que as duas direções fazem perguntas diferentes: a de entrada casa *sete números* contra protótipos; a de saída casa *uma classe C++ mais uma string de tipo* contra códigos.

```
1 ( DisNetIO
2     // ... siteID, applicationID, exerciseID, netInput, netOutput ...
3
4     // Rede -> simulacao: que player criar para cada tipo DIS recebido
5     inputEntityTypes: {
6         ( DisNtm
7             disEntityType: [ 1 2 225 1 1 1 0 ] // kind domain country cat sub spec extra
8             template: ( Aircraft
9                 type: "F-16C"
10                signature: ( SigConstant rcs: ( dB 5.0 ) )
11            )
12        )
13        ( DisNtm
14            disEntityType: [ 1 1 225 1 1 0 0 ]
15            template: ( Tank type: "M1A2" )
16        )
17    }
18
19    // Simulacao -> rede: que codigo DIS emitir para cada player local
20    outputEntityTypes: {
21        ( DisNtm
22            disEntityType: [ 1 2 225 1 1 1 0 ]
23            template: ( Aircraft type: "F-16C" )
24        )
25    }
```

Listing 288: A forma real do mapeamento DIS: `DisNtm` com protótipo e vetor de sete códigos, em duas listas separadas.

Na prática essas listas são longas e vivem em arquivos à parte, incluídos no cenário — o próprio código-fonte cita `DisIncomingEntityTypes.epp` e `DisOutgoingEntityTypes.epp` como exemplos.

A busca de entrada: uma árvore de sete níveis. Casar sete números contra centenas de protótipos com uma varredura linear seria caro a cada PDU recebido. O DIS resolve isso com uma árvore de decisão — os `NtmInputNode` — em que cada nível discrimina por um campo do `EntityType`, na ordem *kind* → *domain* → *country* → *category* → *subcategory* → *specific* → *extra*. É a mesma ordem hierárquica que o `operator==` acima usa, agora explorada como índice.

A busca de saída: melhor casamento por classe e prefixo de tipo. A direção contrária não pode ser uma árvore, porque a chave é um objeto C++. A regra, documentada no próprio cabeçalho, tem duas partes: a classe do protótipo tem de bater com a do *player* (ou ser uma classe base dele), e a *string* de tipo do protótipo tem de ser **prefixo** da do *player*. Entre os que casam, vence o mais específico. Para um `Aircraft` de tipo "F-16C":

Protótipo declarado	Casa?	Por quê
(<code>Aircraft type: "F-16C"</code>)	sim (melhor)	exato
(<code>Aircraft type: "F-16"</code>)	sim	prefixo
(<code>AirVehicle type: "F-16C"</code>)	sim	classe base, tipo exato
(<code>AirVehicle type: "F-16"</code>)	sim (pior)	classe base, prefixo
(<code>Aircraft type: "F-16A"</code>)	não	<i>string</i> diverge
(<code>Aircraft type: "F-16C1"</code>)	não	mais longa que a do alvo
(<code>Ship type: "F-16C"</code>)	não	classe incompatível

Table 22: A regra de melhor casamento na saída, com o exemplo do próprio `include/mixr/interop/common/Ntm.hpp`.

Armadilha — sem `Ntm` de entrada, a rede é ignorada em silêncio

Uma entidade cujo tipo não case com nenhum `Ntm` da lista de entrada **não é criada**. Não vira um *player* genérico, não gera aviso: é simplesmente descartada. E o caso extremo é o mais fácil de cair: **sem o *slot* `inputEntityTypes`, todas as entidades da rede são ignoradas** — a simulação conecta, recebe PDUs e não mostra ninguém. Há ainda um segundo filtro depois desse, o *slot* `maxEntityRange`, que descarta entidades novas além de uma distância.

Aparência e estado de dano. `dis::Nib` mantém o campo `appearance` atualizado a cada quadro a partir do estado do *player*: nível de dano (0–3), fumaça (0–3), chamadas, mobilidade comprometida, potência comprometida e tipo de camuflagem. Para `LifeForm`, os bits 16–19 mapeiam diretamente para `ActionState` (Seção VI.14). Receptores DIS conformes usam esses bits para renderizar o estado visual correto — um *tank* fumegante com mobilidade comprometida aparece na representação do receptor exatamente como está no emissor, sem mensagem adicional.

No cenário da interceptação

O cenário do Seção II.6 publica o `falcao1` no exercício DIS 17, e o faz com muito menos tráfego do que se poderia supor.

Em voo reto sob autopiloto, o *dead reckoning* do receptor acerta a posição do caça com folga: o erro não chega aos 3m de `maxPositionError`. O que dispara o envio, então, não é o erro — é o *heartbeat* de 5s. Frente a um envio a cada quadro, isso é duas ordens de grandeza a menos de pacotes.

O quadro muda no instante da interceptação: a manobra faz o erro estourar o limiar a cada poucos décimos de segundo, e a taxa de *Entity State PDUs* sobe. É o comportamento desejado — a rede gasta banda exatamente quando há algo acontecendo.

E há a outra direção: as duas listas de `Ntm` declaradas no cenário são o que permite ao `falcao1` ver entidades de outros simuladores no mesmo exercício. Sem a lista de entrada, ele publicaria sem nunca receber ninguém.

libmixr_interop_common.so — agnóstico de protocolo

Nib — estado de *dead reckoning*

drPos · drVel · drAccel (ECEF)
drAngles · drAngVel · drTime

dez algoritmos, e _DRM é sufixo:

OTHER · STATIC

FPW · RPW · RVW · FVW (mundo)

FPB · RPB · RVB · FVB (corpo)

reenvia acima de 3 m ou 3°,
ou a cada 5 s de *heartbeat*

NetIO — repositório de NIBs

inputList[] — recebidos da rede
outputList[] — *players* locais
arrays fixos de MAX_OBJECTS = 5000

findNib() monta NibKey{id, fName}
e faz bsearch() na lista ordenada — $O(\log n)$
a chave é *player* + nome do federado,
não a tripla *site/app/player*

inputFrame() · outputFrame() — separados

libmixr_interop_dis.so — DIS IEEE 1278

dis::NetIO

siteID · appID
exerciseID
version: 5 · 6 · 7 (padrão 6)
UdpMulticastHandler
netInputHandler() despacha

Entity State PDU

ECEF: double 64 bits
Euler: float 32 bits
appearance: 32 bits
dano · fumaça · chamadas

Ntm — MIXR ↔ DIS

slot template: o *player*
NtmOutputNode /
NtmInputNode
EntityType: 7 campos
até 1000 mapeamentos

Quais PDU o despachante realmente trata

Quatro chegam a fazer trabalho: 1 Entity State · 2 Fire · 3 Detonation · 23 EM Emission

Onze têm case, mas o corpo é return true; — *stubs*:

13 Start	15 Acknowledge	20 Data	26 Signal
14 Stop	16 Action Req	22 Comment	56 · 57 Act R
	18 Data Query	25 Transmitter	

24 Designator e 27 Receiver não têm sequer um case — o MIXR os descarta em silêncio.

netInputHandler é a grafia do próprio fonte (interop/common/NetIO.hpp:237) — falta o *I* de *handler*, e é assim que o método se sobreescreve.

Figure 44: Arquitetura do módulo interop: Nib com o estado de *dead reckoning* e NetIO como repositório de NIBs (interop_common, topo); dis::NetIO e o Entity State PDU (interop_dis, centro); o tipo EntityType e o Ntm como mapeamento MIXR↔DIS (baixo). Tipos de PDU implementados no rodapé.

XI Sistema de compilação: Meson e Conan

A integração do MIXR num projeto externo envolve três decisões interdependentes: gerência de dependências, descrição do *build* e ligação das bibliotecas. Este projeto adota **Conan 2** para dependências e **Meson** para a descrição do *build*, com a comunicação entre eles feita por `PkgConfigDeps` e `MesonToolchain`.

A seção começa pelo lado de dentro — como o próprio MIXR é construído e empacotado — porque é isso que determina o que um projeto consumidor encontra do outro lado.

XI.1 Compilando o próprio MIXR

O repositório do MIXR é ele próprio um pacote Conan construído por Meson. O `Makefile` da raiz é um invólucro fino sobre os dois. Dos seis alvos que declara, quatro descrevem o ciclo de *build* (os outros dois são `clean` e `help`):

```
1 make configure # conan install + meson setup (buildtype debug, prefix ../dist)
2 make build     # meson compile -C ./build
3 make install   # meson install -C ./build
4 make package   # conan create, em Debug e Release
```

Listing 289: Os alvos do `Makefile` da raiz do MIXR.

O `meson.build` da raiz fixa os parâmetros globais do projeto. Dois deles merecem registro por afetarem diretamente quem estende o *framework*:

```
1 project('mixr', 'cpp',
2     version: '1.0.5',
3     license: 'LGPL-3.0',
4     default_options: [
5         'default_library=shared',
6         'c_std=c11',
7         'cpp_std=c++11',      # <-- o MIXR compila como C++11
8         'c_args=-fPIC',
9         'cpp_args=-fPIC',
10    ],
11 )
12
13 add_project_arguments('-fpermissive', language: 'cpp') # <-- afrouxa a conformidade
```

Listing 290: `meson.build` da raiz: padrão de linguagem e *flags* globais.

O primeiro é o `cpp_std=c++11`: apesar do uso extensivo de inicialização uniforme com chaves e de `nullptr` no código, o MIXR é construído contra o padrão C++11. O segundo é o `-fpermissive`, que rebaixa a erros de conformidade a categoria de aviso — é o que permite ao código-fonte, herdado de uma base bem mais antiga, compilar sem uma revisão completa. Nenhum dos dois impede que uma aplicação consumidora use um padrão mais recente (a Seção XI.3 usa C++17), já que a fronteira entre elas são cabeçalhos e símbolos, não unidades de tradução compartilhadas.

As dependências externas são três, declaradas no `conanfile.py` e resolvidas por *pkg-config* do lado do Meson:

O `package_info()` do `conanfile.py` expõe o resultado como nove componentes Conan (`base`, `interop_common`, `interop_dis`, `linearsystem`, `linkage`, `models`, `recorder`, `simulation`, `terrain`), e o `pkg.generate()` da raiz produz um `mixr.pc` agregador que lista as nove bibliotecas na ordem correta de ligação. É esse arquivo que a Seção XI.3 consome com uma única chamada a `dependency('mixr')`.

Dependência	Versão	Usada por
jsbsim	1.1.11	models — JSBSimModel (Seção VI.4.4)
protobuf	3.21.12	recorder — serialização de DataRecord
openrti	(commit fixo)	declarada para HLA, que não é compilado (Seção X.1)

Table 23: Dependências externas do MIXR e quem as consome.

XI.2 Estrutura do `conanfile.py`

O `conanfile.py` de um projeto que consome o MIXR segue o padrão canônico do Conan 2 com quatro métodos obrigatórios:

```

1 from conan import ConanFile
2 from conan.tools.meson import MesonToolchain
3 from conan.tools.gnu import PkgConfigDeps
4
5 class MixrProjectConan(ConanFile):
6     settings = "os", "compiler", "build_type", "arch"
7     generators = [] # gerado manualmente em generate()
8
9     def requirements(self):
10         self.requires("mixr/VERSION")
11         # demais dependências diretas do projeto
12
13     def layout(self):
14         # build/ e generators/ no mesmo diretório
15         self.folders.build = "build"
16         self.folders.generators = "build"
17
18     def generate(self):
19         # 1. Gera os arquivos .pc para cada biblioteca MIXR
20         pc = PkgConfigDeps(self)
21         pc.generate()
22
23         # 2. Gera cross_file/native_file com flags de compilador
24         tc = MesonToolchain(self)
25         tc.generate()

```

Listing 291: Estrutura canônica do `conanfile.py` para projetos que consomem o MIXR via Conan 2 e Meson.

Os pontos críticos do `conanfile.py`:

- `requirements()` — todas as dependências diretas do projeto são declaradas aqui com `self.requires(...)`. O MIXR arrasta consigo as suas (`jsbsim`, `protobuf`, `openrti`, e `zlib` por transitividade do `protobuf`); não é necessário declará-las explicitamente a menos que o projeto as use diretamente. O `jsbsim` é declarado com `transitive_headers=True`, de modo que os seus cabeçalhos ficam visíveis a quem inclua `JSBSimModel.hpp`.
- `layout()` — `build` e `generators` devem apontar para o **mesmo diretório**. O Meson espera que o `native_file` e os arquivos `.pc` gerados pelo Conan estejam no mesmo local que a pasta de `build` configurada.
- `generate()` — `PkgConfigDeps` gera um arquivo `.pc` por biblioteca MIXR instalada. `MesonToolchain` gera o `conan_meson_native.ini` (ou `cross`), que carrega os caminhos corretos de compilador e *flags*.

XI.3 Descrição do *build* com Meson

O `meson.build` do projeto declara uma única dependência sobre o MIXR usando o método `pkg-config`:

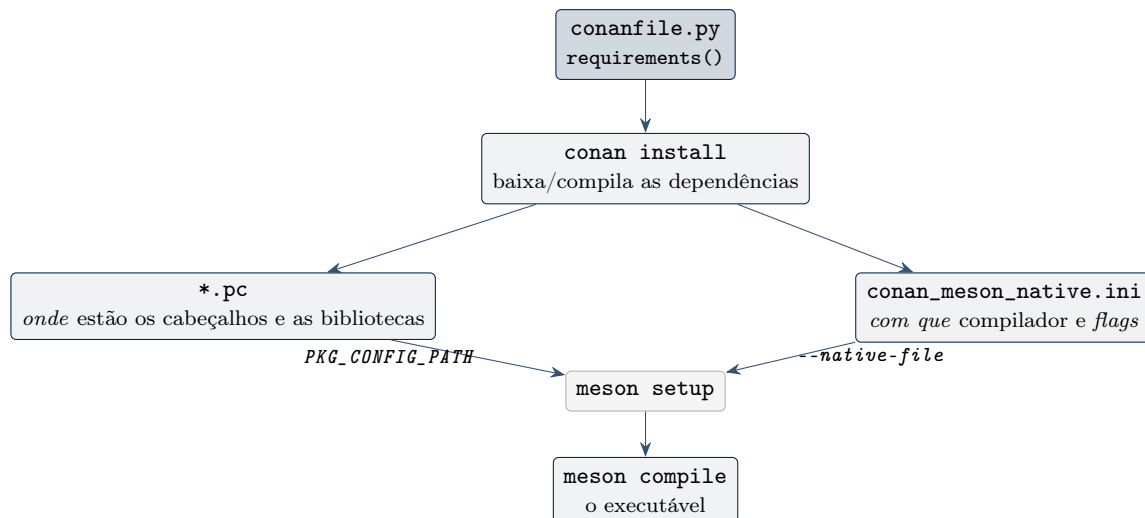
```
1 project('poc-mixr', 'cpp',
2         version      : '0.1.0',
3         default_options : ['cpp_std=c++17'])
4
5 # Uma unica declaracao cobre todas as bibliotecas MIXR
6 # expostas pelo arquivo .pc gerado pelo Conan
7 mixr_dep = dependency('mixr', method : 'pkg-config')
8
9 executable('poc-mixr',
10           sources : ['main.cpp'],
11           dependencies : [mixr_dep])
```

Listing 292: `meson.build` mínimo para um projeto que usa o MIXR.

A decisão de usar uma **única** chamada a `dependency()` em vez de uma por módulo (`mixr_base`, `mixr_simulation` etc.) é deliberada: o arquivo `.pc` gerado pelo Conan para o pacote `mixr` já lista todas as **Requires**: das sub-bibliotecas na ordem correta de ligação. Declarar cada módulo separadamente no Meson seria redundante e quebraria a precedência de ligação em plataformas que exigem ordem explícita de `-l` flags.

XI.4 Fluxo de configuração e compilação

Conan e Meson não se conhecem. O que os liga são dois artefatos gerados pelo primeiro e lidos pelo segundo — e entender *quais* é o que faz o fluxo deixar de ser uma receita decorada:



Os dois arquivos respondem a perguntas diferentes, e por isso são passados ao Meson por caminhos diferentes: os `.pc` dizem *onde* está cada biblioteca, e chegam pela variável de ambiente; o *native file* diz *com que* compilador e opções construir, e chega por um argumento de linha de comando.

O fluxo de trabalho completo, do repositório limpo à aplicação compilada, é o seguinte:

```
1 # 1. Instala as dependencias e gera os arquivos de integracao
2 conan install . --output-folder=build --build=missing
```

```

3
4 # 2. Configura o Meson apontando para o native file do Conan
5 # e para o diretório onde os .pc foram gerados
6 PKG_CONFIG_PATH="$(pwd)/build" \
7 meson setup build \
8     --native-file build/conan_meson_native.ini \
9     --buildtype=release
10
11 # 3. Compila
12 meson compile -C build
13
14 # 4. (opcional) Roda os testes
15 meson test -C build

```

Listing 293: Comandos de configuração e compilação. O `PKG_CONFIG_PATH` deve apontar para o diretório `build/` onde o Conan gerou os arquivos `.pc`.

Armadilha — esquecer o `PKG_CONFIG_PATH` dá um erro enganoso

Sem a variável, o `pkg-config` procura os `.pc` apenas nos caminhos do sistema — e o Conan os gerou dentro de `build/`. O Meson então falha com Dependency "mixr" not found, que se lê naturalmente como “o MIXR não está instalado”.

Não é isso: o MIXR está compilado e o `.pc` existe; o que falta é dizer onde procurar. O sintoma é idêntico ao de uma dependência genuinamente ausente, e é a primeira coisa a conferir antes de reinstalar qualquer coisa.

Regra — o `layout()` do `conanfile.py` não é decorativo

A Seção XI.2 pede que `folders.build` e `folders.generators` apontem para o **mesmo** diretório. O diagrama acima mostra por quê: os dois artefatos gerados precisam estar onde `meson setup` vai procurá-los — um pela variável de ambiente, o outro por caminho relativo. Separá-los faz uma das duas metades do elo se perder, e o erro resultante aponta para o lado errado do problema.

XI.5 Dependências com código C++ interdependente: o caso `models/`

O módulo `models/` apresenta dependências circulares internas que exigem uma estratégia de compilação diferente da simples ordenação topológica. As classes `Player`, `System`, `Navigation` e os modelos de arma formam um ciclo: `Player` referencia `System`; `System` usa ponteiros de volta para o `Player`; `Navigation` e o gerenciador de armas (`StoresMgr`) dependem de ambos.

A solução adotada é compilar **todos esses arquivos num único alvo de biblioteca** em vez de incrementalmente, eliminando a circularidade do ponto de vista do enlace. O `meson.build` interno do MIXR já reflete isso: toda a pasta `src/models/` é compilada como um único `shared_library('mixr_models', ...)`.

Para projetos que estendem o MIXR com novas subclasses (por exemplo, um `DynamicsModel` personalizado), a mesma regra se aplica: **nunca compilar a extensão num alvo separado que ligue contra `mixr_models` se a extensão e `mixr_models` tiverem dependências mútuas**. O arquivo da extensão deve entrar na mesma `sources` list que os demais arquivos de `models/`.

Além disso, arquivos específicos de plataforma devem sempre ser incluídos ao lado do arquivo genérico correspondente; omiti-los gera erros de símbolo não resolvido que só aparecem no enlace final. No MIXR

esses arquivos ficam todos em `base` e `linkage` — `src/base/util/platform/system_linux.cpp` ao lado de `src/base/util/system_utils.cpp`, e assim por diante (Tabela 10). O módulo `models` não tem código de plataforma próprio.

O `meson.build` real de `models` é, portanto, uma lista plana seguida de uma única chamada a `library()`, sem qualquer condicional:

```
1 source_files = [  
2     './WorldModel.cpp',  
3     './factory.cpp',  
4     './player/Player.cpp',  
5     './system/System.cpp',  
6     './navigation/Navigation.cpp',  
7     # ... 107 arquivos no total, em uma única lista ...  
8 ]  
9  
10 mixr_models = library(  
11     'mixr_models',  
12     source_files,  
13     include_directories: public_headers,  
14     dependencies: [  
15         mixr_base_dep,  
16         mixr_simulation_dep,  
17         mixr_terrain_dep,  
18         jsbsim_dep,  
19     ],  
20     link_args : [ static_flags ],  
21     install : true,  
22 )  
23  
24 mixr_models_dep = declare_dependency(link_with : mixr_models)  
25 pkg.generate(mixr_models, name : 'mixr-models')
```

Listing 294: `src/models/meson.build` (estrutura real): um alvo único para todo o módulo, com as quatro dependências declaradas.

Note que a ordem dos arquivos dentro de `source_files` não tem significado — `factory.cpp` aparece perto do início da lista, e isso é irrelevante: cada unidade de tradução é compilada isoladamente e as dependências entre elas se resolvem no enlace.

No cenário da interceptação

O cenário do Seção II.6 fecha o círculo aqui, porque nada dele roda sem este capítulo.

O `main()` que carrega `intercepcao.edl` não é um programa MIXR: é um programa *seu* que se liga a bibliotecas MIXR. E a lista de bibliotecas de que ele precisa não é arbitrária — sai direto do arquivo de configuração. Um cenário com `QuadMap` exige `mixr_terrain`; com `DataRecorder`, exige `mixr_recorder` e, por transitividade, `protobuf`; com `dis::NetIO`, exige `mixr_interop_dis`. Trocar uma linha de EDL pode mudar o que se liga.

É por isso que o `pkg-config` importa tanto: cada `pkg.generate()` deste capítulo produz o `.pc` que faz `dependency('mixr-models')` funcionar no `build` da sua aplicação, com as dependências transitivas já resolvidas. O `meson.build` do apêndice tem cinco linhas por causa disso.

O apêndice traz o arquivo completo e os comandos exatos (Seção XII).

XII Apêndice A: uma aplicação completa

Os capítulos anteriores mostraram cada mecanismo isoladamente. Este apêndice junta tudo num único cenário que compila, carrega e roda — do `main()` ao arquivo de configuração, passando pelos comandos de *build*.

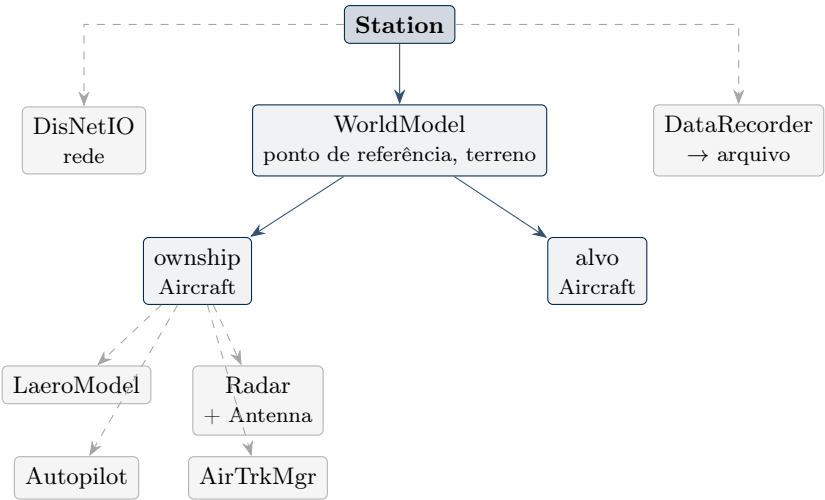
O cenário é deliberadamente pequeno mas completo: **uma interceptação**. Uma aeronave de caça com radar e mísseis, seguindo uma rota sob autopiloto, detecta um alvo que voa em rota de colisão. Há terreno, há gravação de dados, e há rede DIS. Cada peça foi apresentada num capítulo diferente; aqui elas aparecem juntas, e é a montagem que interessa.

Regra — todo EDL deste manual é verificado contra o fonte

Os arquivos deste apêndice — e todos os demais blocos EDL do manual — passam por um verificador que confere, contra o código-fonte do MIXR, se cada nome de fábrica existe e está registrado, e se cada *slot* pertence à classe ou a um ancestral seu. O procedimento está descrito na Seção XII.6.

XII.1 A árvore do cenário

Antes do texto, a forma. Toda simulação MIXR é uma árvore cuja raiz é uma *Station*, e vale ter o desenho em mente ao ler o arquivo:



Duas relações diferentes aparecem nesse desenho, e confundi-las é o erro mais comum de quem escreve o primeiro cenário. A *Station* guarda os seus subsistemas em **slots nomeados** (*simulation*, *dataRecorder*, *networks*). Já um *player* guarda os seus subsistemas em **components** — e é por tipo, não por nome, que o *Player* os encontra depois (Seção VI.3.2).

XII.2 O arquivo de configuração

```
1 ( Station
2
3 //-----
```

```

4 // Taxas das threads (Secao sec:sim-station-edl)
5 //-----
6 tcRate: 50 // fisica e sensores a 50 Hz
7 tcPriority: 0.8 // SCHED_FIFO no Linux
8 bgRate: 10 // tarefas de fundo a 10 Hz
9 netRate: 20 // rede a 20 Hz
10
11 ownship: "falcao1" // de quem e a perspectiva da aplicacao
12
13 //-----
14 // O executive, situado no globo
15 //-----
16 simulation: ( WorldModel
17
18 // Ponto de referencia: centro da gaming area (Vale do Paraiba)
19 latitude: ( LatLon direction: s degrees: 23 minutes: 10 )
20 longitude: ( LatLon direction: w degrees: 45 minutes: 50 )
21 gamingAreaRange: ( KiloMeters 150 )
22
23 earthModel: wgs84
24 gamingAreaUseEarthModel: true
25
26 // Terreno: quatro tiles em grade 2x2 (Secao sec:terrain-quadmap)
27 terrain: ( QuadMap
28 components: {
29 sw: ( DtedFile path: "/data/terrain" file: "s24w046.dt1" )
30 se: ( DtedFile path: "/data/terrain" file: "s24w045.dt1" )
31 nw: ( DtedFile path: "/data/terrain" file: "s23w046.dt1" )
32 ne: ( DtedFile path: "/data/terrain" file: "s23w045.dt1" )
33 }
34 )
35
36 players: {
37
38 //-----
39 // OWNSHIP -- o caçador
40 //-----
41 falcao1: ( Aircraft
42 type: "F-16C"
43 side: blue
44
45 initLatitude: ( LatLon direction: s degrees: 23 minutes: 30 )
46 initLongitude: ( LatLon direction: w degrees: 46 minutes: 10 )
47 initAlt: ( Feet 25000 )
48 initHeading: ( Degrees 45 )
49 initVelocityKts: 420
50
51 // Assinatura radar: constante, 5 m2 (Secao sec:models-rfsig)
52 signature: ( SigConstant rcs: ( SquareMeters 5.0 ) )
53
54 components: {
55
56 // --- Dinamica de voo (Secao sec:models-laero) ---
57 dinamica: ( LaeroModel )
58
59 // --- Cadeia RF: Radar contem a Antenna (Secao sec:models-rfsensor)
60 radar1: ( Radar
61 // O radar acha o gerente de pistas PELO NOME:
62 trackManagerName: "gerentePistas"
63 antennaName: "antena1"
64
65 frequency: ( GigaHertz 10 )
66 powerPeak: ( KiloWatts 100 )
67 bandwidth: ( MegaHertz 1 )
68 ranges: [ 20 40 80 160 ]
69 initRangeIdx: 3
70
71 components: {
72 antena1: ( Antenna
73 polarization: vertical
74 gain: 8000.0
75 scanMode: horizontal

```

```

76         scanWidth: ( Degrees 120 )
77         numBars: 4
78     )
79 }
80 )
81
82 // --- Gerente de pistas (Secao sec:models-trackmanagers) ---
83 gerentePistas: ( AirTrkMgr
84     maxTracks: 20
85     maxTrackAge: 3.0
86     firstTrackId: 1000
87     alpha: 1.0
88     beta: 0.5      // gamma nao teria efeito: ver o quadro
89                   // da Secao sec:models-trackmanagers
90 )
91
92 // --- Navegacao e rota (Secao sec:models-route) ---
93 nav1: ( Navigation
94     route: ( Route
95         autoSequence: true
96         autoSeqDistance: ( NauticalMiles 2.0 )
97         to: 1
98         components: {
99             wp1: ( Steerpoint
100                 latitude: ( LatLon direction: s degrees: 23 minutes: 20 )
101                 longitude: ( LatLon direction: w degrees: 46 minutes: 0 )
102                 altitude: ( Feet 25000 )
103                 airspeed: 420.0
104             )
105             wp2: ( Steerpoint
106                 latitude: ( LatLon direction: s degrees: 23 minutes: 0 )
107                 longitude: ( LatLon direction: w degrees: 45 minutes: 40 )
108                 altitude: ( Feet 28000 )
109                 airspeed: 420.0
110             )
111         }
112     )
113 )
114
115 // --- Autopiloto (Secao sec:models-autopilot) ---
116 // Os limites NAO tem default util: ver o quadro daquela secao.
117 piloto: ( Autopilot
118     navMode: true
119     altitudeHoldMode: true
120     velocityHoldMode: true
121     maxBankAngle: 30.0
122     maxRateOfTurnDps: 3.0
123     maxPitchAngle: 15.0
124     maxClimbRateMps: 10.0
125     maxAcceleration: 5.0
126 )
127
128 // --- Armamento (Secao sec:models-storesmgr) ---
129 // "StoresMgr" constroi um SimpleStoresMgr. Ver o quadro
130 // da Secao sec:base-fabrica-nomes.
131 sms: ( StoresMgr
132     stores: {
133         1: ( Missile type: "AIM-120C" maxSpeed: 1400.0 )
134         3: ( Missile type: "AIM-120C" maxSpeed: 1400.0 )
135     }
136 )
137
138 //-----
139 // ALVO -- rota de colisao, sem sensores
140 //-----
141
142 alvo1: ( Aircraft
143     type: "Su-27"
144     side: red
145
146     initLatitude: ( LatLon direction: s degrees: 22 minutes: 50 )
147     initLongitude: ( LatLon direction: w degrees: 45 minutes: 30 )

```

```

148         initAlt:      ( Feet 26000 )
149         initHeading:  ( Degrees 225 )
150         initVelocityKts: 450
151
152         signature: ( SigConstant rcs: ( dB 8.0 ) ) // 8 dBsm
153
154         components: {
155             // RacModel: convergencia de primeira ordem (Secao sec:models-rac)
156             dinamica: ( RacModel
157                 cmdAltitude: ( Feet 26000 )
158                 cmdHeading: ( Degrees 225 )
159                 cmdSpeed: 450.0
160             )
161         }
162     )
163 }
164 )
165
166 //-----
167 // Gravador (Secao sec:recorder) -- note o nome de fabrica do FileWriter
168 //-----
169 dataRecorder: ( DataRecorder
170     eventName: "intercepcao"
171     caseNum: 1
172     missionNum: 1
173     outputHandler: ( RecorderFileWriter
174         pathname: "/data/gravacoes"
175         filename: "intercepcao.rcd"
176     )
177 )
178
179 //-----
180 // Rede DIS (Secao sec:interop)
181 //-----
182 networks: {
183     dis1: ( DisNetIO
184         siteID: 10
185         applicationID: 1
186         exerciseID: 17
187
188         netInput: ( UdpMulticastHandler
189             multicastGroup: "225.0.0.17" port: 3000 localPort: 3000 )
190         netOutput: ( UdpMulticastHandler
191             multicastGroup: "225.0.0.17" port: 3000 )
192
193         // Sem inputEntityType, TODA entidade da rede e ignorada.
194         // Ver o quadro da Secao sec:interop-ntm.
195         inputEntityType: {
196             ( DisNtm
197                 disEntityType: [ 1 2 225 1 1 1 0 ]
198                 template: ( Aircraft type: "F-16C" )
199             )
200         }
201         outputEntityType: {
202             ( DisNtm
203                 disEntityType: [ 1 2 225 1 1 1 0 ]
204                 template: ( Aircraft type: "F-16C" )
205             )
206         }
207     )
208 }
209 )

```

Listing 295: `intercepcao.edl` — o cenário completo.

XII.3 A aplicação em C++

O arquivo acima não faz nada sozinho: alguém precisa construí-lo e girar o laço. Esse alguém é um `main()` curto, e as suas quatro responsabilidades são sempre as mesmas.

```
1  #include "mixr/base/edl_parser.hpp"
2  #include "mixr/base/Pair.hpp"
3  #include "mixr/base/util/system_utils.hpp"
4  #include "mixr/simulation/Station.hpp"
5
6  #include "mixr/base/factory.hpp"
7  #include "mixr/simulation/factory.hpp"
8  #include "mixr/models/factory.hpp"
9  #include "mixr/terrain/factory.hpp"
10 #include "mixr/interop/dis/factory.hpp"
11 #include "mixr/recorder/factory.hpp"
12 #include "mixr/linkage/factory.hpp"
13
14 #include <cstdlib>
15 #include <iostream>
16
17 namespace mixr {
18
19 //-----
20 // 1. A FABRICA -- a ordem AQUI define a precedencia dos nomes.
21 //    Nao existe registro global: o encadeamento e este codigo.
22 //    (Secao sec:models-factory)
23 //-----
24 base::Object* factory(const std::string& name)
25 {
26     base::Object* obj {};
27
28     // Classes proprias da aplicacao viriam primeiro, para poder
29     // sobrepôr um nome de fabrica do framework:
30     // obj = minhaApp::factory(name);
31
32     if (obj == nullptr) obj = models::factory(name);
33     if (obj == nullptr) obj = dis::factory(name);
34     if (obj == nullptr) obj = recorder::factory(name);
35     if (obj == nullptr) obj = linkage::factory(name);
36     if (obj == nullptr) obj = simulation::factory(name);
37     if (obj == nullptr) obj = terrain::factory(name);
38     if (obj == nullptr) obj = base::factory(name);
39
40     return obj;
41 }
42
43 //-----
44 // 2. CARGA -- construir a arvore a partir do arquivo, e CONFERIR os erros
45 //-----
46 simulation::Station* carrega(const char* const arquivo)
47 {
48     int nErros {};
49     base::Object* raiz { base::edl_parser(arquivo, factory, &nErros) };
50
51     // Um ponteiro nao-nulo NAO significa que o arquivo estava correto:
52     // slots mal escritos apenas incrementam o contador. Ver Secao sec:base-parser.
53     if (nErros > 0) {
54         std::cerr << "erros na configuracao: " << nErros << std::endl;
55         return nullptr;
56     }
57
58     // O parser devolve um Pair quando o arquivo começa com "nome: ( ... )",
59     // e o objeto nu quando começa com "( ... )". Aceitar os dois:
60     if (const auto par = dynamic_cast<base::Pair*>(raiz)) {
61         raiz = par->object();
62     }
63
64     const auto station = dynamic_cast<simulation::Station*>(raiz);
65     if (station == nullptr) {
```



```

66         std::cerr << "a raiz do arquivo nao e uma Station" << std::endl;
67         return nullptr;
68     }
69     return station;
70 }
71
72 } // namespace mixr
73
74 //-----
75 int main(int argc, char* argv[])
76 {
77     const char* arquivo { (argc > 1) ? argv[1] : "intercepcao.edl" };
78
79     mixr::simulation::Station* station { mixr::carrega(arquivo) };
80     if (station == nullptr) return EXIT_FAILURE;
81
82     //-----
83     // 3. RESET -- condicoes iniciais. So DEPOIS disso a arvore esta viva.
84     // E aqui que o terreno e lido do disco e o pool de threads e criado.
85     //-----
86     station->event(mixr::base::Component::RESET_EVENT);
87
88     // A thread de tempo critico e criada EXPLICITAMENTE pela aplicacao --
89     // ao contrario das de fundo e de rede. Ver Secao sec:sim-station-data.
90     station->createTimeCriticalProcess();
91
92     //-----
93     // 4. O LACO -- aqui so restam as tarefas de fundo: a thread TC ja roda.
94     //-----
95     const double dt { 1.0 / 10.0 }; // 10 Hz de fundo
96     for (;;) {
97         station->updateData(dt);
98         mixr::base::msleep(100); // 100 ms
99     }
100
101     return EXIT_SUCCESS;
102 }

```

Listing 296: main.cpp — a aplicação mínima completa.

Quatro observações sobre esse `main()`, cada uma remetendo a uma armadilha já vista:

1. **A fábrica é código, não configuração.** Não há registro global nem descoberta automática; a ordem das linhas é a precedência (Seção VI.16.3).
2. **nErros tem de ser conferido.** O *parser* devolve a árvore mesmo com dezenas de *slots* malsucedidos; ignorar o contador é subir uma simulação silenciosamente malconfigurada (Seção III.3).
3. **O RESET_EVENT não é opcional.** É ele que carrega o terreno, monta a lista de *players* e cria o *pool* de *threads*. Antes dele, a árvore existe mas não está inicializada.
4. **A thread de tempo crítico é criada à mão.** Omitir `createTimeCriticalProcess()` é uma escolha válida — significa que a aplicação vai chamar `station->updateTC(dt)` ela mesma, de um laço próprio ou de uma interrupção externa (Seção IV.5.3).

XII.4 Compilando e executando

```

1 project('intercepcao', 'cpp',
2         version : '0.1.0',
3         default_options : ['cpp_std=c++17'])
4
5 # Uma unica declaracao cobre as nove bibliotecas do MIXR:

```

```

6  # o .pc gerado pelo Conan ja as lista na ordem correta de ligacao.
7  mixr_dep = dependency('mixr', method : 'pkg-config')
8
9  executable('intercepcao',
10     sources      : ['main.cpp'],
11     dependencies : [mixr_dep],
12     install      : true)

```

Listing 297: meson.build da aplicação.

```

1  # 1. Dependencias e arquivos de integracao (gera os .pc e o native file )
2  conan install . --output-folder=build --build=missing
3
4  # 2. Configuracao do Meson, apontando para o que o Conan gerou
5  PKG_CONFIG_PATH="$(pwd)/build" \
6  meson setup build \
7     --native-file build/conan_meson_native.ini \
8     --buildtype=release
9
10 # 3. Compilacao
11 meson compile -C build
12
13 # 4. Execucao
14 ./build/intercepcao intercepcao.edl

```

Listing 298: Do repositório limpo à simulação rodando.

XII.5 O que observar rodando

O cenário foi montado para que cada mecanismo do manual se manifeste. Vale percorrer o que acontece, quadro a quadro, na primeira dezena de segundos:

1. **Fase 0.** `LaeroModel` integra a posição do *ownship*; `RacModel` converge o alvo para o seu rumo e altitude comandados. Ambos escrevem via `setPosition()`, que sincroniza NED, ECEF e geodésicas (Seção VI.3.1).
2. **Fundo.** `RfSystem::updateData()` monta o Tdb com os alvos de interesse — e note que isso é a *thread* de fundo, não a fase 1 (Seção VI.5.2).
3. **Fase 1.** `Antenna::rfTransmit()` calcula o ganho na direção do alvo, monta uma `Emission` com a perda de percurso já embutida e a entrega ao alvo (Seção VI.5.3).
4. **Fase 2.** O eco volta; `Radar::receive()` aplica a RCS e a segunda perda de percurso, compara com o limiar em dB e enfileira o relatório (Seção VI.5.4).
5. **Fase 3.** `Radar::process()` consolida os relatórios da varredura e chama `AirTrkMgr::newReport()`; o filtro α - β passa a manter a pista (Seção VI.6). `Autopilot::process()` lê a rota e comanda rumo, altitude e velocidade (Seção VI.10.1).
6. **Em paralelo.** A *thread* de rede publica o *Entity State PDU* do *ownship* quando o erro de *dead reckoning* passar de 3 m ou 3°, ou a cada 5 s (Seção X.3). O gravador drena a sua fila e escreve o arquivo (Seção IX.4).

Regra — o roteiro de depuração

Quando um cenário não se comporta como esperado, a ordem de investigação que poupa mais tempo é esta:

1. `nErros` do *parser* é zero?
2. O `RESET_EVENT` foi enviado?
3. Os subsistemas estão em `components`, e não em *slots* inventados? (Um filho de tipo errado some sem aviso — Seção III.2.1.)
4. Os nomes cruzados batem? `trackManagerName` e `antennaName` são resolvidos por *string*, e um erro de digitação neles não produz erro de carga.
5. Ligue `enableMessageType: DEBUG` no nó suspeito — a política desce por toda a subárvore (Seção III.2.6).

XII.6 Como este manual é verificado

Um manual técnico envelhece pior do que o código que documenta, e a forma mais rápida de perder a confiança do leitor é um exemplo que não funciona. Este manual acompanha um verificador que fecha esse laço automaticamente.

O script `scripts/check-edl.py` extrai do código-fonte três coisas: os nomes de fábrica de todo `IMPLEMENT_*SUBCLASS`, a cadeia de herança de todo `DECLARE_SUBCLASS`, e os *slots* de todo `BEGIN_SLOTTABLE`. Depois percorre *todos* os blocos `language=EDL` dos capítulos e verifica, com a estrutura de aninhamento correta:

1. que cada `(Nome ...)` corresponde a um nome de fábrica que existe e está registrado em alguma `factory.cpp`;
2. que cada `nome:` é um *slot* da classe corrente ou de um ancestral — respeitando que nomes dentro de `{ }` são nomes de `Pair`, e não *slots*.

```
1 $ python3 scripts/check-edl.py
2 nomes de fabrica no fonte: 334   registrados: 224
3 blocos EDL no manual: 41
4
5 EDL: nenhuma violacao.
```

Listing 299: Rodando o verificador.

Foi esse verificador que revelou a maior parte dos erros de EDL corrigidos nesta edição — inclusive a divergência sistemática entre nome de classe e nome de fábrica da Seção III.3.1, que não era evidente lendo o texto. Ao atualizar o manual para uma versão nova do MIXR, rodá-lo é a primeira coisa a fazer: ele aponta exatamente quais exemplos deixaram de ser válidos.

XIII Apêndice B: nomes de fábrica que divergem do nome da classe

A Seção III.3.1 estabeleceu a regra: o nome de fábrica é uma *string* independente do nome da classe C++, e nas 28 classes listadas abaixo os dois divergem. Escrever o nome da classe onde o *framework* espera o nome de fábrica é a causa mais comum de arquivos EDL que não carregam — e, no caso de `StoresMgr`, de arquivos que carregam a classe errada sem avisar.

Esta tabela cobre apenas as classes **registradas** em alguma `factory.cpp` deste *fork*: são as que podem de fato ser instanciadas a partir de um arquivo de configuração. Os módulos omitidos (`simulation`, `terrain`, `linkage`) não têm nenhuma divergência.

base

Nome de fábrica (EDL)	Classe C++
*	Multiply
+	Add
-	Subtract
/	Divide
UbfAgent	Agent
UbfArbiter	Arbiter
boolean	Boolean
cie	Cie
cmv	Cmv
dB	Decibel
float	Float
hls	Hls
hsv	Hsv
hsva	Hsva
int	Integer
rgb	Rgb
rgba	Rgba
yi	Yi

models

Nome de fábrica (EDL)	Classe C++
AamMissile	Aam
AgmMissile	Agm
StoresMgr	SimpleStoresMgr

recorder

Nome de fábrica (EDL)	Classe C++
RecorderFileReader	FileReader
RecorderFileWriter	FileWriter
RecorderNetInput	NetInput
RecorderNetOutput	NetOutput
RecorderOutputHandler	OutputHandler

interop

Nome de fábrica (EDL)	Classe C++
DisNetIO	NetIO
DisNtm	Ntm

Regra — como conferir

A tabela acima foi extraída do código-fonte, não digitada. Para regenerá-la contra outra versão do MIXR, ou para conferir um nome específico:

```

1  # O nome de fabrica de uma classe especifica:
2  grep -rn "IMPLEMENT_.*SUBCLASS(NomeDaClasse" tmp/mixr/src/
3
4  # Todas as divergencias de uma vez:
5  grep -rhno 'IMPLEMENT_[A-Z_]*SUBCLASS([A-Za-z0-9_]*, *"[^"]*")' tmp/mixr/src/ \
6  | sed 's/.*SUBCLASS(/;/ s/, */ -> /; s/"/)/' \
7  | awk -F' -> ' '{ $1 != $2 }
```

Listing 300: Localizando o nome de fábrica de qualquer classe.

As três outras razões para um nome não funcionar. Além da divergência, um nome correto ainda pode falhar por não estar **registrado**. Dos 334 nomes de fábrica declarados no fonte, 224 estão registrados. Os casos notáveis de ausência, todos discutidos no texto:

- **UbfAgentTC** — a classe existe, o nome existe, e `base/factory.cpp` registra apenas `UbfAgent` e `UbfArbiter` (Seção III.10.2).
- **BaseIoHandler** — além de não registrado, é abstrato e não tem implementação concreta em lugar nenhum (Seção VIII.6).
- **Todo o módulo linearsystem** — não possui `factory.cpp` (Seção VII).

Em todos esses casos a saída é a mesma: registrar a classe na fábrica da própria aplicação, como mostra o Seção XII.3.

Referências

- [1] MIXR Project, “MIXR: Mixed Reality Simulation,” Sítio oficial do projeto, 2026. URL <https://www.mixr.dev/>, acessado em 21 ago. 2026.

Glossário

ciclo

Conjunto de 16 *frames* consecutivos. Serve de relógio para tarefas de baixa frequência, que se agendam por `frame() % N`

DRM

Dead Reckoning Model. Um dos oito algoritmos padronizados pelo DIS para extrapolar a posição de uma entidade remota entre atualizações de rede

ECEF

Earth-Centered, Earth-Fixed. Sistema cartesiano com origem no centro da Terra e eixos fixos a ela. É o referencial em que o estado trafega na rede

EDL

English Description Language. A linguagem de configuração do MIXR, na qual se descreve a estrutura de uma simulação — quais objetos existem, como se aninham e com que parâmetros

ERP

Effective Radiated Power. A potência efetivamente irradiada numa direção: potência do transmissor multiplicada pelo ganho da antena naquela direção

fase

Uma das quatro subdivisões de um *frame* de tempo crítico (dinâmica, transmissão, recepção, lógica). A lista de *players* é percorrida uma vez por fase, mas cada método de fase recebe o Δt integral do *frame*

flyout

O clone de uma arma que efetivamente voa. A arma declarada em EDL permanece presa ao cabide e passa a LAUNCHED; o clone entra na lista de *players* em PRE_RELEASE e vira ACTIVE no quadro seguinte

gaming area

A região de interesse da simulação, definida por um ponto de referência geodésico e um raio. Serve de origem do plano tangente local (NED) em que os *players* se movem

LFI

Linear Function Interpolation. A interpolação linear multidimensional do MIXR, usada em toda função tabelada — coeficientes aerodinâmicos, padrões de antena, transmitância atmosférica

LLA

Latitude, Longitude, Altitude. Coordenadas geodésicas: latitude e longitude em graus, altitude em metros acima do elipsoide

NED

North-East-Down. Plano tangente local, ancorado no ponto de referência da *gaming area*. É o referencial de trabalho da dinâmica de voo e dos sensores

NIB

Network Interface Block. O objeto que representa a relação entre um *player* e a rede: guarda o estado do último *update* e extrapola a posição entre eles

nome de fábrica

A *string* pela qual uma classe é instanciável a partir de um arquivo EDL. É definida no segundo argumento de `IMPLEMENT_SUBCLASS` e **não precisa coincidir com o nome da classe C++** — em 56 classes do MIXR ela difere

Ntm

Network Type Mapper. O objeto que associa um tipo de entidade da rede a uma classe de *player* do MIXR, em cada direção (entrada e saída) separadamente

ownship

O *player* sob controle da estação — aquele cuja perspectiva a aplicação apresenta. É indicado pelo *slot ownship* da *Station* e pode ser trocado em tempo de execução

PDU

Protocol Data Unit. A unidade de mensagem do protocolo DIS. Cada tipo (Entity State, Fire, Detonation, ...) tem um leiaute binário fixo, transmitido em *big-endian*

PLA

Power Lever Angle. Posição normalizada da manete: 0,0 = *idle*, 1,0 = potência militar, 2,0 = pós-combustor. É a variável de entrada das tabelas de assinatura IR de escapamento

RCS

Radar Cross Section. A área equivalente, em m², que descreve quanta energia de radar um alvo reflete de volta ao emissor. Frequentemente expressa em dBsm

slot

Atributo nomeado que um objeto expõe ao arquivo de configuração. Cada classe declara os seus numa *slot table*; o *parser* resolve o nome para um índice global e despacha para a função C++ correspondente. É o mecanismo que liga o texto EDL ao código

Tdb

O *buffer* de geometria que um *gimbal* preenche com os alvos de interesse de um quadro. Apesar do nome (*Track Data Block*), não tem relação com **Track**: guarda alvos candidatos, não pistas rastreadas

TOF

Time of Flight. Tempo decorrido desde o lançamento de uma arma. Governa o início da guiagem, a janela de queima do motor e a autodestruição

Índice remissivo

AbstractGenerator, 206
AbstractWeapon, 172
AirTrkMgr, 186
AngleOnlyTrackManager, 152
Antenna, 186
Autopilot, 186, 201
AvionicsPod, 157

Bullet, 157

Color, 27
Component, 18, 94, 144

Datalink, 154
DataRecorder, 234
DataRecordHandle, 217
DynamicsModel, 144

Effect, 172
Emission, 186
ExternalStore, 156

FuelTank, 157
Func1, 71
Func5, 71

Gun, 156

Image, 153
IrTrack, 180

Jammer, 154

LaeroModel, 186
LagFilter, 201
LatLon, 85, 94

MergingIrSensor, 152

Ntm, 228
Number, 27, 85

Object, 18
OnboardComputer, 155
OutputHandler, 20

Pair, 242
PairStream, 19
Polynomial, 72

QuadMap, 130, 234

RacModel, 186

Radar, 153, 186
Referenced, 18
RfSensor, 150
RfSystem, 154
RfTrack, 180
Route, 144

safe_ptr, 20, 108
ScanGimbal, 148
SigConstant, 186
SimpleStoresMgr, 30, 156
slots
 a, 164
 actorComponentName, 182
 actorPlayerName, 182
 adapters, 204
 ai, 206
 alpha, 152
 angle, 180
 antenna, 225
 application, 213
 armDelay, 180
 autoSeqDistNM, 158
 autoSequence, 158
 azimuth, 180
 azimuthBin, 152
 b, 164
 backlog, 89
 beta, 152
 bulletType, 157
 burstRate, 157
 caseNum, 213
 channel, 205
 chipSize, 153
 compareToValD, 217
 compareToValI, 217
 compareToValS, 217
 completed, 180
 components, 19
 condition, 217
 day, 105, 213
 defaultIn, 225
 defaultOut, 225
 degrees, 85
 di, 205, 206
 direction, 85
 disableMessageType, 27
 dragIndex, 172

- elevationBin, 152
- emissionPduHandlers, 224
- enabled, 180
- enableMode1, 155
- enableMode2, 155
- enableMode3a, 155
- enableMode4, 155
- enableModeC, 155
- enableTimingStats, 26
- eventName, 213
- fieldName, 217
- file, 121
- filename, 81
- frequency, 206
- fuzeTime, 156
- gamma, 152
- generators, 205, 206
- ignoreSourcePort, 89
- initAlt, 135
- initGearPos, 174
- initGeocentric, 135
- initLatitude, 135
- initLongitude, 135
- initXPos, 135
- initYPos, 135
- inputData, 203
- inputEntityTypes, 226
- interval, 180
- inverted, 205
- ioData, 203
- irSignature, 173
- jettisonable, 157
- level, 205
- localPort, 88
- manualAssign, 180
- maxEntityRange, 227
- maxMissDistance, 180
- maxOrientationError, 220
- maxPlayersOfInterest, 146
- maxPositionError, 220
- maxTimeDR, 220
- messageToken, 217
- minutes, 85
- missionNum, 213
- mode1, 155
- mode2, 155
- mode3a, 155
- mode4a, 155
- mode4b, 155
- month, 105, 213
- msgHdrOptn, 215
- outputData, 203
- outputEntityTypes, 226
- outputHandler, 213
- path, 121
- pathname, 81
- phase, 206
- pitch, 157
- port, 88, 89
- position, 157
- printTimingStats, 26
- quantity, 180
- rate, 157
- recordLength, 81
- roll, 157
- rounds, 157
- runNum, 213
- seconds, 85
- select, 26
- sensor, 225
- signal, 206
- signature, 173
- simulationTime, 105
- stickDistance, 180
- stickType, 180
- subjectNum, 213
- table, 71, 206
- template, 226
- type, 156, 157
- typeId, 225
- unlimited, 157
- usePerceivedPosVel, 152
- value, 206
- velocity, 180
- version, 222
- weaponType, 180
- whichMode4, 155
- yaw, 157
- year, 105, 213
- SpaceDynamicsModel, 177
- Station, 27, 119, 235
- Statistic, 22, 26
- Steerpoint, 144
- String, 27
- SynchronizedState, 222
- System, 144
- Table1, 20, 27
- TargetData, 180
- Timer, 27
- Track, 155, 172

Transform, 44